IEEE P802.1CB-2017-Rev/D1.2
20 August 2026

Revision of IEEE P802.1CB™-2017 as amended by IEEE Std 802.1CBcv-2021
and IEEE Std 802.1CBdb-2021
and corrected by IEEE Std 802.1CB-2017/Cor1-2026

Draft Standard for
Local and metropolitan area networks —

Frame Replication and Elimination for Reliability

Prepared by the Maintenance Task Group of IEEE 802.1

LAN MAN Standards Committee
of the
IEEE Computer Society

Copyright © 2026 by the IEEE.
Three Park Avenue
New York, New York 10016-5997, USA

Important Notice

This document is an unapproved draft of a proposed IEEE Standard. IEEE hereby grants the named IEEE SA Working Group or Standards Committee Chair permission to distribute this document to participants in the receiving IEEE SA Working Group or Standards Committee, for purposes of review for IEEE standardization activities. No further use, reproduction, or distribution of this document is permitted without the express written permission of IEEE Standards Association (IEEE SA). Prior to any review or use of this draft standard, in part or in whole, by another standards development organization, permission must first be obtained from IEEE SA (stds-copyright@ieee.org). This page is included as the cover of this draft, and shall not be modified or deleted.

IEEE Standards Association
445 Hoes Lane
Piscataway, NJ 08854, USA

All rights reserved.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Abstract: This revision of IEEE Std 802.1CB™-2017 incorporates the amendments .

Keywords: TSN, Time-Sensitive Networking, Redundancy, Bridging, Bridges, Frame Replication, Frame Elimination, Stream identification, Extended Stream identification, Bridged Local Area Networks, IEEE 802®, IEEE 802.1Q™, IEEE 802.1CB™, IEEE 802.1CBcv™, IEEE 802.1CBdb™, local area networks (LANs), MAC Bridges, Virtual Bridged Local Area Networks (virtual LANs), amendment.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Important Notices and Disclaimers Concerning IEEE Standards Documents

IEEE Standards documents are made available for use subject to important notices and legal disclaimers. These notices and disclaimers, or a reference to this page (https://standards.ieee.org/ipr/disclaimers.html), appear in all IEEE standards and may be found under the heading "Important Notices and Disclaimers Concerning IEEE Standards Documents."

Notice and Disclaimer of Liability Concerning the Use of IEEE Standards Documents

IEEE Standards documents are developed within IEEE Societies and subcommittees of IEEE Standards Association (IEEE SA) Board of Governors. IEEE develops its standards through an accredited consensus development process, which brings together volunteers representing varied viewpoints and interests to achieve the final product. IEEE Standards are documents developed by volunteers with scientific, academic, and industry-based expertise in technical working groups. Volunteers involved in technical working groups are not necessarily members of IEEE or IEEE SA and participate without compensation from IEEE. While IEEE administers the process and establishes rules to promote fairness in the consensus development process, IEEE does not independently evaluate, test, or verify the accuracy of any of the information or the soundness of any judgments contained in its standards.

IEEE makes no warranties or representations concerning its standards, and expressly disclaims all warranties, express or implied, concerning all standards, including but not limited to the warranties of merchantability, fitness for a particular purpose and non-infringement. IEEE Standards documents do not guarantee safety, security, health, or environmental protection, or compliance with law, or guarantee against interference with or from other devices or networks. In addition, IEEE does not warrant or represent that the use of the material contained in its standards is free from patent infringement. IEEE standards documents are supplied "AS IS" and "WITH ALL FAULTS."

Use of an IEEE standard is wholly voluntary. The existence of an IEEE standard does not imply that there are no other ways to produce, test, measure, purchase, market, or provide other goods and services related to the scope of the IEEE standard. Furthermore, the viewpoint expressed at the time a standard is approved and issued is subject to change brought about through developments in the state of the art and comments received from users of the standard.

In publishing and making its standards available, IEEE is not suggesting or rendering professional or other services for, or on behalf of, any person or entity, nor is IEEE undertaking to perform any duty owed by any other person or entity to another. Any person utilizing any IEEE Standards document should rely upon their own independent judgment in the exercise of reasonable care in any given circumstances or, as appropriate, seek the advice of a competent professional in determining the appropriateness of a given IEEE standard.

IN NO EVENT SHALL IEEE BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO: THE NEED TO PROCURE SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE PUBLICATION, USE OF, OR RELIANCE UPON ANY STANDARD, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE AND REGARDLESS OF WHETHER SUCH DAMAGE WAS FORESEEABLE.

Translations

The IEEE consensus balloting process involves the review of documents in English only. In the event that an IEEE standard is translated, only the English language version published by IEEE is the approved IEEE standard.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Use by artificial intelligence systems

In no event shall material in any IEEE Stadards documents be used for the purpose of creating, training, enhancing, developing, maintaining, or contributing to any artificial intelligence systems without the express, written consent of the IEEE SA in advance. “Artificial intelligence” refers to any software, application, or other system that uses artificial intelligence, machine learning, or similar technologies, to analyze, train, process, or generate content. Requests for consent can be submitted using the Contact Us form.[1]

Official statements

A statement, written or oral, that is not processed in accordance with the IEEE SA Standards Board Operations Manual is not, and shall not be considered or inferred to be, the official position of IEEE, IEEE SA, or any of its committees, and is not, and shall not be considered to be or be relied upon as, a formal position of IEEE or IEEE SA. At lectures, symposia, seminars, or educational courses, an individual presenting information on IEEE standards shall make it clear that the presenter's views should be considered the personal views of that individual rather than the formal position of IEEE, IEEE SA, the Standards Committee, or the Working Group. Statements made by volunteers may not represent the formal position of their employer(s) or affiliation(s). News releases about IEEE standards issued by entities other than IEEE SA should be considered the view of the entity issuing the release rather than the formal position of IEEE or IEEE SA.

Comments on standards

Comments for revision of IEEE Standards documents are welcome from any interested party, regardless of membership affiliation with IEEE or IEEE SA. However, **IEEE does not provide interpretations, consulting information, or advice pertaining to IEEE Standards documents**.

Suggestions for changes in documents should be in the form of a proposed change of text, together with appropriate supporting comments. Since IEEE standards represent a consensus of concerned interests, it is important that any responses to comments and questions also receive the concurrence of a balance of interests. For this reason, IEEE and the members of its Societies and subcommittees of the IEEE SA Board of Governors are not able to provide an instant response to comments or questions, except in those cases where the matter has previously been addressed. For the same reason, IEEE does not respond to interpretation requests. Any person who would like to participate in evaluating comments or revisions to an IEEE standard is welcome to join the relevant IEEE SAworking group. You can indicate interest in a working group using the Interests tab in the Manage Profile & Interests area of the IEEE SA myProject system.[2]

Comments on standards should be submitted using the Contact Us form.

Laws and regulations

Users of IEEE Standards documents should consult all applicable laws and regulations. Compliance with the provisions of any IEEE Standards document does not constitute compliance to any applicable regulatory requirements. Implementers of the standard are responsible for observing or referring to the applicable regulatory requirements. IEEE does not, by the publication of its standards, intend to urge action that is not in compliance with applicable laws, and these documents may not be construed as doing so.

[1] Available at: https://standards.ieee.org/content/ieee-standards/en/about/contact/index.html.

[2] Available at: https://development.standards.ieee.org/myproject-web/public/view.html#landing.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Data privacy

Users of IEEE Standards documents should evaluate the standards for considerations of data privacy and data ownership in the context of assessing and using the standards in compliance with applicable laws and regulations.

Copyrights

IEEE draft and approved standards are copyrighted by IEEE under U.S. and international copyright laws. They are made available by IEEE and are adopted for a wide variety of both public and private uses. These include both use by reference, in laws and regulations, and use in private self-regulation, standardization, and the promotion of engineering practices and methods. By making these documents available for use and adoption by public authorities and private users, neither IEEE nor its licensors waive any rights in copyright to the documents.

Photocopies

Subject to payment of the appropriate licensing fees, IEEE will grant users a limited, non-exclusive license to photocopy portions of any individual standard for company or organizational internal use or individual, non-commercial use only. To arrange for payment of licensing fees, please contact Copyright Clearance Center, Customer Service, 222 Rosewood Drive, Danvers, MA 01923 USA; +1 978 750 8400; https://www.copyright.com/. Permission to photocopy portions of any individual standard for educational classroom use can also be obtained through the Copyright Clearance Center.

Updating of IEEE Standards documents

Users of IEEE Standards documents should be aware that these documents may be superseded at any time by the issuance of new editions or may be amended from time to time through the issuance of amendments, corrigenda, or errata. An official IEEE document at any point in time consists of the current edition of the document together with any amendments, corrigenda, or errata then in effect.

Every IEEE standard is subjected to review at least every 10 years. When a document is more than 10 years old and has not undergone a revision process, it is reasonable to conclude that its contents, although still of some value, do not wholly reflect the present state of the art. Users are cautioned to check to determine that they have the latest edition of any IEEE standard.

In order to determine whether a given document is the current edition and whether it has been amended through the issuance of amendments, corrigenda, or errata, visit IEEE Xplore or contact IEEE.[3] For more information about the IEEE SA or IEEE's standards development process, visit the IEEE SA Website.

Errata

Errata, if any, for all IEEE standards can be accessed on the IEEE SA Website.[4] Search for standard number and year of approval to access the web page of the published standard. Errata links are located under the Additional Resources Details section. Errata are also available in IEEE Xplore. Users are encouraged to periodically check for errata.

[3] Available at: https://ieeexplore.ieee.org/browse/standards/collection/ieee.

[4] Available at: https://standards.ieee.org/standard/index.html.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Patents

IEEE standards are developed in compliance with the IEEE SA Patent Policy.[5]

Attention is called to the possibility that implementation of this standard may require use of subject matter covered by patent rights. By publication of this standard, no position is taken by the IEEE with respect to the existence or validity of any patent rights in connection therewith. If a patent holder or patent applicant has filed a statement of assurance via an Accepted Letter of Assurance, then the statement is listed on the IEEE SA Website at https://standards.ieee.org/about/sasb/patcom/patents.html. Letters of Assurance may indicate whether the Submitter is willing or unwilling to grant licenses under patent rights without compensation or under reasonable rates, with reasonable terms and conditions that are demonstrably free of any unfair discrimination to applicants desiring to obtain such licenses.

Essential Patent Claims may exist for which a Letter of Assurance has not been received. The IEEE is not responsible for identifying Essential Patent Claims for which a license may be required, for conducting inquiries into the legal validity or scope of Patents Claims, or determining whether any licensing terms or conditions provided in connection with submission of a Letter of Assurance, if any, or in any licensing agreements are reasonable or non-discriminatory. Users of this standard are expressly advised that determination of the validity of any patent rights, and the risk of infringement of such rights, is entirely their own responsibility. Further information may be obtained from the IEEE Standards Association.

IMPORTANT NOTICE

Technologies, application of technologies, and recommended procedures in various industries evolve over time. The IEEE standards development process allows participants to review developments in industries, technologies, and practices, and to determine what, if any, updates should be made to the IEEE standard. During this evolution, the technologies and recommendations in IEEE standards may be implemented in ways not foreseen during the standard's development. IEEE standards development activities consider research and information presented to the standards development group in developing any safety recommendations. Other information about safety practices, changes in technology or technology implementation, or impact by peripheral systems also may be pertinent to safety considerations during implementation of the standard. Implementers and users of IEEE Standards documents are responsible for determining and complying with all appropriate safety, security, environmental, health, data privacy, and interference protection practices and all applicable laws and regulations.

[5] Available at: https://standards.ieee.org/about/sasb/patcom/materials.html.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Participants

The following is a list of participants in the IEEE 802.1 Working Group.

Glenn Parsons, *Chair*
Jessy Rouyer, *Vice-Chair*
Mark Hantel, *Chair, Maintenance Task Group*
Christophe Mangin, *Editor*

The following members of the individual balloting committee voted on this standard. Balloters may have voted for approval, disapproval, or abstention.

When the IEEE-SA Standards Board approved this standard on XX Month 20XX, it had the following membership:

Name, *Chair*
Name, *Vice Chair*
Name, *Past Chair*

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Name, *Secretary*

SBMember1	SBMember9	SBMember17
SBMember2	SBMember10	SBMember18
SBMember3	SBMember11	SBMember19
SBMember4	SBMember12	SBMember20
SBMember5	SBMember13	SBMember21
SBMember6	SBMember14	SBMember22
SBMember7	SBMember15	SBMember23
SBMember8	SBMember16	

*Member Emeritus

Also included are the following nonvoting IEEE-SA Standards Board liaisons:

Name, *DOE Representative*
Name, *NIST Representative*

Name
IEEE Standards Program Manager, Document Development

Name
IEEE Standards Program Manager, Technical Program Development

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Introduction

This introduction is not part of IEEE P802.1CB-2017-Rev-20XX, IEEE Draft Standard for Local and metropolitan area networks – Frame Replication and Elimination for Reliability

The first edition of IEEE Std 802.1CB™ was published in 2017. A first amendment IEEE Std 802.1CBcv™-2021 added the YANG data model that allows configuration and state reporting for all managed objects of the base standard. A second amendment IEEE Std 802.1CBdb™-2021 added the mask-and-match Stream identification function and provided technical and editorial corrections. A corrigendum, IEEE 802.1CB-2017/Cor1-2026, implemented the changes necessitated by various maintenance items that have been notified to the IEEE 802.1 Working Group.

This revision incorporates all the changes introduced by the amendements, the corrigendum, the implementation of maintenance item #343, as well as technical and editorial corrections to existing functionalities.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Editor's Foreword

<<Notes>>

<<Throughout this document, all notes such as this one, presented between angle braces, are temporary notes inserted by the Editors for a variety of purposes; these notes and the Editors' Foreword will all be removed prior to publication and are not part of the normative text.>>

<<Comments and participation in 802.1 standards development

Comments on this draft are encouraged. **PLEASE NOTE: All issues related to IEEE standards presentation style, formatting, spelling, etc. are routinely handled between the 802.1 Editor and the IEEE Staff Editors prior to publication, after balloting and the process of achieving agreement on the technical content of the standard is complete.** Readers are urged to devote their valuable time and energy only to comments that materially affect either the technical content of the document or the clarity of that technical content. Comments should not simply state what is wrong, but also what might be done to fix the problem.

Full participation in the development of this draft requires individual attendance at IEEE 802 meetings. Information on 802.1 activities, working papers, and email distribution lists etc. can be found on the 802.1 website:

https://1.ieee802.org/

Use of the email distribution list is not presently restricted to 802.1 members, and the working group has had a policy of considering ballot comments from all who are interested and willing to contribute to the development of the draft. Individuals not attending meetings have helped to identify sources of misunderstanding and ambiguity in past projects. Non-members are advised that the email lists exist primarily to allow the members of the working group to develop standards, and are not a general forum.

All participants in IEEE standards development have responsibilities under the IEEE patent policy and should familiarize themselves with that policy. See http://standards.ieee.org/about/sasb/patcom/materials.html.

Comments on this document may be sent to the 802.1 email exploder, to the editors, or to the Chairs of the 802.1 Working Group and Time-Sensitive Networking Task Group.

Christophe Mangin
Editor,
Email: c.mangin@fr.merce.mee.com

Mark Hantel
Chair, 802.1 Maintenance Task Group
Email: mrhantel@ra.rockwell.com

Glenn Parsons
Chair, 802.1 Working Group
Email: glenn.parsons@ericsson.com

PLEASE NOTE: Comments whose distribution is restricted in any way cannot be considered, and may not be acknowledged. >>

<<The draft text and accompanying information

This document currently comprises:

— A title page for the proposed standard including an Abstract and Keywords. This title page will be retained following working group approval of this draft, i.e. prior to sponsor ballot.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

- The editors' forewords, including this text. These include an unofficial and informal appraisal of history and status, introductory notes to each draft that summarize the progress and focus of each successive draft, and requests for comments and contributions on major issues.
- IEEE boilerplate text.
- A record of participants (not included in early drafts but added prior to publication).
- The introduction to this standard.
- The proposed standard proper.

During the early stages of draft development, 802.1 editors have a responsibility to attempt to craft technically coherent drafts from the resolutions of ballot comments and the other discussions that take place in the working group meetings. Preparation of drafts often exposes inconsistencies in editors instructions or exposes the need to make choices between approaches that were not fully apparent in the meeting. Choices and requests by the editors' for contributions on specific issues will be found in the editors' introductory notes to the current draft, at appropriate points in the draft, and in Annex Z. Significant discussion of more difficult topics will be found in the last of these.

The ballot comments received on each draft, and the editors' proposed and final disposition of comments, are part of the audit trail of the development of the standard and are available, along with all the revisions of the draft on the 802.1 web site (for address see above).

>>

<< **Introductory notes to draft 1.2**

Draft 1.2 was prepared for the second Working Group recirculation ballot. It incorporates the resolutions of the comments made against Draft 1.1, resolved during the July 2026 plenary meeting:

- Clause 7.5 (Individual recovery function) moved to clause 7.4.5
- Corrections of Annex A table contents
- Text clarifications in 7.4 and 7.4.2
- Move of autoconfiguration to an informative annex
- Still no change in the text color codes compared to D1.0

As proposed in https://www.ieee802.org/1/files/public/docs2026/cb-rev-autoconfig-info-annex-mangin-0726-v01.pdf. the descriptions and specifications related to the FRER autoconfiguration mechanism were moved to an informative Annex (Annex E). The modifications made to the original text, tables and figures are signaled by revision marks (~~striked~~ and added).
The YANG model was refactored by extracting the autoconfiguration capabilities from the original YANG model and re-introducing them through augmentations specified in Annex E.
The MIB was not modified.
The description/reference statements in the MIB and YANG files attached to this draft may still differ of those in clauses 11.5 and 12.6. They will be harmonized when the YANG and MIB are finalized.

The YANG modules could not be validated with pyang due to reasons independent of the editor's will (recent changes in company rules regarding software authorized on company PCs). This unfortunate situation can be considered as temporary.

Some extra typos and disoriented cross-references were also corrected.

<< **Introductory notes to draft 1.1**

Draft 1.1 was prepared for the first Working Group recirculation ballot. It incorporates the resolutions of the comments made against Draft 1.0, resolved during the May 2026 interim meeting and during a maintenance TG call on May 26, 2026.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

- No change in the text color codes compared to D1.0.
- Wording clarifications and consistency improvements in the conformance descriptions in Clause 5
- Removal of the remaining text and figures changes signaled by revision marks and associated editor's instructions (Cl. 6, 8, and Annex C)
- Revision marks are kept in the MIB description (Clause 11) and YANG modules (Clause 12) to keep track of the changes to be reflected in the corresponding files. There are some minor discrepancies between the MIB description of Clause 11 and the MIB file attached to D1.1. There are also some minor discrepancies between the YANG modules of Clause 12 and the YANG files attached to D1.1. Since these differences are only located in "revision" and "description" and "reference" statements, they will be processed once the corresponding text they originate from is finalized.
- Various text clarifications, text layout inconsistencies and typos corrections.

>>

<< **Introductory notes to draft 1.0**

Draft 1.0 was prepared for the initial Working Group ballot. It incorporates the resolutions of the comments made against Draft 0.3, resolved during the January 2026 interim meeting and during a maintenance TG call on February 17, 2026.

- Text color codes remain the same as for D0.3.
- Text of Clauses 9.1.2.3, 9.1.3.3 and 9.1.4.3 is reverted to Draft 0.1's : the indication primitive's vlan_identifier parameter is ignored when the tsnCpeXxxYyyVlan object is equal to 0.
- Text of Clause 9.1.5.2 (tsnCpeIpIdTagged) is aligned to the definitions of the tsnCpeXxxYyyTagged objects of the other passive Stream identification functions (semantics of the object was similar to the one of an active Stream identification function).
- The definition of the tsnCpeIpIdVlan (9.1.5.3) is aligned the those of the other passive Stream identification functions': the indication primitive's vlan_identifier parameter is ignored when the tsnCpeIpIdVlan object is equal to 0.
- MIB and YANG (references) are updated according to the above changes.
- The definitions of the Base (7.4.2) and Sequence recovery (7.4.3) functions are reorganised; the text of both clauses seems to have been mixed up.
- Text clarifications and typos.

>>

<< **Introductory notes to draft 0.3**

Draft 0.3 was prepared for a third Task Group ballot. It incorporates the resolutions of the comments made against Draft 0.2, resolved during the November 2025 plenary meeting and during a maintenance TG call on December 9, 2025.

- Text color codes remain the same as for D0.2.
- YANG source files are now provided. The changes made to the original 802.1CBcv YANG modules and MIB are reflected in clauses 11 and 12 through the same color codes as for D0.2 (pine green and violet). Clauses 11 and 12, rolled up from 802.1CBcv, are not magenta highlighted except their clause header.
- Text of clauses 9.1.5.1 and 9.1.5.4 is reverted to the text present in Draft 0.1, with a slight different wording (comment #5).
- Changes made to the ieee802-dot1cb-stream-identification YANG module (cl. 12.6.2.2) are removed (comment #5).

>>

<< **Introductory notes to draft 0.2**

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Draft 0.2 was prepared for a second Task Group ballot. It incorporates the resolutions of the comments made against Draft 0.1, resolved during the September interim meeting.

- Text color codes remain the same as for D0.1.
- Neither MIB nor YANG source file is provided. The changes made to the original 802.1CBcv YANG modules and MIB are reflected in clauses 11 and 12 through the same color codes as for D0.1 (pine green and violet). Clauses 11 and 12, rolled up from 802.1CBcv, are not magenta highlighted except their clause header.
- So far, the changes made in the YANG modules have not been reflected in the MIB.
- Changes due to the resolution of comments against D0.1 on managed object definitions (clause 9) and their associated YANG models (clause 12) are signaled using addition/deletion revision marks.
- Changes made to the figures and text of clauses 6 and 8, as well as annex C (incorporated from 802.1CB-2017/Cor1 and maintenance item #343's implementation) are highlighted using addition/deletion revision marks.

>>

<< **Introductory notes to draft 0.1**

Draft 0.1 was prepared for a first Task Group ballot. The following color codes are used in this draft to ease the visualization of changes made to IEEE 802.1CB-2017:

- all changes incorporated from IEEE 802.1CBcv™-2021 are magenta highlighted
- all changes incorporated from IEEE 802.1CBdb™-2021 are violet highlighted;
- all changes incorporated from IEEE 802.1CB™-2017/Cor1 are pine green highlighted.
- changes resulting from the implementation of maintenance item #343 are signaled using addition/ ~~deletion~~ revision marks

>>

<< Project Authorization Request, Scope, Purpose, and Five Criteria

A PAR (Project Authorization Request) for P802.1CB-2017-Rev was approved by the IEEE Standards Association on February 13, 2024. The following information is taken from the 802.1CB-2017-Rev PAR.

Need for the Project:

This revision project is needed to incorporate approved amendments and corrigenda, to incorporate technical and editorial corrections to existing functionality, and to maintain consistency in the consolidated text.

Purpose of Proposed Project:

This document will not include a purpose clause.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Contents

1. Overview 24
1.1 Scope 24
1.2 Rationale 24
1.3 State diagram conventions 24
1.4 Specification model 24
1.5 Specification precedence 25
1.6 Introduction 25

2. Normative references 24

3. Definitions 26

4. Acronyms and abbreviations 28

5. Conformance 30
5.1 Requirements terminology 30
5.2 Conformant components and equipment 30
5.3 Stream identification component required behaviors 30
5.4 Stream identification component recommended behavior 31
5.5 Stream identification component optional behaviors 31
5.6 Talker end system required behaviors 31
5.7 Talker end system recommended behaviors 31
5.8 Talker end system optional behaviors 31
5.9 Listener end system required behaviors 32
5.10 Listener end system recommended behavior 32
5.11 Listener end system optional behaviors 32
5.12 Relay system required behaviors 33
5.13 Relay system recommended behaviors 33
5.14 Relay system optional behaviors 33
5.15 FRER C-component required and optional behaviors 34

6. Stream identification 35
6.1 Stream service subparameters 36
6.2 Stream identification function 37
6.3 Stream identification in systems 38
6.4 Null Stream identification 40
6.5 Source MAC and VLAN Stream identification 40
6.6 Active Destination MAC and VLAN Stream identification 40
6.7 IP Stream identification 41
6.8 Mask-and-match Stream identification 41

7. Frame Replication and Elimination for Reliability 43
7.1 Overview of Frame Replication and Elimination for Reliability 43
7.1.1 Goals and objectives 43
7.2 Use of the term Stream 45
7.3 Frame Replication and Elimination for Reliability functions 45
7.4 Sequencing function 46
7.4.1 Sequence generation function 46
7.4.1.1 Events for sequence generation 47

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

7.4.1.2 Variables for sequence generation 47
7.4.1.2.1 GenSeqSpace 47
7.4.1.2.2 GenSeqNum 47
7.4.1.3 SequenceGenerationReset 47
7.4.1.4 SequenceGenerationAlgorithm 48
7.4.2 Base recovery function 48
7.4.3 Sequence recovery function 48
7.4.3.1 Events for sequence recovery 49
7.4.3.2 Variables for sequence recovery 49
7.4.3.2.1 RecovSeqSpace 49
7.4.3.2.2 SequenceHistory 50
7.4.3.2.3 RecovSeqNum 50
7.4.3.2.4 RemainingTicks 50
7.4.3.2.5 TicksPerSecond 50
7.4.3.2.6 TakeAny 50
7.4.3.3 SequenceRecoveryReset 50
7.4.3.4 VectorRecoveryAlgorithm 51
7.4.3.5 MatchRecoveryAlgorithm 53
7.4.3.6 ShiftSequenceHistory 54
7.4.4 Latent error detection function 54
7.4.4.1 Events for latent error detection 55
7.4.4.2 Variables for latent error detection 55
7.4.4.2.1 CurBaseDifference 56
7.4.4.3 LatentErrorReset 56
7.4.4.4 LatentErrorTest 56
7.4.5 Individual recovery function 56
7.5 Sequence encode/decode function 57
7.6 Stream splitting function 57
7.7 Redundancy tag 58
7.7.1 Redundancy tag EtherType 59
7.7.2 Redundancy tag information 59
7.8 HSR sequence tag 59
7.9 PRP sequence trailer 60
7.10 Autoconfiguration 61
7.10.1 Introduction to autoconfiguration 61
7.10.2 Autoconfigured Stream identity table entry creation and destruction 62

8. Frame Replication and Elimination for Reliability in Bridges 66

8.1 Limiting options 66
8.2 FRER C-component input transformations 69
8.3 Frame Replication and Elimination for Reliability and VLAN tags 69
8.4 Configuring Frame Replication and Elimination for Reliability in Bridges 70

9. Stream Identification Management 71

9.1 Stream identity table 71
9.1.1 tsnStreamIdEntry 71
9.1.1.1 tsnStreamIdHandle 71
9.1.1.2 tsnStreamIdInFacOutputPortList 71
9.1.1.3 tsnStreamIdOutFacOutputPortList 71
9.1.1.4 tsnStreamIdInFacInputPortList 72
9.1.1.5 tsnStreamIdOutFacInputPortList 72
9.1.1.6 tsnStreamIdIdentificationType 72
9.1.1.7 tsnStreamIdParameters 72

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

9.1.2 Managed objects for Null Stream identification 72
9.1.2.1 tsnCpeNullDownDestMac 73
9.1.2.2 tsnCpeNullDownTagged 73
9.1.2.3 tsnCpeNullDownVlan 73
9.1.3 Managed objects for Source MAC and VLAN Stream identification 73
9.1.3.1 tsnCpeSmacVlanDownSrcMac 73
9.1.3.2 tsnCpeSmacVlanDownTagged 73
9.1.3.3 tsnCpeSmacVlanDownVlan 73
9.1.4 Managed objects for Active Destination MAC and VLAN Stream identifications . 74
9.1.4.1 tsnCpeDmacVlanDownDestMac 74
9.1.4.2 tsnCpeDmacVlanDownTagged 74
9.1.4.3 tsnCpeDmacVlanDownVlan 74
9.1.4.4 tsnCpeDmacVlanDownPriority 74
9.1.4.5 tsnCpeDmacVlanUpDestMac 74
9.1.4.6 tsnCpeDmacVlanUpTagged 75
9.1.4.7 tsnCpeDmacVlanUpVlan 75
9.1.4.8 tsnCpeDmacVlanUpPriority 75
9.1.5 Managed objects for IP Stream identification 75
9.1.5.1 tsnCpeIpIdDestMac 75
9.1.5.2 tsnCpeIpIdTagged 75
9.1.5.3 tsnCpeIpIdVlan 75
9.1.5.4 tsnCpeIpIdIpSource 76
9.1.5.5 tsnCpeIpIdIpDestination 76
9.1.5.6 tsnCpeIpIdDscp 76
9.1.5.7 tsnCpeIpIdNextProtocol 76
9.1.5.8 tsnCpeIpIdSourcePort 76
9.1.5.9 tsnCpeIpIdDestinationPort 76
9.1.6 Managed objects for Mask-and-match Stream identification 76
9.1.6.1 tsnCpeMmIdDestMacMask 76
9.1.6.2 tsnCpeMmIdDestMacMatch 77
9.1.6.3 tsnCpeMmIdSrcMacMask 77
9.1.6.4 tsnCpeMmIdSrcMacMatch 77
9.1.6.5 tsnCpeMmIdMsduMaskLength 77
9.1.6.6 tsnCpeMmIdMsduMask 77
9.1.6.7 tsnCpeMmIdMsduMatch 77
9.2 Operational per-port per-Stream Stream identification counters 78
9.2.1 tsnCpsSidInputPackets 78
9.2.2 tsnCpsSidOutputPackets 78
9.3 Operational per-port Stream identification counters 78
9.3.1 tsnCpSidInputPackets 78
9.3.2 tsnCpSidOutputPackets 78
9.4 Per-port managed object 78
9.4.1 tsnPpMmIdMsduMaskMaxLength 78

10. Frame Replication and Elimination for Reliability management 79

10.1 Counter behavior 79
10.2 Additional tsnStreamIdEntry managed objects 79
10.2.1 tsnStreamIdAutoconfigured 80
10.2.2 tsnStreamIdLanPathId 80
10.3 Sequence generation table 80
10.3.1 frerSeqGenEntry 80
10.3.1.1 frerSeqGenStreamList 80
10.3.1.2 frerSeqGenDirection 80

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

10.3.1.3 frerSeqGenReset 80
10.4 Sequence recovery table 80
10.4.1 frerSeqRcvyEntry 81
10.4.1.1 frerSeqRcvyStreamList 81
10.4.1.2 frerSeqRcvyPortList 81
10.4.1.3 frerSeqRcvyDirection 81
10.4.1.4 frerSeqRcvyReset 81
10.4.1.5 frerSeqRcvyAlgorithm 81
10.4.1.6 frerSeqRcvyHistoryLength 81
10.4.1.7 frerSeqRcvyResetMSec 82
10.4.1.8 frerSeqRcvyInvalidSequenceValue 82
10.4.1.9 frerSeqRcvyTakeNoSequence 82
10.4.1.10 frerSeqRcvyIndividualRecovery 82
10.4.1.11 frerSeqRcvyLatentErrorDetection 82
10.4.1.12 Latent error detection managed objects 82
10.4.1.12.1 frerSeqRcvyLatentErrorDifference 82
10.4.1.12.2 frerSeqRcvyLatentErrorPeriod 83
10.4.1.12.3 frerSeqRcvyLatentErrorPaths 83
10.4.1.12.4 frerSeqRcvyLatentResetPeriod 83
10.5 Sequence identification table 83
10.5.1 frerSeqEncEntry 83
10.5.1.1 frerSeqEncStreamList 83
10.5.1.2 frerSeqEncPort 83
10.5.1.3 frerSeqEncDirection 83
10.5.1.4 frerSeqEncActive 83
10.5.1.5 frerSeqEncEncapsType 84
10.5.1.6 frerSeqEncPathIdLanId 84
10.6 Stream split table 84
10.6.1 frerSplitEntry 84
10.6.1.1 frerSplitPort 84
10.6.1.2 frerSplitDirection 84
10.6.1.3 frerSplitInputIdList 85
10.6.1.4 frerSplitOutputIdList 85
10.7 Managed objects for autoconfiguration 85
10.7.1 Sequence autoconfiguration table 85
10.7.1.1 frerAutSeqEntry 85
10.7.1.1.1 frerAutSeqSeqEncaps 85
10.7.1.1.2 frerAutSeqReceivePortList 85
10.7.1.1.3 frerAutSeqTagged 85
10.7.1.1.4 frerAutSeqVlan 85
10.7.1.1.5 frerAutSeqRecoveryPortList 85
10.7.1.1.6 frerAutSeqDestructMSec 86
10.7.1.1.7 frerAutSeqResetMSec 86
10.7.1.1.8 frerAutSeqAlgorithm 86
10.7.1.1.9 frerAutSeqHistoryLength 86
10.7.1.1.10 frerAutSeqCreateIndividual 86
10.7.1.1.11 frerAutSeqCreateRecovery 86
10.7.1.1.12 frerAutSeqLatErrDetection 86
10.7.1.1.13 frerAutSeqLatErrDifference 86
10.7.1.1.14 frerAutSeqLatErrPeriod 86
10.7.1.1.15 frerAutSeqLatErrResetPeriod 86
10.7.2 Output autoconfiguration table 87
10.7.2.1 frerAutOutEntry 87
10.7.2.1.1 frerAutOutPortList 87

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

10.7.2.1.2 frerAutOutEncaps 87
10.7.2.1.3 frerAutOutLanPathId 87
10.8 Operational per-port and per-Stream FRER counters 87
10.8.1 Per-Stream vs. per-Stream-per-port counters 87
10.8.2 frerCpsSeqGenResets 88
10.8.3 frerCpsSeqRcvyOutOfOrderPackets 88
10.8.4 frerCpsSeqRcvyRoguePackets 88
10.8.5 frerCpsSeqRcvyPassedPackets 88
10.8.6 frerCpsSeqRcvyDiscardedPackets 88
10.8.7 frerCpsSeqRcvyLostPackets 88
10.8.8 frerCpsSeqRcvyTaglessPackets 88
10.8.9 frerCpsSeqRcvyResets 88
10.8.10frerCpsSeqRcvyLatentErrorResets 89
10.8.11frerCpsSeqEncErroredPackets 89
10.9 Operational per-port FRER counters 89
10.9.1 frerCpSeqRcvyPassedPackets 89
10.9.2 frerCpSeqRcvyDiscardPackets 89
10.9.3 frerCpSeqEncErroredPackets 89

11. Management Information Base (MIB) 90

11.1 Internet Standard Management Framework 90
11.2 Structure of the MIB 90
11.2.1 Structure of the IEEE8021-STREAM-IDENTIFICATION-MIB 92
11.2.2 Structure of the IEEE8021-FRER-MIB 94
11.3 Relationship to other MIBs 94
11.3.1 Relationship of the IEEE8021-STREAM-IDENTIFICATION-MIB to other MIBs 94
11.3.2 Relationship of the IEEE8021-FRER-MIB to other MIBs 99
11.4 Security considerations 99
11.4.1 Security considerations of the IEEE8021-STREAM-IDENTIFICATION-MIB 99
11.4.2 Security considerations of the IEEE8021-FRER-MIB 100
11.5 MIB modules, ... 100
11.5.1 Definitions for the IEEE8021-STREAM-IDENTIFICATION-MIB 100
11.5.2 Definitions for the IEEE8021-FRER-MIB 131

12. YANG Data Model 168

12.1 YANG Framework 168
12.2 IEEE Std 802.1CB YANG model 169
12.2.1 Stream Identification model 169
12.2.2 Frame Replication and Elimination for Reliability model 171
12.3 Structure of the YANG model 174
12.3.1 Structure of the ieee802-dot1cb-stream-identification YANG module 175
12.3.2 Structure of the ieee802-dot1cb-mask-and-match YANG module 176
12.3.3 Structure of the ieee802-dot1cb-frer YANG module 177
12.4 Relationship to other YANG modules 178
12.4.1 IEEE 802 Types Module 178
12.4.2 IEEE 802.1Q Types Module 178
12.4.3 IETF Inet Types Module 178
12.4.4 IETF Interfaces YANG Module 178
12.4.5 IEEE 802.1CB Stream Identification Types YANG module 178
12.4.6 IEEE 802.1CB Stream Identification YANG module 178
12.4.7 IEEE 802.1CB Extended Stream identification module 178
12.4.8 IEEE 802.1CB FRER Types YANG module 179
12.5 Security Considerations 179

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

12.5.1 Security Considerations of the ieee802-dot1cb-stream-identification YANG module 179
12.5.2 Security Considerations of the ieee802-dot1cb-frer YANG module 179
12.6 Definition of IEEE Std 802.1CB YANG modules 179
12.6.1 YANG data scheme tree definitions 180
12.6.1.1 YANG data scheme definition for ieee802-dot1cb-stream-identification YANG module 180
12.6.1.2 YANG data scheme definition for ieee802-dot1cb-mask-and-match YANG module 181
12.6.1.3 YANG data scheme definition for ieee802-dot1cb-frer YANG module ... 181
12.6.2 YANG data module definitions 184
12.6.2.1 Definition for the ieee802-dot1cb-stream-identification-types YANG module 184
12.6.2.2 Definition for the ieee802-dot1cb-stream-identification YANG module . 186
12.6.2.3 Definition for the ieee802-dot1cb-mask-and-match module 200
12.6.2.4 Definition for the ieee802-dot1cb-frer-types YANG module 204
12.6.2.5 Definition for the ieee802-dot1cb-frer YANG module 206

Annex A (normative) Protocol Implementation Conformance Statement (PICS) proforma 228

A.1 Introduction 228
A.1.1 Abbreviations and special symbols 228
A.1.2 Instructions for completing the PICS proforma 229
A.1.3 Additional information 229
A.1.4 Exceptional information 229
A.1.5 Conditional items 230
A.1.6 Identification 230
A.2 PICS proforma for Frame Replication and Elimination for Reliability 231
A.2.1 Major capabilities/options 231
A.2.2 Stream identification component 231
A.2.3 Talker end system 232
A.2.4 Listener end system 233
A.2.5 Relay system 235
A.2.6 FRER 802.1Q C-component 237
A.2.7 Common requirements 237
A.2.8 Management Information Base (MIB) 238
A.2.9 YANG 238

Annex B (informative) Interoperability with other standards 239

B.1 Sequence number size 239
B.2 Per-Stream versus per-source sequencing 239

Annex C (informative) Frame Replication and Elimination for Reliability in systems 240

C.1 Example 1: End-to-end FRER 240
C.2 Example 2: Various stack positions 241
C.3 Example 3: Ladder redundancy 244
C.4 Example 4: Multicast trees 245
C.5 Example 5: Protocol interworking 245
C.6 Example 6: Chained two-port end systems 246
C.7 Cautions 247

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

C.8 Balancing tag insertion and removal 248
C.9 FRER and reserved bandwidth 248
C.10 Use of the Individual recovery function 249
C.11 Use of autoconfiguration 249
C.11.1 Routing and labeling Member Streams 250
C.11.2 Recognizing packets that trigger autoconfiguration 251
C.11.3 Per-port packet decoding and encoding 251
C.11.4 Individual and Sequence recovery functions 251

Annex D (informative) Mask-and-match Stream identification function 253

D.1 Example 1: Generic Stream identification for industrial communication networks 253
D.2 Example 2: MPLS DetNet flow identification 253

Annex E (informative) Autoconfiguration 255

E.1 Introduction to autoconfiguration 255
E.2 Autoconfigured Stream identity table entry creation and destruction 256
E.3 Autoconfiguration management 259
E.3.1 Additional tsnStreamIdEntry managed objects 259
E.3.2 Managed objects for autoconfiguration 259
E.4 Use of autoconfiguration 261
E.4.1 Routing and labeling Member Streams 262
E.4.2 Recognizing packets that trigger autoconfiguration 263
E.4.3 Per-port packet decoding and encoding 263
E.4.4 Individual and Sequence recovery functions 263

Annex F (informative) Bibliography 265

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

List of figures

Figure 6-1—Stream identification service 35
Figure 6-2—A Stream with three Listeners 35
Figure 6-3—Stream identification function: single upper SAP 37
Figure 6-4—Stream identification function: array of upper SAPs 37
Figure 6-5—Stream functions in a relay system (three views of same system) 38
Figure 6-6—In- and out-facing functions 39
Figure 7-1—Compound Stream built from four Member Streams 43
Figure 7-2—Frame Replication and Elimination for Reliability functions 45
Figure 7-3—Base recovery functions and Individual recovery functions 57
Figure 7-4—R-TAG format 58
Figure 8-1—Split functions in an FRER C-component 66
Figure 8-2—Merge functions in an FRER C-component 67
Figure 8-3—Augmented Forwarding Process does sequence recovery 68
Figure 8-4—Example Ethernet frame format 69
Figure 11-1—MIB modules structure 91
Figure 12-1—General YANG hierarchy 168
Figure 12-2—YANG root hierarchy with IEEE Std 802.1CB YANG modules 169
Figure 12-3—Stream identification model 170
Figure 12-4—Stream identification counters model 171
Figure 12-5—FRER model 172
Figure 12-6—FRER Stream identification augmentation 173
Figure 12-7—FRER counters model 174
Figure C-1—Dual-homed end systems using Link Aggregation 240
Figure C-2—Protocol stack for End System B in Figure C-1 241
Figure C-3—Protocol stack for End System G in Figure C-1 and Figure C-4 241
Figure C-4—IEEE Std 802.1CB flexible positioning 242
Figure C-5—Protocol stack for relay system B, proxying for End System A, in Figure C-4 243
Figure C-6—Protocol stack for relay system C in Figure C-4 243
Figure C-7—Protocol stack for relay system F in Figure C-4 244
Figure C-8—Ladder redundancy 244
Figure C-9—Multicast trees 245
Figure C-10—Protocol interworking 246
Figure C-11—Dual-homed end systems using 3-port bridge 246
Figure C-12—Protocol stacks for Systems B and G in Figure C-11 247
Figure C-13—Explicit path causing a loop 247
Figure C-14—Example of Long and short paths 248
Figure C-15—Autoconfiguration example 250
Figure E-1—Autoconfiguration example 262
Figure E-2—FRER augmentation 265
Figure E-3—FRER Stream identification augmentation 266

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

List of tables

Table 6-1—Stream identification functions 36
Table 7-1—R-TAG EtherType 59
Table 8-1—Managed objects for FRER in an FRER C-component 70
Table 9-1—Stream identification types 72
Table 10-1—Enumerated values for frerSeqRcvyAlgorithm 81
Table 10-2—Sequence Encode/Decode types 84
Table 11-1—Structure of the MIB modules 90
Table 11-2—IEEE8021-STREAM-IDENTIFICATION-MIB structure and relationship to this standard 92
Table 11-3—IEEE8021-FRER-MIB structure and relationship to this standard 95
Table 12-2—ieee802-dot1cb-stream-identification structure and relationship to this standard 175
Table 12-3—YANG module dependencies for the Stream identification model 175
Table 12-1—Description of the YANG modules 175
Table 12-4—ieee802-dot1cb-mask-and-match structure and relationship to this standard 176
Table 12-5—YANG module dependencies for the Extended Stream identification model 176
Table 12-6—ieee802-dot1cb-frer structure and relationship to this standard 177
Table 12-7—YANG module dependencies for the FRER model 177
Table D-1—Managed objects values for example 1 253
Table D-2—Managed objects values for example 2 254
Table E-4—ieee802-dot1cb-autoconfiguration structure and relationship to this standard 266
Table E-5—YANG module dependencies for the autoconfiguration model 266

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

IEEE Standard for
Local and metropolitan area networks—

Frame Replication and Elimination for Reliability

1. Overview

1.1 Scope

This standard specifies procedures, managed objects, and protocols for bridges and end systems that provide identification and replication of packets for redundant transmission, identification of duplicate packets, and elimination of duplicate packets. It is not concerned with the creation of the multiple paths over which the duplicates are transmitted.

1.2 Rationale

The reason for Frame Replication and Elimination for Reliability (FRER) is to increase the probability that a given packet will be delivered. It is expected that, in many applications, other means to increase the probability of delivery are likely to be used as well. When FRER is used over paths that are fixed to a specific topology, and that are protected against congestion loss (e.g., by using techniques described by IEEE Std 802.1BA™ [B2]), FRER can substantially reduce the probability of packet loss due to equipment failures.[6]

1.3 State diagram conventions

This document uses the programming language C (ISO/IEC 9899:2011) to document the operation of conformant systems.[7] C functions are distinguished with `this special fixed-width font` (e.g., 7.4.3.3). Each C function is executed when a given event occurs, as described for that code segment or in the accompanying text. Events are assumed to take place sequentially, not simultaneously, and code routines execute instantaneously.

1.4 Specification model

The model of operation documented by this standard is simply a basis for describing the functionality of compliant equipment. Implementations can adopt any internal model of operation compatible with the

[6]The numbers in brackets correspond to those of the bibliography in Annex F.
[7]Information on references can be found in Clause 2.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

externally visible behavior that this standard specifies. Conformance of equipment to this standard is purely in respect of observable protocol.

1.5 Specification precedence

If any conflict among parts of this standard become apparent, C functions (see 1.3) take precedence over other parts of the standard, followed by information in normative tables, followed by that in normative text, followed by that in normative figures. Non-normative tables, figures, and text are in annexes and are clearly marked as such.

1.6 Introduction

This standard is one of a number of IEEE 802.1™ and other standards suitable for Time-Sensitive Networking (TSN) that together have the overall goal of providing extremely low packet loss rates and finite, low, and stable end-to-end latencies. TSN supports unicast and multicast Streams of packets that implement a wide range of demanding real-time applications including audio/video studios, industrial processes, and the control of machines and vehicles. The TSN goals are not achieved at the expense of hampering the ability of the network to carry traffic for non-time-critical applications.

At the highest level, this standard posits the existence of one Talker end system and one or more Listener end systems per Stream. A Stream is characterized by a maximum packet size and number of packets transmitted per time interval. Because the Stream's maximum throughput is known, the resources, including link bandwidth, buffer space, and control parameters, required at every hop along the Stream's path to guarantee that Stream zero congestion loss and finite latency, can be provided (by other standards, e.g., Clause 35 of IEEE Std 802.1Q™-2022). This provisioned path carrying the Stream is called a *Reservation*.

On the assumption that the time required for a dynamic network control protocol to recover from an equipment failure is unacceptable in certain applications, this standard defines Frame Replication and Elimination for Reliability (FRER), which divides a Stream into one or more linked Member Streams, thus making the original Stream a Compound Stream. It replicates the packets of the Stream, splitting the copies into the multiple Member Streams, and then rejoins those Member Streams at one or more other points, eliminates the replicates, and delivers the reconstituted Stream from those points.

In order to accommodate existing applications and to promote interoperability with similar standards, this standard defines a number of schemes for identifying packets belonging to Streams and distinguishing them from other packets.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

2. Normative references

The following referenced documents are indispensable for the application of this document (i.e., they must be understood and used, so each referenced document is cited in text and its relationship to this document is explained). For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments or corrigenda) applies. Non-normative references (i.e., that provide additional information not required for the application of this document) are given in Annex F.

IEC 62439-3:2021, Industrial communication networks—High availability automation networks—Part 3: Parallel Redundancy Protocol (PRP) and High-availability Seamless Redundancy (HSR).[8]

IEEE Std 802®-2024, IEEE Standard for Local and metropolitan area networks: Overview and Architecture.[9, 10]

IEEE Std 802.1AC™-2026, IEEE Standard for Local and metropolitan area networks—Media Access Control (MAC) Service Definition.

IEEE Std 802.1AX™-2020, IEEE Standard for Local and metropolitan area networks—Link Aggregation.

IEEE Std 802.1Q™-2022, IEEE Standard for Local and metropolitan area networks—Bridges and Bridged Networks.

IETF RFC 768, User Datagram Protocol, Postel, J., August 1980.[11]

IETF RFC 791, Internet Protocol, Postel, J., Ed., September 1981.

IETF RFC 793, Transmission Control Protocol, Postel, J., Ed., September 1981.

IETF RFC 2460, Internet Protocol, Version 6 (IPv6) Specification, Deering, S. and R. Hinden, December 1998.

IETF RFC 2474, Definition of the Differentiated Services Field (DS Field) in the IPv4 and IPv6 Headers, Nichols, K., et al., December 1998.

IETF RFC 2578 (STD 58), Structure of Management Information Version 2 (SMIv2), McCloghrie, K., D. Perkins, and J. Schoenwaelder, Apr. 1999.

IETF RFC 2579 (STD 58), Textual Conventions for SMIv2, McCloghrie, K., D. Perkins, and J. Schoenwaelder, Apr. 1999.

IETF RFC 2580 (STD 58), Conformance Statements for SMIv2, McCloghrie, K., D. Perkins, and J. Schoenwaelder, Apr. 1999.

IETF RFC 3410 (informational), Introduction and Applicability Statements for Internet-Standard Management Framework, Case, J., R. Mundy, D. Partain, and B. Stewart, Dec. 2002.

IETF RFC 4960, Stream Control Transmission Protocol, Stewart, R., Ed., September 2007.

[8]IEC publications are available from the International Electrotechnical Commission (http://www.iec.ch) and the American National Standards Institute (http://www.ansi.org/).

[9]The IEEE standards or products referred to in Clause 2 are trademarks owned by The Institute of Electrical and Electronics Engineers, Incorporated.

[10]IEEE publications are available from The Institute of Electrical and Electronics Engineers (http://standards.ieee.org).

[11]IETF RFCs are available from the Internet Engineering Task Force (https://www.rfc-archive.org/).

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

IETF RFC 8343 (proposed standard), A YANG Data Model for Interface Management, Bjorklund, M., Mar. 2018.

ISO/IEC 9899:2011, Information technology—Programming languages—C.[12]

[12]ISO/IEC publications are available from the International Organization for Standardization (http://www.iso.org/) and the American National Standards Institute (http://www.ansi.org/).

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

3. Definitions

For the purposes of this document, the following terms and definitions apply. The *IEEE Standards Dictionary Online* should be consulted for terms not defined in this clause. [13]

This standard makes use of the following terms defined in IEEE Std 802®:

- End station
- Organizationally Unique Identifier (OUI)
- Company ID (CID)

bridge: A layer 2 interconnection device that conforms to IEEE Std 802.1Q.

Company ID (CID): CIDs allow a general means of assuring unique identifiers for a number of purposes. See Clause 8 of IEEE Std 802-2014.

Compound Stream: A Compound Stream is a Stream composed of one or more Member Streams linked together via Frame Replication and Elimination for Reliability (FRER).

down: The direction through the protocol stack from a sublayer using a service provided by another sublayer to the sublayer whose services it uses; the direction of output packets.

end station: A device attached to a local area network (LAN) or metropolitan area network (MAN), which acts as a source of and/or destination for data traffic carried on the LAN or MAN. (From IEEE Std 802)

end system: A system attached to a network that is an initial source or a final destination of packets transmitted across that network.

NOTE—The term *end system* is often used in this document in places where the reader of IEEE 802 standards would expect the term *end station* in order to avoid confusion caused by standards relating to routers. For example, a router, as defined by IETF, is an *IEEE 802 end station*, but not an end system. Where this standard specifically refers to the use of IEEE 802 services, the term *end station* is used. Where it refers to more generalized instances of connectionless services, the term *end system* is used.[14]

in-facing: In a system that includes a Stream Transfer Function, the "in-facing" protocol functions are those functions that are below the Stream Transfer Function on the port that is not above the physical layer. On a system that does not include a Stream Transfer Function, there are no in-facing protocol functions.

NOTE—See Figure 6-6.

input: Input packets are those moving up the protocol stack, regardless of the position of receiving function relative to the physical media, for example, by the M_UNITDATA.indication primitive of the Internal Sublayer Service (ISS).

Internal local area network (LAN): An instance of a connectionless packet service with two Service Access Points (SAPs), both internal to a single system, that relays packets from one SAP to the other. It is not observable externally to that system.

Internal Sublayer Service (ISS): An augmented version of the MAC Service, defined in IEEE Std 802.1AC.

[13] *IEEE Standards Dictionary Online* is available at: https://ieeexplore.ieee.org/browse/standards/dictionary.
[14] Notes in text, tables, and figures of a standard are given for information only and do not contain requirements needed to implement this standard.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Media Access Control (MAC): The data link sublayer that is responsible for transferring data to and from the Physical Layer.

Member Stream: A Stream that is linked with other Member Streams via mechanisms specified by this standard to form a Compound Stream.

Non-Stream Transfer Function (NSTF): A two-port function that relays service indications on one port to service requests on the other port.

Organizationally Unique Identifier (OUI): OUIs allow a general means of assuring unique identifiers for a number of purposes. See Clause 9 and Clause 10 of IEEE Std 802-2014.

out-facing: In a system that includes a Stream Transfer Function, the "out-facing" protocol functions are those functions that are below the Stream Transfer Function on the port that is above the physical layer. On a system that does not include a Stream Transfer Function, all TSN functions are out-facing.

NOTE—See Figure 6-6.

output: Output packets are those moving down the protocol stack, regardless of the position of receiving function relative to the physical media, for example, by the M_UNITDATA.request primitive of the Internal Sublayer Service (ISS).

packet: A unit of data carried over a network, comprising, at least, one or more pairs of destination and source addresses and a payload. A packet carried on an Ethernet local area network (LAN) is a frame.

NOTE—The term *packet* is often used in this document in places where the reader of IEEE 802 standards would expect the term *frame*. Where the standard specifically refers to the use of IEEE 802 services, the term *frame* is used. Where the standard refers to more generalized instances of connectionless services, the term *packet* is used.

Quality of Service (QoS): Overall performance of a packet Stream as it relates to packet loss probability, latency, and latency variation.

relay system: A router or a bridge.

NOTE—The term *relay system* is often used in this document in places where the reader of IEEE 802 standards would expect the term *bridge*. A relay system can, in theory, be a router, a bridge, or some other kind of forwarding device. Where this standard specifically refers to one or the other, the terms *router* or *bridge* are used. Where it refers to more generalized instances of connectionless services, the term *relay system* is used.

Reservation: The collection of state information and resources allocated, in a chain of end systems and relay systems, for one or more Streams.

router: A packet forwarding device operating on Internet Protocol (IP) packets.

Stream: A unidirectional flow of time-sensitive data from one source to one or more destinations, and at the highest level, one Talker end system to one or more Listener end systems.

Stream Transfer Function: A two-port function that relays service indications on one port to service requests on the other port, where the service includes all TSN service subparameters.

subparameter: A service parameter encoded into the value of the connection_identifier parameter of the Internal Sublayer Service (ISS).

system: An end system or a relay system.

up: The direction through the protocol stack from a sublayer offering a service to the sublayer making use of that service; the direction of input packets.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

4. Acronyms and abbreviations

This standard contains the following abbreviations:

CID	Company ID
DRNI	Distributed Resilient Network Interconnect
EISS	Enhanced Internal Sublayer Service
FRER	Frame Replication and Elimination for Reliability
IEC	International Electrotechnical Commission
IP	Internet Protocol
ISO	International Organization for Standardization
ISS	Internal Sublayer Service
LAN	Local Area Network
MAC	Medium Access Control
MIB	Management Information Base
NETCONF	Network Configuration Protocol
NSTF	Non-Stream Transfer Function
OUI	Organizationally Unique Identifier
PDU	Protocol Data Unit
PICS	Protocol Implementation Conformance Statement
QoS	Quality of Service
R-TAG	Redundancy tag
SAP	Service Access Point
SMI	Structure of Management Information
SNMP	Simple Network Management Protocol
TSN	Time-Sensitive Networking
UML	Unified Modeling Language
VLAN	Virtual Local Area Network
YANG	Yet Another Next Generation[15]

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

[15] YANG is best viewed as a name, not an acronym

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

5. Conformance

This clause specifies the mandatory and optional capabilities provided by conformant implementations of this standard.

5.1 Requirements terminology

For consistency with existing IEEE and IEEE 802.1 standards, requirements placed upon conformant implementations of this standard are expressed using the following terminology:

a) *Shall* is used for mandatory requirements;
b) *May* is used to describe implementation or administrative choices ("may" means "is permitted to," and hence, "may" and "may not" mean precisely the same thing);
c) *Should* is used for recommended choices (the behaviors described by "should" and "should not" are both permissible but not equally desirable choices).

The Protocol Implementation Conformance Statement (PICS) proformas (see Annex A) reflect the occurrences of the words "shall," "may," and "should" within the standard.

The standard avoids needless repetition and apparent duplication of its formal requirements by using *is*, *is not*, *are*, and *are not* for definitions and the logical consequences of conformant behavior. Behavior that is permitted but is neither always required nor directly controlled by an implementor or administrator, or whose conformance requirement is detailed elsewhere, is described by *can*. Behavior that never occurs in a conformant implementation or system of conformant implementations is described by *cannot*. The word *allow* is used as a replacement for the phrase "support the ability for," and the word *capability* means "can be configured to."

5.2 Conformant components and equipment

Conformance for Frame Replication and Elimination for Reliability (FRER) is defined for five types of components and equipment:

a) Stream identification components (5.3, 5.4, 5.5), which provide a useful subset of the FRER capabilities for systems that are not talker end systems, listener end systems, or relay systems.
b) Talker end systems (5.6, 5.7, 5.8), which originate Compound Streams.
c) Listener end systems (5.9, 5.10, 5.11), which consume Compound Streams.
d) Relay systems (5.12, 5.13, 5.14), which transfer or discard packets belonging to Compound Streams.
e) One particular type of Relay system, an FRER C-component (5.15).

The Stream identification component can be useful in components or equipment that are not talker end systems, listener end systems, or relay systems as defined in this standard, e.g., bridges or routers that carry packets belonging to Compound Streams, and must identify them, but do not process FRER sequence numbers. Other useful systems can be created using the preceding components listed. For example, a two-port Talker could be modeled and implemented as two Talker end systems in a single chassis, as a single one-port Talker end system that uses IEEE 802.1AX Link Aggregation to combine two physical ports into a single logical port (C.1), or as a single-port Talker end system connected to a three-port Bridge (C.6).

5.3 Stream identification component required behaviors

A Stream identification component shall be able to instantiate the following out-facing functions on at least one port, for at least one Compound Stream:

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

a) Stream identification (Clause 6);
b) A Null Stream identification function (6.4);
c) The managed objects in Clause 9.

5.4 Stream identification component recommended behavior

A Stream identification component should be able to instantiate one or more of the following out-facing functions on at least one port, for at least one Compound Stream:

a) A Source MAC and VLAN Stream identification function (6.5);
b) An Active Destination MAC and VLAN Stream identification function (6.6).

5.5 Stream identification component optional behaviors

In addition to the requirements of 5.3 and 5.4, a Stream identification component may perform the following functions:

a) The items in 5.3 and 5.4 on more than one port;
b) The items in 5.3 and 5.4 for some number of Compound Streams greater than 1;
c) An IP Stream identification function (6.7);
d) A Mask-and-match Stream identification function (6.8);
e) Additional types of Stream identification functions.

5.6 Talker end system required behaviors

A Talker end system shall be able to instantiate the following out-facing functions on at least one port, for at least one Compound Stream:

a) Stream identification (Clause 6);
b) A Null Stream identification function (6.4);
c) A Sequence generation function (7.4.1);
d) The Redundancy tag Sequence encode/decode function specified in 7.5 and 7.7;
e) The managed objects in Clause 9 and Clause 10, excepting 10.7.

5.7 Talker end system recommended behaviors

A Talker end system should be able to instantiate one or more of the following out-facing functions on at least one port, for at least one Compound Stream:

a) A Source MAC and VLAN Stream identification function (6.5);
b) An Active Destination MAC and VLAN Stream identification function (6.6);
c) A Stream splitting function (7.6).

5.8 Talker end system optional behaviors

In addition to the requirements of 5.6, a Talker end system may perform the following functions:

a) The items in 5.6 and 5.7 on more than one port;
b) The items in 5.6 and 5.7 for some number of Compound Streams greater than 1;

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

c) An IP Stream identification function (6.7);
d) A Mask-and-match Stream identification function (6.8);
e) Additional types of Stream identification functions;
f) The HSR sequence tag (7.8);
g) The PRP sequence trailer (7.9);
h) Additional types of Sequence encode/decode functions;
i) Support SMIv2 MIB modules for the management of talker end system functions (Clause 11) as required by item e in 5.6; and/or
j) Support YANG modules for the management of talker end system functions (Clause 12) as required by item e in 5.6.

5.9 Listener end system required behaviors

A Listener end system shall be able to instantiate the following out-facing functions on at least one port, for at least one Compound Stream:

a) Stream identification (Clause 6);
b) A Null Stream identification function (6.4);
c) A Sequence recovery function (7.4.3) that supports the MatchRecoveryAlgorithm (7.4.3.5) and supports the VectorRecoveryAlgorithm (7.4.3.4) with a value of at least 2 for the managed object frerSeqRcvyHistoryLength (10.4.1.6);
d) At least two instances of Individual recovery functions (7.4.5), each using the MatchRecoveryAlgorithm (7.4.3.5);
e) The Redundancy tag Sequence encode/decode function specified in 7.5 and 7.7.
f) The managed objects in Clause 9 and Clause 10, excepting 10.7.

5.10 Listener end system recommended behavior

A Listener end system should be able to instantiate one or more of the following out-facing functions on at least one port, for at least one Compound Stream:

a) A Source MAC and VLAN Stream identification function (6.5);
b) An Active Destination MAC and VLAN Stream identification function (6.6).

5.11 Listener end system optional behaviors

In addition to the requirements of 5.9 and 5.10, a Listener end system may perform the following functions:

a) The items in 5.9 and 5.10 on more than one port;
b) The items in 5.9 and 5.10 for some number of Compound Streams greater than 1;
c) An IP Stream identification function (6.7);
d) A Mask-and-match Stream identification function (6.8);
e) Additional types of Stream identification functions;
f) The HSR sequence tag (7.8);
g) The PRP sequence trailer (7.9);
h) Additional types of Sequence encode/decode functions;
i) At least two instances of Individual recovery functions (7.4.5), each using the VectorRecoveryAlgorithm (7.4.3.4);

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

j) Support SMIv2 MIB modules for the management of listener end system functions (Clause 11) as required by item f in 5.9; and/or

k) Support YANG modules for the management of listener end system functions (Clause 12) as required by item f in 5.9.

5.12 Relay system required behaviors

A relay system shall be able to instantiate the following in-facing functions on at least two ports, for both transmit and receive, for at least one Stream:

a) Stream identification (Clause 6).

b) A Null Stream identification function (6.4);

c) A Sequence generation function (7.4.1);

d) The Redundancy tag Sequence encode/decode function specified in 7.5 and 7.7;

e) A Sequence recovery function (7.4.3) that supports the MatchRecoveryAlgorithm (7.4.3.5) and supports the VectorRecoveryAlgorithm (7.4.3.4) with a value of at least 2 for the managed object frerSeqRcvyHistoryLength (10.4.1.6);

f) At least two instances of Individual recovery functions (7.4.5), each using the MatchRecoveryAlgorithm (7.4.3.5);

g) The managed objects in Clause 9 and Clause 10, excepting 10.7.

5.13 Relay system recommended behaviors

A relay system should be able to instantiate one or more of the following in-facing functions on at least two ports, for both transmit and receive, for at least one Stream:

a) A Source MAC and VLAN Stream identification function (6.5);

b) Active Destination MAC and VLAN Stream identification functions (6.6) for encoding and decoding packets; and

c) either of the following for identifying packets:

1) IP Stream identification functions (6.7);

2) Mask-and-match Stream identification functions (6.8).

NOTE—IP Stream identification and Mask-and-match Stream identification enable a relay system to proxy for an FRER-unaware end system.

5.14 Relay system optional behaviors

In addition to the requirements of 5.12 and 5.13, a relay system may perform the following functions:

a) The items in 5.12 or 5.13 on more than two ports;

b) The items in 5.12 or 5.13 for some number of Streams greater than 1;

c) Additional types of Stream identification functions;

d) The Stream splitting function (7.6);

e) The HSR sequence tag (7.8);

f) The PRP sequence trailer (7.9);

g) Additional types of Sequence encode/decode functions;

h) Some or all of the functions in 5.12 or 5.13 as both in- and out-facing functions;

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

i) At least two instances of Individual recovery functions (7.4.5), each using the VectorRecoveryAlgorithm (7.4.3.4);
j) ~~Autoconfiguration (7.10) and the associated managed objects (10.7);~~
k) Support SMIv2 MIB modules for the management of relay systems functions (Clause 11) as required by item g in 5.12; and/or
l) Support YANG modules for the management of relay systems functions (Clause 12) as required by item g in 5.12.

5.15 FRER C-component required and optional behaviors

An FRER C-component is an IEEE 802.1Q C-VLAN component that is FRER-capable. An FRER C-component:

a) Shall meet all of the required and any or none of the optional behaviors for an IEEE 802.1Q C-VLAN component (5.5 of IEEE Std 802.1Q-2022).
b) Shall meet all of the required behaviors for a relay system (5.12);
c) May meet any or none of the optional behaviors for a relay system (5.14); and
d) Shall conform to the requirements for configuring FRER C-components (8.4).

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

6. Stream identification

Clause 7 of IEEE Std 802.1AC-2026 describes the IEEE 802.1 layering model, that Frame Replication and Elimination for Reliability (FRER) follows. Stream identification utilizes a single Service Access Point (SAP) to a connectionless packet service offere-2026d by the layer below it [e.g., the Intermediate Sublayer Service (ISS) of Clause 11 of IEEE Std 802.1AC-2026, or the Enhanced Internal Sublayer Service (EISS) of 6.8 of IEEE Std 802.1Q-2022], and offers an array of SAPs to the layers above it, corresponding to different Streams. The Stream identification model is illustrated in Figure 6-1.

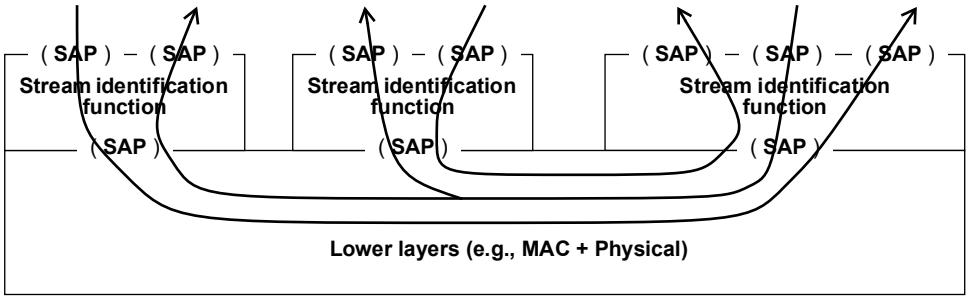


Figure 6-1—Stream identification service

Stream identification can be active (e.g., 6.6) or passive (e.g., 6.4). When accepting packets for transmission from the upper layers, an active Stream identification function (6.2) modifies the data parameters to encode the choice of SAPs, and offer the encapsulated packets to the lower layers. In the receive direction, the active Stream identification function accepts packets from the lower layers, decapsulates them, and passes them to the upper layers through the appropriate SAPs according to the Stream identification derived from the packet. A passive Stream identification function does nothing to packets passed down from the upper layers, but examines packets received from lower layers to identify the packet's Stream and determine to which SAP to pass it.

NOTE—In principle, any number of different methods for identifying and encoding Streams can be defined. Several methods are specified in the following subclauses (6.4, 6.5, 6.6, 6.7, 6.8).

A Stream is the entity to which the Qualities of Service (QoSs) are offered. It is a sequence of packets, either unicast or multicast, from a Talker to one or more Listeners. Figure 6-2 illustrates a network carrying a single Stream.

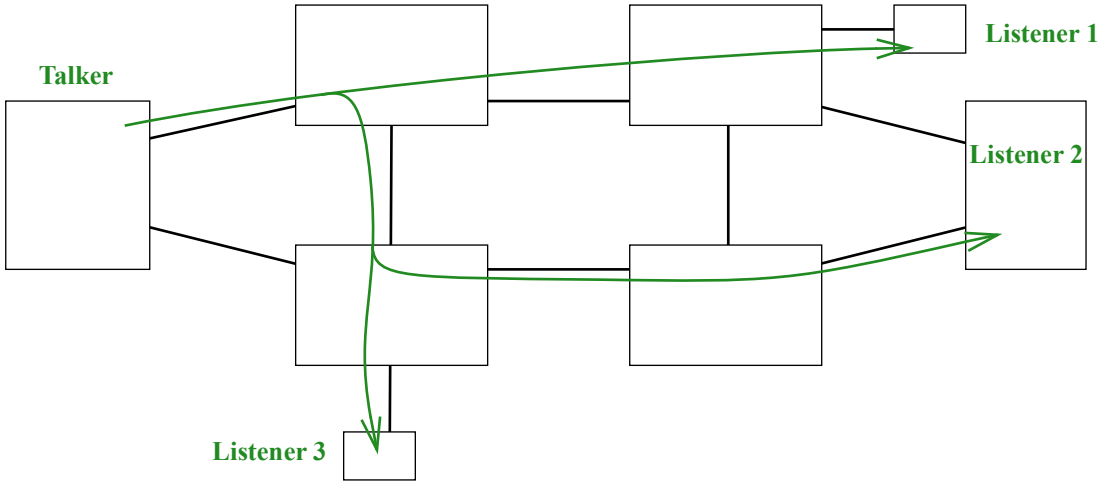


Figure 6-2—A Stream with three Listeners

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Stream identification is described in the following subclauses as follows:

a) Additional service subparameters required by Stream identification are in 6.1.
b) The Stream identification function is described in 6.2, and its placement in the protocol stack of a system in 6.3.
c) Five specific Stream identification functions are described: Null Stream identification (6.4), Source MAC and VLAN Stream identification (6.5), Active Destination MAC and VLAN Stream identification (6.6), IP Stream identification (6.7), and Mask-and-match Stream identification (6.8).

These Stream identification functions are summarized in Table 6-1.

Table 6-1—Stream identification functions

Stream identification function	Active/passive	Examines	Overwrites	Reference
Null Stream identification	Passive	destination_address, vlan_identifier	None	6.4, 9.1.2
Source MAC and VLAN Stream identification	Passive	source_address, vlan_identifier	None	6.5, 9.1.3
Active Destination MAC and VLAN Stream identification	Active	destination_address, vlan_identifier	destination_address, vlan_identifier, priority	6.6, 9.1.4
IP Stream identification	Passive	destination_address, vlan_identifier, IP source address, IP destination address, DSCP, IP next protocol, source port, destination port	None	6.7, 9.1.5
Mask-and-match Stream identification function	Passive	destination_address, source_address, mac_service_data_unit	None	6.8, 9.1.6

6.1 Stream service subparameters

The ISS defined in IEEE Std 802.1AC and the EISS defined in IEEE Std 802.1Q include a connection_identifier parameter that is of local significance (to a system) only. The parameter is not carried across the underlying service. Stream identification makes use of this parameter to carry parametrized information. Stream identification has need for more than one subparameter, but an implementor can create mathematical algorithms to combine those subparameters (and/or other subparameters for other layers) into a single connection_identifier parameter, especially since the connection_identifier's values are undefined outside the system implementing them. In this document, parameters that are assumed to be encoded in the connection_identifier are deemed *subparameters*.

The parameters of the service offered by Stream identification include the following subparameters required by the Stream identification function and other functions defined in this standard:

a) **stream_handle:** An integer identifying the Stream to which the packet belongs.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

b) **sequence_number:** An unsigned integer identifying the order in which the packet was transmitted relative to other packets in the same Compound Stream.

The numerical value of the stream_handle subparameter is local to a system; this subparameter is not to be confused with, for example, a field in a Protocol Data Unit (PDU) with a similar function. The value of a stream_handle is connected to protocol fields by means of protocol actions and network management. It can, but does not necessarily, have a 1:1 relationship with an explicit field in the packet.

Not every sequence of packets of interest to FRER (Clause 7) requires the relay systems along its path to identify to which Stream the packets belong. FRER does require the stream_handle and sequence_number subparameters in systems where a Sequence recovery function (7.4.3) or Individual recovery function (7.4.5) is configured for a Stream.

The numerical value of the sequence_number subparameter can be explicitly encoded in the packet by either the Stream identification function (6.2) or the Sequence encode/decode function (7.5).

6.2 Stream identification function

As illustrated in Figure 6-3, the Stream identification function can be described as having two SAPs (see IEEE Std 802.1AC). One SAP connects a Stream identification function to the upper layers. This SAP includes a stream_handle subparameter and can include a sequence_number subparameter. The other SAP connects to the lower layers. This SAP can, but typically does not, include the stream_handle or sequence_number subparameters.

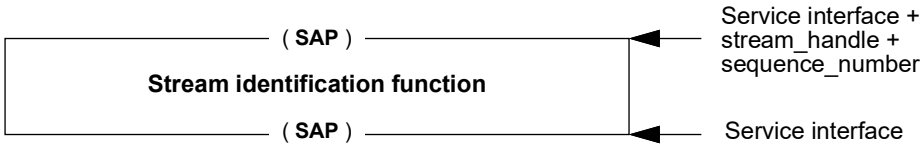


Figure 6-3—Stream identification function: single upper SAP

Equivalently, the Stream identification function can be described as having the same SAP to the lower layers, but as having an array of SAPs to the upper layers. A unique SAP corresponds to each value of the stream_handle subparameter, including one SAP for packets belonging to no known Stream. An SAP can serve more than one stream_handle value. This is illustrated in Figure 6-4.

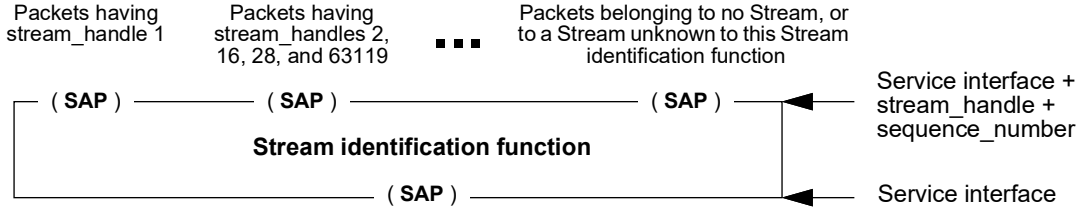


Figure 6-4—Stream identification function: array of upper SAPs

The Stream identification function performs two functions:

a) Packets received from the lower layer are examined by the Stream identification function to determine a value for the stream_handle subparameter for that packet, i.e., to determine to which Stream the packet belongs.

   1) If the packet belongs to a Stream known to this Stream identification function the resultant packet is passed, along with the stream_handle and any other subparameters (e.g.,

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

sequence_number) extracted from the packet, to the upper layers. Depending on the particular Stream identification method (or methods) employed, parameters can be modified (e.g., a tag can be removed or an address changed) by the Stream identification function.

2) Otherwise (the packet belongs to no known Stream), the packet is passed unchanged to the next upper layer with null values for the stream_handle and sequence_number subparameters.

b) For packets passed down from the upper layers for transmission, the Stream identification function uses the packet's stream_handle subparameter to determine how to handle the packet:

1) If the packet belongs to a Stream known to this Stream identification function the resultant packet is passed to the lower layers. Depending on the particular Stream identification method (or methods) employed, parameters can be modified (e.g., a tag can be added or an address changed) by the Stream identification function.

2) Otherwise (the packet belongs to no known Stream), the packet is passed unchanged to the next lower layer with all parameters intact.

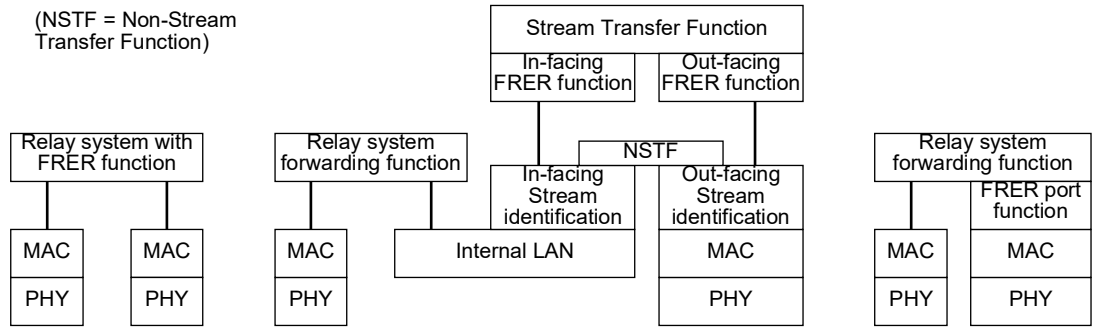
The null-value is a value known from the management system as identifying the stream_handle assigned to packets that belong to no known Stream.

NOTE 1—The Stream identification method does not necessarily involve an actual transformation of the packet. See 6.4 and 6.6 for examples of both cases.

NOTE 2—The order of precedence of Stream identification functions and the invocation of their instances, when more than one matches the same frame, is out of scope of this standard.

6.3 Stream identification in systems

FRER (Clause 7) capabilities generally require Stream identification in order to function. How the Stream identification function (6.2) and the various FRER functions (7.4, 7.5) are arranged to accomplish a given task, and how they are described for the purposes of standards specification, can vary. Diagram a in Figure 6-5 illustrates a relay system (e.g., 8.6 of IEEE Std 802.1Q-2022) with two ports, that has its FRER capabilities embedded in its forwarding function, and not shown explicitly in the diagram. This formulation is the simplest, but does not make clear the ordering of operations, at least in the diagram.



a. Relay system with FRER capability

b. Relay system with separated FRER capability, e.g., packet counting, on an output port.

c. Shorthand for an FRER function on a port

Figure 6-5—Stream functions in a relay system (three views of same system)

Diagram b in Figure 6-5 shows that same relay system, this time with the FRER functions placed explicitly in one of its ports, the picture of which is greatly expanded. In Diagram b, the forwarding function has no

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

FRER capabilities. Stream identification functions extract Stream identification function subparameters in order to enable an FRER function. The connectivity between the Relay system's forwarding and the Stream identification functions is modelled by an Internal LAN that provides transparent connectivity between ISS, or EISS, SAPs supporting the Stream service subparameters (6.1). The *Stream Transfer Function* acts as a two-port packet relay, residing entirely inside the one port of the relay system, relaying packets belonging to Streams. The *Non-Stream Transfer Function* (NSTF) does the same, but attaches to the "unknown" SAP of Figure 6-4, and thus relays packets not recognized as belonging to Streams. The *In-facing* and *Out-facing Stream identification* functions separate Stream and non-Stream packets; the latter are relayed across the NSTF as if the FRER capabilities were not present. Among the Stream packets, the *In-facing* and *Out-facing Stream identification* functions further identify the FRER packets' Streams so that the FRER functions can perform their tasks. Each of the two-port transfer functions have a pair of SAPs, and transfer all packet receive indications into packet requests on the other SAP. This formulation illustrates exactly the peering relationships among the various functions, but is too complex for many purposes.

Diagram c in Figure 6-5 shows a way of illustrating the function in diagram b in a more compact manner. The operations to be performed can be more explicit than for diagram a, but the FRER sublayers are not at the right peering level.

It is sometimes necessary to differentiate between functions that are on one side or the other of the expanded port diagram (diagram b in Figure 6-5), Figure 6-6 illustrates a relay system with two ports, one of which has in- and out-facing FRER functions, and an end system with one port, with out-facing FRER functions. Every function communicates to its peers by sending and receiving packets through the real or virtual service layers below it. Thus, in-facing functions are on the side of the Stream Transfer Function towards the relay system's forwarding function, and out-facing functions are on either the side of the Stream Transfer Function away from the forwarding function, or on a simple port, between the forwarding function and the underlying real or virtual service layer. Typically, in-facing FRER functions are used for proxy functions operating on behalf of end systems that are not FRER-capable. See Annex C for examples of the use of in- and out-facing FRER functions. (The MAC and PHY layers are the Media Access Control and physical layers, respectively.)

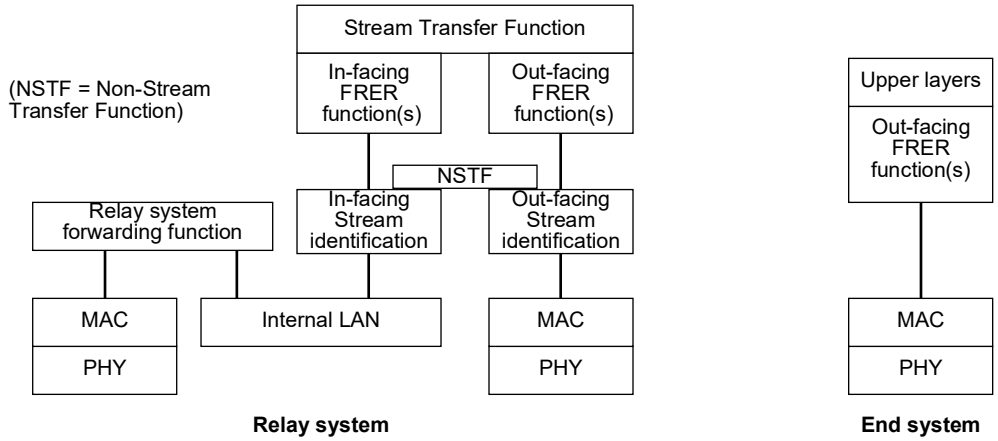


Figure 6-6—In- and out-facing functions

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

6.4 Null Stream identification

The Null Stream identification is a passive Stream identification function that operates at the frame level. It can be defined using the Enhanced Internal Sublayer Service (EISS) described in 6.9 of IEEE Std 802.1Q-2022, in which case it is enhanced with the extra stream_handle subparameter of the connection_identifier, specified in 6.1 of the present standard. It generates a stream_handle subparameter on frames passed up the stack based on the frame's destination MAC address and VLAN ID. It does not change any of a packet's other parameters. It is suitable for applications in which all data packets to a particular {MAC address, VLAN} pair are Stream packets. For example, AVB Streams (IEEE Std 802.1BA [B2]) have a unique {destination MAC address, VLAN} pair per Stream. In order to instantiate the Null Stream identification function, the tsnStreamIdIdentificationType managed object (9.1.1.6) is encoded using the OUI (00-80-C2) and the type values as shown in Table 9-1.

The managed objects for Null Stream identification are described in 9.1.2.

NOTE—The drop_eligible parameter is also present, along with the VLAN identifier and priority, in an IEEE 802.1Q VLAN tag. FRER does not affect the use of this parameter. It passes through Null Stream identification unchanged, and defaults to False when not present.

6.5 Source MAC and VLAN Stream identification

The Source MAC and VLAN Stream identification is a passive Stream identification function that operates at the frame level. It can be defined using the EISS described in 6.9 of IEEE Std 802.1Q, in which case it is enhanced with the extra stream_handle subparameter of the connection_identifier, specified in 6.1 of the present standard. It generates a stream_handle subparameter on frames passed up the stack based on the frame's source MAC address and VLAN ID. It does not change any of a packet's parameters. It is suitable for applications in which all data packets from a particular {source MAC address, VLAN} pair are Stream packets. In order to instantiate the Source MAC and VLAN Stream identification function, the tsnStreamIdIdentificationType managed object (9.1.1.6) is encoded using the OUI (00-80-C2) and the type values as shown in Table 9-1.

The managed objects for Source MAC and VLAN Stream identification are described in 9.1.3.

NOTE—The drop_eligible parameter is also present, along with the VLAN identifier and priority, in an IEEE 802.1Q VLAN tag. FRER does not affect the use of this parameter. It passes through Source MAC and VLAN Stream identification unchanged, and defaults to False when not present.

6.6 Active Destination MAC and VLAN Stream identification

The Active Destination MAC and VLAN Stream identification is an active Stream identification function that operates at the frame level. It can be defined using the EISS described in 6.9 of IEEE Std 802.1Q-2022, in which case it is enhanced with the extra stream_handle subparameter of the connection_identifier, specified in 6.1 of the present standard. In order to instantiate the Active Destination MAC and VLAN Stream identification function, the tsnStreamIdIdentificationType managed object (9.1.1.6) is encoded using the OUI (00-80-C2) and the type values as shown in Table 9-1.

In the Active Destination MAC and VLAN Stream identification, the destination_address, vlan_identifier, and priority parameters of the frame passed down the stack from the upper layers or up the stack from the lower layers are replaced with alternate values. The replacement values for frames transmitted down the stack to the Active Destination MAC and VLAN Stream identification, and used to recognize frames passed up the stack to the Active Destination MAC and VLAN Stream identification function, as well as the replacement values for frames passed up the stack are described in 9.1.4. The replacement values for frames

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

transmitted down the stack to the Active Destination MAC and VLAN Stream identification function are determined by the stream_handle subparameter passed down in the EISS request's connection_identifier.

Active Destination MAC and VLAN Stream identification is useful for translating a particular Stream, within a Talker, to use a particular {MAC address, VLAN} pair to identify the Stream to IEEE 802.1Q Bridges in the network, and within a Listener, to recover the original addressing information before passing the packet up the protocol stack. It is also useful in a relay system that is providing that restoration as a proxy service for a Listener.

The managed objects for Active Destination MAC and VLAN Stream identification are described in 9.1.4.

NOTE 1—The drop_eligible parameter is also present, along with the VLAN identifier and priority, in an IEEE 802.1Q VLAN tag. FRER does not affect the use of this parameter. It passes through Active Destination MAC and VLAN Stream identification unchanged, and defaults to False when not present.

NOTE 2—Changing the destination MAC address and/or VLAN must be done carefully, if the receiver is to recognize the packet. For example, if Active Destination MAC and VLAN Stream identification is used along with IP Stream identification (6.7), the user can configure Active Destination MAC and VLAN Stream identification at the receiving end to restore the original destination MAC address and VLAN before delivery up the protocol stack.

6.7 IP Stream identification

The IP Stream identification is a passive Stream identification function that operates at the transport layer and Internet Protocol (IP) interface layer. It can be defined using the union of the IP address primitives listed in 9.1.5 and parameters defined by the EISS described in 6.9 of IEEE Std 802.1Q-2022, in which case it is enhanced with the stream_handle subparameter of the connection_identifier, specified in 6.1 of the present standard. In IP Stream identification, the IP and higher layer address parameters and the EISS parameters are used to determine the stream_handle subparameter of packets passed up the stack. It does not change any of a packet's other parameters. In order to instantiate the IP Stream identification function, the tsnStreamIdIdentificationType managed object (9.1.1.6) is encoded using the OUI (00-80-C2) and the type values as shown in Table 9-1.

IP Stream identification can be coupled, for example, with Active Destination MAC and VLAN Stream identification (6.6) to assign a particular {MAC address, VLAN, priority} triple to packets belonging to a particular unicast IP Stream, as shown in Figure 8-1, Port A, where IP Stream identification would be in the box labeled "Passive Out-facing Stream identification functions (6.3)."

Practically, the IP Stream identification function requires locating the beginning of the IP packet encapsulated in the frame, preceded by an IP EtherType field, e.g., 0x0800 or 0x86DD. The location of the IP EtherType field within the frame is dependent on the combination of tag(s) following the addresses in a frame. The method used to determine this location is beyond the scope of this standard.

The managed objects for IP Stream identification are described in 9.1.5.

NOTE—The drop_eligible parameter is also present, along with the VLAN identifier and priority, in an IEEE 802.1Q VLAN tag. FRER does not affect the use of this parameter. It passes through IP Stream identification unchanged, and defaults to False when not present.

6.8 Mask-and-match Stream identification

The Mask-and-match Stream identification is a passive Stream identification that operates at the frame level. It can be defined using the Internal Sublayer Service (ISS) described in IEEE Std 802.1AC, in which case it is enhanced with the extra stream_handle subparameter of the connection_identifier, specified in 6.1 of the present standard.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

It generates a stream_handle subparameter on frames passed up the stack based on information extracted from a subset of the frame's ISS parameters. It does not change any of a packet's parameters, other than the connection_identifier.

Input information for the Mask-and-match Stream identification function is obtained by masking one or more of the following ISS parameters:

- source_address,
- destination_address, and
- the first N octets of mac_service_data_unit, N being a system-dependent variable.

Stream identification is then done by comparing these masked ISS parameters with specific match values.

The Mask-and-match Stream identification function supports Stream identification schemes ranging from very simple, using a single mask applied to one of its input ISS parameters, to complex schemes involving masks applied to all input ISS parameters, and in particular a specific mac_service_data_unit mask to identify communication flows supported by various higher-layer protocols.

In order to instantiate the Mask-and-match Stream identification function, the tsnStreamIdIdentificationType managed object (9.1.1.6) is encoded using the OUI (00-80-C2) and the type values as shown in Table 9-1.

The managed objects for Mask-and-match Stream identification are described in 9.1.6.

Mask-and-match Stream identification can be coupled, for example, with Active Destination MAC and VLAN Stream identification (6.6) to assign a particular {MAC address, VLAN, priority} triplet to packets belonging to a particular unicast application Stream identified by the combination of the values of particular fields in the MAC service data unit, as shown in Figure 8-1, Port A, where Mask-and-match Stream identification would be in the box labeled "Passive Out-facing Stream identification functions (6.3)."

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

7. Frame Replication and Elimination for Reliability

7.1 Overview of Frame Replication and Elimination for Reliability

7.1.1 Goals and objectives

FRER, as specified in this clause, provides increased reliability (reduced packet loss rates) for a Stream by sequence numbering and replicating every packet, in the source end system and/or in relay systems in the network, and eliminating those replicates in the destination end system and/or in other relay systems.

Figure 7-1 illustrates an example of a Compound Stream with four component Member Streams. In this example, a sequence_number subparameter (6.1) is generated and encoded into each packet in the leftmost box. Sequence recovery functions (7.4.3) eliminate duplicate packets, and the non-duplicate packets copied as a new Member Stream (with sequence numbers unchanged), at two intermediate points. The final two Member Streams are brought together and the duplicates eliminated at the destination at right. This configuration protects against all 7 possible one-link failures, and against 16 of 21 possible two-link failures.

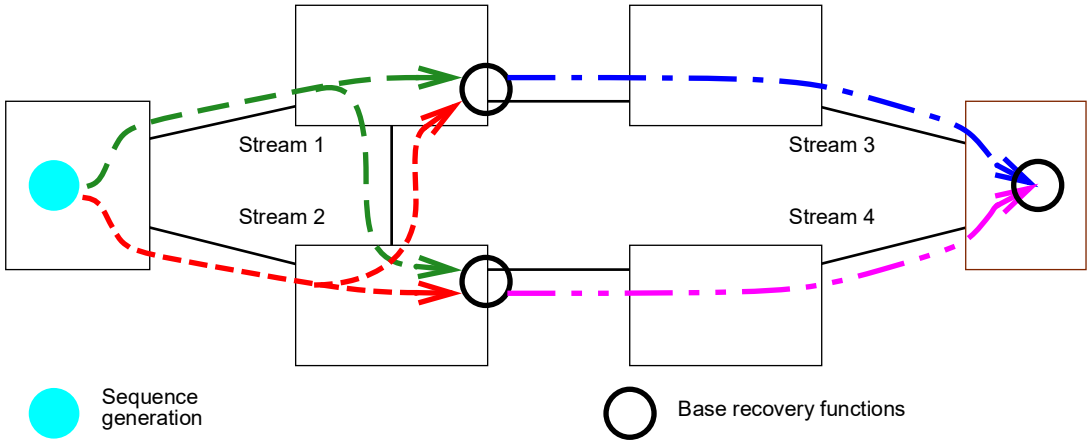


Figure 7-1—Compound Stream built from four Member Streams

FRER has the following goals and objectives:

a) **Packet replication:** By replicating packets, sending them on separate paths, and then using the sequence_number to eliminate replicates, the effective probability of packet loss is reduced.

NOTE 1—The means by which the multiple paths are created is the subject of other standards, including IEEE Std 802.1Q

b) **Multicast or unicast:** A path on which a Stream is sent can be a point-to-point path or a point-to-multipoint tree.

c) **Intermittent Streams:** A Stream such that there can be no more than one packet in flight on any given path than on any other path.

d) **Bulk Streams:** A Stream such that there can be many more packets in flight on any given path than on any other path.

NOTE 2—An equivalent distinction between bulk and intermittent Streams is that, at the point at which the Streams are combined, the difference between the sequence numbers of an intermittent Stream along the two paths can be no greater than 1. Bulk Streams, for which the difference can be greater than 1, require a somewhat more complex replicate deletion capability than intermittent Streams.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

e) **Flexible positioning:** The FRER functions have to be usable in a number of configurations, as explained in Annex C, including (but not limited to):

1) In an end system's protocol stack.

2) In a relay system's protocol stack (e.g., systems C and F in Figure C-4).

3) In a relay system's protocol stack, where the FRER functions are acting in proxy for an adjacent end system that lacks an FRER capability (e.g., system B in Figure C-4).

f) **Latent error detection:** When replicating a Stream, if a component along one path fails, it is not immediately apparent to the receiver that something is wrong, since the other path continues to deliver data. Given the higher reliability achievable with FRER, some means of detecting a failure to actually deliver copies of each packet is required at the point that the replicated packets are discarded.

g) **Interoperability:** There exist protocols antecedent to this standard that provide a similar function, but have details that are different from this standard that would make it difficult or impossible to design an interworking function that translates between the different standards' packet formats. This standard defines a small number of controls that make such interoperation possible (see Annex B).

h) **Backward compatibility:** To increase the number of uses available for FRER, it is desirable for a set of end systems conforming to this standard to obtain most or all of the benefits of FRER when connected to a network that is not aware of FRER, and for a network of conformant relay systems to offer these benefits to unaware end systems.

i) **Dynamic capability:** Some applications will need to turn FRER on or off on an operational Stream, rather than only when the Stream is quiescent.

j) **Robustness:** At packet loss rates targeted by FRER, certain classes of errors, e.g., a stuck transmitter repeatedly sending the same packet, become more important than, for example, simple packet loss events. FRER has to be proof against such errors.

k) **Zero congestion loss:** Various techniques (e.g., IEEE 802.1BA [B2], Audio Video Bridging) can be used to provide a Stream with zero (or very low) packet loss due to congestion. FRER has to be compatible with such techniques.

NOTE 3—There are additional QoS features provided by Time-Sensitive Networking (TSN). FRER offers the most benefit when used in combination with other features, including especially zero congestion loss.

l) **Ease of use:** It must be possible to employ FRER without per-Stream configuration in each relay system.

In addition to the above goals, there are other possible capabilities that are explicitly not goals of this standard:

m) **In-order delivery**: FRER can cause packets belonging to a Compound Stream to be delivered in a sequence other than that in which they were transmitted. Rectifying this is beyond the scope of this standard.

If the paths of the Member Streams comprising a Compound Stream have significantly different lengths, and if the Streams are bulk Streams (item d), then a lost packet on the shorter path can easily give rise to an out-of-order delivery when its duplicate arrives via the longer path, even though in-order delivery is maintained for each Member Stream along its own path. Delivering packets in their original order for the Compound Stream, as indicated by the sequence numbers, would require buffering the packets. Because of this buffering requirement, in-order delivery for a Compound Stream is explicitly not a goal of FRER. Users of Bridged Virtual LANs, as defined in IEEE Std 802.1Q, need to understand that the usual expectations of in-order delivery can be compromised by FRER.

FRER can be configured to apply recursively, so that a Compound Stream composed of Member Streams can itself be a Member Stream to an enclosing Compound Stream.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

7.2 Use of the term *Stream*

Readers of this standard who are familiar with IEEE Std 802.1Q will recognize that the distinction between a Member Stream and a Compound Stream is required by the present standard. For example, in IEEE Std 802.1Q, the Stream is both the entire Talker-to-Listener path and that for which a bandwidth reservation must be made before the Stream can obtain the desired QoS. It is expected that bandwidth reservations using the mechanisms of IEEE Std 802.1Q could be made for each of the four Member Streams illustrated in Figure 7-1, while the end-to-end Talker-to-Listener path is the whole Compound Stream. (See C.9.)

In addition, IEEE Std 802.1Q defines a StreamID that is used to identify a stream between a Talker and one or more Listener(s). In contrast, the present standard defines a stream_handle subparameter that is used internally to identify a Stream.

Some end systems sequence number their packets per system, rather than per-Stream. Such an end system can transmit any number of separate Streams (in the sense of data flows), each with different destination_mac_address, IP address, TCP port numbers, VLANs, etc., each with its own entries in a relay system's forwarding database and/or each with its own bandwidth reservation. In receiving end systems or in relay systems, Source MAC and VLAN Stream identification (6.5) is useful for organizing Individual recovery functions and Sequence recovery functions, because the sequence_number parameter is per-source MAC address, not per-destination MAC address; each state machine can operate on all of the individual Streams transmitted by a single source as a single Stream. Therefore, any number of individual Streams (in the bandwidth reservation sense) can be considered a single Stream (in the source MAC address sense) by different systems in a network. See also B.2.

7.3 Frame Replication and Elimination for Reliability functions

FRER provides five of the functions illustrated in Figure 7-2. Not all functions are required in every protocol stack.

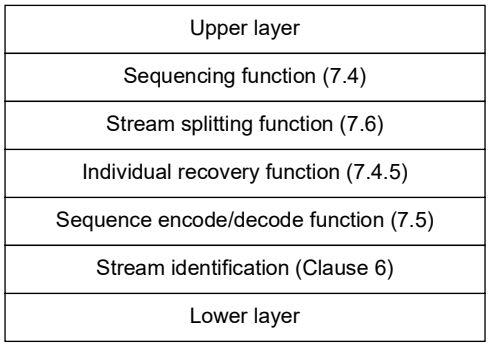


Figure 7-2—Frame Replication and Elimination for Reliability functions

a) **Sequencing:** This function (see 7.4):

   1) Supplies a sequential value for the sequence_number subparameter for packets passed down from the upper layer for transmission (7.4.1);

   2) Examines the sequence numbers of received packets passed up to it (from multiple Streams), and discards packets whose sequence_number subparameter indicates that it is a duplicate of a packet received previously (7.4.2); and

   3) Monitors counter variables to detect latent errors of Streams passed up to it (7.4.4).

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

b) **Stream splitting:** This function (see 7.6):

1) Replicates each packet passed down to it, assigning each replicate a different stream_handle, at most one of which can be the same as the original.

2) Passes packets unchanged up the stack.

c) **Individual recovery:** This function (see 7.4.5):

1) Examines the sequence numbers of received packets passed up to it (belonging to Member Streams), and discards packets whose sequence_number subparameter indicates that it is a duplicate of a packet received previously.

d) **Sequence Encode/Decode:** This function (see 7.5):

1) Encodes the sequence_number subparameter into the packet in a manner such that it can be decoded by its peer Sequencing function(s) by altering the other packet parameters, typically by encoding the sequence_number in the packet data by some means, e.g., an R-TAG (7.7).

2) Extracts the sequence_number from a received packet passed up to it. Depending on the Stream identification function used, the Sequencing function can remove the sequence_number encapsulation from the packet.

The Sequence encode/decode function is not required if the Stream identification function used in the stack encodes all of the required subparameters.

FRER also requires the Stream identification function (Clause 6):

e) **Stream identification:** This function (see Clause 6):

1) Passes each packet down the stack and, if required by the particular encapsulation method configured, uses the stream_handle to alter the packet; and

2) Derives a stream_handle from a received packet and passes it up the stack, altering the packet if required by the encapsulation method.

7.4 Sequencing function

The Sequencing function makes use of the stream_handle and sequence_number service subparameters of 6.1. The Sequencing function consists of two types of functions: the Sequence generation functions (7.4.1) and the Sequence recovery functions (7.4.3). Sequence generation functions (7.4.1) operate on packets passed down the protocol stack towards the physical layer in Figure 7-2 and generate a value for the sequence_number subparameter. Sequence recovery functions (7.4.3) operate on packets passed up the stack towards the higher layer functions and use the sequence_number subparameter to decide which packets to pass and which to discard.

On any given port, zero or more instances of the Sequence generation function, and/or Sequence recovery function can be instantiated. These functions can be instantiated as in-facing or out-facing functions. For both kinds of functions, the stream_handle subparameter of a packet determines through which function, if any, the packet is passed. Each instance of each function has its own set of state variables.

7.4.1 Sequence generation function

The sequence generation function resides in the Sequencing function (7.4). It generates a sequence_number subparameter for each packet of a Stream passed down to the lower layers. There is at most one sequence generation function per port per stream_handle value per direction (in-facing or out-facing), as specified through managed objects (10.3).

The sequence generation function is described in terms of the following:

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

a) Managed objects that control the sequence generation function and that are affected by the sequence generation function (10.3, 10.8);
b) Events, the occurrence of which trigger the execution of code routines, and that can be triggered by the code routines (7.4.1.1);
c) Variables that are manipulated by the code routines and maintain the state of the sequence generation function between routine executions (7.4.1.2); and
d) Two code routines that interact with the managed objects, events, and variables (7.4.1.4, 7.4.1.3).

7.4.1.1 Events for sequence generation

There are two events that can affect the sequence generation function, and one that it produces.

a) **BEGIN:** The global event that resets all functions, including the sequence generation function. This event triggers the execution of the SequenceGenerationReset routine (7.4.1.3).
b) **DATA_REQUEST:** The event that presents a packet from the upper layers to the sequence generation function for transmission to the lower layers, e.g., the M_UNITDATA.request primitive of the ISS of IEEE Std 802.1AC. This event triggers the execution of the SequenceGenerationAlgorithm routine (7.4.1.4).
c) **SEND_DATA:** The event that presents a packet down the stack from the sequence generation function to the lower layers, e.g., the M_UNITDATA.request primitive of the ISS of IEEE Std 802.1AC.

7.4.1.2 Variables for sequence generation

Each instance of the sequence generation function has its own set of variables, independent from any other instance.

7.4.1.2.1 GenSeqSpace

GenSeqSpace specifies the range of values for the sequence_number subparameter and other variables that depend on it. It is a constant with the value 65 536.

7.4.1.2.2 GenSeqNum

The GenSeqNum variable contains the value that the sequence generation function increments after passing down to the lower layer as the sequence_number subparameter. The variable is an unsigned integer in the range 0 to (GenSeqSpace – 1). GenSeqSpace is defined in 7.4.1.2.1. GenSeqNum is initialized to 0 whenever the function is reset (7.4.1.3), and incremented by 1 after its value is copied to the sequence_number subparameter. When incremented past its maximum value, the new value is 0.

7.4.1.3 SequenceGenerationReset

The SequenceGenerationReset function is called whenever the BEGIN event occurs (item a in 7.4.1.1) or the value True is written to the frerSeqGenReset managed object (10.3.1.3). It resets GenSeqNum (7.4.1.2.2) and increments frerCpsSeqGenResets (10.8.2).

```
void SequenceGenerationReset () {
   GenSeqNum = 0;
   frerCpsSeqGenResets = frerCpsSeqGenResets + 1;
}
```

NOTE —SequenceGenerationReset can cause packet drop.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

7.4.1.4 SequenceGenerationAlgorithm

The SequenceGenerationAlgorithm function is called whenever the DATA_REQUEST (item b in 7.4.1.1) event occurs. It copies GenSeqNum (7.4.1.2.2) to the sequence_number subparameter (item b in 6.1) and increments GenSeqNum. It then triggers the SEND_DATA primitive (item c in 7.4.1.1) to pass the packet on to the next lower layer.

```
void SequenceGenerationAlgorithm () {
   unsigned int sequence_number = GenSeqNum;
   if (GenSeqNum >= (GenSeqSpace - 1))
      GenSeqNum = 0;
   else
      GenSeqNum = GenSeqNum + 1;
   SEND_DATA; // Pass packet & sequence_number subparameter down stack
}
```

7.4.2 Base recovery function

A Base recovery function evaluates the sequence_number subparameter of a packet of one or more Member Streams passed up from the lower layers, in order to discard duplicated packets. A given instantiation of a Base recovery function can function as either a Sequence recovery function (7.4.3) or an Individual recovery function (7.4.5). This choice, the selection of stream_handle subparameter value(s) to which an instantiation applies, the port and direction (in-facing or out-facing) on which it resides, and other operational parameters are specified through managed objects (10.4).

7.4.3 Sequence recovery function

The Sequence recovery function operates on a merged set of Member Streams originally marked with sequence_number (6.1) values from a single instance of the Sequence generation function (7.4.1). It is not to be confused with the Individual recovery function (7.4.5) that applies to only one Member Stream, although both use the same basic algorithms.

An instantiation of the Sequence recovery function consists of the following:

a) An instantiation of the Base recovery function (7.4.2), with its frerSeqRcvyIndividualRecovery object (10.4.1.10) set to False, implementing either the VectorRecoveryAlgorithm (7.4.3.4) or the MatchRecoveryAlgorithm (7.4.3.5), configured to apply to one or more values of the sequence_number subparameter; and
b) An instantiation of the Latent error detection function (7.4.4).

Managed objects (Clause 9 and Clause 10) are used to instantiate these functions on ports of a system.

NOTE—The managed objects in Clause 9 and Clause 10 create an instantiation of the Sequence recovery function per port. In fact, a relay system can choose to create a single instance of the Sequence recovery function as part of its forwarding function, or one instance per line card, or in some other, distributed fashion, without violating the externally visible behaviors specified by this standard. The per-instantiation variables are internal to the system, so their definitions are not affected by this choice. See 10.8.1 for a discussion of what this choice means for the counters in Clause 9 and Clause 10 that are referenced by instantiations of the Sequence recovery function.

The sequence recovery function is described in terms of the following:

a) Managed objects that control the sequence recovery function and that are affected by the sequence recovery function (10.4, 10.8);
b) Events, the occurrence of which trigger the execution of code routines and which can be triggered by the code routines (7.4.3.1);

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

c) Variables that are manipulated by the code routines and maintain the state of the sequence recovery function between routine executions (7.4.3.2); and

d) Four code routines that interact with the managed objects, events, and variables (7.4.3.3, 7.4.3.4, 7.4.3.5, 7.4.3.6).

The sequence recovery function has two sequence recovery algorithms (7.4.3.4, 7.4.3.5) so that it can be used either for Intermittent Streams (item c in 7.1.1) or for Bulk Streams (item d in 7.1.1). As long as the maximum difference in the number of packets in flight among all of the paths taken by the Member Streams served by a given sequence recovery function does not exceed the size of the SequenceHistory variable (7.4.3.2.2, 10.4.1.6), the packets are delivered without duplication, but can be delivered out-of-order. If the difference exceeds this value, then packets can be discarded.

NOTE 1—The requirements for conformance to this standard, particularly item c in 5.9 and item e in 5.12, specify that the sequence recovery function need only support a minimum difference in path length for the correct elimination of duplicates in Bulk Streams. If the actual path difference in a given network exceeds the capability of a sequence recovery function, then packets will be discarded. It is up to the user to see that the capabilities of the systems purchased match the needs of the particular network in which they are employed.

An FRER recovery function shall not process a packet unless the lower-level packet validity checks (e.g., IEEE Std 802.3™ [B4], Frame Check Sequence) have been completed successfully.

NOTE 2—This restriction has the effect of disallowing cut through forwarding on a port on which the FRER recovery function is discarding packets.

7.4.3.1 Events for sequence recovery

There are three events that can affect the sequence recovery function, and one that the Base recovery function can trigger.

a) **BEGIN:** The global event that resets all functions, including the sequence recovery function. This event triggers the execution of the SequenceRecoveryReset routine (7.4.3.3).

b) **DATA_INDICATION:** The event that presents a packet up the stack to the sequence recovery function from the lower layers, e.g., the M_UNITDATA.indication primitive of the ISS of IEEE Std 802.1AC.

c) **RECOVERY_TIMEOUT:** The event that occurs when the RemainingTicks (7.4.3.2.4) variable reaches 0. It triggers the execution of the SequenceRecoveryReset routine (7.4.3.3).

d) **PRESENT_DATA:** The event that presents a packet up the stack from the sequence recovery function to the upper layers, e.g., the M_UNITDATA.indication primitive of the ISS of IEEE Std 802.1AC. This event can be triggered by the VectorRecoveryAlgorithm routine (7.4.3.4) and the MatchRecoveryAlgorithm routine (7.4.3.5).

7.4.3.2 Variables for sequence recovery

Each instance of the sequence recovery function has its own set of variables, independent from any other instance.

7.4.3.2.1 RecovSeqSpace

RecovSeqSpace specifies the range of values for the sequence_number subparameter and other variables that depend on it. It is a constant with the value 65 536.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

7.4.3.2.2 SequenceHistory

The SequenceHistory variable maintains a history of the sequence_number subparameters of recently received packets. The SequenceHistory variable is a bit vector, with one bit for each value from 0 to (frerSeqRcvyHistoryLength – 1), corresponding to sequence_numbers in the range RecovSeqNum (7.4.3.2.3) through (RecovSeqNum – frerSeqRcvyHistoryLength + 1), inclusive, with the arithmetic in both expressions performed modulo frerSeqRcvyHistoryLength. A 1 in the SequenceHistory indicates that the corresponding sequence_number has been received, and a 0 that it has not.

7.4.3.2.3 RecovSeqNum

The RecovSeqNum variable holds the highest sequence_number value received (modulo RecovSeqSpace), or the value (RecovSeqSpace – 1), if none have been received since the sequence recovery function was reset. The variable is an unsigned integer in the range 0 to (RecovSeqSpace – 1). RecovSeqSpace is defined in 7.4.3.2.1. RecovSeqNum is initialized to (RecovSeqSpace – 1) whenever the function is reset (7.4.3.3). When incremented past its maximum value, the new value is 0.

7.4.3.2.4 RemainingTicks

An unsigned integer variable that holds the number of clock ticks remaining until a RECOVERY_TIMEOUT event (item c in 7.4.3.1) occurs. It is decremented regularly, at the rate indicated by TicksPerSecond (7.4.3.2.5), and can be reset by the VectorRecoveryAlgorithm (7.4.3.4) and the MatchRecoveryAlgorithm (7.4.3.5). When RemainingTicks decrements from 1 to 0, a RECOVERY_TIMEOUT event occurs, and RemainingTicks remains at 0.

7.4.3.2.5 TicksPerSecond

An unsigned integer value, supplied by the implementation, that indicates the number of times per second that the variable RemainingTicks (7.4.3.2.4) is decremented. The value of TicksPerSecond shall be 100 or greater.

7.4.3.2.6 TakeAny

A Boolean value indicating whether the recovery algorithm (7.4.3.4 or 7.4.3.5) is to accept the next packet, no matter what the value of its sequence_number subparameter.

7.4.3.3 SequenceRecoveryReset

SequenceRecoveryReset is called whenever the BEGIN event (item a in 7.4.3.1) or the RECOVERY_TIMEOUT event (item c in 7.4.3.1) occurs. It resets the RecovSeqNum (7.4.3.2.3) and SequenceHistory (7.4.3.2.2) variables to their initial states, increments frerCpsSeqRcvyResets (10.8.9), and sets TakeAny (7.4.3.2.6). Note that RecovSeqNum and SequenceHistory are reset only if the VectorRecoveryAlgorithm (7.4.3.4) is configured.

```
void SequenceRecoveryReset (
   if (frerSeqRcvyAlgorithm == Vector_Alg) {
      int i;
      RecovSeqNum = RecovSeqSpace - 1;
      for (i = 0; i < frerSeqRcvyHistoryLength; i = i + 1)
         SequenceHistory[i] = 0; // Set all bits 0 (packet not seen)
   }
   frerCpsSeqRcvyResets = frerCpsSeqRcvyResets + 1;
   TakeAny              = true;
}
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

NOTE—Reset of Sequence recovery function may cause temporary duplicate delivery.

7.4.3.4 VectorRecoveryAlgorithm

VectorRecoveryAlgorithm is called whenever the DATA_INDICATION event occurs (item b in 7.4.3.1) and the Sequence recovery function is configured to use the VectorRecoveryAlgorithm (frerSeqRcvyAlgorithm = **Vector_Alg**, 10.4.1.5). If it terminates without triggering the PRESENT_DATA event (item d in 7.4.3.1), then the packet is effectively discarded. Each instance of the function is controlled by the constant RecovSeqSpace (7.4.3.2.1) and the managed object frerSeqRcvyHistoryLength (10.4.1.6).

Immediately after SequenceRecoveryReset (7.4.3.3) is called, the VectorRecoveryAlgorithm accepts the first packet received as valid. After the first packet has been accepted, all subsequent packets that are in the recovery window (i.e., last packet number accepted - frerSeqRcvyHistoryLength + 1 to last packet number accepted + frerSeqRcvyHistoryLength) are accepted, and those packets with sequence_number values outside that range are discarded. Each packet accepted and passed up the stack resets the timer variable RemainingTicks (7.4.3.2.4). If that variable ticks down to 0, meaning that no packet has been accepted in frerSeqRcvyResetMSec milliseconds (10.4.1.7), then SequenceRecoveryReset again resets the algorithm, and the next packet received is accepted.

This timeout mechanism means that:

a) "Rogue" packets, meaning packets outside the recovery window, are discarded as invalid.
b) If a Base recovery function somehow gets out of step with its corresponding Sequence generation function, then after frerSeqRcvyResetMSec milliseconds, the Base recovery function will be reset and data will again be passed.
c) If a Sequence generation function is reset, perhaps by rebooting a system, then Base recovery functions that have not been reset are likely to discard packets until the timeout has occurred.
d) If a Talker fails in such a way as to repeatedly transmit packets with the same sequence_number subparameter (perhaps repeating exactly the same packet), then in case of Individual Recovery (7.4.5), all repeated packets are discarded. If only Sequence Recovery (7.4.2) is used, then all except one repeated packets are discarded per RemainingTicks (7.4.3.2.4) interval. If a relay system fails the same way, then in case of Individual Recovery (7.4.5), all packets are discarded. If only Sequence Recovery (7.4.2) is used and if there is a correct Member Stream, then the failed Member Stream pollutes the correct Member Stream.

NOTE—Individual recovery functions (7.4.5) can be configured to discard repeated sequence_number values (or sequences of values) and prevent even this wrap around problem.

As a result of item d, it is advisable for a Listener or relay system to be configured to discard packets in a single Member Stream. See 7.4.5 and C.10.

```
void VectorRecoveryAlgorithm () {
   //  Check that sequence number is present in the packet
   unsigned int sequence_number;
   if (sequence_number == frerSeqRcvyInvalidSequenceValue) {
      frerCpsSeqRcvyTaglessPackets = frerCpsSeqRcvyTaglessPackets + 1;
      if (frerSeqRcvyTakeNoSequence) {
         frerCpsSeqRcvyPassedPackets = frerCpsSeqRcvyPassedPackets + 1;
         frerCpSeqRcvyPassedPackets  = frerCpSeqRcvyPassedPackets  + 1;
         RemainingTicks              =
                     ((frerSeqRcvyResetMSec*TicksPerSecond)+999)/1000;
         PRESENT_DATA;
      } else {
         frerCpsSeqRcvyDiscardedPackets =
                                    frerCpsSeqRcvyDiscardedPackets + 1;
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
      frerCpSeqRcvyDiscardPackets = frerCpSeqRcvyDiscardPackets + 1;
   }
   return;
}
// Compute signed difference modulo RecovSeqSpace.
int delta = (sequence_number-RecovSeqNum) & (RecovSeqSpace - 1);
if (0 != (delta & (RecovSeqSpace/2)))
   delta = delta - RecovSeqSpace;
// Here, -(RecovSeqSpace/2)<=delta<=((RecovSeqSpace/2)-1)
// After reset, accept any packet
if (TakeAny) {
   TakeAny                     = false;
   SequenceHistory[0]          = 1;    // Shift, adding a "seen" bit
   RecovSeqNum                 = sequence_number;
   frerCpsSeqRcvyPassedPackets = frerCpsSeqRcvyPassedPackets + 1;
   frerCpSeqRcvyPassedPackets  = frerCpSeqRcvyPassedPackets  + 1;
   RemainingTicks              =
      ((frerSeqRcvyResetMSec * TicksPerSecond) + 999) / 1000;
   PRESENT_DATA;
} else if (delta > frerSeqRcvyHistoryLength  ||
           delta <= -frerSeqRcvyHistoryLength     )
{
   // Packet is out-of-range.  Count and discard it.
   frerCpsSeqRcvyRoguePackets  = frerCpsSeqRcvyRoguePackets  + 1;
   frerCpSeqRcvyDiscardPackets = frerCpSeqRcvyDiscardPackets + 1;
   // Reset timer if working on an individual Stream
   if (frerSeqRcvyIndividualRecovery)
      RemainingTicks           =
         ((frerSeqRcvyResetMSec * TicksPerSecond) + 999) / 1000;
} else if (delta <= 0) {
   // Packet is old and in SequenceHistory; have we seen it before?
   if (0 == SequenceHistory[-delta]) {
      // Packet has not been seen.  Take it.
      SequenceHistory[-delta]     = 1;
      frerCpsSeqRcvyOutOfOrderPackets =
                                 frerCpsSeqRcvyOutOfOrderPackets  + 1;
      frerCpsSeqRcvyPassedPackets = frerCpsSeqRcvyPassedPackets + 1;
      frerCpSeqRcvyPassedPackets  = frerCpSeqRcvyPassedPackets  + 1;
      RemainingTicks              =
         ((frerSeqRcvyResetMSec * TicksPerSecond) + 999) / 1000;
      PRESENT_DATA;
   } else {
      // Packet has been seen.  Do not forward.  Count the discard.
      frerCpsSeqRcvyDiscardedPackets =
                                   frerCpsSeqRcvyDiscardedPackets + 1;
      frerCpSeqRcvyDiscardPackets = frerCpSeqRcvyDiscardPackets + 1;
      // Reset timer if working on an individual Stream
      if (frerSeqRcvyIndividualRecovery)
         RemainingTicks           =
            ((frerSeqRcvyResetMSec * TicksPerSecond) + 999) / 1000;
   }
} else {
   // Packet is not too far ahead of the one we want.
   // Packet is out-of-order unless it directly follows RecovSeqNum
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
      if (delta != 1)
         frerCpsSeqRcvyOutOfOrderPackets =
                                    frerCpsSeqRcvyOutOfOrderPackets + 1;
      // Shift the history until bit 0 refers to sequence_number.
      while (0 != (delta = delta - 1))
         ShiftSequenceHistory(0);   // Shift, adding a "not seen" bit
      ShiftSequenceHistory(1);      // Shift, adding a "seen" bit
      RecovSeqNum                 = sequence_number;
      frerCpsSeqRcvyPassedPackets = frerCpsSeqRcvyPassedPackets + 1;
      frerCpSeqRcvyPassedPackets  = frerCpSeqRcvyPassedPackets  + 1;
      RemainingTicks              =
         ((frerSeqRcvyResetMSec * TicksPerSecond) + 999) / 1000;
      PRESENT_DATA;
   }
}
```

7.4.3.5 MatchRecoveryAlgorithm

MatchRecoveryAlgorithm is called whenever the DATA_INDICATION event occurs (item b in 7.4.3.1) and the Sequence recovery function is configured to use the MatchRecoveryAlgorithm (frerSeqRcvyAlgorithm = **Match_Alg**, 10.4.1.5). If it terminates without triggering the PRESENT_DATA event (item d in 7.4.3.1), then the packet is effectively discarded.

Immediately after SequenceRecoveryReset (7.4.3.3) is called, the MatchRecoveryAlgorithm accepts the first packet received as valid. After the first packet has been accepted, all subsequent packets either match the last packet number accepted, and are therefore discarded, or do not, in which case they are accepted. Each packet accepted and passed up the stack resets the timer variable RemainingTicks (7.4.3.2.4). Dropped packets also reset the timer variable RemainingTicks (7.4.3.2.4) in case of Individual recovery. If that variable ticks down to 0, meaning that no packet has been accepted in frerSeqRcvyResetMSec milliseconds (10.4.1.7), then SequenceRecoveryReset again resets the algorithm, and the next packet received is accepted.

If a Talker or a relay system fails in such a way as to repeatedly transmit packets with the same sequence_number subparameter (perhaps repeating exactly the same packet), those packets will continue to be discarded, at least until the sequence_number wraps around. At that point, either the stuck packet or the right packet could be passed. Therefore, it is advisable for a Listener or relay system to be configured to discard packets in a single Member Stream. See 7.4.5 and C.10.

NOTE—Individual recovery functions (7.4.5) can be configured to discard repeated sequence_number values (or sequences of values) and prevent even this wrap around problem.

```
void MatchRecoveryAlgorithm () {
   //  Check that sequence number is present in the packet
   unsigned int sequence_number;
   if (sequence_number == frerSeqRcvyInvalidSequenceValue) {
      frerCpsSeqRcvyTaglessPackets = frerCpsSeqRcvyTaglessPackets + 1;
      frerCpsSeqRcvyPassedPackets  = frerCpsSeqRcvyPassedPackets  + 1;
      frerCpSeqRcvyPassedPackets   = frerCpSeqRcvyPassedPackets   + 1;
      PRESENT_DATA;
      return;
   }
   // After reset, accept any packet
   if (TakeAny) {
      TakeAny                     = false;
      RecovSeqNum                 = sequence_number;
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
      frerCpsSeqRcvyPassedPackets = frerCpsSeqRcvyPassedPackets + 1;
      frerCpSeqRcvyPassedPackets  = frerCpSeqRcvyPassedPackets  + 1;
      RemainingTicks              =
         ((frerSeqRcvyResetMSec * TicksPerSecond) + 999) / 1000;
      PRESENT_DATA;
      return;
   }
   // Compute signed difference modulo RecovSeqSpace.
   int delta = (sequence_number-RecovSeqNum) & (RecovSeqSpace - 1);
   if (delta == 0) {
      // Packet has been seen.  Do not forward.  Count the discard.
      frerCpsSeqRcvyDiscardedPackets =
                                    frerCpsSeqRcvyDiscardedPackets + 1;
      frerCpSeqRcvyDiscardPackets    = frerCpSeqRcvyDiscardPackets + 1;
      // Reset timer if working on an individual Stream
      if (frerSeqRcvyIndividualRecovery)
         RemainingTicks          =
            ((frerSeqRcvyResetMSec * TicksPerSecond) + 999) / 1000;
   } else {
      // Packet has not been seen; accept it.
      // Packet is out-of-order unless it directly follows RecovSeqNum
      if (delta != 1)
         frerCpsSeqRcvyOutOfOrderPackets =
                                 frerCpsSeqRcvyOutOfOrderPackets + 1;
      RecovSeqNum                 = sequence_number;
      frerCpsSeqRcvyPassedPackets = frerCpsSeqRcvyPassedPackets + 1;
      frerCpSeqRcvyPassedPackets  = frerCpSeqRcvyPassedPackets  + 1;
      RemainingTicks              =
         ((frerSeqRcvyResetMSec * TicksPerSecond) + 999) / 1000;
      PRESENT_DATA;
   }
}
```

7.4.3.6 ShiftSequenceHistory

This routine is called by the VectorRecoveryAlgorithm routine (7.4.3.4) to advance the SequenceHistory bit array (7.4.3.2.2) and to count lost packets (frerCpsSeqRcvyLostPackets, 10.8.7). ShiftSequenceHistory takes one parameter, which is the new value for index 0 in the SequenceHistory bit array.

```
void ShiftSequenceHistory (int newZeroValue) {
   int i;
   if (0 == SequenceHistory[frerSeqRcvyHistoryLength - 1])
      frerCpsSeqRcvyLostPackets = frerCpsSeqRcvyLostPackets + 1;
   for (i = frerSeqRcvyHistoryLength - 1; i != 0; i = i - 1)
      SequenceHistory[i] = SequenceHistory[i - 1];
   SequenceHistory[0] = newZeroValue;
}
```

7.4.4 Latent error detection function

Each instance of a Latent error detection function is part of a Sequence recovery function (7.4.3). It monitors the managed objects associated with a single instance of the Base recovery function (7.4.2), in order to detect the condition that relatively few packets are being discarded by that function. Latent error detection operates on the assumption that, in a properly functioning Compound Stream employing *n* paths into the

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

current system, there will be $n - 1$ packets discarded for every packet passed through the Base recovery function (7.4.2). The latent error detection function issues a SIGNAL_LATENT_ERROR (item d in 7.4.4.1) when that assumption is violated.

There is at most one latent error detection function per port per stream_handle value per direction (in-facing or out-facing), as specified through the managed objects (10.4) that control both functions. For every Base recovery function configured, there can be only zero or one associated latent error detection function. Any implementation that supports the Base recovery function shall support the Latent error detection function.

Latent error detection is accomplished by two periodic functions. The first (LatentErrorTest, 7.4.4.4) examines the number of packets passed and discarded, and reports a latent error if the differences among those counters exceed a set threshold. The other periodic function (LatentErrorReset, 7.4.4.3) resets the variables used by the first, so that occasional random packet losses do not accumulate forever. These functions are driven by timers, not by the receipt or transmission of packets.

The latent error detection function is described in terms of the following:

a) Managed objects that control the latent error detection function and that are affected by the latent error detection function (10.4, 10.8);

b) Events, the occurrence of which trigger the execution of code routines, and that can be triggered by the code routines (7.4.4.1);

c) Variables that are manipulated by the code routines, and maintain the state of the latent error detection function between routine executions (7.4.4.2); and

d) Two code routines that interact with the managed objects, events, and variables (7.4.4.3, 7.4.4.4).

NOTE—The latent error detection algorithm (7.4.4.3, 7.4.4.4) cannot catch all errors. For example, if one path erroneously produces twice the proper number of copies and the other path fails, LatentErrorTest will generate no error. For this reason, the LatentErrorTest should be used in conjunction with Individual recovery functions (7.4.5).

7.4.4.1 Events for latent error detection

There are three events that can affect the latent error detection function:

a) **BEGIN:** The global event that resets all functions, including the latent error detection function. This event triggers the execution of the LatentErrorReset routine (7.4.4.3).

b) **RESET_LATENT_ERROR:** This event is generated periodically and is controlled by frerSeqRcvyLatentResetPeriod (10.4.1.12.4). It triggers the execution of the LatentErrorReset routine (7.4.4.3).

c) **TRIGGER_LATENT_ERROR_TEST:** This event is generated periodically and is controlled by frerSeqRcvyLatentErrorPeriod (10.4.1.12.2). It triggers the execution of the LatentErrorTest routine (7.4.4.4).

The latent error detection function can trigger one event:

d) **SIGNAL_LATENT_ERROR:** The event generated by the latent error detection function that indicates to the upper layers a possible error condition.

7.4.4.2 Variables for latent error detection

Each instance of the latent error detection function has its own set of variables, independent from any other instance.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

7.4.4.2.1 CurBaseDifference

The CurBaseDifference variable is an unsigned integer that is the same size as the counters (e.g., frerCpsSeqRcvyPassedPackets, 10.8.5) from which its value is computed. It contains the offset between the expected and actual number of discarded packets as it was the last time that the LatentErrorReset function (7.4.4.3) was called.

7.4.4.3 LatentErrorReset

LatentErrorReset is called whenever the BEGIN event (item a in 7.4.4.1) or the RESET_LATENT_ERROR (item b in 7.4.4.1) event occur. It recomputes the CurBaseDifference (7.4.4.2.1) variable based on frerCpsSeqRcvyPassedPackets (10.8.5), frerSeqRcvyLatentErrorPaths (10.4.1.12.3), and frerCpsSeqRcvyDiscardedPackets (10.8.6), and increments frerCpsSeqRcvyLatentErrorResets (10.8.10).

```
void LatentErrorReset () {
   CurBaseDifference = (frerCpsSeqRcvyPassedPackets *
                           (frerSeqRcvyLatentErrorPaths - 1)) -
                                    frerCpsSeqRcvyDiscardedPackets;
   frerCpsSeqRcvyLatentErrorResets = frerCpsSeqRcvyLatentErrorResets +1;
}
```

7.4.4.4 LatentErrorTest

LatentErrorTest is called whenever the TRIGGER_LATENT_ERROR_TEST event occurs (item c in 7.4.4.1). It tests to see whether the number of packets discarded by the sequence recovery function is approximately the expected number, and triggers an event if not, using CurBaseDifference (7.4.4.2.1), frerCpsSeqRcvyPassedPackets (10.8.5), frerSeqRcvyLatentErrorPaths (10.4.1.12.3), frerCpsSeqRcvyDiscardedPackets (10.8.6), frerSeqRcvyLatentErrorPeriod (10.4.1.12.2), and frerSeqRcvyLatentErrorDifference (10.4.1.12.1).

```
void LatentErrorTest () {
   int diff = CurBaseDifference - ((frerCpsSeqRcvyPassedPackets *
                                    (frerSeqRcvyLatentErrorPaths - 1)) -
                                        frerCpsSeqRcvyDiscardedPackets);
   if (frerSeqRcvyLatentErrorPaths  > 1  &&   // There are multiple paths
       frerSeqRcvyLatentErrorPeriod > 0     ) // LE detection is turned on
   {
      if (diff < 0)
         diff = - diff;
      if (diff > frerSeqRcvyLatentErrorDifference) {
         SIGNAL_LATENT_ERROR;
      }
   }
}
```

NOTE—This algorithm has, in effect, a quantization error of 2. That is, if the user is unlucky in the timing of LatentErrorTest and LatentErrorReset, it may take a `diff` of 2*frerSeqRcvyLatentErrorDifference to trigger a fault.

7.4.5 Individual recovery function

The Individual recovery function is defined in order to meet the Robustness goal (item j in 7.1.1). It accomplishes this by removing repeating sequence numbered packets received from a stuck transmitter. An instantiation of the Individual recovery function consists of an instantiation of the Base recovery function (7.4.2) with its frerSeqRcvyIndividualRecovery object (10.4.1.10) set to True, configured to apply to a single Member Stream. [A Sequence recovery function (7.4.3) operates on all Member Streams of a

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Compound Stream.] An instantiation of the Individual recovery function does not include an instance of the Latent error detection function (7.4.4). An instantiation of the Individual recovery function can employ either the VectorRecoveryAlgorithm (7.4.3.4) or the MatchRecoveryAlgorithm (7.4.3.5).

By applying the Individual recovery function (7.4.5) to Member Streams, whether before or without the application of the Sequence recovery function (7.4.3) to Compound Streams, errored Streams can be discovered early, and pollution of the merged Stream avoided. Figure 7-3 shows the example network of Figure 7-1, to which instances of the Individual recovery function have been added. See also C.10.

Unlike the Sequence recovery function, which is modeled as being instantiated on a specific port, a single instantiation of an Individual recovery function can be applied to any number of ports. See C.11.

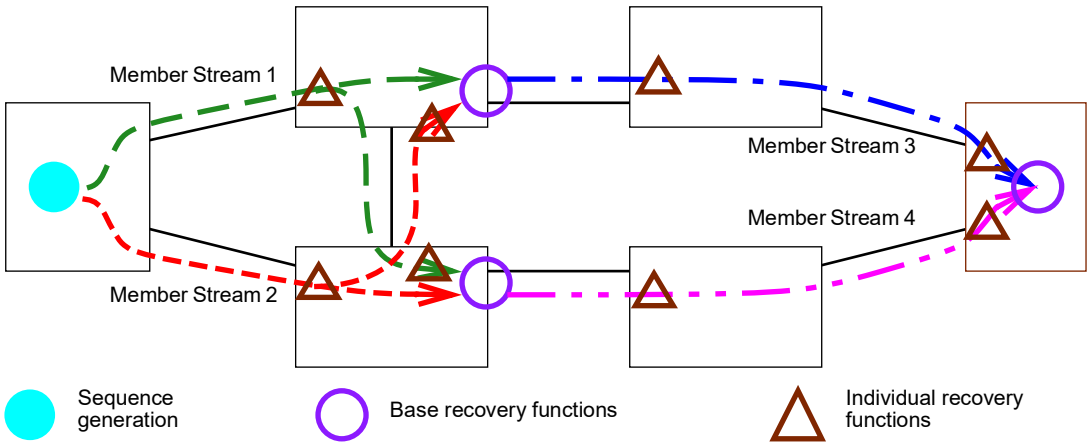


Figure 7-3—Base recovery functions and Individual recovery functions

7.5 Sequence encode/decode function

The Sequence encode/decode function is responsible for inserting the sequence_number subparameter (item b in 6.1) into the packet, and extracting it from the packet. If the Stream identification function (6.2) also encodes and decodes the sequence_number subparameter for a given Stream on a particular port and direction, then no Sequence encode/decode sublayer is needed. Instances of the Sequence encode/decode function are instantiated via managed objects (10.5).

The only Sequence encode/decode format specified as required, separately from any Stream identification function, is the Redundancy tag (R-TAG, 7.7). In addition, two optional formats (7.8, 7.9) are defined.

7.6 Stream splitting function

The Stream splitting function accepts a packet from the upper layers with a stream_handle subparameter (item a in 6.1), makes zero or more copies of that packet, each with a stream_handle subparameter that can be different from the original stream_handle, and passes those packets to the next-lower layer. This effectively creates Member Stream(s) from a Compound Stream. Packets passing up the stack are unchanged by the Stream splitting function.

Instances of the Stream splitting function are created to process specific Streams by means of the Stream split table (10.6) managed object. A packet passed down from the upper layers is acted upon by a Stream splitting function on a particular port (10.6.1.1) and direction (10.6.1.2) only if its stream_handle subparameter is in the frerSplitInputIdList (10.6.1.3) configured for that port and direction in some entry in the Stream split table (10.6). If the stream_handle matches any of the items in that list, then one copy of the

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

packet is generated for each item in that same frerSplitEntry's frerSplitOutputIdList (10.6.1.4), each copy with one of those values for its stream_handle subparameter.

7.7 Redundancy tag

The Redundancy tag (R-TAG) is an example of a Sequence encode/decode function (7.5). It operates at the frame level and can be defined using the ISS defined by IEEE Std 802.1AC, or the EISS described in 6.9 of IEEE Std 802.1Q-2022, enhanced with the extra stream_handle and sequence_number subparameters specified in 6.1. In order to instantiate the Sequence encode/decode function using the R-TAG, the frerSeqEncEncapsType managed object (10.5.1.5) is encoded using the OUI (00-80-C2) and the type values as shown in Table 10-2. The R-TAG is illustrated in Figure 7-4. The value for the EtherType for the R-TAG is given in Table 7-1.

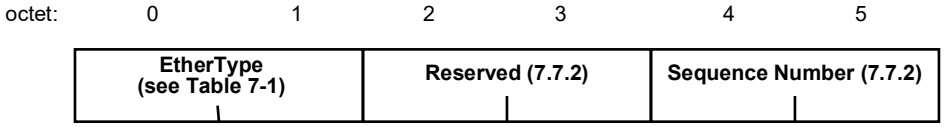


Figure 7-4—R-TAG format

When presented by the upper layers with a service request (ISS M_UNITDATA.request or EISS EM_UNITDATA.request), the Redundancy tag Sequence encode/decode function:

a) Creates an Redundancy tag information (7.7.2) word by copying the low-order 16 bits from the sequence_number subparameter to the Sequence Number field of the Redundancy tag information, and fills the Reserved field with zeros. The most-significant octet of the low-order 16 bits of the sequence_number goes in the first octet (octet 4 in Figure 7-4).

NOTE 1—Any high-order bits above the low-order 16 bits of the sequence_number subparameter are ignored.

b) Inserts an EtherType (7.7.1) and the Redundancy tag information as the first octets of the mac_service_data_unit parameter, thus increasing the size of the mac_service_data_unit parameter by 6 octets.

When presented by the lower layers with a service indication (ISS M_UNITDATA.indication or EISS EM_UNITDATA.indication), the Redundancy tag Sequence encode/decode function:

c) Examines the first two octets of the mac_service_data_unit parameter for equality with the Redundancy tag EtherType specified in 7.7.1. If it is equal, and if the mac_service_data_unit is at least 6 octets in length, then the Redundancy tag Sequence encode/decode function:

   1) Removes the first 6 octets of the mac_service_data_unit, shortening it by 6 octets; and

   2) Copies the Sequence Number field of the Redundancy tag information to the sequence_number subparameter, ignoring the contents of the Reserved field.

d) If the first two octets of the mac_service_data_unit parameter are not equal to the Redundancy tag EtherType, or if the mac_service_data_unit is less than 6 octets in length, then the Redundancy tag Sequence encode/decode function:

   1) Sets the sequence_number subparameter to frerSeqRcvyInvalidSequenceValue (10.4.1.8).

   2) Increments the frerCpsSeqEncErroredPackets and frerCpSeqEncErroredPackets managed objects (10.8.11, 10.9.3).

NOTE 2—The position of the R-TAG in a frame relative to other tags depends on the number and relative position of the Sequence encode/decode functions and other functions in the protocol stacks of the various components of a network. The managed objects in Clause 9 and Clause 10 support placing the Sequence encode/decode functions in any position to suit the needs of an application. See Clause 8 for a typical application of FRER to an IEEE 802.1Q Bridge.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

7.7.1 Redundancy tag EtherType

The Tag Protocol Identifier (TPID) of the R-TAG shall use the value shown in Table 7-1.

Table 7-1—R-TAG EtherType

Purpose	EtherType
Redundancy tag (R-TAG)	F1-C1

7.7.2 Redundancy tag information

The R-TAG information consists of two fields:

a) The first two octets of the R-TAG information is a 16-bit Reserved field. This field shall be transmitted with all zeros and shall be ignored on receipt. It is intended that future revisions of this standard can use the most-significant bits of the Reserved field for sub-typing purposes, as described in 9.2.1 of IEEE Std 802-2024.
b) The last two octets of the R-TAG information are a 16-bit value, the Sequence Number field.

7.8 HSR sequence tag

The optional High-availability Seamless Redundancy (HSR) sequence tag is an example of a Sequence encode/decode function (7.5). It operates at the frame level and can be defined using the ISS defined by IEEE Std 802.1AC, or the EISS described in 6.9 of IEEE Std 802.1Q-2022, enhanced with the extra stream_handle and sequence_number subparameters specified in 6.1. In order to instantiate the Sequence encode/decode function using the HSR sequence tag, the frerSeqEncEncapsType managed object (10.5.1.5) is encoded using the OUI (00-80-C2) and the type values as shown in Table 10-2. The HSR sequence tag is described in 5.7.1 of IEC 62439-3:2021.

When presented by the upper layers with a service request (ISS M_UNITDATA.request or EISS EM_UNITDATA.request), the HSR sequence tag Sequence encode/decode function:

a) Creates an HSR sequence tag by:
    1) Copying the low-order 16 bits from the sequence_number subparameter to the SeqNr field of the HSR sequence tag;
    2) Setting the LSDU Size field in accordance with 5.7.1 of IEC 62439-3:2021;
    3) Setting the PathId field according to the per-port per-Stream managed object frerSeqEncPathIdLanId (10.5.1.6); and
    4) Setting the EtherType field according to 5.7.1 of IEC 62439-3:2021.

NOTE 1—Any high-order bits above the low-order 16 bits of the sequence_number subparameter are ignored.

b) Inserts the HSR sequence tag as the first octets of the mac_service_data_unit parameter, thus increasing the size of the mac_service_data_unit parameter by 6 octets.

When presented by the lower layers with a service indication (ISS M_UNITDATA.indication or EISS EM_UNITDATA.indication), the HSR sequence tag Sequence encode/decode function:

c) Examines the first two octets of the mac_service_data_unit parameter for equality with the HSR sequence tag EtherType specified in 5.7.1 of IEC 62439-3:2021. If it is equal, and if the mac_service_data_unit is at least 6 octets in length, then the HSR sequence tag Sequence encode/decode function:

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

1) Removes the first 6 octets of the mac_service_data_unit, shortening it by 6 octets; and

2) Copies the SeqNr field to the sequence_number subparameter.

d) If the first two octets of the mac_service_data_unit parameter are not equal to the HSR sequence tag EtherType, or if the mac_service_data_unit is less than 6 octets in length, then the HSR sequence tag Sequence encode/decode function:

1) Sets the sequence_number subparameter to frerSeqRcvyInvalidSequenceValue (10.4.1.8).

2) Increments the frerCpsSeqEncErroredPackets and frerCpSeqEncErroredPackets managed objects (10.8.11, 10.9.3).

NOTE 2—No part of this standard is to be construed in a manner so as to alter the specifications in, or the intent of, IEC 62439-3:2016, or to restrict its future development in any way. The purpose of 7.8 is to enable the creation of interworking functions between end systems employing the R-TAG (7.7) and the HSR sequence tag.

NOTE 3—See B.2 for additional considerations when interworking between FRER and HSR.

NOTE 4—The position of the Sequence encode/decode function in the protocol stack does not need to change depending on whether the R-TAG, the HSR tag, or the PRP trailer is used.

7.9 PRP sequence trailer

The optional Parallel Redundancy Protocol (PRP) sequence trailer is an example of a Sequence encode/decode function (7.5). It operates at the frame level and can be defined using the ISS defined by IEEE Std 802.1AC, or the EISS described in 6.9 of IEEE Std 802.1Q-2022, enhanced with the extra stream_handle and sequence_number subparameters specified in 6.1. In order to instantiate the Sequence encode/decode function using the PRP sequence trailer, the frerSeqEncEncapsType managed object (10.5.1.5) is encoded using the OUI (00-80-C2) and the type values as shown in Table 10-2. The PRP sequence trailer is described in 4.2.7.3 of IEC 62439-3:2021.

When presented by the upper layers with a service request (ISS M_UNITDATA.request or EISS EM_UNITDATA.request), the PRP sequence trailer Sequence encode/decode function:

a) Creates an PRP sequence trailer by:

1) Copying the low-order 16 bits from the sequence_number subparameter to the SeqNr field of the PRP sequence trailer;

2) Setting the LSDU Size field in accordance with 4.2.7.3 of IEC 62439-3:2021;

3) Setting the LanId field according to the per-port per-Stream managed object frerSeqEncPathIdLanId (10.5.1.6); and

4) Setting the PRPsuffix field according to 4.2.7.3 of IEC 62439-3:2021.

NOTE 1—Any high-order bits above the low-order 16 bits of the sequence_number subparameter are ignored.

b) Inserts the PRP sequence trailer as the final octets of the mac_service_data_unit parameter, thus increasing the size of the mac_service_data_unit parameter by 6 octets.

When presented by the lower layers with a service indication (ISS M_UNITDATA.indication or EISS EM_UNITDATA.indication), the PRP sequence trailer Sequence encode/decode function:

c) Examines the last two octets of the mac_service_data_unit parameter for equality with the PRP sequence trailer PRPsuffix field specified in 4.2.7.3 of IEC 62439-3:2021. If it is equal, and if the mac_service_data_unit is at least 8 octets in length, then the PRP sequence trailer Sequence encode/decode function:

1) Removes the last 6 octets of the mac_service_data_unit, shortening it by 6 octets; and

2) Copies the SeqNr field to the sequence_number subparameter.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

d) If the last two octets of the mac_service_data_unit parameter are not equal to the PRP sequence trailer EtherType, or if the mac_service_data_unit is less than 8 octets in length, then the PRP sequence trailer Sequence encode/decode function:

1) Sets the sequence_number subparameter to frerSeqRcvyInvalidSequenceValue (10.4.1.8).

2) Increments the frerCpsSeqEncErroredPackets and frerCpSeqEncErroredPackets managed objects (10.8.11, 10.9.3).

NOTE 2—No part of this standard is to be construed in a manner so as to alter the specifications in, or the intent of, IEC 62439-3:2016, or to restrict its future development in any way. The purpose of 7.9 is to enable the creation of interworking functions between end systems employing the R-TAG (7.7) and the PRP sequence trailer.

NOTE 3—See B.2 for additional considerations when interworking between FRER and PRP.

7.10 ~~Autoconfiguration~~

7.10.1 ~~Introduction to autoconfiguration~~

~~In order to satisfy the goal of Ease of use (item l in 7.1.1), there are two methods by which Stream identification functions (6.2), Individual recovery functions (7.4.5), Base recovery functions (7.4.2), and Sequence encode/decode functions (7.5) can be configured in a system:~~

~~a) Explicit configuration of entries in the Stream identity table (9.1), Sequence recovery table (10.4), Sequence identification table (10.5); and~~

~~b) Autoconfiguration of entries in these tables using the Sequence autoconfiguration table (10.7.1) and Output autoconfiguration table (10.7.2).~~

~~Autoconfiguration can only be used with Source MAC and VLAN Stream identification (6.5) and the MatchRecoveryAlgorithm (7.4.3.5).~~

~~Whenever a packet is received that has a sequence_number subparameter encoded in a manner identified in an entry in the Sequence autoconfiguration table, but for which there is no tsnStreamIdEntry (9.1.1) in the Stream identity table, a new entry in the Stream identity table is created for the packet's Stream, and optionally, a new entry in the Sequence recovery table for an Individual recovery function for that Stream. If a second or subsequent such Stream identity table entry is created that shares all the same key parameters as another autoconfigured tsnStreamIdEntry, except for port number (tsnStreamIdOutFacInputPortList, 9.1.1.5) and/or LanId/PathId (tsnStreamIdLanPathId, 10.2.2), then another entry in the Sequence recovery table, this time for a Sequence recovery function, is created (for the second Member Stream) or is expanded (for subsequent Member Streams) for what is now known to be a Compound Stream.~~

~~The Sequence autoconfiguration table controls whether Individual recovery functions, Base recovery functions, neither, or both, are created automatically. Stream identification functions are always created. If the R-TAG (7.7) is used to encode the sequence_number, entries autoconfigured in the Sequence identification table, and thus the Individual recovery functions (7.4.5) created, are keyed to the individual ports and VLANs on which the first packet of each Member Stream is received. If the HSR sequence tag (7.8) or PRP sequence trailer (7.9) is used, entries autoconfigured in the Sequence identification table and the autocreated Individual recovery functions are keyed to the PathId or LanId of the first-received packet of each Member Stream, and span all autoconfigurable ports.~~

~~In order to support the goals of Ease of use (item l in 7.1.1) and Interoperability (item g in 7.1.1) among systems using the HSR sequence tag (7.8), the PRP sequence trailer (7.9), and the R-TAG (7.7), passive (packet decoding) entries are also automatically created in the Sequence identification table to instantiate Sequence encode/decode functions as governed by the Sequence autoconfiguration table (10.7.1). Typically,~~

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

~~the Output autoconfiguration table (10.7.2) is configured to construct active (packet encoding) Sequence encode/decode functions on every port on which a Stream might be output. See C.11.~~

7.10.2 ~~Autoconfigured Stream identity table entry creation and destruction~~

~~NOTE 1—The following description, if implemented naïvely, might well result in a relay system delaying the transmission of the first-received packet of a Member Stream for an unacceptable length of time. The purpose of the following description is to describe the externally visible behavior of a system solely in terms of the managed objects in Clause 9 and Clause 10, not to constrain an implementation. It is expected that an actual implementation is likely to construct databases in software, firmware, and/or hardware that combine elements of the managed objects in Clause 9 and Clause 10, and to perform up-front preliminary configuration, so as to optimize the reaction time when a packet is actually received.~~

~~Autoconfiguration is triggered by the receipt of a packet that matches an entry in the Sequence autoconfiguration table (10.7.1) as follows:~~

a) ~~If a packet received on an out-facing port matches an entry in the Stream identity table (9.1), it is processed according to the managed objects in 10.3, 10.4, 10.5, and 10.6.~~

~~NOTE 2—It does not matter whether the Stream identity table entry was configured explicitly, or was created via Autoconfiguration.~~

b) ~~If the packet does not match any entry in the Stream identity table at all, the packet is examined to see if it matches an entry in the Sequence autoconfiguration table. A match between the packet and a frerAutSeqEntry (10.7.1.1) in the Sequence autoconfiguration table occurs if all of the following match conditions are met:~~

   1) ~~The packet has a sequence_number encapsulated in the manner specified by frerAutSeqSeqEncaps (10.7.1.1.1).~~
   2) ~~The packet was received on a port in frerAutSeqReceivePortList (10.7.1.1.2).~~
   3) ~~The packet's VLAN tag (or lack thereof) matches frerAutSeqTagged (10.7.1.1.3).~~
   4) ~~Either the vlan_identifier parameter is in the list in frerAutSeqVlan (10.7.1.1.4), or that list is empty.~~

c) ~~If the packet does not match an entry in the Sequence autoconfiguration table, no further Autoconfiguration processing takes place. Otherwise (the packet did not match any tsnStreamIdEntry, but does match a frerAutSeqEntry), Autoconfiguration processing proceeds.~~

d) ~~The system creates (autoconfigures) new entries in the Stream identity table, the Sequence recovery table (10.4). and the Sequence identification table (10.5) using the matched frerAutSeqEntry as follows:~~

   1) ~~One new tsnStreamIdEntry (9.1.1) is created in the Stream identity table as follows:~~
      i) ~~tsnStreamIdHandle (9.1.1.1) = an unused stream_handle value chosen by the implementation.~~
      ii) ~~tsnStreamIdInFacOutputPortList (9.1.1.2) = empty.~~
      iii) ~~tsnStreamIdOutFacOutputPortList (9.1.1.3) = empty.~~
      iv) ~~tsnStreamIdInFacInputPortList (9.1.1.4) = empty.~~
      v) ~~If frerAutSeqSeqEncaps equals either HSR or PRP, tsnStreamIdOutFacInputPortList (9.1.1.5) = frerAutSeqReceivePortList (10.7.1.1.2). Otherwise, frerSeqRcvyPortList (10.4.1.2) = only the port on which the packet was received.~~
      vi) ~~tsnStreamIdIdentificationType (9.1.1.6) = 00-80-C2, 2 = Source MAC and VLAN Stream identification (6.5).~~
      vii) ~~tsnStreamIdAutoconfigured (10.2.1) = True.~~
      viii) ~~If frerAutSeqSeqEncaps equals either HSR or PRP, tsnStreamIdLanPathId (10.2.2) = the PathId or LanId field from the packet. Otherwise, tsnStreamIdLanPathId is not used.~~
      ix) ~~tsnCpeSmacVlanDownSrcMac (9.1.3.1) = the source_mac_address parameter of the received packet.~~

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

x) ~~tsnCpeSmacVlanDownTagged (9.1.3.2) = frerAutSeqTagged (10.7.1.1.3.~~
xi) ~~tsnCpeSmacVlanDownVlan (9.1.3.3) = the vlan_identifier parameter of the received packet.~~

2) ~~If and only if frerAutSeqCreateIndividual (10.7.1.1.10) is True, a frerSeqRcvyEntry (10.4.1) is created in the Sequence recovery table (10.4) as follows:~~

i) ~~frerSeqRcvyStreamList (10.4.1.1) = the stream_handle from item d:2:i, above.~~
ii) ~~frerSeqRcvyPortList (10.4.1.2) = the same list as tsnStreamIdOutFacInputPortList (9.1.1.5), described in item d:2:v, above. (See C.11.)~~
iii) ~~frerSeqRcvyDirection (10.4.1.3) = True (out-facing).~~
iv) ~~frerSeqRcvyReset (10.4.1.4) = True. (The state machine is reset when created.)~~
v) ~~frerSeqRcvyAlgorithm (10.4.1.5) = Match_Alg (00-80-C2, 1, see Table 10-1).~~
vi) ~~frerSeqRcvyHistoryLength (10.4.1.6) is not used.~~
vii) ~~frerSeqRcvyResetMSec (10.4.1.7) = frerAutSeqResetMSec (10.7.1.1.7).~~
viii) ~~frerSeqRcvyInvalidSequenceValue (10.4.1.8) is a read-only value chosen by the system.~~
ix) ~~frerSeqRcvyTakeNoSequence (10.4.1.9) = True.~~
x) ~~frerSeqRcvyIndividualRecovery (10.4.1.10) = True.~~
xi) ~~frerSeqRcvyLatentErrorDetection (10.4.1.11) = False.~~

3) ~~One new passive frerSeqEncEntry (10.5.1) is created in the Sequence identification table to decode further received packets as follows:~~

i) ~~frerSeqEncStreamList (10.5.1.1) = the stream_handle from item d:2:i, above.~~
ii) ~~frerSeqEncPort (10.5.1.2) = frerAutSeqReceivePortList (10.7.1.1.2).~~
iii) ~~frerSeqEncDirection (10.5.1.3) = True (out-facing).~~
iv) ~~frerSeqEncActive (10.5.1.4) = False (passive).~~
v) ~~frerSeqEncEncapsType (10.5.1.5) = frerAutSeqSeqEncaps (10.7.1.1.1).~~
vi) ~~If frerAutSeqSeqEncaps == 00-80-C2, 2 (HSR), frerSeqEncPathIdLanId (10.5.1.6) = the PathId field of the HSR sequence tag (7.8) found in the packet. If frerAutSeqSeqEncaps == 00-80-C2, 3 (PRP), frerSeqEncPathIdLanId = the LanId field of the PRP sequence trailer (7.9) found in the packet. Otherwise, frerSeqEncPathIdLanId is not used.~~

~~If a frerSeqEncEntry with the same port list, direction, and encapsulation type is present, a new entry need not be created; the packet's stream_handle can be added to that frerSeqEncEntry's frerSeqEncStreamList.~~

4) ~~One new active frerSeqEncEntry (10.5.1) is created in the Sequence identification table for each frerAutOutEntry (10.7.2.1) in the Output autoconfiguration table (10.7.2) to format transmitted packets for this packet's Stream as follows:~~

i) ~~frerSeqEncStreamList (10.5.1.1) = the stream_handle from item d:2:i, above.~~
ii) ~~frerSeqEncPort (10.5.1.2) = frerAutOutPortList (10.7.2.1.1).~~
iii) ~~frerSeqEncDirection (10.5.1.3) = True (out-facing).~~
iv) ~~frerSeqEncEncapsType (10.5.1.5) = frerAutOutEncaps (10.7.2.1.2).~~
v) ~~frerSeqEncPathIdLanId (10.5.1.6) = frerAutOutLanPathId (10.7.2.1.3).~~

~~If a frerSeqEncEntry with the same port list, direction, encapsulation type, and PathId/LanId is present, a new entry need not be created; the packet's stream_handle can be added to that frerSeqEncEntry's frerSeqEncStreamList.~~

e) ~~If none of the ports to which the received packet is to be output are listed in the frerAutSeqEntry's frerAutSeqRecoveryPortList (10.7.1.1.5), or if frerAutSeqCreateRecovery (10.7.1.1.11) is false, Autoconfiguration processing is terminated. No Sequence recovery function need be instantiated.~~

f) ~~The system next scans the Stream identity table looking for a tsnStreamIdEntry that nearly matches the received packet, in order to discover whether there are other Streams similar to the received packet's Stream such that they are all Member Streams of a Compound Stream. "Nearly matches" means that all of the following conditions are met:~~

1) ~~tsnStreamIdIdentificationType (9.1.1.6) == 00-80-C2, 2 == Source MAC and VLAN Stream identification (6.5).~~

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

2) ~~The packet's stream_handle is not equal to tsnStreamIdHandle (9.1.1.1).~~
3) ~~The source_mac_address parameter of the packet == tsnCpeSmacVlanDownSreMac (9.1.3.1).~~
4) ~~Either tsnCpeSmacVlanDownVlan (9.1.3.3) is in the frerAutSeqVlan (10.7.1.1.4) list, or that list is empty.~~

g) ~~If no tsnStreamIdEntry is found that meets these criteria, then Autoconfiguration processing is terminated. No Sequence recovery function need be instantiated.~~

h) ~~If a tsnStreamIdEntry is found that does meet these criteria, then either Base recovery functions need to be instantiated, or existing instantiations need to be modified to include the received packet's Member Stream. To determine which, the system searches the Sequence recovery table for a frerSeqRcvyEntry that meets all of the following criteria:~~

1) ~~frerSeqRcvyStreamList contains the stream_handle of the found tsnStreamIdEntry.~~
2) ~~frerSeqRcvyIndividualRecovery (10.4.1.10) == False == Sequence recovery function.~~
3) ~~frerSeqRcvyPortList (10.4.1.2) == frerAutSeqRecoveryPortList (10.7.1.1.5).~~
4) ~~For a relay system, frerSeqRcvyDirection (10.4.1.3) is False (in-facing). For an end system, True (out-facing).~~

i) ~~If the Sequence recovery table search (item h, above) is found, then the following steps are taken:~~

1) ~~The packet's stream_handle is added to the frerSeqRcvyEntry's frerSeqRcvyStreamList (10.4.1.1).~~
2) ~~If frerAutSeqLatErrDetection (10.7.1.1.12) is True, then the following steps are taken:~~
   i) ~~frerSeqRcvyLatentErrorPaths (10.4.1.12.3) is incremented by 1.~~
   ii) ~~The LatentErrorReset function (7.4.4.3) is called for the instantiation of the Latent error detection function (7.4.4) on every port in the frerSeqRcvyEntry's frerSeqRcvyPortList (10.4.1.2).~~

j) ~~Otherwise (the Sequence recovery table search in item h, above, failed to find a frerSeqRcvyEntry), a new frerSeqRcvyEntry is created as follows:~~

1) ~~frerSeqRcvyStreamList (10.4.1.1) = two stream_handle values, the stream_handle from the received packet (item d:2:i, above) and the tsnStreamIdHandle (9.1.1.1) from the tsnStreamIdEntry found in item f, above.~~
2) ~~frerSeqRcvyPortList (10.4.1.2) = frerAutSeqRecoveryPortList (10.7.1.1.5).~~
3) ~~frerSeqRcvyDirection (10.4.1.3) = False (in-facing) for a relay system, or True (out-facing) for an end system.~~
4) ~~frerSeqRcvyReset (10.4.1.4) = True. (The state machine is reset when created.)~~
5) ~~frerSeqRcvyAlgorithm (10.4.1.5) = frerAutSeqAlgorithm (10.7.1.1.8).~~
6) ~~frerSeqRcvyHistoryLength (10.4.1.6) = frerAutSeqAlgorithm (10.7.1.1.8). This value is not used if frerSeqRcvyAlgorithm (10.4.1.5) == Match_Alg (00-80-C2, 1, see Table 10-1).~~
7) ~~frerSeqRcvyResetMSec (10.4.1.7) = frerAutSeqResetMSec (10.7.1.1.7).~~
8) ~~frerSeqRcvyInvalidSequenceValue (10.4.1.8) is a read-only value chosen by the system.~~
9) ~~frerSeqRcvyTakeNoSequence (10.4.1.9) = True.~~
10) ~~frerSeqRcvyIndividualRecovery (10.4.1.10) = False.~~
11) ~~frerSeqRcvyLatentErrorDetection (10.4.1.11) = frerAutSeqLatErrDetection (10.7.1.1.12).~~
12) ~~if frerAutSeqLatErrDetection is True, then the following assignments are made:~~
   i) ~~frerSeqRcvyLatentErrorDifference (10.4.1.12.1) = frerAutSeqLatErrDifference (10.7.1.1.13).~~
   ii) ~~frerSeqRcvyLatentErrorPeriod (10.4.1.12.2) = frerAutSeqLatErrPeriod (10.7.1.1.14).~~
   iii) ~~frerSeqRcvyLatentErrorPaths (10.4.1.12.3) = 2.~~
   iv) ~~frerSeqRcvyLatentResetPeriod (10.4.1.12.4) = frerAutSeqLatErrResetPeriod (10.7.1.1.15).~~

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

k) ~~The packet is then processed according to the newly created entries.~~

l) ~~After the reception of the packet that triggers Autoconfiguration for a stream_handle, if none of the Individual recovery functions (7.4.5) or Base recovery functions (7.4.2) instantiated by a given frerAutSeqEntry (10.7.1.1) increment any packet counters (10.8) for longer than the period specified by the frerAutSeqEntry's frerAutSeqDestructMSec (10.7.1.1.6) object, the system can destroy the state machines created by that frerAutSeqEntry.~~

~~NOTE 3—As Member Streams are found, instances of the Sequence recovery function are created, and the number of Member Flows increased to enable Latent Error Detection. If the destruct timer (frerAutSeqDestructMSec, 10.7.1.1.6) is not much larger than the normal reset period for the state machines (frerSeqRevyResetMSec, 10.4.1.7), there is a risk that the state machines will be destroyed due to a temporary service interruption, that one or more of the Member Streams will fail to be restored after the interruption, and the purpose of Latent Error Detection will be defeated.~~

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

8. Frame Replication and Elimination for Reliability in Bridges

8.1 Limiting options

The arrangements of the various components of Time-Sensitive Networking (TSN) and Frame Replication and Elimination for Reliability (FRER) in Annex C show that the model described in Clause 6 and Clause 7 has the advantages that proper layering principles are observed, that the descriptions of the individual functions are simple, and most importantly, that the definition of a relay system's forwarding function does not have to be modified in order to add TSN or FRER functions; all of the TSN and FRER capabilities can be provided as add-ons in the ports. However, this model offers a multitude of options for the placement of functions in an actual system. Experience shows that such flexibility is often not necessary, and if implemented naively, can increase the complexity of the system both for the implementor and the user.

Change the text as shown:

Most of the flexibility of the model described in Clause 6 and Clause 7 can be achieved in an IEEE 802.1Q Bridge, if we limit the Bridge to two stages of activity:

1) **Input transformations:** An expanded port (as in, e.g., Figure C-5) that includes Sequence generation functions (7.4.1) and active In-facing Stream identification functions (6.6), shown in white boxes with boldface type in Figure 8-1.
2) **Augmented forwarding:** The Bridge Forwarding Process, described in 8.6 of IEEE Std 802.1Q-2022 (shown in white boxes in Figure 8-1 and Figure 8-2), augmented with Stream identification functions (6.2, both incoming and outgoing), Sequence encode/decode functions (7.5, both incoming and outgoing), Sequence recovery functions (7.4.2), Individual recovery functions (7.4.5), and their associated infrastructure, all shown in shaded boxes in Figure 8-1 and Figure 8-2.

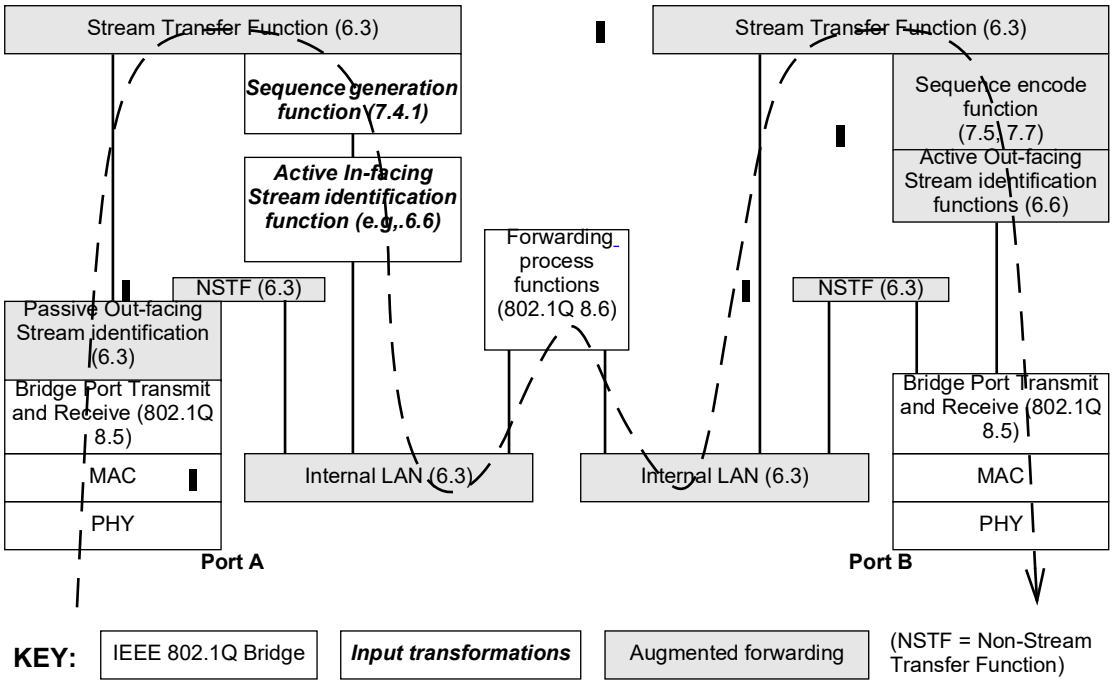


Figure 8-1—Split functions in an FRER C-component

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

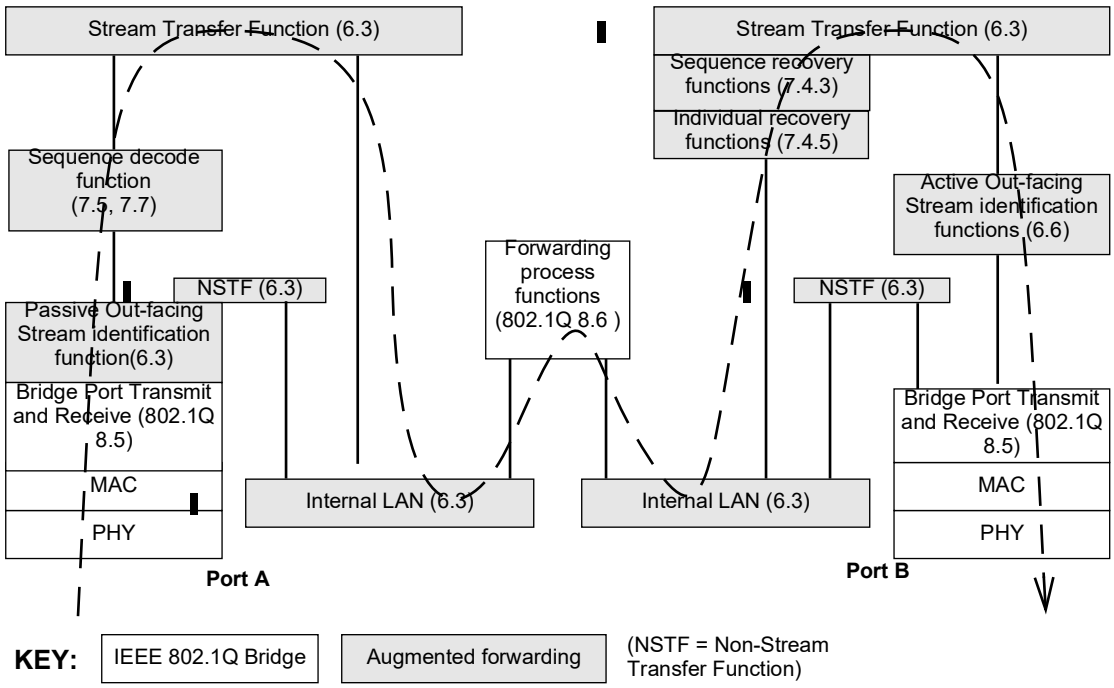


Figure 8-2—Merge functions in an FRER C-component

Item 1, Input transformations, allows the Bridge to:

a) Proxy for FRER-unaware Talkers; and
b) Perform Stream identification or Sequence encapsulation transformations on Streams entering the Bridge.

Item 2, Augmented forwarding, allows the Bridge to:

c) Serve as an intermediate sequence recovery point (e.g., system F in Figure C-4) in a network providing for multiple failures;
d) Proxy for FRER-unaware Listeners; and
e) Perform Stream identification or Sequence encapsulation transformations on Streams exiting the Bridge.

As shown in Figure 8-2, the paradigm is that a given Stream is recognized on input, meaning that the stream_handle and sequence_number subparameters are extracted. The frame is then forwarded normally through the Bridge. On the output port, the stream_handle and sequence_number subparameters found on the input port are used to drive the Individual recovery functions, Sequence recovery functions, Sequence encoding, and Stream identification functions.

Figure 8-1 and Figure 8-2 do not imply that the Augmented forwarding functions have to be placed in expanded ports; they can be integrated with the Bridge forwarding function (8.6 of IEEE 802.1Q) as shown in Figure 8-3. The ordering of these functions with respect to 8.6 of IEEE 802.1Q can be important. In particular:

f) As modeled, the stream_handle subparameter is extracted only once, on ingress. A frame is not re-identified, even if Input transformations modifies the frame.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

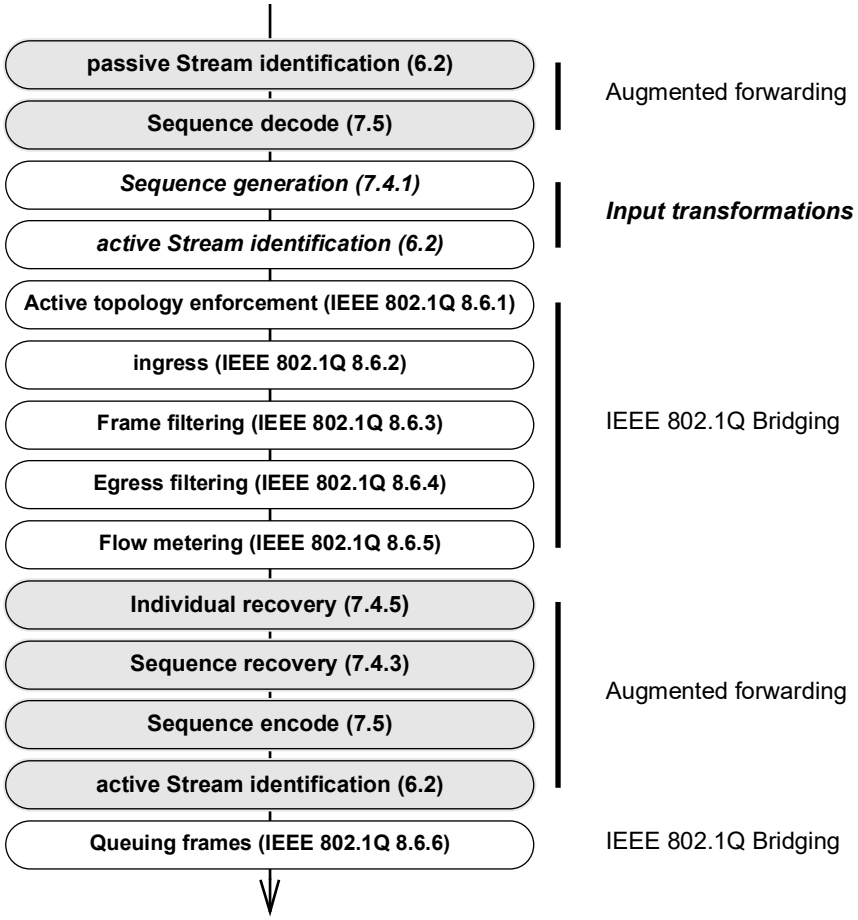


Figure 8-3—Augmented Forwarding Process does sequence recovery

g) As modeled, the sequence_number subparameter is not encoded into the frame during Input transformations; the final output encoding determines the frame format.

NOTE—Whether an implementation encodes the stream_handle and sequence_number subparameters into the frame's mac_service_data_unit parameter and decodes them again after forwarding, or carries them through the forwarding process as separate subparameters, does not matter as long as the externally visible behavior conforms to that of the model.

h) If Input transformations are performed, the frame is forwarded (8.6 of IEEE Std 802.1Q-2022) according to its post-transformation parameters.

i) Flow metering (8.6.5 of IEEE Std 802.1Q-2022) is placed after the passive Stream identification function (6.2) and before the Individual recovery functions (7.4.5). This makes the stream_handle subparameter available for Flow metering, and means that Flow metering can be applied to the individual Streams feeding an instance of the Sequence recovery function. As a result, Flow metering can be applied to frames that will be discarded by the Individual recovery functions or Sequence recovery functions.

j) Output transformations following the Sequence recovery function take place after all forwarding (except Queuing frames, 8.6.6 of IEEE Std 802.1Q-2022). Thus, output transformations can appear to cause violations of the normal forwarding rules, e.g., Egress filtering (8.6.4 of IEEE Std 802.1Q-2022).

With the model of Figure 8-3, no explicit Stream splitting function (7.6) is required. Frames in a single Compound Stream can be replicated using the normal multicast mechanisms, and active Stream identification functions (6.2) on different output ports can cause these to be recognized as different Member

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Streams at the next hop. Assigning a single Sequence encode/decode function and Stream identification function to more than one stream_handle value can cause multiple Member Streams to have the same exact Stream identification method on output, and thus be merged into a single Stream, as seen by the next receiver.

In theory, if using the model of Figure 8-3, there exist instantiations of Individual recovery functions (7.4.5) and Sequence recovery function (7.4.3) for each distinct set of ports for which an identical set of Streams is to be output. The vector of output ports that, in principle, accompanies the packet through the IEEE 802.1Q Forwarding Process, guides the allocation of frames to these functions. After this stage, sets of Sequence encode/decode functions (7.5), and/or Stream identification functions (6.2) can be configured to further transform the packet(s).

8.2 FRER C-component input transformations

The Input transformations, marked with white box and boldface type in Figure 8-1, enable a Bridge to proxy for a non-FRER-capable end system. The expanded input port identifies packets belonging to a Stream, e.g., using IP Stream identification (6.7) or Mask-and-match Stream identification (6.8), serializes the packets with a Sequence generation function (7.4.1), encodes the sequence number with an R-TAG (7.7), and then gives the packets belonging to this Stream a {vlan_identifier, destination_mac_address} pair that is unique, at least inside this Bridge, using Active Destination MAC and VLAN Stream identification (6.6). The IEEE 802.1Q Forwarding Process, enhanced with the Individual recovery function (7.4.5) and Sequence recovery function (7.4.3), then forwards the frame.

8.3 Frame Replication and Elimination for Reliability and VLAN tags

As illustrated in Figure 8-1, Figure 8-2 and Figure 8-4, FRER in an IEEE 802.1Q C-VLAN Component is above the Bridge Port Transmit and Receive function (8.5 of IEEE Std 802.1Q-2022) in the protocol stack. As a consequence of this placement, a frame containing both an IEEE 802.1Q C-VLAN tag and an R-TAG but no other IEEE 802.1Q tags, would be as illustrated in Figure 8-4.

Field	Offset	Length
Destination MAC address	**0**	**6**
Source MAC address	**6**	**6**
C-TAG EtherType	**12**	**2**
Priority, DE, VLAN ID	**14**	**2**
R-TAG EtherType	**16**	**2**
Reserved	**18**	**2**
Sequence number	**20**	**2**
Payload Length/EtherType	**22**	**2**
Data	**24**	***n***
Frame Check Sequence	**24+*n***	**4**

Figure 8-4—Example Ethernet frame format

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

8.4 Configuring Frame Replication and Elimination for Reliability in Bridges

As described in 8.1, the output transformations, including Individual recovery functions (7.4.5), Sequence recovery functions (7.4.2), Sequence encode/decode functions (7.5), and Stream identification functions (6.2), can be considered as being embedded in the IEEE 802.1Q Forwarding Process. However, since different activities can be required by an application on different output ports, the managed objects in Clause 9 and Clause 10 can be used to configure an FRER C-component. Figure 8-1 and Figure 8-2 illustrate an allowed arrangement of FRER functions on an output Bridge port as perceived by the managed objects in Clause 9 and Clause 10.

An FRER C-component shall implement all of the managed objects in Clause 9 and Clause 10 that are needed to meet the conformance requirements of 5.12 and 5.15 with the exceptions noted in Table 8-1.

Table 8-1—Managed objects for FRER in an FRER C-component

Variable	Reference	Limitation	Reason
tsnStreamIdInFacOutputPortList	9.1.1.2	Only Active Destination MAC and VLAN Stream identification (or nothing) can be configured.	Only needed for Input transformations.
tsnStreamIdOutFacOutputPortList	9.1.1.3	Only Active Destination MAC and VLAN Stream identification (or nothing) can be configured.	A Bridge can modify the vlan_identifier and destination_mac_address of packets belonging to a particular Stream
tsnStreamIdInFacInputPortList	9.1.1.4	Not allowed	There is no need to re-identify Streams on their way out of the Bridge.
tsnStreamIdOutFacInputPortList	9.1.1.5	No limitations	—
tsnCpeDmacVlanDownTagged	9.1.4.2	Not required	The decision whether to tag an outgoing frame is determined, in an IEEE 802.1Q Bridge, by other managed objects.
frerSeqGenDirection	10.3.1.2	Must be False (in-facing)	Sequence numbers can be added only to incoming packets.
frerSeqRcvyDirection	10.4.1.3	Must be False (in-facing)	Duplicate packets can be discarded only on outgoing packets.
frerSeqEncDirection	10.5.1.3	Must be True (out-facing)	Packets' sequence_numbers are encoded/decoded as they leave.
Stream split table	10.6	Not required	—

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

9. Stream Identification Management

The description of the managed objects that control Stream identification are described in the following subclauses:

a) The Stream identity table (9.1) assigns packets a stream_handle (6.1);
b) The per-port, per-Stream packet counters that are kept by Stream identification functions for inspection by network management entities are described in 9.2, and the per-port (totaled over all Streams) counters in 9.3.

The general requirements on the behavior of the Stream Identification counters in 9.2 and 9.3 are described in 10.1.

9.1 Stream identity table

The Stream identity table consists of a set of tsnStreamIdEntry objects (9.1.1), each relating to a single Stream, specifying the points in the system where Stream identification functions (6.2) are to be instantiated. Each entry in the Stream identity table has a tsnStreamIdHandle object (9.1.1.1) specifying a stream_handle value and one or more tsnStreamIdEntry objects (9.1.1) describing one identification method for that Stream. If a single Stream has multiple identification methods, perhaps (but not necessarily) on different ports, then there can be multiple tsnStreamIdEntry objects with the same value for the tsnStreamIdHandle. If the HSR or PRP method or the Sequence encode/decode function is applied to a packet, then the LanId or PathId fields are also used to identify the Stream to which the packet belongs.

9.1.1 tsnStreamIdEntry

A set of managed objects, all applying to the Stream specified by tsnStreamIdHandle (9.1.1.1), and all using the same Stream identification types and parameters (9.1.1.6, 9.1.1.7).

~~See 10.2 for additional managed objects that are present in the tsnStreamIdEntry only if Autoconfiguration (7.10) is used.~~

9.1.1.1 tsnStreamIdHandle

The objects in a given entry of the Stream identity table are used to control packets whose stream_handle subparameter is equal to the entry's tsnStreamIdHandle object. The specific values used in the tsnStreamIdHandle object are not necessarily used in the system; they are used only to relate the various management objects in Clause 9 and Clause 10.

NOTE—It is also possible to assign a packet that was identified as belonging to a known Stream (i.e., a specific tsnStreamIdEntry) a stream_handle equal to 0.

9.1.1.2 tsnStreamIdInFacOutputPortList

The list of ports on which an in-facing Stream identification function (6.2) using this identification method (9.1.1.6, 9.1.1.7) is to be placed for this Stream (9.1.1.1) in the output (towards the system forwarding function) direction. At most one tsnStreamIdEntry can list a given port for a given tsnStreamIdHandle in its tsnStreamIdInFacOutputPortList.

9.1.1.3 tsnStreamIdOutFacOutputPortList

The list of ports on which an out-facing Stream identification function (6.2) using this identification method (9.1.1.6, 9.1.1.7) is to be placed for this Stream (9.1.1.1) in the output (towards the physical interface)

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

direction. At most one tsnStreamIdEntry can list a given port for a given tsnStreamIdHandle in its tsnStreamIdOutFacOutputPortList.

9.1.1.4 tsnStreamIdInFacInputPortList

The list of ports on which an in-facing Stream identification function (6.2) using this identification method (9.1.1.6, 9.1.1.7) is to be placed for this Stream (9.1.1.1) in the input (coming from the system forwarding function) direction. Any number of tsnStreamIdEntry objects can list the same port for the same tsnStreamIdHandle in its tsnStreamIdInFacInputPortList.

9.1.1.5 tsnStreamIdOutFacInputPortList

The list of ports on which an out-facing Stream identification function (6.2) using this identification method (9.1.1.6, 9.1.1.7) is to be placed for this Stream (9.1.1.1) in the input (coming from the physical interface) direction. Any number of tsnStreamIdEntry objects can list the same port for the same tsnStreamIdHandle in its tsnStreamIdOutFacInputPortList.

9.1.1.6 tsnStreamIdIdentificationType

An enumerated value indicating the method used to identify packets belonging to the Stream. The enumeration includes an Organizationally Unique Identifier (OUI) or Company ID (CID) to identify the organization defining the enumerated type. The values defined by this standard are shown in Table 9-1.

Table 9-1—Stream identification types

OUI/CID	Type number	Stream identification function	Controlling parameters
00-80-C2	0	Reserved	—
00-80-C2	1	Null Stream identification (6.4)	9.1.2
00-80-C2	2	Source MAC and VLAN Stream identification (6.5)	9.1.3
00-80-C2	3	Active Destination MAC and VLAN Stream identification (6.6)	9.1.4
00-80-C2	4	IP Stream identification (6.7)	9.1.5
00-80-C2	5	Mask-and-match Stream identification (6.8)	9.1.6
00-80-C2	6−255	Reserved	—
other	—	Defined by entity owning the OUI or CID	—

9.1.1.7 tsnStreamIdParameters

The number of controlling parameters for a Stream identification method, their types and values, are specific to the tsnStreamIdIdentificationType (9.1.1.6) and are referenced in Table 9-1.

9.1.2 Managed objects for Null Stream identification

When instantiating an instance of the Null Stream identification function (6.4) for a particular input Stream, the managed objects in the following subclauses serve as the tsnStreamIdParameters managed object (9.1.1.7).

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

9.1.2.1 tsnCpeNullDownDestMac

Specifies the destination_address that identifies a packet in an EISS indication primitive, to the Null Stream identification function.

9.1.2.2 tsnCpeNullDownTagged

An enumerated value indicating whether a packet in an EISS indication primitive to the Null Stream identification function is permitted to have a VLAN tag. It can take the following values:

1) **tagged:** A frame must have a VLAN tag to be recognized as belonging to the Stream.
2) **priority:** A frame must be untagged, or have a VLAN tag with a VLAN ID = 0 to be recognized as belonging to the Stream.
3) **all:** A frame is recognized as belonging to the Stream whether tagged or not.

9.1.2.3 tsnCpeNullDownVlan

Specifies the vlan_identifier parameter that identifies a packet in an EISS indication primitive to the Null Stream identification function. A value of 0 indicates that the vlan_identifier parameter is ignored on EISS indication primitives.

NOTE—When the tsnCpeNullDownTagged object is set to “all” or “tagged”, the tagged frames carrying a VLAN-ID of 0 cannot be matched. The matching of tagged frames carrying a VLAN-ID of 0 is achieved by setting the tsnCpeNullDownTagged object to “priority”, causing the tsnCpeNullDownVlan object to be ignored.

9.1.3 Managed objects for Source MAC and VLAN Stream identification

When instantiating an instance of the Source MAC and VLAN Stream identification function (6.5) for a particular input Stream, the managed objects in the following subclauses serve as the tsnStreamIdParameters managed object (9.1.1.7).

9.1.3.1 tsnCpeSmacVlanDownSrcMac

Specifies the source_address that identifies a packet in an EISS indication primitive, to the Source MAC and VLAN Stream identification function.

9.1.3.2 tsnCpeSmacVlanDownTagged

An enumerated value indicating whether a packet in an EISS indication primitive to the Source MAC and VLAN Stream identification function is permitted to have a VLAN tag. It can take the following values:

1) **tagged:** A frame must have a VLAN tag to be recognized as belonging to the Stream.
2) **priority:** A frame must be untagged, or have a VLAN tag with a VLAN ID = 0 to be recognized as belonging to the Stream.
3) **all:** A frame is recognized as belonging to the Stream whether tagged or not.

9.1.3.3 tsnCpeSmacVlanDownVlan

Specifies the vlan_identifier parameter that identifies a packet in an EISS indication primitive to the Source MAC and VLAN Stream identification function. A value of 0 indicates that the vlan_identifier parameter is ignored on EISS indication primitives.

NOTE—When the tsnCpeSmacVlanDownTagged object is set to “all” or “tagged”, the tagged frames carrying a VLAN-ID of 0 cannot be matched. The matching of tagged frames carrying a VLAN-ID of 0 is achieved by setting the tsnCpeSmacVlanDownTagged object to “priority”, causing the tsnCpeSmacVlanDownVlan object to be ignored.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

9.1.4 Managed objects for Active Destination MAC and VLAN Stream identifications

When instantiating an instance of the Active Destination MAC and VLAN Stream identification function (6.6) for a particular output Stream, the managed objects in the following subclauses, serve as the tsnStreamIdParameters managed object (9.1.1.7).

9.1.4.1 tsnCpeDmacVlanDownDestMac

Specifies the destination_address parameter to use in the EISS request primitive for output packets sent to lower layers by the Active Destination MAC and VLAN Stream identification function, and the destination_address that identifies an input packet in an EISS indication primitive to the Active Destination MAC and VLAN Stream identification function.

9.1.4.2 tsnCpeDmacVlanDownTagged

An enumerated value indicating whether a packet in an EISS indication or request primitive between the Active Destination MAC and VLAN Stream identification function and the lower layers is to have a VLAN tag. It can take the following values:

1) **tagged:** An input frame must have a VLAN tag to be recognized as belonging to the Stream. An output frame receives a VLAN tag.
2) **priority:** An input frame must be untagged, or have a VLAN tag with a VLAN ID = 0 to be recognized as belonging to the Stream. An output frame is marked with a VLAN tag with VLAN ID = 0.
3) **all:** A frame is recognized as belonging to the Stream whether tagged or not. An output frame is to be untagged.

This variable is not used in an FRER C-component. See 8.4.

9.1.4.3 tsnCpeDmacVlanDownVlan

Specifies the vlan_identifier parameter to use in the EISS request primitive for output packets sent to lower layers by the Active Destination MAC and VLAN Stream identification function, and the vlan_identifier that identifies an input packet in an EISS indication primitive to the Active Destination MAC and VLAN Stream identification function. A value of 0 indicates that the vlan_identifier parameter is ignored on EISS indication primitives.

NOTE—When the tsnCpeDmacVlanDownTagged object is set to “all” or “tagged”, the tagged frames carrying a VLAN-ID of 0 cannot be matched. The matching of tagged frames carrying a VLAN-ID of 0 is achieved by setting the tsnCpeDmacVlanDownTagged object to “priority”, causing the tsnCpeDmacVlanDownVlan object to be ignored.

9.1.4.4 tsnCpeDmacVlanDownPriority

Specifies the priority parameter to use in the EISS request primitive for output packets sent to lower layers by the Active Destination MAC and VLAN Stream identification function for all packets in a particular Stream.

9.1.4.5 tsnCpeDmacVlanUpDestMac

Specifies the destination_address parameter to use in the EISS indication primitive for input packets offered to upper layers by the Active Destination MAC and VLAN Stream identification layer. This address replaces the address that was used to identify the packet (tsnCpeDmacVlanDownDestMac, 9.1.4.1).

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

9.1.4.6 tsnCpeDmacVlanUpTagged

An enumerated value indicating whether a packet in an EISS indication or request primitive between the Active Destination MAC and VLAN Stream identification function and the upper layers is to have a VLAN tag. It can take the following values:

1) **tagged:** An output frame must have a VLAN tag to be recognized as belonging to the Stream. An input frame receives a VLAN tag.
2) **priority:** An output frame must be untagged, or have a VLAN tag with a VLAN ID = 0 to be recognized as belonging to the Stream. An input frame is marked with a VLAN tag with VLAN ID = 0.
3) **all:** A frame is recognized as belonging to the Stream whether tagged or not. An input frame is to be untagged.

This variable is used only by an end system and not by a relay system. See 8.4.

9.1.4.7 tsnCpeDmacVlanUpVlan

Specifies the vlan_identifier parameter to use in the EISS indication primitive for packets offered to upper layers, or the VLAN ID field for an IEEE 802.1Q tag in an ISS mac_service_data_unit. This address replaces the VLAN ID that was used to identify the packet (tsnCpeDmacVlanDownVlan, 9.1.4.3).

9.1.4.8 tsnCpeDmacVlanUpPriority

Specifies the priority parameter to use in the EISS indication primitive for packets offered to upper layers.

9.1.5 Managed objects for IP Stream identification

When instantiating an instance of the IP Stream identification function (6.7), the parameters in the following subclauses replace the tsnStreamIdParameters managed object (9.1.1.7).

9.1.5.1 tsnCpeIpIdDestMac

Specifies the destination_address parameter that identifies a packet in an EISS indication primitive. A value of all 0 indicates that the MAC destination address is to be ignored on packets received from lower layers.

9.1.5.2 tsnCpeIpIdTagged

An enumerated value indicating whether a packet in an EISS indication or request primitive to the IP Stream identification function is to have a VLAN tag. It can take the following values:

1) **tagged:** An input frame must have a VLAN tag to be recognized as belonging to the Stream.
2) **priority:** An input frame must be untagged, or have a VLAN tag with a VLAN ID = 0 to be recognized as belonging to the Stream.
3) **all:** A frame is recognized as belonging to the Stream whether tagged or not.

9.1.5.3 tsnCpeIpIdVlan

Specifies the vlan_identifier parameter that identifies a packet in an EISS indication primitive.A value of 0 indicates that the vlan_identifier parameter is ignored on EISS indication primitives.

NOTE—When the tsnCpeIpIdTagged object is set to “all” or “tagged”, the tagged frames carrying a VLAN-ID of 0 cannot be matched. The matching of tagged frames carrying a VLAN-ID of 0 is achieved by setting the tsnCpeIpIdTagged object to “priority”, causing the tsnCpeIpIdVlan object to be ignored.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

9.1.5.4 tsnCpeIpIdIpSource

Specifies the IPv4 (RFC 791) or IPv6 (RFC 2460) source address parameter that must be matched to identify packets coming up from lower layers. A value of all 0 indicates that the IP source address is to be ignored on packets received from lower layers.

9.1.5.5 tsnCpeIpIdIpDestination

Specifies the IPv4 (RFC 791) or IPv6 (RFC 2460) destination address parameter that must be matched to identify packets coming up from lower layers.

9.1.5.6 tsnCpeIpIdDscp

Specifies the IPv4 (RFC 791) or IPv6 (RFC 2460) differentiated services codepoint (DSCP, RFC 2474) that must be matched to identify packets coming up from the lower layers. A value of 64 decimal indicates that the DSCP is to be ignored on packets received from lower layers.

9.1.5.7 tsnCpeIpIdNextProtocol

Specifies the IP next protocol parameter that must be matched to identify packets coming up from lower layers. The value of this parameter must specify either none, UDP (RFC 768), TCP (RFC 793), or SCTP (RFC 4960). If “none,” then the tsnCpeIpIdSourcePort (9.1.5.8) and tsnCpeIpIdDestinationPort (9.1.5.9) managed objects are not used.

9.1.5.8 tsnCpeIpIdSourcePort

Specifies the TCP or UDP Source Port parameter that must be matched to identify packets coming up from lower layers. A value of 0 indicates that the Source Port number of the packet is to be ignored on packets received from lower layers.

9.1.5.9 tsnCpeIpIdDestinationPort

Specifies the TCP or UDP Destination Port parameter that must be matched to identify packets coming up from lower layers. A value of 0 indicates that the Destination Port number of the packet is to be ignored on packets received from lower layers.

9.1.6 Managed objects for Mask-and-match Stream identification

When instantiating an instance of the Mask-and-match Stream identification function (6.8) for a particular input Stream, the managed objects in the following subclauses serve as the tsnStreamIdParameters managed object (9.1.1.7).

9.1.6.1 tsnCpeMmIdDestMacMask

Specifies a 48-bit mask. A bitwise AND operation is performed between tsnCpeMmIdDestMacMask and the destination_address parameter passed by the ISS indication primitive to the Mask-and-match Stream identification function. The resulting 48-bit information is the masked destination_address that is used as input for the instance of the Mask-and-match Stream identification function. If tsnCpeMmIdDestMacMask has a value of 0, the destination_address parameter is ignored.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

9.1.6.2 tsnCpeMmIdDestMacMatch

Specifies the 48-bit value of the masked destination_address, as defined in 9.1.6.1, to be matched by the instance of the Mask-and-match Stream identification function.

9.1.6.3 tsnCpeMmIdSrcMacMask

Specifies a 48-bit mask. A bitwise AND operation is performed between tsnCpeMmIdSrcMacMask and the source_address parameter passed by the ISS indication primitive to the Mask-and-match Stream identification function. The resulting 48-bit information is the masked source_address that is used as input for the instance of the Mask-and-match Stream identification function. If tsnCpeMmIdSrcMacMask has a value of 0, the source_address parameter is ignored.

9.1.6.4 tsnCpeMmIdSrcMacMatch

Specifies the 48-bit value of the masked source_address, as defined in 9.1.6.3, to be matched by the instance of the Mask-and-match Stream identification function.

9.1.6.5 tsnCpeMmIdMsduMaskLength

Is an integer value that specifies the length, in octets, of tsnCpeMmIdMsduMask (9.1.6.6) and tsnCpeMmIdMsduMatch (9.1.6.7). The behavior of the Mask-and-match Stream identification function is undefined on any port where the value of tsnCpeMmIdMsduMaskLength exceeds the per-port mac_service_data_unit mask maximum length (9.4.1). tsnCpeMmIdMsduMaskLength has a minimum value of 2. If the length of the frame's mac_service_data_unit is shorter than tsnCpeMmIdMsduMaskLength, then the identification fails.

9.1.6.6 tsnCpeMmIdMsduMask

Specifies a mask of tsnCpeMmIdMsduMaskLength octets. A bitwise AND operation is performed between tsnCpeMmIdMsduMask and the tsnCpeMmIdMsduMaskLength (9.1.6.5) first octets of the mac_service_data_unit parameter passed by the ISS indication primitive to the Mask-and-match Stream identification function. The resulting tsnCpeMmIdMsduMaskLength octets of information are the masked mac_service_data_unit that is used as input for the instance of the Mask-and-match Stream identification function. If tsnCpeMmIdMsduMask has a value of 0, the mac_service_data_unit parameter is ignored.

9.1.6.7 tsnCpeMmIdMsdu Match

Specifies the value of the masked mac_service_data_unit, as defined in 9.1.6.5 and 9.1.6.6, to be matched by the instance of the Mask-and-match Stream identification function.

NOTE—The Mask-and-match Stream identification function takes as input the ISS indication primitive parameters, which include the mac_service_data_unit parameter. If specific part(s) of the mac_service_data_unit are required for the identification of a Stream, the ad-hoc tsnCpeMmIdMsduMask and tsnCpeMmIdMsduMatch have to be defined. For instance, when a VLAN ID is required for Stream identification, it is to be masked within the mac_service_data_unit by setting 16 bits to 1 at the offset where the tag is expected (to isolate the EtherType), followed by 4 bits set to 0 (to ignore the PCP and DEI fields), and ending with 12 bits set to 1 (to mask the full VID) in tsnCpeMmIdMasduMask. In a Customer Bridged Network, the C-VLAN Tag is expected at an offset of 0, and the tsnCpeMmIdMsduMatch value used in the Mask-and-match Stream identification function instance contains, at the same offset, a pattern matching the C-VLAN EtherType (81-00), followed by 4 don’t-care bits (set to 0), followed by 12 bits representing the VID value to be matched.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

9.2 Operational per-port per-Stream Stream identification counters

The counters in this subclause are instantiated for each port on which the Stream identification function (6.2) is configured. They are indexed by port number, facing (in-facing or out-facing), and stream_handle value (tsnStreamIdHandle, 9.1.1.1). Encoded as unsigned integers, they shall be 64 bits in length to insure against excessively short wrap times when used on links on which transmission rate exceeds 650 000 000 bits per second.

9.2.1 tsnCpsSidInputPackets

The tsnCpsSidInputPackets counter is incremented once for each packet identified by the Stream identification function (6.2).

9.2.2 tsnCpsSidOutputPackets

The tsnCpsSidOutputPackets counter is incremented once for each packet passed down the stack by the Stream identification function (6.2).

9.3 Operational per-port Stream identification counters

The counters in this subclause are instantiated for each port on which the Stream identification function (6.2) is configured. They are indexed by port number. Encoded as unsigned integers, they shall be 64 bits in length to insure against excessively short wrap times when used on links on which transmission rate exceeds 650 000 000 bits per second.

9.3.1 tsnCpSidInputPackets

The tsnCpSidInputPackets counter is incremented once for each packet identified by any Stream identification function (6.2) on this port. Its value equals the sum (modulo the size of the counters) of all of the tsnCpsSidInputPackets (9.2.1) counters on this same port.

9.3.2 tsnCpSidOutputPackets

The tsnCpSidOutputPackets counter is incremented once for each packet passed down the stack by any Stream identification function (6.2) on this port. Its value equals the sum (modulo the size of the counters) of all of the tsnCpsSidOutputPackets (9.2.2) counters on this same port.

9.4 Per-port managed object

9.4.1 tsnPpMmIdMsduMaskMaxLength

tsnPpMmIdMsduMaskMaxLength is a read-only variable defined for each port the Mask-and-match Stream identification function is to be run on. It defines the maximum length a mac_service_data_unit mask can have. It is an integer value, in octets, with a minimum value of 2 and a maximum value of 1984 (maximum ISS mac_service_data_unit length for an IEEE 802.3 frame).

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

10. Frame Replication and Elimination for Reliability management

The managed objects that control Stream identification are described in Clause 9. The managed objects that control FRER are described in this Clause 10 as follows:

a) General requirements on the behavior of counters are in 10.1.
b) The various tables of managed objects that can manage, in detail, each individual Stream, are described in five subclauses, including:
   1) ~~Additions (10.2) to the Stream identity table (9.1) required for Autoconfiguration (7.10, 10.7).~~
   2) The Sequence generation table (10.3) that configures instances of the Sequence generation function (7.4.1);
   3) The Sequence recovery table (10.4) that configures instances of the Individual recovery function (7.4.5), the Sequence recovery function (7.4.3), and the Latent error detection function (7.4.4);
   4) The Sequence identification table (10.5) that configures instances of the Sequence encode/decode function (7.5); and
   5) The Stream split table (10.6) that configures instances of the Stream splitting function (7.6).
c) ~~The managed objects that support the automatic configuration, upon receipt of a packet, of entries in the first four of the preceding tables (10.2 through 10.5), are described in the subclause on Autoconfiguration (10.7).~~
d) The per-port, per-Stream packet counters that are kept by FRER functions for inspection by network management entities are described in 10.8, and the per-port (totaled over all Streams) counters in 10.9.

The managed objects in the subclauses under 9.1 make it possible to configure more than one encapsulation for the same stream_handle subparameter on the same port. Similarly, the managed objects in the subclauses under 10.3 and 10.4 make it possible to configure more than one Sequence encode/decode function (7.5) or more than one Sequence generation function (7.4.1) for the same stream_handle subparameter. [The same value of stream_handle can be in the frerSeqGenStreamList (10.3.1.1) of more than one frerSeqGenEntry (10.3.1) or in the frerSeqRcvyStreamList (10.4.1.1) of more than one frerSeqRcvyEntry (10.4.1).] A system shall return an error if an attempt is made to configure conflicting requirements upon that system.

10.1 Counter behavior

All counters defined by this standard (e.g., frerCpsSeqEncErroredPackets, 10.8.11) when incremented past the maximum value representable shall roll over to 0 and continue. There is no provision for resetting any counter to any specific value, e.g., 0. A counter can be set to any value when the system is reset. This standard assumes the following:

a) Any management entity will read any counters often enough that at most one rollover can occur between examinations;
b) There exists a “time of last counter reset” or “time since last counter reset” or an equivalent managed object in the system, that the management entity can examine to determine whether the counters have been reset since the last time they were examined.
c) A counter managed object is large enough to take at least one minute to roll over from 0 to 0.

10.2 ~~Additional tsnStreamIdEntry managed objects~~

~~Two managed objects augment each tsnStreamIdEntry (9.1.1) in the Stream identity table (9.1) when Managed objects for autoconfiguration (7.10, 10.7) is implemented.~~

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

10.2.1 ~~tsnStreamIdAutoconfigured~~

~~A read-only Boolean value, supplied by the system, specifying whether this entry was created explicitly (False) or via the Sequence autoconfiguration table (10.7.1, True).~~

10.2.2 ~~tsnStreamIdLanPathId~~

~~An integer specifying a path or LAN. If and only if a packet matches an entry in the Sequence identification table (10.5) that specifies HSR or PRP in its frerSeqEncEncapsType (10.5.1.5) object, tsnStreamIdLanPathId specifies the LanId or PathId value that must be matched for this tsnStreamIdEntry to apply. A value of –1 indicates that the LanId or PathId are to be ignored.~~

10.3 Sequence generation table

There is one Sequence generation table in a system, and one entry in the Sequence generation table for each Sequence generation function (7.4.1).

10.3.1 frerSeqGenEntry

Each frerSeqGenEntry lists the Streams (10.3.1.1) and direction (10.3.1.2) for which a single instance of the Sequence generation function (7.4.1) is to be placed.

10.3.1.1 frerSeqGenStreamList

A list of stream_handle values, corresponding to the values of the tsnStreamIdHandle objects (9.1.1.1) in the Stream identity table (9.1), on which this instance of the Sequence generation function (7.4.1) is to operate. The single instance of the Sequence generation function created by this frerSeqGenEntry operates every packet belonging to this Stream, regardless of the port on which it is received.

10.3.1.2 frerSeqGenDirection

A Boolean object indicating whether the Sequence generation function (7.4.1) is to be placed on the out-facing (True) or in-facing (False) side of the port (Figure 6-6).

10.3.1.3 frerSeqGenReset

A Boolean object indicating that the Sequence generation function (7.4.1) is to be reset by calling its corresponding SequenceGenerationReset function (7.4.1.3). Writing the value True to frerSeqGenReset triggers a reset; writing the value False has no effect. When read, frerSeqGenReset always returns the value False.

10.4 Sequence recovery table

There is one Sequence recovery table in a system, and one entry in the Sequence recovery table for each Sequence recovery function (7.4.3) or Individual recovery function (7.4.5) that can also be present. The entry describes a set of managed objects for the single instance of a Base recovery function (7.4.2) and Latent error detection function (7.4.4) included in the Sequence recovery function or Individual recovery function.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

10.4.1 frerSeqRcvyEntry

Each frerSeqRcvyEntry lists the Streams (10.4.1.1), ports (10.4.1.2), and direction (10.4.1.3) for which instances of a Sequence recovery function (7.4.3) or Individual recovery function (7.4.5) are to be instantiated.

10.4.1.1 frerSeqRcvyStreamList

A list of the stream_handle values, corresponding to the values of the tsnStreamIdHandle objects (9.1.1.1) in the Stream identity table (9.1), to which the system is to apply the instance of the Sequence recovery function (7.4.3) or Individual recovery function (7.4.5).

10.4.1.2 frerSeqRcvyPortList

The list of ports on each of which the system is to instantiate the Sequence recovery function (7.4.3), or from which received packets are to be fed to a single instance of the Individual recovery function (7.4.5).

10.4.1.3 frerSeqRcvyDirection

A Boolean object indicating whether the Sequence recovery function (7.4.2) or Individual recovery function (7.4.5) is to be placed on the out-facing (True) or in-facing (False) side of the port (Figure 6-6).

10.4.1.4 frerSeqRcvyReset

A Boolean object indicating that the Sequence recovery function (7.4.3) or Individual recovery function (7.4.5) is to be reset by calling its corresponding SequenceRecoveryReset function (7.4.3.3). Writing the value True to frerSeqRcvyReset triggers a reset; writing the value False has no effect. When read, frerSeqRcvyReset always returns the value False.

10.4.1.5 frerSeqRcvyAlgorithm

This object is an enumerated value specifying which sequence recovery algorithm is to be used for this instance of the Sequence recovery function (7.4.3). The enumeration uses an OUI or CID as shown in Table 10-1. The default value for frerSeqRcvyAlgorithm is **Vector_Alg** (00-80-C2, 0).

Table 10-1—Enumerated values for frerSeqRcvyAlgorithm

Enumeration	OUI/CID	Type number	Sequence recovery function algorithm
Vector_Alg	00-80-C2	0	VectorRecoveryAlgorithm (7.4.3.4)
Match_Alg	00-80-C2	1	MatchRecoveryAlgorithm (7.4.3.5)
—	00-80-C2	2–255	Reserved
—	Other	—	Defined by entity owning the OUI or CID

10.4.1.6 frerSeqRcvyHistoryLength

An integer specifying how many bits of the SequenceHistory variable (7.4.3.2.2) are to be used. The minimum and the default value is 2, maximum is the maximum allowed by the implementation. [Not used if frerSeqRcvyAlgorithm (10.4.1.5) = **Match_Alg** (00-80-C2, 1).]

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

10.4.1.7 frerSeqRcvyResetMSec

An unsigned integer specifying the timeout period in milliseconds for the RECOVERY_TIMEOUT event (item c in 7.4.3.1).

10.4.1.8 frerSeqRcvyInvalidSequenceValue

A read-only unsigned integer value that cannot be encoded in a packet as a value for the sequence_number subparameter (item b in 6.1), i.e., frerSeqRcvyInvalidSequenceValue is larger than or equal to RecovSeqSpace (7.4.3.2.1).

10.4.1.9 frerSeqRcvyTakeNoSequence

A Boolean value specifying whether packets with no sequence_number subparameter are to be accepted (True) or not (False). Default value False. See item i in 7.1.1.

10.4.1.10 frerSeqRcvyIndividualRecovery

A Boolean value specifying whether this entry describes a Sequence recovery function (7.4.3) or Individual recovery function (7.4.5).

a) **True:** The entry describes an Individual recovery function (7.4.5). Packets discarded by the VectorRecoveryAlgorithm (7.4.3.4) or the MatchRecoveryAlgorithm (7.4.3.5) will cause the variable RemainingTicks (7.4.3.2.4) to be reset. There is no Latent error detection function (7.4.4) associated with this entry, so frerSeqRcvyLatentErrorDetection (10.4.1.11) cannot also be True.

b) **False:** The entry describes a Sequence recovery function (7.4.3). Packets discarded by the VectorRecoveryAlgorithm (7.4.3.4) or the MatchRecoveryAlgorithm (7.4.3.5) will not cause the variable RemainingTicks (7.4.3.2.4) to be reset.

10.4.1.11 frerSeqRcvyLatentErrorDetection

A Boolean value indicating whether an instance of the Latent error detection function (7.4.4) is to be instantiated along with the Base recovery function (7.4.2) in this Sequence recovery function (7.4.3) or Individual recovery function (7.4.5). frerSeqRcvyLatentErrorDetection cannot be set True if frerSeqRcvyIndividualRecovery (10.4.1.10) is also True; an Individual recovery function does not include a Latent error detection function.

10.4.1.12 Latent error detection managed objects

The objects in the following subclauses are present if and only if frerSeqRcvyIndividualRecovery (10.4.1.10) is False.

10.4.1.12.1 frerSeqRcvyLatentErrorDifference

An integer specifying the maximum difference between frerCpsSeqRcvyDiscardedPackets (10.8.6), and the product of frerCpsSeqRcvyPassedPackets (10.8.5) and (frerSeqRcvyLatentErrorPaths – 1) (10.4.1.12.3) that is allowed. Any larger difference will trigger the detection of a latent error by the LatentErrorTest function (7.4.4.4).

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

10.4.1.12.2 frerSeqRcvyLatentErrorPeriod

The integer number of milliseconds that are to elapse between instances of running the LatentErrorTest function (7.4.4.4). An implementation can have a minimum value for frerSeqRcvyLatentErrorPeriod, below which it cannot be set, but this minimum shall be no larger than 1000 ms (1 s). Default value 2000 (2 s).

10.4.1.12.3 frerSeqRcvyLatentErrorPaths

The integer number of paths over which FRER is operating for this instance of the Base recovery function (7.4.2) and Latent error detection function (7.4.4).

10.4.1.12.4 frerSeqRcvyLatentResetPeriod

The integer number of milliseconds that are to elapse between instances of running the LatentErrorReset function (7.4.4.3). An implementation can have a minimum value for LatentErrorReset, below which it cannot be set, but this minimum shall be no larger than 1000 ms (1 s). Default value 30000 (30 s).

10.5 Sequence identification table

There is one Sequence identification table per system, and one entry in the Sequence identification table for each port and direction for which an instance of the Sequence encode/decode function (7.5) is to be created.

10.5.1 frerSeqEncEntry

Each entry in the Sequence identification table specifies a port (10.5.1.2) and direction (10.5.1.3) on which an instance of the Sequence encode/decode function is to be instantiated for a list of Streams (10.5.1.1).

10.5.1.1 frerSeqEncStreamList

A list of stream_handles, corresponding to the values of the tsnStreamIdHandle objects (9.1.1.1) in the Stream identity table (9.1), for which the system is to use the same encapsulation (10.5.1.5) for the Sequence encode/decode function.

10.5.1.2 frerSeqEncPort

The port on which the system is to place an instance of the Sequence encode/decode function (7.5).

10.5.1.3 frerSeqEncDirection

A Boolean object indicating whether the Sequence encode/decode function (7.5) is to be placed on the out-facing (True) or in-facing (False) side of the port (Figure 6-6).

10.5.1.4 frerSeqEncActive

A Boolean value specifying whether this frerSeqEncEntry is passive (False), and therefore is used only to decode (extract information from) input packets passing up the protocol stack, or active (True), and therefore is used both for recognizing input packets and for encoding output packets being passed down the protocol stack.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

10.5.1.5 frerSeqEncEncapsType

An enumerated value indicating the type of encapsulation used for this instance of the Sequence encode/decode function (7.5). The type includes an OUI or CID. The values defined by this standard are shown in Table 10-2.

Table 10-2—Sequence Encode/Decode types

OUI/CID	Type number	Sequence encode/decode method
00-80-C2	0	Reserved
00-80-C2	1	R-TAG (7.7)
00-80-C2	2	HSR sequence tag (7.8)
00-80-C2	3	PRP sequence trailer (7.9)
00-80-C2	4–255	Reserved
Other	—	Defined by entity owning the OUI or CID

10.5.1.6 frerSeqEncPathIdLanId

A 4-bit integer value to be placed in the PathId field of an HSR sequence tag (7.8) or the LanId field of a PRP sequence trailer (7.9) added to an output packet. This managed object is used only if:

a) The HSR sequence tag or the PRP sequence trailer is selected by the frerSeqEncEncapsType object (10.5.1.5); and
b) frerSeqEncActive (10.5.1.4) is False (passive)

10.6 Stream split table

There is one Stream split table per system, with one frerSplitEntry (10.6.1) per Stream splitting function (7.6) per set of stream_handle values.

10.6.1 frerSplitEntry

Each entry in the Stream split table specifies a port (10.6.1.1) and direction (10.6.1.2) on which an instance of the Stream splitting function (7.6) is to be instantiated, and the list of stream_handles specifying its operation.

10.6.1.1 frerSplitPort

The port on which the system is to place an instance of the Stream splitting function (7.6) performing the stream_handle translations specified by frerSplitInputIdList and frerSplitOutputIdList (10.6.1.3, 10.6.1.4) is to be placed.

10.6.1.2 frerSplitDirection

A Boolean object indicating whether the instance of the Stream splitting function (7.6) performing the stream_handle translations specified by frerSplitInputIdList and frerSplitOutputIdList (10.6.1.3, 10.6.1.4) is to be placed on the out-facing (True) or in-facing (False) side of the port (Figure 6-6).

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

10.6.1.3 frerSplitInputIdList

A list of stream_handles (tsnStreamIdHandle values, 9.1.1.1) that are to be split.

10.6.1.4 frerSplitOutputIdList

A list of stream_handles (tsnStreamIdHandle values, 9.1.1.1) into which the input packet is to be split, one copy per item in the frerSplitOutputIdList.

10.7 ~~Managed objects for autoconfiguration~~

10.7.1 ~~Sequence autoconfiguration table~~

~~There is one Sequence autoconfiguration table per system. It contains any number of table entries (10.7.1.1). No two (or more) entries in the Sequence autoconfiguration table can have the same values for frerAutSeqSeqEncaps (10.7.1.1.1), frerAutSeqTagged (10.7.1.1.3), and frerAutSeqVlan (10.7.1.1.4) on any given port (10.7.1.1.2).~~

10.7.1.1 ~~frerAutSeqEntry~~

~~Each frerAutSeqEntry objects (10.7.1.1) relates to a single class of Streams, and specifies how entries are created (and destroyed) in the Stream identity table (9.1), the Sequence recovery table (10.4), and the Sequence identification table (10.5).~~

10.7.1.1.1 ~~frerAutSeqSeqEncaps~~

~~An enumerated value from Table 10-2, specifying which Sequence encode/decode function, and therefore, which type sequence_number encoding, is to be recognized for the purposes of Autoconfiguration.~~

10.7.1.1.2 ~~frerAutSeqReceivePortList~~

~~The list of ports to which this frerAutSeqEntry applies, and on which Stream identification functions (6.2), Sequence encode/decode functions (7.5), and Individual recovery functions (7.4.5) are to be autocreated.~~

10.7.1.1.3 ~~frerAutSeqTagged~~

~~An enumerated value indicating whether packets to be matched by this frerAutSeqEntry are permitted to have a VLAN tag. It can take the following values:~~

1) ~~**tagged**: A frame must have a VLAN tag to be matched.~~
2) ~~**priority**: A frame must be untagged, or have a VLAN tag with a VLAN ID = 0 to be matched.~~
3) ~~**all**: A frame is matched whether tagged or not.~~

10.7.1.1.4 ~~frerAutSeqVlan~~

~~A list of vlan_identifiers for the packet to match. A null list matches all vlan_identifiers.~~

10.7.1.1.5 ~~frerAutSeqRecoveryPortList~~

~~The list of ports on which Base recovery functions (7.4.2) are to be autocreated by this frerAutSeqEntry.~~

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

10.7.1.1.6 ~~frerAutSeqDestructMSec~~

~~An integer number of milliseconds after which an idle set of functions created by this frerAutSeqEntry can be destroyed. A value of 0 indicates that idle autoconfigured functions are not to be destroyed. Default value is 86 400 000 decimal (one day).~~

10.7.1.1.7 ~~frerAutSeqResetMSec~~

~~The value used to fill frerSeqRcvyResetMSec (10.4.1.7) when autoconfiguring entries in the Sequence recovery table.~~

10.7.1.1.8 ~~frerAutSeqAlgorithm~~

~~The value used to fill frerSeqRcvyAlgorithm (10.4.1.5) when autoconfiguring entries in the Sequence recovery table.~~

10.7.1.1.9 ~~frerAutSeqHistoryLength~~

~~The value used to fill frerSeqRcvyHistoryLength (10.4.1.6) when autoconfiguring entries in the Sequence recovery table.~~

10.7.1.1.10 ~~frerAutSeqCreateIndividual~~

~~A Boolean value. If True, the receipt of a packet that triggers the autoconfiguration of a new tsnStreamIdEntry also triggers the instantiation of a frerSeqRcvyEntry for an Individual recovery function.~~

10.7.1.1.11 ~~frerAutSeqCreateRecovery~~

~~A Boolean value. If True, the receipt of a packet that triggers the autoconfiguration of a new tsnStreamIdEntry can also trigger the instantiation of a frerSeqRcvyEntry for a Sequence recovery function.~~

10.7.1.1.12 ~~frerAutSeqLatErrDetection~~

~~A Boolean value. If True, the autoconfiguration of a new Sequence recovery function also creates an associated Latent Error Detection function.~~

10.7.1.1.13 ~~frerAutSeqLatErrDifference~~

~~The value used to fill frerSeqRcvyLatentErrorDifference (10.4.1.12.1) when autoconfiguring entries in the Sequence recovery table.~~

10.7.1.1.14 ~~frerAutSeqLatErrPeriod~~

~~The value used to fill frerSeqRcvyLatentErrorPeriod (10.4.1.12.2) when autoconfiguring entries in the Sequence recovery table.~~

10.7.1.1.15 ~~frerAutSeqLatErrResetPeriod~~

~~The value used to fill frerSeqRcvyLatentResetPeriod (10.4.1.12.4) when autoconfiguring entries in the Sequence recovery table.~~

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

10.7.2 ~~Output autoconfiguration table~~

~~There is one Output autoconfiguration table per system. It contains any number of frerAutOutEntry objects (10.7.2.1), each relating to a single class of Streams specifying how active entries are created in the Sequence identification table (10.5).~~

10.7.2.1 ~~frerAutOutEntry~~

~~No two (or more) entries in the Output autoconfiguration table can include the same port in their frerAutSeqReceivePortList objects (10.7.1.1.2).~~

10.7.2.1.1 ~~frerAutOutPortList~~

~~The list of ports to which this frerAutOutEntry applies, and on which active Sequence encode/decode functions (7.5) are to be autocreated.~~

10.7.2.1.2 ~~frerAutOutEncaps~~

~~An enumerated value from Table 10-2, specifying which Sequence encode/decode function, and therefore, which type sequence_number encoding, is to be used for autoconfigured Streams on the ports in frerAutSeqReceivePortList (10.7.1.1.2).~~

10.7.2.1.3 ~~frerAutOutLanPathId~~

~~An integer specifying a path or LAN. If and only if frerAutOutEncaps (10.7.2.1.2) specifies HSR or PRP frerAutOutLanPathId specifies the LanId or PathId value to be inserted into the HSR sequence tag or PRP sequence trailer of autoconfigured packets transmitted on the ports in frerAutSeqReceivePortList (10.7.1.1.2).~~

10.8 Operational per-port and per-Stream FRER counters

The following counters are instantiated for each port on which any of the Stream identification function (6.2), Sequencing function (7.4), or Sequence encode/decode function (7.5) is configured. The counters are indexed by port number, facing (in-facing or out-facing), and stream_handle value (tsnStreamIdHandle, 9.1.1.1). All counters are unsigned integers. If used on links faster than 650 000 000 bits per second, they shall be 64 bits in length to insure against excessively short wrap times.

A Stream identification component (5.3) shall implement the first two counters tsnCpsSidInputPackets (9.2.1) and tsnCpsSidOutputPackets (9.2.2); the remainder of the counters in 10.8 are optional for such a system.

10.8.1 Per-Stream vs. per-Stream-per-port counters

The preceding managed objects create an instantiation of the Sequence recovery function (7.4.3) per port. In fact, a relay system can choose to create a single instance of the Sequence recovery function as part of its forwarding function, or one instance per line card, or in some other, distributed fashion, without violating the externally visible behaviors specified by this standard. The per-instantiation variables defined in Clause 7 are internal to the system, so their definitions are not affected by this choice.

However, when the algorithms of Clause 7 [e.g., the MatchRecoveryAlgorithm (7.4.3.5)] reference counters [e.g., frerCpsSeqRcvyPassedPackets (10.8.5)] defined as follows, this choice becomes important. Whenever an algorithm in Clause 7 increments a counter in the following subclauses, it increments each of the relevant counters, according to the ports on which the packet is (or would have been) output, no matter how many

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

instantiations of the Sequence recovery function actually exist. Thus, if a duplicate packet that would have been output on ports 1, 6, and 27 is discarded by a single instantiation of the Sequence recovery function residing in a relay system's forwarding function, three instances of the frerCpsSeqRcvyDiscardedPackets (10.8.6) counter are incremented, one for each of those ports.

10.8.2 frerCpsSeqGenResets

The frerCpsSeqGenResets counter is incremented each time the SequenceGenerationReset function (7.4.1.3) is called.

10.8.3 frerCpsSeqRcvyOutOfOrderPackets

The frerCpsSeqRcvyOutOfOrderPackets counter is incremented once for each packet accepted out-of-order by the VectorRecoveryAlgorithm (7.4.3.4) or MatchRecoveryAlgorithm (7.4.3.5). Out-of-order means that the packet's sequence number is not one more than the previous packet received. (See item m in 7.1.1.)

10.8.4 frerCpsSeqRcvyRoguePackets

The frerCpsSeqRcvyRoguePackets counter is incremented once for each packet discarded by the VectorRecoveryAlgorithm (7.4.3.4) because its sequence_number subparameter is more than frerSeqRcvyHistoryLength (10.4.1.6) from RecovSeqNum (7.4.3.2.3).

10.8.5 frerCpsSeqRcvyPassedPackets

The frerCpsSeqRcvyPassedPackets counter is incremented once for each packet passed up the stack by the VectorRecoveryAlgorithm (7.4.3.4) or MatchRecoveryAlgorithm (7.4.3.5).

10.8.6 frerCpsSeqRcvyDiscardedPackets

The frerCpsSeqRcvyDiscardedPackets counter is incremented once for each packet discarded due to a duplicate sequence number by the VectorRecoveryAlgorithm (7.4.3.4) or MatchRecoveryAlgorithm (7.4.3.5).

10.8.7 frerCpsSeqRcvyLostPackets

The frerCpsSeqRcvyLostPackets counter is incremented once for each packet lost by the VectorRecoveryAlgorithm (7.4.3.4). A packet is counted as lost if its sequence number is not received on any ingress port.

NOTE—If per-source sequence numbering is used, frerCpsSeqRcvyLostPackets can count, as lost, packets that were sent to another destination, but not lost. See B.2.

10.8.8 frerCpsSeqRcvyTaglessPackets

The frerCpsSeqRcvyTaglessPackets counter is incremented once for each packet received by the VectorRecoveryAlgorithm (7.4.3.4) or the MatchRecoveryAlgorithm (7.4.3.5) that has no sequence_number subparameter (item b in 6.1).

10.8.9 frerCpsSeqRcvyResets

The frerCpsSeqRcvyResets counter is incremented once each time the SequenceRecoveryReset function (7.4.3.3) is called.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

10.8.10 frerCpsSeqRcvyLatentErrorResets

The frerCpsSeqRcvyLatentErrorResets counter is incremented once each time the LatentErrorReset function (7.4.4.3) is called.

10.8.11 frerCpsSeqEncErroredPackets

The frerCpsSeqEncErroredPackets counter is incremented once each time the Sequence encode/decode function (7.5) receives a packet that it is unable to decode successfully.

10.9 Operational per-port FRER counters

The following counters are instantiated for each port on which any of the Stream identification function (6.2), Sequencing function (7.4), or Sequence encode/decode function (7.5) is configured. The counters are indexed by port number. Encoded as unsigned integers, they shall be 64 bits in length to insure against excessively short wrap times when used on links on which transmission rate exceeds 650 000 000 bits per second.

A Stream identification component (5.3) shall implement the first two counters tsnCpSidInputPackets (9.3.1) and tsnCpSidOutputPackets (9.3.2); the remainder of the counters in 10.9 are optional for such a system.

10.9.1 frerCpSeqRcvyPassedPackets

The frerCpSeqRcvyPassedPackets counter is incremented once for each packet passed up the stack by the VectorRecoveryAlgorithm (7.4.3.4) or MatchRecoveryAlgorithm (7.4.3.5). Its value equals the sum (modulo the size of the counters) of all of the frerCpsSeqRcvyPassedPackets (10.8.5) counters on this same port.

10.9.2 frerCpSeqRcvyDiscardPackets

The frerCpSeqRcvyDiscardPackets counter is incremented once for each packet discarded due to a duplicate sequence number or for being a rogue packet by any VectorRecoveryAlgorithm (7.4.3.4) or MatchRecoveryAlgorithm (7.4.3.5) on this port. Its value equals the sum (modulo the size of the counters) of all of the frerCpsSeqRcvyRoguePackets (10.8.4) and frerCpsSeqRcvyDiscardedPackets (10.8.6) counters on this same port.

10.9.3 frerCpSeqEncErroredPackets

The frerCpSeqEncErroredPackets counter is incremented once each time the Sequence encode/decode function (7.5) receives a packet that it is unable to decode successfully. Its value equals the sum (modulo the size of the counters) of all of the frerCpsSeqEncErroredPackets (10.8.11) counters on this same port.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

11. Management Information Base (MIB)

This clause contains a complete SMIv2 MIB set for all features of this standard.

11.1 Internet Standard Management Framework

The MIBs in this standard are designed to operate in a manner consistent with the principles of the Internet Standard Management Framework, which describes the separation of a data modeling language (for example, SMIv2) from content-specific data models (for example, the LLDP remote systems MIB), and from messages and protocol operations used to manipulate the data (for example, SNMPv3).

For a detailed overview of the documents that describe the current Internet Standard Management Framework, please refer to section 7 of IETF RFC 3410 (2002).

Managed objects are accessed via a virtual information store, termed the Management Information Base or MIB. MIB objects are generally accessed through the Simple Network Management Protocol (SNMP). Objects in the MIB are defined using the mechanisms defined in the Structure of Management Information (SMI). This clause specifies a MIB module that is compliant to the SMIv2, which is described in IETF RFC 2578 (STD 58), IETF RFC 2579 (STD 58), and IETF RFC 2580 (STD 58).

11.2 Structure of the MIB

The IEEE Std 802.1CB MIB is divided into a number of MIB modules. A summary of the modules contained in this clause is represented in Table 11-1. The general MIB table structures for the MIB modules contained in this clause are illustrated in Figure 11-1.

Table 11-1—Structure of the MIB modules

Module	Subclause	References	Notes
IEEE8021-STREAM-IDENTIFICATION-MIB	11.5.1	Clause 9	MIB model for Stream identification.
IEEE8021-FRER-MIB	11.5.2	Clause 10	MIB model for FRER.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

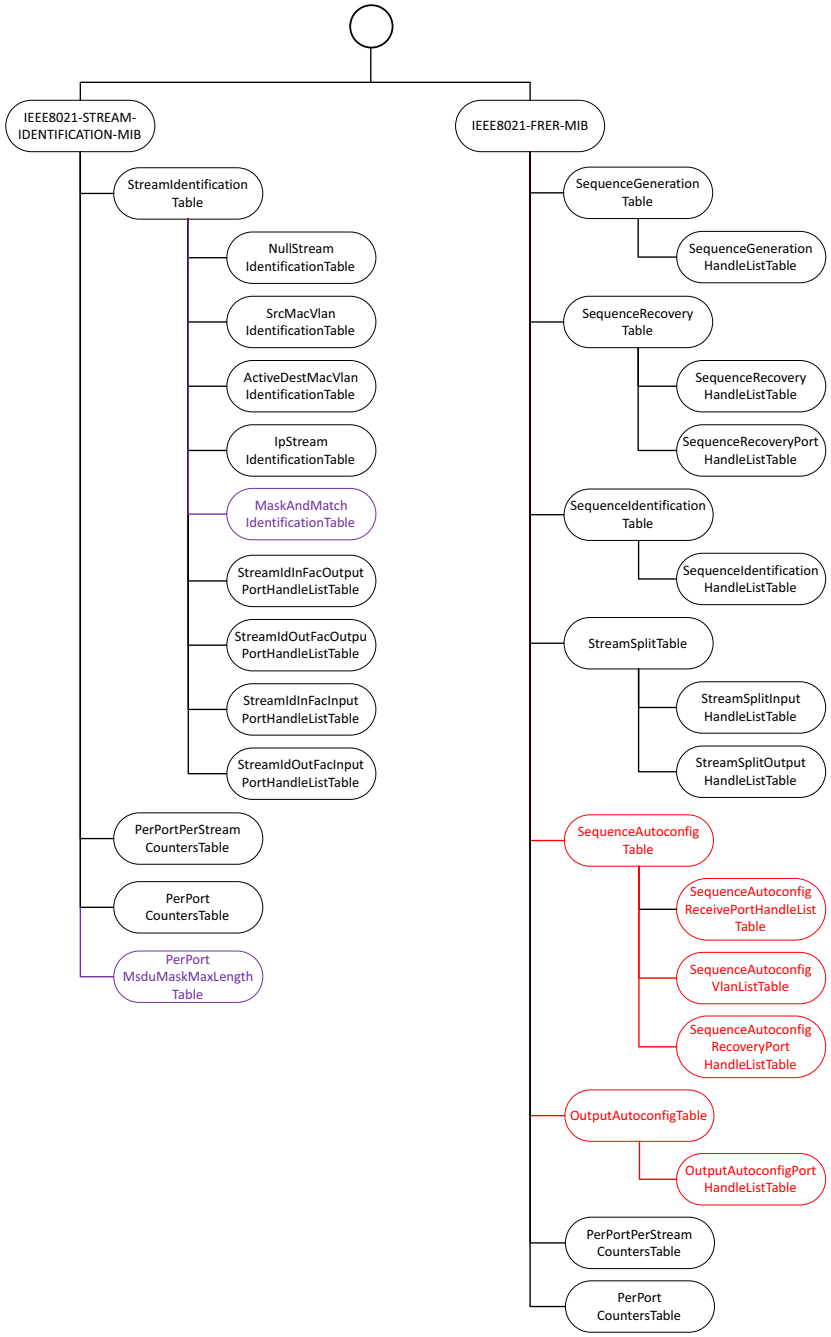


Figure 11-1—MIB modules structure

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

11.2.1 Structure of the IEEE8021-STREAM-IDENTIFICATION-MIB

The IEEE8021-STREAM-IDENTIFICATION-MIB module is divided into a number of branches (e.g., subtrees). Each subtree is organized as a set of related objects. Where appropriate, the corresponding Clause 9 management reference is also included.

Table 11-2 indicates the structure of the IEEE8021-STREAM-IDENTIFICATION-MIB module.

Table 11-2—IEEE8021-STREAM-IDENTIFICATION-MIB structure and relationship to this standard

Clause 11 MIB table/object	Reference
ieee8021StreamIdStreamIdentificationTable	9.1, 9.1.1
ieee8021StreamIdStreamIdentificationIndex	—
ieee8021StreamIdStreamIdIdentificationType	—
ieee8021StreamIdStreamIdIdentificationTypeOUI	—
ieee8021StreamIdStreamIdIdentificationCustomType	—
ieee8021StreamIdStreamIdIdentificationCustomTypeOUI	—
ieee8021StreamIdStreamIdIdentificationTypeSelect	—
ieee8021StreamIdStreamIdIdentificationTypeSelect	—
ieee8021StreamIdStreamIdInFacOutputPortList	—
ieee8021StreamIdStreamIdOutFacOutputPortList	—
ieee8021StreamIdStreamIdInFacInputPortList	—
ieee8021StreamIdStreamIdOutFacInputPortList	—
ieee8021StreamIdAutoConfigured	—
ieee8021StreamIdLanPathId	—
ieee8021StreamIdStatus	—
ieee8021StreamIdNullStreamIdentificationTable	9.1.2
ieee8021StreamIdCpeNullDownDestMac	—
ieee8021StreamIdCPENullDownTagged	—
ieee8021StreamIdCpeNullDownVlan	—
ieee8021StreamIdSrcMacVlanIdentificationTable	9.1.3
ieee8021StreamIdCpeSmacVlanDownSrcMac	—
ieee8021StreamIdCpeSmacVlanDownTagged	—
ieee8021StreamIdCpeSmacVlanDownVlan	—
ieee8021StreamIdActiveDestMacVlanIdentificationTable	9.1.4
ieee8021StreamIdCpeDmacVlanDownDestMac	—
ieee8021StreamIdCpeDmacVlanDownTagged	—

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Table 11-2—IEEE8021-STREAM-IDENTIFICATION-MIB structure and relationship to this standard *(continued)*

Clause 11 MIB table/object	Reference
ieee8021StreamIdCpeDmacVlanDownVlan	—
ieee8021StreamIdCpeDmacVlanDownPriority	—
ieee8021StreamIdCpeDmacVlanUpDestMac	—
ieee8021StreamIdCpeDmacVlanUpTagged	—
ieee8021StreamIdCpeDmacVlanUpVlan	—
ieee8021StreamIdCpeDmacVlanUpPriority	—
Ieee8021StreamIdIpStreamIdentificationTable	9.1.5
ieee8021StreamIdCpeIpIdDestMac	—
ieee8021StreamIdCpeIpIdTagged	—
ieee8021StreamIdCpeIpIdVlan	—
ieee8021StreamIdCpeIpIdIpSourceType	—
ieee8021StreamIdCpeIpIdIpSource	—
ieee8021StreamIdCpeIpIdIpDestinationType	—
ieee8021StreamIdCpeIpIdIpDestination	—
ieee8021StreamIdCpeIpIdDscp	—
ieee8021StreamIdCpeIpIdNextProtocol	—
ieee8021StreamIdCpeIpIdSourcePort	—
ieee8021StreamIdCpeIpIdDestinationPort	—
ieee8021StreamIdMaskAndMatchIdentificationTable	9.1.6
ieee8021StreamIdCpeMmIdDestMacMask	—
ieee8021StreamIdCpeMmIdDestMacMatch	—
ieee8021StreamIdCpeMmIdSrcMacMask	—
ieee8021StreamIdCpeMmIdSrcMacMatch	—
ieee8021StreamIdCpeMmIdMsduMaskLength	—
ieee8021StreamIdCpeMmIdMsduMask	—
ieee8021StreamIdCpeMmIdMsduMatch	—
ieee8021StreamIdStreamIdInFacOutputPortHandleListTable	9.1.1.2
ieee8021StreamIdStreamIdInFacOutputPortHandleListIndex	—
ieee8021StreamIdStreamIdInFacOutputPortHandle	—
ieee8021StreamIdStreamIdInFacOutputPortHandleListStatus	—
ieee8021StreamIdStreamIdOutFacOutputPortHandleListTable	9.1.1.3
ieee8021StreamIdStreamIdOutFacOutputPortHandleListIndex	—

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Table 11-2—IEEE8021-STREAM-IDENTIFICATION-MIB structure and relationship to this standard *(continued)*

Clause 11 MIB table/object	Reference
ieee8021StreamIdStreamIdOutFacOutputPortHandle	—
ieee8021StreamIdStreamIdOutFacOutputPortHandleListStatus	—
ieee8021StreamIdStreamIdInFacInputPortHandleListTable	9.1.1.2
ieee8021StreamIdStreamIdInFacInputPortHandleListIndex	—
ieee8021StreamIdStreamIdInFacInputPortHandle	—
ieee8021StreamIdStreamIdInFacInputPortHandleListStatus	—
ieee8021StreamIdStreamIdOutFacInputPortHandleListTable	9.1.1.5
ieee8021StreamIdStreamIdOutFacInputPortHandleListIndex	—
ieee8021StreamIdStreamIdOutFacInputPortHandle	—
ieee8021StreamIdStreamIdOutFacInputPortHandleListStatus	—
ieee8021StreamIdPerPortPerStreamCountersTable	9.2
ieee8021StreamIdPerPortPerStreamDirection	—
ieee8021StreamIdPerPortPerStreamInputPackets	—
ieee8021StreamIdPerPortPerStreamOutputPackets	—
ieee8021StreamIdPerPortCountersTable	9.3
ieee8021StreamIdPerPortInputPackets	—
ieee8021StreamIdPerPortOutputPackets	—
ieee8021StreamIdMaskAndMatchPerPortMsduMaskMaxLengthTable	9.4.1
ieee8021StreamIdMaskAndMatchMsduMaskMaxLength	—

11.2.2 Structure of the IEEE8021-FRER-MIB

The IEEE8021-FRER-MIB module is divided into a number of branches (e.g., subtrees). Each subtree is organized as a set of related objects. Where appropriate, the corresponding Clause 9 management reference is also included.

Table 11-3 indicates the structure of the IEEE8021-FRER-MIB module.

11.3 Relationship to other MIBs

In order to facilitate interoperable management of the features defined in this standard by remote means, 11.5 contains a complete set of SMIv2 MIB modules. This subclause describes how the MIB modules defined in this standard are related to the MIB modules that are imported.

11.3.1 Relationship of the IEEE8021-STREAM-IDENTIFICATION-MIB to other MIBs

The IEEE8021-STREAM-IDENTIFICATION-MIB identifies the interfaces for the counters provided by this standard by an ifIndex and therefore imports from the IF-MIB.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Table 11-3—IEEE8021-FRER-MIB structure and relationship to this standard

Clause 11 MIB table/object	Reference
ieee8021FrerSequenceGenerationTable	10.3
ieee8021FrerSequenceGenerationIndex	—
ieee8021FrerSequenceGenerationStreamList	—
ieee8021FrerSequenceGenerationDirection	—
ieee8021FrerSequenceGenerationReset	—
ieee8021FrerSequenceGenerationHandleListTable	10.3.1.1
ieee8021FrerSequenceGenerationHandleListIndex	—
ieee8021FrerSequenceGenerationStreamHandle	—
ieee8021FrerSequenceGenerationHandleListStatus	—
ieee8021FrerSequenceRecoveryTable	10.4
ieee8021FrerSequenceRecoveryIndex	—
ieee8021FrerSequenceRecoveryStreamList	—
ieee8021FrerSequenceRecoveryPortList	—
ieee8021FrerSequenceRecoveryDirection	—
ieee8021FrerSequenceRecoveryReset	—
ieee8021FrerSequenceRecoveryAlgorithm	—
ieee8021FrerSequenceRecoveryAlgorithmOUI	—
ieee8021FrerSequenceCustomRecoveryAlgorithm	—
ieee8021FrerSequenceCustomRecoveryAlgorithmOUI	—
ieee8021FrerSequenceRecoveryHistoryLength	—
ieee8021FrerSequenceRecoveryResetMSec	—
ieee8021FrerSequenceRecoveryInvalidSequenceValue	—
ieee8021FrerSequenceRecoveryTakeNoSequence	—
ieee8021FrerSequenceRecoveryIndividualRecovery	—
ieee8021FrerSequenceRecoveryLatentErrorDetection	—
ieee8021FrerSequenceRecoveryLatentErrorDifference	—
ieee8021FrerSequenceRecoveryLatentErrorPeriod	—
ieee8021FrerSequenceRecoveryLatentErrorPaths	—
ieee8021FrerSequenceRecoveryLatentResetPeriod	—
ieee8021FrerSequenceRecoveryStatus	—
ieee8021FrerSequenceRecoveryHandleListTable	10.4.1.1
ieee8021FrerSequenceRecoveryHandleListIndex	—
ieee8021FrerSequenceRecoveryStreamHandle	—

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Table 11-3—IEEE8021-FRER-MIB structure and relationship to this standard *(continued)*

Clause 11 MIB table/object	Reference
ieee8021FrerSequenceRecoveryHandleListStatus	—
ieee8021FrerSequenceRecoveryPortHandleListTable	10.4.1.2
ieee8021FrerSequenceRecoveryPortHandleListIndex	—
ieee8021FrerSequenceRecoveryPortHandle	—
ieee8021FrerSequenceRecoveryPortHandleListStatus	—
ieee8021FrerSequenceIdentificationTable	10.5
ieee8021FrerSequenceIdentificationEncodePort	—
ieee8021FrerSequenceIdentificationEncodeDirection	—
ieee8021FrerSequenceIdentificationStreamList	—
ieee8021FrerSequenceIdentificationEncodeActive	—
ieee8021FrerSequenceIdentificationEncodeEncapsulationType	—
ieee8021FrerSequenceIdentificationEncodeEncapsulationOUI	—
ieee8021FrerSequenceIdentificationCustomEncodeEncapsulationType	—
ieee8021FrerSequenceIdentificationCustomEncodeEncapsulationOUI	—
ieee8021FrerSequenceIdentificationEncodePathIdLanId	—
ieee8021FrerSequenceIdentificationStatus	—
ieee8021FrerSequenceIdentificationHandleListTable	10.5.1.1
ieee8021FrerSequenceIdentificationHandleListIndex	—
ieee8021FrerSequenceIdentificationStreamHandle	—
ieee8021FrerSequenceIdentificationHandleListStatus	—
ieee8021FrerStreamSplitTable	10.6
ieee8021FrerStreamSplitIndex	—
ieee8021FrerStreamSplitPort	—
ieee8021FrerStreamSplitDirection	—
ieee8021FrerStreamSplitInputIdList	—
ieee8021FrerStreamSplitOutputIdList	—
ieee8021FrerStreamSplitInputHandleListTable	10.6.1.3
ieee8021FrerStreamSplitInputHandleListIndex	—
ieee8021FrerStreamSplitInputIdHandle	—
ieee8021FrerStreamSplitInputHandleListStatus	—
ieee8021FrerStreamSplitOutputHandleListTable	10.6.1.4
ieee8021FrerStreamSplitOutputHandleListIndex	—
ieee8021FrerStreamSplitOutputIdHandle	—

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Table 11-3—IEEE8021-FRER-MIB structure and relationship to this standard *(continued)*

Clause 11 MIB table/object	Reference
ieee8021FrerStreamSplitOutputHandleListStatus	—
ieee8021FrerSequenceAutoconfigTable	10.7, 10.7.1
ieee8021FrerSequenceAutoconfigIndex	—
ieee8021FrerSequenceAutoconfigSequenceEncapsulation	—
ieee8021FrerSequenceAutoconfigSequenceEncapsulationOUI	—
ieee8021FrerSequenceAutoconfigCustomSequenceEncapsulation	—
ieee8021FrerSequenceAutoconfigCustomSequenceEncOUI	—
ieee8021FrerSequenceAutoconfigReceivePortList	—
ieee8021FrerSequenceAutoconfigTagged	—
ieee8021FrerSequenceAutoconfigVlanList	—
ieee8021FrerSequenceAutoconfigRecoveryPortList	—
ieee8021FrerSequenceAutoconfigDestructMSec	—
ieee8021FrerSequenceAutoconfigResetMSec	—
ieee8021FrerSequenceAutoconfigAlgorithm	—
ieee8021FrerSequenceAutoconfigAlgorithmOUI	—
ieee8021FrerSequenceAutoconfigCustomAlgorithm	—
ieee8021FrerSequenceAutoconfigCustomAlgorithmOUI	—
ieee8021FrerSequenceAutoconfigHistoryLength	—
ieee8021FrerSequenceAutoconfigCreateIndividual	—
ieee8021FrerSequenceAutoconfigCreateRecovery	—
ieee8021FrerSequenceAutoconfigLatentErrorDetection	—
ieee8021FrerSequenceAutoconfigLatentErrorDifference	—
ieee8021FrerSequenceAutoconfigLatentErrorPeriod	—
ieee8021FrerSequenceAutoconfigLatentErrorResetPeriod	—
ieee8021FrerSequenceAutoconfigStatus	—
ieee8021FrerSequenceAutoconfigReceivePortHandleListTable	10.7.1.1.2
ieee8021FrerSequenceAutoconfigReceivePortHandleListIndex	—
ieee8021FrerSequenceAutoconfigReceivePortHandle	—
ieee8021FrerSequenceAutoconfigReceivePortHandleListStatus	—
ieee8021FrerSequenceAutoconfigVlanListTable	10.7.1.1.4
ieee8021FrerSequenceAutoconfigVlanListIndex	—
ieee8021FrerSequenceAutoconfigVlan	—

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Table 11-3—IEEE8021-FRER-MIB structure and relationship to this standard *(continued)*

Clause 11 MIB table/object	Reference
ieee8021FrerSequenceAutoconfigVlanListStatus	—
ieee8021FrerSequenceAutoconfigRecoveryPortHandleListTable	10.7.1.1.5
ieee8021FrerSequenceAutoconfigRecoveryPortHandleListIndex	—
ieee8021FrerSequenceAutoconfigRecoveryPortHandle	—
ieee8021FrerSequenceAutoconfigRecoveryPortHandleListStatus	—
ieee8021FrerOutputAutoconfigTable	10.7, 10.7.2
ieee8021FrerOutputAutoconfigIndex	—
ieee8021FrerOutputAutoconfigPortList	—
ieee8021FrerOutputAutoconfigEncapsulation	—
ieee8021FrerOutputAutoconfigEncapsulationOUI	—
ieee8021FrerOutputAutoconfigCustomEncapsulation	—
ieee8021FrerOutputAutoconfigCustomEncapsulationOUI	—
ieee8021FrerOutputAutoconfigLanPathId	—
ieee8021FrerOutputAutoconfigStatus	—
ieee8021FrerOutputAutoconfigPortHandleListTable	10.7.2.1.1
ieee8021FrerOutputAutoconfigPortHandleListIndex	—
ieee8021FrerOutputAutoconfigPortHandle	—
ieee8021FrerOutputAutoconfigPortHandleListStatus	—
ieee8021FrerPerPortPerStreamCountersTable	10.8
ieee8021FrerPerPortPerStreamDirection	—
ieee8021FrerPerPortPerStreamSeqGenResets	—
ieee8021FrerPerPortPerStreamSeqRecoveryOutOfOrderPackets	—
ieee8021FrerPerPortPerStreamSeqRecoveryRoguePackets	—
ieee8021FrerPerPortPerStreamSeqRecoveryPassedPackets	—

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Table 11-3—IEEE8021-FRER-MIB structure and relationship to this standard *(continued)*

Clause 11 MIB table/object	Reference
ieee8021FrerPerPortPerStreamSeqRecoveryDiscardedPackets	—
ieee8021FrerPerPortPerStreamSeqRecoveryLostPackets	—
ieee8021FrerPerPortPerStreamSeqRecoveryTaglessPackets	—
ieee8021FrerPerPortPerStreamSeqRecoveryResets	—
ieee8021FrerPerPortPerStreamSeqRecoveryLatentErrorResets	—
ieee8021FrerPerPortPerStreamSeqEncErroredPackets	—
ieee8021FrerPerPortCountersTable	10.9
ieee8021FrerPerPortSeqRecoveryPassedPackets	—
ieee8021FrerPerPortfrerCpSeqRecoveryDiscardPackets	—
ieee8021FrerPerPortfrerCpSeqEncErroredPackets	—

It also imports items from the SNMPv2-SMI, SNMPv2-TC, INET-ADDRESS-MIB, and IEEE8021-TC-MIB and requires SNMPv2-CONF for conformance.

11.3.2 Relationship of the IEEE8021-FRER-MIB to other MIBs

The IEEE8021-FRER-MIB identifies the interfaces for the counters provided by this standard by an ifIndex and therefore imports from the IF-MIB.

It also imports items from the IEEE8021-STREAM-IDENTIFICATION-MIB, SNMPv2-SMI, SNMPv2-TC, and IEEE8021-TC-MIB and requires SNMPv2-CONF for conformance.

11.4 Security considerations

SNMP versions prior to SNMPv3 did not include adequate security. Even if the network itself is secure (for example, by using IPsec), there is no control about who on the secure network is allowed to access and GET/SET (read/change/create/delete) the objects in these MIB modules.

It is recommended that implementers consider the security features as provided by the SNMPv3 framework (see IETF RFC 3410 (2002), section 8), including full support for the SNMPv3 cryptographic mechanisms (for authentication and privacy).

Further, deployment of SNMP versions prior to SNMPv3 is not recommended. Instead, it is recommended to deploy SNMPv3 and to enable cryptographic security. It is then a customer/operator responsibility to ensure that the SNMP entity giving access to an instance of these MIB modules is properly configured to give access to the objects only to those principals (users) that have legitimate rights to indeed GET or SET (change/create/delete) them.

11.4.1 Security considerations of the IEEE8021-STREAM-IDENTIFICATION-MIB

There are a number of management objects defined in the IEEE8021-STREAM-IDENTIFICATION-MIB module that have a MAX-ACCESS clause of read-write and/or read-create. Such objects may be considered

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

sensitive or vulnerable in some network environments. The support for SET operations in a nonsecure environment without proper protection can have a negative effect on network operations.

Some of the readable objects in this MIB module (i.e., objects with a MAX-ACCESS other than not-accessible) may be considered sensitive or vulnerable in some network environments. It is thus important to control all types of access (including GET and/or NOTIFY) to these objects and possibly to even encrypt the values of these objects when sending them over the network via SNMP.

11.4.2 Security considerations of the IEEE8021-FRER-MIB

There are a number of management objects defined in the IEEE8021-FRER-MIB module that have a MAX-ACCESS clause of read-write and/or read-create. Such objects may be considered sensitive or vulnerable in some network environments. The support for SET operations in a nonsecure environment without proper protection can have a negative effect on network operations.

Some of the readable objects in this MIB module (i.e., objects with a MAX-ACCESS other than not-accessible) may be considered sensitive or vulnerable in some network environments. It is thus important to control all types of access (including GET and/or NOTIFY) to these objects and possibly to even encrypt the values of these objects when sending them over the network via SNMP.

11.5 MIB modules[16, 17]

In this subclause, certain terms (e.g., “SHOULD” and “MUST”) denote requirements terminology according to the usage specified in IETF RFC 2119, as is customary for IETF MIB module definitions. In the MIB modules definitions below, if any discrepancy between the DESCRIPTION text and the corresponding definition in any other part of this standard occur, the definitions outside this subclause take precedence.

11.5.1 Definitions for the IEEE8021-STREAM-IDENTIFICATION-MIB

```
IEEE8021-STREAM-IDENTIFICATION-MIB DEFINITIONS ::= BEGIN

-- ================================================================
-- MIB for support of stream identification  of 802.1CB
-- Frame Replication and Elimination for Reliability
-- ================================================================

IMPORTS
    OBJECT-GROUP,
    MODULE-COMPLIANCE
        FROM SNMPv2-CONF
    MODULE-IDENTITY,
    OBJECT-TYPE,
    OBJECT-IDENTITY,
    Counter64,
    Unsigned32,
    Integer32
        FROM SNMPv2-SMI
    TEXTUAL-CONVENTION,
    TruthValue,
```

[16] Copyright release for MIBs: Users of this standard may freely reproduce the MIBs contained in this subclause so that they can be used for their intended purpose.

[17] An ASCII version of the MIB modules are attached to the PDF version of this standard, and can be obtained by Web browser from the IEEE 802.1 Website at https://1.ieee802.org/mib-modules/.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
    RowStatus,
    MacAddress,
    AutonomousType,
    VariablePointer
        FROM SNMPv2-TC
    ifIndex
        FROM IF-MIB
    InetAddressType,
    InetAddress,
    InetPortNumber
        FROM INET-ADDRESS-MIB
    IEEE8021PriorityValue,
    ieee802dot1mibs
        FROM IEEE8021-TC-MIB
    ;

ieee8021StreamIdMib MODULE-IDENTITY
    LAST-UPDATED "202603060000Z" -- March 6, 2026
~~LAST-UPDATED "202107220000Z" -- July 22, 2021~~
    ORGANIZATION "IEEE 802.1 Working Group"
    CONTACT-INFO
                "WG-URL: http://ieee802.org/1/
                WG-EMail: stds-802-1-l@ieee.org

    Contact: IEEE 802.1 Working Group Chair
    Postal: C/O IEEE 802.1 Working Group
            IEEE Standards Association
            445 Hoes Lane
            Piscataway, NJ 08854
            USA

    E-mail: stds-802-1-chairs@ieee.org"

    DESCRIPTION
        "The Management Information Base module for IEEE 802.1CB stream
         identification. Unless otherwise indicated, the references in
         this MIB module are to IEEE Std 802.1CB-2017 as amended by
         IEEE Std 802.1CBcv-2021 and IEEE Std 802.1CBdb-2021. Copyright
         (C) IEEE (2021).
         This version of this MIB module is part of IEEE Std 802.1CB-2021;
         see the standard itself for full legal notices."

    REVISION "202603060000Z" -- March 6, 2026
    DESCRIPTION
        "Published as part of IEEE Std 802.1CB-2017-Rev. Removes
         the use of the 0 value for the VLAN-ID-related parameters of the
         Stream identification functions."

    REVISION "202107220000Z" -- July 22, 2021
    DESCRIPTION
        "Published as part of IEEE Std 802.1CBdb-2021. Adds extended
         Stream identification function objects."

    REVISION "202107210000Z" -- July 21, 2021
    DESCRIPTION
        "Initial revision published as part of IEEE Std 802.1CBcv-2021."

::= { ieee802dot1mibs 34 }

```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
-- ================================================================
-- Textual Conventions
-- ================================================================

Ieee8021CBStreamIdentificationType ::= TEXTUAL-CONVENTION
    STATUS       current
    DESCRIPTION
        "An enumerated value indicating the method used to identify
         packets belonging to the Stream. The values 0-255 are reserved
         for use by IEEE 802."
    REFERENCE    "9.1.1.6 and Table 9-1"
    SYNTAX      INTEGER {
                    reserved(0),
                    nullStreamIdentification(1),
                    srcMacVlanStreamIdentification(2),
                    activeDstMacVlanStreamIdentification(3),
                    ipStreamStreamIdentification(4),
                    maskAndMatchStreamIdentification(5),
                    nonIEEESpecified(256)
                }

Ieee8021CBTaggedType ::= TEXTUAL-CONVENTION
    STATUS       current
    DESCRIPTION
        "An enumerated value indicating whether a packet in an EISS
         indication primitive to a stream identification function is
         permitted to have a VLAN tag."
    REFERENCE    "9.1.2.2, 9.1.3.2, 9.1.4.2, 9.1.4.6, 9.1.5.2, and
                  10.7.1.1.3"
    SYNTAX      INTEGER {
                    tagged(1),
                    priority(2),
                    all(3)
                }

Ieee8021CBVlanIdentifier ::= TEXTUAL-CONVENTION
    DISPLAY-HINT "d"
    STATUS   current
    DESCRIPTION
        "Specifies the vlan_identifier. A value of 0 indicates a special
         treatment, depending on the use of Ieee8021CBVlanIdentifier."
    SYNTAX      Integer32 (0|1..4094)

Ieee8021CBIpDscpType ::= TEXTUAL-CONVENTION
    DISPLAY-HINT "d"
    STATUS   current
    DESCRIPTION
        "The code point used for discriminating a traffic
         stream."
    SYNTAX      Integer32 (-1 | 0..63)

Ieee8021CBIdNextProtocolType ::= TEXTUAL-CONVENTION
    STATUS       current
    DESCRIPTION
        "An enumerated value indicating an IP next protocol."
    SYNTAX      INTEGER {
                    none(1),
                    udp(2),
                    tcp(3),
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
                    sctp(4)
                }

Ieee8021CBLanPathIdType ::= TEXTUAL-CONVENTION
    DISPLAY-HINT "d"
    STATUS       current
    DESCRIPTION
        "An integer specifying a path or LAN. If and only if a packet
        matches an entry in the Sequence identification table that
        specifies HSR or PRP in its frerSeqEncEncapsType  object,
        tsnStreamIdLanPathId specifies the LanId or PathId value that
        must be matched for this tsnStreamIdEntry to apply. A value
        of -1 indicates that the LanId or PathId are to be ignored."
    REFERENCE    "10.22"
    SYNTAX      Integer32 (-1 | 0..15)

-- ================================================================
-- subtrees in the stream identification MIB
-- ================================================================

ieee8021StreamIdNotifications
    OBJECT IDENTIFIER ::= { ieee8021StreamIdMib 0 }

ieee8021StreamIdObjects
    OBJECT IDENTIFIER ::= { ieee8021StreamIdMib 1 }

ieee8021StreamIdConformance
    OBJECT IDENTIFIER ::= { ieee8021StreamIdMib 2 }

ieee8021StreamIdStreamIdentification
    OBJECT IDENTIFIER ::= { ieee8021StreamIdObjects 1 }

ieee8021StreamIdStreamIdInFacOutputPortHandleList
    OBJECT IDENTIFIER ::= { ieee8021StreamIdObjects 2 }

ieee8021StreamIdStreamIdOutFacOutputPortHandleList
    OBJECT IDENTIFIER ::= { ieee8021StreamIdObjects 3 }

ieee8021StreamIdStreamIdInFacInputPortHandleList
    OBJECT IDENTIFIER ::= { ieee8021StreamIdObjects 4 }

ieee8021StreamIdStreamIdOutFacInputPortHandleList
    OBJECT IDENTIFIER ::= { ieee8021StreamIdObjects 5 }

ieee8021StreamIdPerPortPerStreamCounters
    OBJECT IDENTIFIER ::= { ieee8021StreamIdObjects 6 }

ieee8021StreamIdPerPortCounters
    OBJECT IDENTIFIER ::= { ieee8021StreamIdObjects 7 }

ieee8021StreamIdMaskAndMatchPerPortMsduMaskMaxLength
    OBJECT IDENTIFIER ::= { ieee8021StreamIdObjects 8 }

-- ================================================================
-- the ieee8021StreamIdStreamIdentification table
-- ================================================================

-- ================================================================
-- the ieee8021StreamIdStreamIdentificationTypes
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
-- ================================================================

ieee8021StreamIdStreamIdentificationTypes
    OBJECT IDENTIFIER ::= { ieee8021StreamIdStreamIdentification 1 }

ieee8021StreamIdNullStream OBJECT-IDENTITY
       STATUS current
       DESCRIPTION
           "The stream identification type for the Null Stream
            identification."
       ::= { ieee8021StreamIdStreamIdentificationTypes 1 }

ieee8021StreamIdSrcMacVlan OBJECT-IDENTITY
       STATUS current
       DESCRIPTION
           "The stream identification type for the Source MAC and VLAN
            Stream identification."
       ::= { ieee8021StreamIdStreamIdentificationTypes 2 }

ieee8021StreamIdActiveDestMacVlan OBJECT-IDENTITY
       STATUS current
       DESCRIPTION
           "The stream identification type for the Active Destination
            and VLAN Stream identification."
       ::= { ieee8021StreamIdStreamIdentificationTypes 3 }

ieee8021StreamIdIpStream OBJECT-IDENTITY
       STATUS current
       DESCRIPTION
           "The stream identification type for the IP Stream
            identification."
       ::= { ieee8021StreamIdStreamIdentificationTypes 4 }

ieee8021StreamIdMaskAndMatch OBJECT-IDENTITY
       STATUS current
       DESCRIPTION
           "The stream identification type for the Mask-and-match
           Stream identification."
       ::= { ieee8021StreamIdStreamIdentificationTypes 5 }

-- ================================================================
-- the ieee8021StreamIdStreamIdentification
-- ================================================================

ieee8021StreamIdStreamIdentificationTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF Ieee8021StreamIdStreamIdentificationEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A table containing a set of tsnStreamIdEntry objects each
         relating to a single Stream, specifying the points in the
         system where Stream identification functions are to be
         instantiated."
    REFERENCE   "9.1"
    ::= { ieee8021StreamIdStreamIdentification 2 }

ieee8021StreamIdStreamIdentificationEntry OBJECT-TYPE
    SYNTAX      Ieee8021StreamIdStreamIdentificationEntry
    MAX-ACCESS  not-accessible
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
    STATUS      current
    DESCRIPTION
        "A set of managed objects, all applying to the Stream specified
        by tsnStreamIdHandle, and all using the same Stream
        identification types and parameters."
    REFERENCE   "9.1.1"
    INDEX  { ieee8021StreamIdStreamIdentificationIndex }
    ::= { ieee8021StreamIdStreamIdentificationTable 1 }

Ieee8021StreamIdStreamIdentificationEntry ::=
    SEQUENCE {
        ieee8021StreamIdStreamIdentificationIndex
            Unsigned32,
        ieee8021StreamIdStreamIdIdentificationType
            Ieee8021CBStreamIdentificationType,
        ieee8021StreamIdStreamIdIdentificationTypeOUI
            OCTET STRING,
        ieee8021StreamIdStreamIdIdentificationCustomType
            Integer32,
        ieee8021StreamIdStreamIdIdentificationCustomTypeOUI
            OCTET STRING,
        ieee8021StreamIdStreamIdIdentificationTypeSelect
            AutonomousType,
        ieee8021StreamIdStreamIdHandle
            Unsigned32,
        ieee8021StreamIdStreamIdInFacOutputPortList
            AutonomousType,
        ieee8021StreamIdStreamIdOutFacOutputPortList
            AutonomousType,
        ieee8021StreamIdStreamIdInFacInputPortList
            AutonomousType,
        ieee8021StreamIdStreamIdOutFacInputPortList
            AutonomousType,
        ieee8021StreamIdAutoConfigured
            TruthValue,
        ieee8021StreamIdLanPathId
            Ieee8021CBLanPathIdType,
        ieee8021StreamIdStatus
            RowStatus
    }

ieee8021StreamIdStreamIdentificationIndex OBJECT-TYPE
    SYNTAX      Unsigned32
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "Index for the Stream Identity table."
    ::= { ieee8021StreamIdStreamIdentificationEntry 1 }

ieee8021StreamIdStreamIdIdentificationType OBJECT-TYPE
    SYNTAX      Ieee8021CBStreamIdentificationType
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "This managed object is an enumerated value indicating the
         method used to identify packets belonging to the Stream.
         If the value used is nonIEEESpecified(256), the managed objects
         ieee8021StreamIdStreamIdIdentificationCustomType and
         ieee8021StreamIdStreamIdIdentificationCustomTypeOUI are used to
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
         specify OUI/CID and the stream identification method defined by
         the entity owning the OUI/CID."
    REFERENCE   "9.1.1.6"
    DEFVAL  { nullStreamIdentification }
    ::= { ieee8021StreamIdStreamIdentificationEntry 2 }

ieee8021StreamIdStreamIdIdentificationTypeOUI OBJECT-TYPE
    SYNTAX      OCTET STRING (SIZE (3))
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
        "This managed object specifies the OUI/CID used together with
         the stream identification method specified in
         ieee8021StreamIdStreamIdIdentificationType.
         It is used if and only if the value for
         ieee8021StreamIdStreamIdIdentificationType is in the range of
         0-255. In this case it always takes the value 00-80-C2."
    REFERENCE   "9.1.1.6"
    ::= { ieee8021StreamIdStreamIdentificationEntry 3 }

ieee8021StreamIdStreamIdIdentificationCustomType OBJECT-TYPE
    SYNTAX      Integer32 (256..2147483647)
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "This managed object is an enumerated value indicating the
         method used to identify packets belonging to the Stream. It is
         used if and only if the value for
         ieee8021StreamIdStreamIdIdentificationType is 256."
    REFERENCE   "9.1.1.6"
    ::= { ieee8021StreamIdStreamIdentificationEntry 4 }

ieee8021StreamIdStreamIdIdentificationCustomTypeOUI OBJECT-TYPE
    SYNTAX      OCTET STRING (SIZE (3))
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "This managed object specifies the OUI/CID used together with
         the stream identification method specified in
         ieee8021StreamIdStreamIdIdentificationType. It is used if and
         only if the value for
         ieee8021StreamIdStreamIdIdentificationType is 256."
    REFERENCE   "9.1.1.6"
    ::= { ieee8021StreamIdStreamIdentificationEntry 5 }

ieee8021StreamIdStreamIdIdentificationTypeSelect OBJECT-TYPE
    SYNTAX      AutonomousType
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
        "An indication of the type of stream identification method that
         is used.

         If this value is
         'ieee8021StreamIdNullStream  ieee8021StreamIdObjects 1' then
         an entry exists in the
         ieee8021StreamIdStreamIdentificationTable
         which corresponds to the Null Stream identification.

```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
        If this value is
        'ieee8021StreamIdSrcMacVlan  ieee8021StreamIdObjects 2' then
        an entry exists in the
        ieee8021StreamIdStreamIdentificationTable
        which corresponds to the Source MAC and VLAN Stream
        identification.

        If this value is
        'ieee8021StreamIdActiveDestMacVlan  ieee8021StreamIdObjects 3'
        then an entry exists in the
        ieee8021StreamIdStreamIdentificationTable
        which corresponds to the Active Destination MAC and VLAN Stream
        identification.

        If this value is
        'ieee8021StreamIdIpStream  ieee8021StreamIdObjects 4' then
        an entry exists in the
        ieee8021StreamIdStreamIdentificationTable
        which corresponds to the IP Stream identification.

        If this value is
        'ieee8021StreamIdMaskAndMatch  ieee8021StreamIdObjects 5' then
        an entry exists in the
        ieee8021StreamIdStreamIdentificationTable
        which corresponds to the Mask-and-match Stream
        identification."
    ::= { ieee8021StreamIdStreamIdentificationEntry 6 }

ieee8021StreamIdStreamIdHandle OBJECT-TYPE
    SYNTAX      Unsigned32
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "The objects in a given entry of the Stream identity table are
         used to control packets whose stream_handle subparameter is
         equal to the entry's tsnStreamIdHandle object."
    REFERENCE   "9.1.1.1"
    ::= { ieee8021StreamIdStreamIdentificationEntry 7 }

ieee8021StreamIdStreamIdInFacOutputPortList OBJECT-TYPE
    SYNTAX      AutonomousType
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
        "The list of ports on which an in-facing Stream identification
         function using this identification method is to be placed for
         this Stream in the output (towards the system forwarding
         function) direction.
         For each port in this list, there exists an entry in the
         ieee8021StreamIdStreamIdInFacOutputPortHandleListTable."
    REFERENCE   "9.1.1.2"
    ::= { ieee8021StreamIdStreamIdentificationEntry 8 }

ieee8021StreamIdStreamIdOutFacOutputPortList OBJECT-TYPE
    SYNTAX      AutonomousType
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
        "The list of ports on which an out-facing Stream identification
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
         function using this identification method is to be placed for
         this Stream in the output (towards the physical interface)
         direction.
         For each port in this list, there exists an entry in the
         ieee8021StreamIdStreamIdOutFacOutputPortHandleListTable."
    REFERENCE   "9.1.1.3"
    ::= { ieee8021StreamIdStreamIdentificationEntry 9 }

ieee8021StreamIdStreamIdInFacInputPortList OBJECT-TYPE
    SYNTAX      AutonomousType
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
        "The list of ports on which an in-facing Stream identification
         function using this identification method is to be placed for
         this Stream in the input (coming from the system forwarding
         function) direction.
         For each port in this list, there exists an entry in the
         ieee8021StreamIdStreamIdInFacInputPortHandleListTable."
    REFERENCE   "9.1.1.4"
    ::= { ieee8021StreamIdStreamIdentificationEntry 10 }

ieee8021StreamIdStreamIdOutFacInputPortList OBJECT-TYPE
    SYNTAX      AutonomousType
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
        "The list of ports on which an out-facing Stream identification
         function using this identification method is to be placed for
         this Stream in the input (coming from the physical interface)
         direction.
         For each port in this list, there exists an entry in the
         ieee8021StreamIdStreamIdOutFacInputPortHandleListTable."
    REFERENCE   "9.1.1.5"
    ::= { ieee8021StreamIdStreamIdentificationEntry 11 }

ieee8021StreamIdAutoConfigured OBJECT-TYPE
    SYNTAX      TruthValue
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
        "A read-only Boolean value, supplied by the system, specifying
         whether this entry was created explicitly (False) or via the
         Sequence autoconfiguration table (True)."
    REFERENCE   "10.2.1"
    ::= { ieee8021StreamIdStreamIdentificationEntry 12 }

ieee8021StreamIdLanPathId OBJECT-TYPE
    SYNTAX      Ieee8021CBLanPathIdType
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
        "An integer specifying a path or LAN. If and only if a packet
         matches an entry in the Sequence identification table that
         specifies HSR or PRP in its frerSeqEncEncapsType object,
         tsnStreamIdLanPathId specifies the LanId or PathId value that
         must be matched for this tsnStreamIdEntry to apply. A value
         of -1 indicates that the LanId or PathId are to be ignored."
    REFERENCE   "10.2.2"
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
    ::= { ieee8021StreamIdStreamIdentificationEntry 13 }

ieee8021StreamIdStatus OBJECT-TYPE
    SYNTAX      RowStatus
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "Row-Status for the Stream Identity table."
    ::= { ieee8021StreamIdStreamIdentificationEntry 14 }

-- ============================================================
-- the ieee8021StreamIdNullStreamIdentificationTable
-- ============================================================

ieee8021StreamIdNullStreamIdentificationTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF Ieee8021StreamIdNullStreamIdentificationEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A table containing a set of controlling parameters for the
         Null Stream identification method."
    REFERENCE   "9.1.2"
    ::= { ieee8021StreamIdStreamIdentification 3 }

ieee8021StreamIdNullStreamIdentificationEntry OBJECT-TYPE
    SYNTAX      Ieee8021StreamIdNullStreamIdentificationEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A set of managed objects that serve as the Stream
         identification parameters when using the Null Stream
         identification method."
    REFERENCE   "9.1.2"
    INDEX  { ieee8021StreamIdStreamIdentificationIndex }
    ::= { ieee8021StreamIdNullStreamIdentificationTable 1 }

Ieee8021StreamIdNullStreamIdentificationEntry ::=
    SEQUENCE {
        ieee8021StreamIdCpeNullDownDestMac
            MacAddress,
        ieee8021StreamIdCPENullDownTagged
            Ieee8021CBTaggedType,
        ieee8021StreamIdCpeNullDownVlan
            Ieee8021CBVlanIdentifier
    }

ieee8021StreamIdCpeNullDownDestMac OBJECT-TYPE
    SYNTAX      MacAddress
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "Specifies the destination_address that identifies a packet in
         an EISS indication primitive, to the Null Stream identification
         function."
    REFERENCE   "9.1.2.1"
    ::= { ieee8021StreamIdNullStreamIdentificationEntry 1 }

ieee8021StreamIdCPENullDownTagged OBJECT-TYPE
    SYNTAX      Ieee8021CBTaggedType
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "An enumerated value indicating whether a packet in an EISS
         indication primitive to the Null Stream identification function
         is permitted to have a VLAN tag."
    REFERENCE   "9.1.2.2"
    ::= { ieee8021StreamIdNullStreamIdentificationEntry 2 }

ieee8021StreamIdCpeNullDownVlan OBJECT-TYPE
    SYNTAX      Ieee8021CBVlanIdentifier
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "Specifies the vlan_identifier parameter that identifies a
         packet in an EISS indication primitive to the Null Stream
         identification function. A value of 0 indicates that the
         vlan_identifier parameter is ignored on EISS indication
         primitives."
    REFERENCE   "9.1.2.3"
    ::= { ieee8021StreamIdNullStreamIdentificationEntry 3 }

-- ============================================================
-- the ieee8021StreamIdSrcMacVlanIdentificationTable
-- ============================================================

ieee8021StreamIdSrcMacVlanIdentificationTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF Ieee8021StreamIdSrcMacVlanIdentificationEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A table containing a set of controlling parameters for the
         Source MAC and VLAN Stream identification method."
    REFERENCE   "9.1.3"
    ::= { ieee8021StreamIdStreamIdentification 4 }

ieee8021StreamIdSrcMacVlanIdentificationEntry OBJECT-TYPE
    SYNTAX      Ieee8021StreamIdSrcMacVlanIdentificationEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A set of managed objects that serve as the Stream
         identification parameters when using the Source MAC and VLAN
         Stream identification method."
    REFERENCE   "9.1.3"
    INDEX  { ieee8021StreamIdStreamIdentificationIndex }
    ::= { ieee8021StreamIdSrcMacVlanIdentificationTable 1 }

Ieee8021StreamIdSrcMacVlanIdentificationEntry ::=
    SEQUENCE {
        ieee8021StreamIdCpeSmacVlanDownSrcMac
            MacAddress,
        ieee8021StreamIdCpeSmacVlanDownTagged
            Ieee8021CBTaggedType,
        ieee8021StreamIdCpeSmacVlanDownVlan
            Ieee8021CBVlanIdentifier
    }

ieee8021StreamIdCpeSmacVlanDownSrcMac OBJECT-TYPE
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
    SYNTAX      MacAddress
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "Specifies the source_address that identifies a packet in an
         EISS indication primitive, to the Source MAC and VLAN Stream
         identification function."
    REFERENCE   "9.1.3.1"
    ::= { ieee8021StreamIdSrcMacVlanIdentificationEntry 1 }

ieee8021StreamIdCpeSmacVlanDownTagged OBJECT-TYPE
    SYNTAX      Ieee8021CBTaggedType
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "An enumerated value indicating whether a packet in an EISS
         indication primitive to the Source MAC and VLAN Stream
         identification function is permitted to have a VLAN tag."
    REFERENCE   "9.1.3.2"
    ::= { ieee8021StreamIdSrcMacVlanIdentificationEntry 2 }

ieee8021StreamIdCpeSmacVlanDownVlan OBJECT-TYPE
    SYNTAX      Ieee8021CBVlanIdentifier
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "Specifies the vlan_identifier parameter that identifies a
         packet in an EISS indication primitive to the Source MAC and
         VLAN Stream identification function. A value of 0 indicates
         that the vlan_identifier parameter is ignored on EISS
         indication primitives."
    REFERENCE   "9.1.3.3"
    ::= { ieee8021StreamIdSrcMacVlanIdentificationEntry 3 }

-- ============================================================
-- the ieee8021StreamIdActiveDestMacVlanIdentificationTable
-- ============================================================

ieee8021StreamIdActiveDestMacVlanIdentificationTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF Ieee8021StreamIdActiveDestMacVlanIdentificationEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A table containing a set of controlling parameters for the
         Active Destination MAC and VLAN Stream identification method."
    REFERENCE   "9.1.4"
    ::= { ieee8021StreamIdStreamIdentification 5 }

ieee8021StreamIdActiveDestMacVlanIdentificationEntry OBJECT-TYPE
    SYNTAX      Ieee8021StreamIdActiveDestMacVlanIdentificationEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A set of managed objects that serve as the Stream
         identification parameters when using the Active Destination
         MAC and VLAN Stream identification method."
    REFERENCE   "9.1.4"
    INDEX  { ieee8021StreamIdStreamIdentificationIndex }
    ::= { ieee8021StreamIdActiveDestMacVlanIdentificationTable 1 }
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
Ieee8021StreamIdActiveDestMacVlanIdentificationEntry ::=
    SEQUENCE {
        ieee8021StreamIdCpeDmacVlanDownDestMac
            MacAddress,
        ieee8021StreamIdCpeDmacVlanDownTagged
            Ieee8021CBTaggedType,
        ieee8021StreamIdCpeDmacVlanDownVlan
            Ieee8021CBVlanIdentifier,
        ieee8021StreamIdCpeDmacVlanDownPriority
            IEEE8021PriorityValue,
        ieee8021StreamIdCpeDmacVlanUpDestMac
            MacAddress,
        ieee8021StreamIdCpeDmacVlanUpTagged
            Ieee8021CBTaggedType,
        ieee8021StreamIdCpeDmacVlanUpVlan
            Ieee8021CBVlanIdentifier,
        ieee8021StreamIdCpeDmacVlanUpPriority
            IEEE8021PriorityValue
    }

ieee8021StreamIdCpeDmacVlanDownDestMac OBJECT-TYPE
    SYNTAX      MacAddress
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "Specifies the destination_address parameter to use in the
         EISS request primitive for output packets sent to lower layers
         by the Active Destination MAC and VLAN Stream identification
         function, and the destination_address that identifies an input
         packet in an EISS indication primitive to the Active
         Destination MAC and VLAN Stream identification function."
    REFERENCE   "9.1.4.1"
    ::= { ieee8021StreamIdActiveDestMacVlanIdentificationEntry 1 }

ieee8021StreamIdCpeDmacVlanDownTagged OBJECT-TYPE
    SYNTAX      Ieee8021CBTaggedType
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "An enumerated value indicating whether a packet in an EISS
         indication or request primitive between the Active Destination
         MAC and VLAN Stream identification function and the lower
         layers is to have a VLAN tag.
         This variable is not used in an FRER C-component."
    REFERENCE   "9.1.4.2"
    ::= { ieee8021StreamIdActiveDestMacVlanIdentificationEntry 2 }

ieee8021StreamIdCpeDmacVlanDownVlan OBJECT-TYPE
    SYNTAX      Ieee8021CBVlanIdentifier
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "Specifies the vlan_identifier parameter to use in the EISS
         request primitive for output packets sent to lower layers by
         the Active Destination MAC and VLAN Stream identification
         function, and the vlan_identifier that identifies an input
         packet in an EISS indication primitive to the Active
         Destination MAC and VLAN Stream identification function.
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
         A value of 0 indicates that the vlan_identifier parameter is
         ignored on EISS indication primitives."
    REFERENCE   "9.1.4.3"
    ::= { ieee8021StreamIdActiveDestMacVlanIdentificationEntry 3 }

ieee8021StreamIdCpeDmacVlanDownPriority OBJECT-TYPE
    SYNTAX      IEEE8021PriorityValue
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "Specifies the priority parameter to use in the EISS request
         primitive for output packets sent to lower layers by the Active
         Destination MAC and VLAN Stream identification function for all
         packets in a particular Stream."
    REFERENCE   "9.1.4.4"
    ::= { ieee8021StreamIdActiveDestMacVlanIdentificationEntry 4 }

ieee8021StreamIdCpeDmacVlanUpDestMac OBJECT-TYPE
    SYNTAX      MacAddress
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "Specifies the destination_address parameter to use in the EISS
         indication primitive for input packets offered to upper layers
         by the Active Destination MAC and VLAN Stream identification
         layer. This address replaces the address that was used to
         identify the packet (tsnCpeDmacVlanDownDestMac)."
    REFERENCE   "9.1.4.5"
    ::= { ieee8021StreamIdActiveDestMacVlanIdentificationEntry 5 }

ieee8021StreamIdCpeDmacVlanUpTagged OBJECT-TYPE
    SYNTAX      Ieee8021CBTaggedType
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "An enumerated value indicating whether a packet in an EISS
         indication or request primitive between the Active Destination
         MAC and VLAN Stream identification function and the upper
         layers is to have a VLAN tag.
         This variable is used only by an end system and not by a
         relay system."
    REFERENCE   "9.1.4.6"
    ::= { ieee8021StreamIdActiveDestMacVlanIdentificationEntry 6 }

ieee8021StreamIdCpeDmacVlanUpVlan OBJECT-TYPE
    SYNTAX      Ieee8021CBVlanIdentifier
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "Specifies the vlan_identifier parameter to use in the EISS
         indication primitive for packets offered to upper layers, or
         the VLAN ID field for an IEEE 802.1Q tag in an
         ISS mac_service_data_unit. This address replaces the VLAN ID
         that was used to identify the packet (tsnCpeDmacVlanDownVlan)."
    REFERENCE   "9.1.4.7"
    ::= { ieee8021StreamIdActiveDestMacVlanIdentificationEntry 7 }

ieee8021StreamIdCpeDmacVlanUpPriority OBJECT-TYPE
    SYNTAX      IEEE8021PriorityValue
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "Specifies the priority parameter to use in the EISS indication
         primitive for packets offered to upper layers."
    REFERENCE   "9.1.4.8"
    ::= { ieee8021StreamIdActiveDestMacVlanIdentificationEntry 8 }

-- ============================================================
-- the ieee8021StreamIdIpStreamIdentificationTable
-- ============================================================

ieee8021StreamIdIpStreamIdentificationTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF Ieee8021StreamIdIpStreamIdentificationEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A table containing a set of controlling parameters for the
         IP Stream identification method."
    REFERENCE   "9.1.5"
    ::= { ieee8021StreamIdStreamIdentification 6 }

ieee8021StreamIdIpStreamIdentificationEntry OBJECT-TYPE
    SYNTAX      Ieee8021StreamIdIpStreamIdentificationEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A set of managed objects that serve as the Stream
         identification parameters when using the IP Stream
         identification method."
    REFERENCE   "9.1.5"
    INDEX  { ieee8021StreamIdStreamIdentificationIndex }
    ::= { ieee8021StreamIdIpStreamIdentificationTable 1 }

Ieee8021StreamIdIpStreamIdentificationEntry ::=
    SEQUENCE {
        ieee8021StreamIdCpeIpIdDestMac
            MacAddress,
        ieee8021StreamIdCpeIpIdTagged
            Ieee8021CBTaggedType,
        ieee8021StreamIdCpeIpIdVlan
            Ieee8021CBVlanIdentifier,
        ieee8021StreamIdCpeIpIdIpSourceType
            InetAddressType,
        ieee8021StreamIdCpeIpIdIpSource
            InetAddress,
        ieee8021StreamIdCpeIpIdIpDestinationType
            InetAddressType,
        ieee8021StreamIdCpeIpIdIpDestination
            InetAddress,
        ieee8021StreamIdCpeIpIdDscp
            Ieee8021CBIpDscpType,
        ieee8021StreamIdCpeIpIdNextProtocol
            Ieee8021CBIdNextProtocolType,
        ieee8021StreamIdCpeIpIdSourcePort
            InetPortNumber,
        ieee8021StreamIdCpeIpIdDestinationPort
            InetPortNumber
    }
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
ieee8021StreamIdCpeIpIdDestMac OBJECT-TYPE
    SYNTAX      MacAddress
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "Specifies the destination_address parameter that identifies a
         packet in an EISS indication primitive. A value of all 0 indicates
         that the MAC destination address is to be ignored on packets
         received from lower layers."
    REFERENCE   "9.1.5.1"
    ::= { ieee8021StreamIdIpStreamIdentificationEntry 1 }

ieee8021StreamIdCpeIpIdTagged OBJECT-TYPE
    SYNTAX      Ieee8021CBTaggedType
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "An enumerated value indicating whether a packet in an EISS
         indication or request primitive to the IP Stream identification
         function is to have a VLAN tag."
    REFERENCE   "9.1.5.2"
    ::= { ieee8021StreamIdIpStreamIdentificationEntry 2 }

ieee8021StreamIdCpeIpIdVlan OBJECT-TYPE
    SYNTAX      Ieee8021CBVlanIdentifier
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "Specifies the vlan_identifier parameter that identifies a
         packet in an EISS indication primitive. A value of 0 indicates
         that the frame is not to have a VLAN tag.A value of 0 indicates
         that the vlan_identifier parameter is ignored on EISS
         indication primitives."
    REFERENCE   "9.1.5.3"
    ::= { ieee8021StreamIdIpStreamIdentificationEntry 3 }

ieee8021StreamIdCpeIpIdIpSourceType OBJECT-TYPE
    SYNTAX      InetAddressType
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "Specifies the type of the source address parameter supplied
         in ieee8021StreamIdCpeIpIdIpSource."
    REFERENCE   "9.1.5.4"
    ::= { ieee8021StreamIdIpStreamIdentificationEntry 4 }

ieee8021StreamIdCpeIpIdIpSource OBJECT-TYPE
    SYNTAX      InetAddress
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "Specifies the IPv4 (RFC 791) or IPv6 (RFC 2460) source address
         parameter that must be matched to identify packets coming up
         from lower layers. An addressA value of all 0 indicates that the
         IP source address is to be ignored on packets received from
         lower layers."
    REFERENCE   "9.1.5.4"
    ::= { ieee8021StreamIdIpStreamIdentificationEntry 5 }
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
ieee8021StreamIdCpeIpIdIpDestinationType OBJECT-TYPE
    SYNTAX      InetAddressType
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "Specifies the type of the destination address parameter
         supplied in ieee8021StreamIdCpeIpIdIpDestination."
    REFERENCE   "9.1.5.4"
    ::= { ieee8021StreamIdIpStreamIdentificationEntry 6 }

ieee8021StreamIdCpeIpIdIpDestination OBJECT-TYPE
    SYNTAX      InetAddress
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "Specifies the IPv4 (RFC 791) or IPv6 (RFC 2460) destination
        address parameter that must be matched to identify packets
        coming up from lower layers."
    REFERENCE   "9.1.5.5"
    ::= { ieee8021StreamIdIpStreamIdentificationEntry 7 }

ieee8021StreamIdCpeIpIdDscp OBJECT-TYPE
    SYNTAX      Ieee8021CBIpDscpType
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "Specifies the IPv4 (RFC 791) or IPv6 (RFC 2460) differentiated
         services codepoint (DSCP, RFC 2474) that must be matched to
         identify packets coming up from the lower layers. A value of
         64 decimal indicates that the DSCP is to be ignored on packets
         received from lower layers."
    REFERENCE   "9.1.5.6"
    ::= { ieee8021StreamIdIpStreamIdentificationEntry 8 }

ieee8021StreamIdCpeIpIdNextProtocol OBJECT-TYPE
    SYNTAX      Ieee8021CBIdNextProtocolType
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "Specifies the IP next protocol parameter that must be matched
         to identify packets coming up from lower layers. The value of
         this parameter must specify either none, UDP (RFC 768),
         TCP (RFC 793), or SCTP (RFC 4960). If 'none', then the
         tsnCpeIpIdSourcePort and tsnCpeIpIdDestinationPort managed
         objects are not used."
    REFERENCE   "9.1.5.7"
    ::= { ieee8021StreamIdIpStreamIdentificationEntry 9 }

ieee8021StreamIdCpeIpIdSourcePort OBJECT-TYPE
    SYNTAX      InetPortNumber
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "Specifies the TCP or UDP Source Port parameter that must be
         matched to identify packets coming up from lower layers.
         A value of 0 indicates that the Source Port number of the
         packet is to be ignored on packets received from lower layers."
    REFERENCE   "9.1.5.8"
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
    ::= { ieee8021StreamIdIpStreamIdentificationEntry 10 }

ieee8021StreamIdCpeIpIdDestinationPort OBJECT-TYPE
    SYNTAX      InetPortNumber
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "Specifies the TCP or UDP Destination Port parameter that must
         be matched to identify packets coming up from lower layers.
         A value of 0 indicates that the Destination Port number of the
         packet is to be ignored on packets received from lower layers."
    REFERENCE   "9.1.5.9"
    ::= { ieee8021StreamIdIpStreamIdentificationEntry 11 }

-- ============================================================
-- the ieee8021StreamIdMaskAndMatchIdentificationTable
-- ============================================================

ieee8021StreamIdMaskAndMatchIdentificationTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF Ieee8021StreamIdMaskAndMatchIdentificationEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A table containing a set of controlling parameters for the
         Mask-and-match Stream identification method."
    REFERENCE   "9.1.6"
    ::= { ieee8021StreamIdStreamIdentification 7 }

ieee8021StreamIdMaskAndMatchIdentificationEntry OBJECT-TYPE
    SYNTAX      Ieee8021StreamIdMaskAndMatchIdentificationEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A set of managed objects that serve as the Stream
         identification parameters when using the Mask-and-match Stream
         identification method."
    REFERENCE   "9.1.6"
    INDEX  { ieee8021StreamIdStreamIdentificationIndex }
    ::= { ieee8021StreamIdMaskAndMatchIdentificationTable 1 }

Ieee8021StreamIdMaskAndMatchIdentificationEntry ::=
    SEQUENCE {
        ieee8021StreamIdCpeMmIdDestMacMask
            MacAddress,
        ieee8021StreamIdCpeMmIdDestMacMatch
            MacAddress,
        ieee8021StreamIdCpeMmIdSrcMacMask
            MacAddress,
        ieee8021StreamIdCpeMmIdSrcMacMatch
            MacAddress,
        ieee8021StreamIdCpeMmIdMsduMaskLength
            Integer32,
        ieee8021StreamIdCpeMmIdMsduMask
            OCTET STRING,
        ieee8021StreamIdCpeMmIdMsduMatch
            OCTET STRING
    }

ieee8021StreamIdCpeMmIdDestMacMask OBJECT-TYPE
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
    SYNTAX      MacAddress
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "Specifies a 48-bit mask. A bitwise AND operation is performed
         between MmIdDestMacMask and the destination_address parameter
         passed by the ISS indication primitive to the Mask-and-match
         Stream identification function. The resulting 48-bit
         information is the masked destination_address that is used as
         input for the instance of the Mask-and-match Stream
         identification function. If MmIdDestMacMask has a value of 0,
         the destination_address parameter is ignored."
    REFERENCE   "9.1.6.1"
    ::= { ieee8021StreamIdMaskAndMatchIdentificationEntry 1 }

ieee8021StreamIdCpeMmIdDestMacMatch OBJECT-TYPE
    SYNTAX      MacAddress
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "Specifies the 48-bit value of the masked destination_address,
         as defined in 9.1.6.1, to be matched by the instance of the
         Mask-and-match Stream identification function."
    REFERENCE   "9.1.6.2"
    ::= { ieee8021StreamIdMaskAndMatchIdentificationEntry 2 }

ieee8021StreamIdCpeMmIdSrcMacMask OBJECT-TYPE
    SYNTAX      MacAddress
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "Specifies a 48-bit mask. A bitwise AND operation is performed
         between CpeMmIdSrcMacMask and the source_address parameter
         passed by the ISS indication primitive to the Mask-and-match
         Stream identification function. The resulting 48-bit
         information is the masked source_address that is used as input
         for the instance of the Mask-and-match Stream identification
         function. If CpeMmIdSrcMacMask has a value of 0, the
         source_address parameter is ignored."
    REFERENCE   "9.1.6.3"
    ::= { ieee8021StreamIdMaskAndMatchIdentificationEntry 3 }

ieee8021StreamIdCpeMmIdSrcMacMatch OBJECT-TYPE
    SYNTAX      MacAddress
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "Specifies the 48-bit value of the masked source_address, as
         defined in 9.1.6.3, to be matched by the instance of the
         Mask-and-match Stream identification function."
    REFERENCE   "9.1.6.4"
    ::= { ieee8021StreamIdMaskAndMatchIdentificationEntry 4 }

ieee8021StreamIdCpeMmIdMsduMaskLength OBJECT-TYPE
    SYNTAX      Integer32(2..1984)
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "Is an integer value that specifies the length, in octets, of
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
        CpeMmIdMsduMask (9.1.6.6) and CpeMmIdMsduMatch (9.1.6.7). The
        behavior of the Mask-and-match Stream identification function
        is undefined on any port where the value of
        CpeMmIdMsduMaskLength exceeds the per-port
        mac_service_data_unit mask maximum length(9.4.1). If the
        length of the frame's mac_service_data_unit is shorter than
        CpeMmIdMsduMaskLength, then the identification fails."
    REFERENCE   "9.1.6.5"
    ::= { ieee8021StreamIdMaskAndMatchIdentificationEntry 5 }

ieee8021StreamIdCpeMmIdMsduMask OBJECT-TYPE
    SYNTAX      OCTET STRING (SIZE(2..1984))
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "Specifies a mask of CpeMmIdMsduMaskLength octets. A bitwise
        AND operation is performed between CpeMmIdMsduMask and the
        CpeMmIdMsduMaskLength (9.1.6.6) first octets of the
        mac_service_data_unit parameter passed by the ISS indication
        primitive to the Mask-and-match Stream identification
        function. The resulting CpeMmIdMsduMaskLength octets of
        information are the masked mac_service_data_unit that is used
        as input for the instance of the Mask-and-match Stream
        identification function. If CpeMmIdMsduMask has a value of 0,
        the mac_service_data_unit parameter is ignored."
    REFERENCE   "9.1.6.6"
    ::= { ieee8021StreamIdMaskAndMatchIdentificationEntry 6 }

ieee8021StreamIdCpeMmIdMsduMatch OBJECT-TYPE
    SYNTAX      OCTET STRING (SIZE(2..1984))
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "Specifies the value of the masked mac_service_data_unit, as
        defined in 9.1.6.5 and 9.1.6.6, to be matched by the instance
        of the Mask-and-match Stream identification function."
    REFERENCE   "9.1.6.7"
    ::= { ieee8021StreamIdMaskAndMatchIdentificationEntry 7 }

-- ================================================================
-- the ieee8021StreamIdStreamIdInFacOutputPortHandleList
-- table
-- ================================================================

ieee8021StreamIdStreamIdInFacOutputPortHandleListTable OBJECT-TYPE
                        SYNTAX                                          SEQUENCE      OF
Ieee8021StreamIdStreamIdInFacOutputPortHandleListEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A table containing a list of ports on which an in-facing Stream
        identification function using this identification method is to
        be placed for this Stream in the output (towards the system
        forwarding function) direction, referenced in
        ieee8021StreamIdStreamIdentificationIndex."
    REFERENCE   "9.1.1.2"
    ::= { ieee8021StreamIdStreamIdInFacOutputPortHandleList 2 }

ieee8021StreamIdStreamIdInFacOutputPortHandleListEntry OBJECT-TYPE
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
    SYNTAX      Ieee8021StreamIdStreamIdInFacOutputPortHandleListEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A set of managed objects providing the ports on which an
         in-facing Stream identification function using this
         identification method is to be placed for this Stream in the
         output (towards the system forwarding function) direction,
         referenced in ieee8021StreamIdStreamIdentificationIndex."
    REFERENCE   "9.1.1.2"
    INDEX   { ieee8021StreamIdStreamIdentificationIndex,
              ieee8021StreamIdStreamIdInFacOutputPortHandleListIndex
            }
    ::= { ieee8021StreamIdStreamIdInFacOutputPortHandleListTable 1 }

Ieee8021StreamIdStreamIdInFacOutputPortHandleListEntry ::=
    SEQUENCE {
        ieee8021StreamIdStreamIdInFacOutputPortHandleListIndex
            Unsigned32,
        ieee8021StreamIdStreamIdInFacOutputPortHandle
            VariablePointer,
        ieee8021StreamIdStreamIdInFacOutputPortHandleListStatus
            RowStatus
    }

ieee8021StreamIdStreamIdInFacOutputPortHandleListIndex OBJECT-TYPE
    SYNTAX      Unsigned32
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "Index for the In-Facing Output Port  handle list table."
    ::= { ieee8021StreamIdStreamIdInFacOutputPortHandleListEntry 1 }

ieee8021StreamIdStreamIdInFacOutputPortHandle OBJECT-TYPE
    SYNTAX      VariablePointer
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "A pointer to an interface that is an element of the
         ieee8021StreamIdStreamIdInFacOutputPortList instance in the
         ieee8021StreamIdStreamIdentificationTable."
    REFERENCE   "9.1.1.2"
    ::= { ieee8021StreamIdStreamIdInFacOutputPortHandleListEntry 2 }

ieee8021StreamIdStreamIdInFacOutputPortHandleListStatus OBJECT-TYPE
    SYNTAX      RowStatus
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "Row-Status for the In-Facing Output Port  handle list table."
    ::= { ieee8021StreamIdStreamIdInFacOutputPortHandleListEntry 3 }

-- ============================================================
-- the ieee8021StreamIdStreamIdOutFacOutputPortList
-- table
-- ============================================================

ieee8021StreamIdStreamIdOutFacOutputPortHandleListTable OBJECT-TYPE
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
                        SYNTAX                                          SEQUENCE       OF
Ieee8021StreamIdStreamIdOutFacOutputPortHandleListEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A table containing a list of ports on which an out-facing
         Stream identification function using this identification method
         is to be placed for this Stream in the output (towards the
         system forwarding function) direction, referenced in
         ieee8021StreamIdStreamIdentificationIndex."
    REFERENCE   "9.1.1.3"
    ::= { ieee8021StreamIdStreamIdOutFacOutputPortHandleList 3 }

ieee8021StreamIdStreamIdOutFacOutputPortHandleListEntry OBJECT-TYPE
    SYNTAX      Ieee8021StreamIdStreamIdOutFacOutputPortHandleListEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A set of managed objects providing the ports on which an
         out-facing Stream identification function using this
         identification method is to be placed for this Stream in the
         output (towards the system forwarding function) direction,
         referenced in ieee8021StreamIdStreamIdentificationIndex."
    REFERENCE   "9.1.1.3"
    INDEX   { ieee8021StreamIdStreamIdentificationIndex,
              ieee8021StreamIdStreamIdOutFacOutputPortHandleListIndex
            }
    ::= { ieee8021StreamIdStreamIdOutFacOutputPortHandleListTable 1 }

Ieee8021StreamIdStreamIdOutFacOutputPortHandleListEntry ::=
    SEQUENCE {
        ieee8021StreamIdStreamIdOutFacOutputPortHandleListIndex
            Unsigned32,
        ieee8021StreamIdStreamIdOutFacOutputPortHandle
            VariablePointer,
        ieee8021StreamIdStreamIdOutFacOutputPortHandleListStatus
            RowStatus
    }

ieee8021StreamIdStreamIdOutFacOutputPortHandleListIndex OBJECT-TYPE
    SYNTAX      Unsigned32
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "Index for the Out-Facing Output Port  handle list table."
    ::= { ieee8021StreamIdStreamIdOutFacOutputPortHandleListEntry 1 }

ieee8021StreamIdStreamIdOutFacOutputPortHandle OBJECT-TYPE
    SYNTAX      VariablePointer
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "A pointer to an interface that is an element of the
         ieee8021StreamIdStreamIdOutFacOutputPortList instance in the
         ieee8021StreamIdStreamIdentificationTable."
    REFERENCE   "9.1.1.3"
    ::= { ieee8021StreamIdStreamIdOutFacOutputPortHandleListEntry 2 }

ieee8021StreamIdStreamIdOutFacOutputPortHandleListStatus OBJECT-TYPE
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
    SYNTAX      RowStatus
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "Row-Status for the Out-Facing Output Port  handle list table."
    ::= { ieee8021StreamIdStreamIdOutFacOutputPortHandleListEntry 3 }

-- ================================================================
-- the ieee8021StreamIdStreamIdInFacInputPortList
-- table
-- ================================================================

ieee8021StreamIdStreamIdInFacInputPortHandleListTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF Ieee8021StreamIdStreamIdInFacInputPortHandleListEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A table containing a list of ports on which an in-facing Stream
         identification function using this identification method is to
         be placed for this Stream in the input (coming from the system
         forwarding function) direction, referenced in
         ieee8021StreamIdStreamIdentificationIndex."
    REFERENCE   "9.1.1.4"
    ::= { ieee8021StreamIdStreamIdInFacInputPortHandleList 4 }

ieee8021StreamIdStreamIdInFacInputPortHandleListEntry OBJECT-TYPE
    SYNTAX      Ieee8021StreamIdStreamIdInFacInputPortHandleListEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A set of managed objects providing the ports on which an
         in-facing Stream identification function using this
         identification method is to be placed for this Stream in the
         input (coming from the system forwarding function) direction,
         referenced in ieee8021StreamIdStreamIdentificationIndex."
    REFERENCE   "9.1.1.4"
    INDEX   { ieee8021StreamIdStreamIdentificationIndex,
              ieee8021StreamIdStreamIdInFacInputPortHandleListIndex
            }
    ::= { ieee8021StreamIdStreamIdInFacInputPortHandleListTable 1 }

Ieee8021StreamIdStreamIdInFacInputPortHandleListEntry ::=
    SEQUENCE {
        ieee8021StreamIdStreamIdInFacInputPortHandleListIndex
            Unsigned32,
        ieee8021StreamIdStreamIdInFacInputPortHandle
            VariablePointer,
        ieee8021StreamIdStreamIdInFacInputPortHandleListStatus
            RowStatus
    }

ieee8021StreamIdStreamIdInFacInputPortHandleListIndex OBJECT-TYPE
    SYNTAX      Unsigned32
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "Index for the In-Facing Input Port handle list table."
    ::= { ieee8021StreamIdStreamIdInFacInputPortHandleListEntry 1 }

```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
ieee8021StreamIdStreamIdInFacInputPortHandle OBJECT-TYPE
    SYNTAX      VariablePointer
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "A pointer to an interface that is an element of the
         ieee8021StreamIdStreamIdOutFacInputPortList instance in the
         ieee8021StreamIdStreamIdentificationTable."
    REFERENCE   "9.1.1.4"
    ::= { ieee8021StreamIdStreamIdInFacInputPortHandleListEntry 2 }

ieee8021StreamIdStreamIdInFacInputPortHandleListStatus OBJECT-TYPE
    SYNTAX      RowStatus
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "Row-Status for the In-Facing Input Port handle list table."
    ::= { ieee8021StreamIdStreamIdInFacInputPortHandleListEntry 3 }

-- ================================================================
-- the ieee8021StreamIdStreamIdOutFacInputPortList
-- table
-- ================================================================

ieee8021StreamIdStreamIdOutFacInputPortHandleListTable OBJECT-TYPE
                        SYNTAX                                          SEQUENCE      OF
Ieee8021StreamIdStreamIdOutFacInputPortHandleListEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A table containing a list of ports on which an out-facing
         Stream identification function using this identification method
         is to be placed for this Stream in the input (coming from the
         physical interface) direction, referenced in
         ieee8021StreamIdStreamIdentificationIndex."
    REFERENCE   "9.1.1.5"
    ::= { ieee8021StreamIdStreamIdOutFacInputPortHandleList 5 }

ieee8021StreamIdStreamIdOutFacInputPortHandleListEntry OBJECT-TYPE
    SYNTAX      Ieee8021StreamIdStreamIdOutFacInputPortHandleListEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A set of managed objects providing the ports on which an
         out-facing Stream identification function using this
         identification method is to be placed for this Stream in the
         input (coming from the physical interface) direction,
         referenced in ieee8021StreamIdStreamIdentificationIndex."
    REFERENCE   "9.1.1.5"
    INDEX   { ieee8021StreamIdStreamIdentificationIndex,
              ieee8021StreamIdStreamIdOutFacInputPortHandleListIndex
            }
    ::= { ieee8021StreamIdStreamIdOutFacInputPortHandleListTable 1 }

Ieee8021StreamIdStreamIdOutFacInputPortHandleListEntry ::=
    SEQUENCE {
        ieee8021StreamIdStreamIdOutFacInputPortHandleListIndex
            Unsigned32,
        ieee8021StreamIdStreamIdOutFacInputPortHandle
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
            VariablePointer,
        ieee8021StreamIdStreamIdOutFacInputPortHandleListStatus
            RowStatus
    }

ieee8021StreamIdStreamIdOutFacInputPortHandleListIndex OBJECT-TYPE
    SYNTAX      Unsigned32
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "Index for the Out-Facing Input Port handle list table."
    ::= { ieee8021StreamIdStreamIdOutFacInputPortHandleListEntry 1 }

ieee8021StreamIdStreamIdOutFacInputPortHandle OBJECT-TYPE
    SYNTAX      VariablePointer
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "A pointer to an interface that is an element of the
         ieee8021StreamIdStreamIdOutFacInputPortList instance in the
         ieee8021StreamIdStreamIdentificationTable."
    REFERENCE   "9.1.1.5"
    ::= { ieee8021StreamIdStreamIdOutFacInputPortHandleListEntry 2 }

ieee8021StreamIdStreamIdOutFacInputPortHandleListStatus OBJECT-TYPE
    SYNTAX      RowStatus
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "Row-Status for the Out-Facing Input Port handle list table."
    ::= { ieee8021StreamIdStreamIdOutFacInputPortHandleListEntry 3 }


-- ================================================================
-- the ieee8021StreamIdPerPortPerStreamCountersTable
-- ================================================================

ieee8021StreamIdPerPortPerStreamCountersTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF Ieee8021StreamIdPerPortPerStreamCountersEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "A table containing a set of counters for stream
             identification that are instantiated per port, stream and
             direction."
    REFERENCE   "9.2"
    ::= { ieee8021StreamIdPerPortPerStreamCounters 6 }

ieee8021StreamIdPerPortPerStreamCountersEntry OBJECT-TYPE
    SYNTAX      Ieee8021StreamIdPerPortPerStreamCountersEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "A set of managed objects for stream identification
             counters that are instantiated per port, stream and
             direction."
    REFERENCE   "9.2"
     INDEX  { ifIndex,
              ieee8021StreamIdStreamIdHandle,
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
              ieee8021StreamIdPerPortPerStreamDirection
            }
    ::= { ieee8021StreamIdPerPortPerStreamCountersTable 1 }

Ieee8021StreamIdPerPortPerStreamCountersEntry ::=
    SEQUENCE {
        ieee8021StreamIdPerPortPerStreamDirection
            TruthValue,
        ieee8021StreamIdPerPortPerStreamInputPackets
            Counter64,
        ieee8021StreamIdPerPortPerStreamOutputPackets
            Counter64
    }

ieee8021StreamIdPerPortPerStreamDirection OBJECT-TYPE
    SYNTAX      TruthValue
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A managed object specifying the facing of a given stream
         on a port. The stream can be either in-facing (False) or
         out-facing (True)."
    REFERENCE   "9.2"
    ::= { ieee8021StreamIdPerPortPerStreamCountersEntry 1 }

ieee8021StreamIdPerPortPerStreamInputPackets OBJECT-TYPE
    SYNTAX      Counter64
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
        "A managed object serving as a counter that is incremented once
         for each packet identified by the Stream identification
         function."
    REFERENCE   "9.2.1"
    ::= { ieee8021StreamIdPerPortPerStreamCountersEntry 2 }

ieee8021StreamIdPerPortPerStreamOutputPackets OBJECT-TYPE
    SYNTAX      Counter64
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
        "A managed object serving as a counter that is incremented once
         for each packet passed down the stack by the Stream
         identification function."
    REFERENCE   "9.2.2"
    ::= { ieee8021StreamIdPerPortPerStreamCountersEntry 3 }

-- ============================================================
-- the ieee8021StreamIdPerPortCountersTable
-- ============================================================

ieee8021StreamIdPerPortCountersTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF Ieee8021StreamIdPerPortCountersEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "A table containing a set of counters for stream
             identification that are instantiated per port and
             direction."
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
    REFERENCE   "9.3"
    ::= { ieee8021StreamIdPerPortCounters 7 }

ieee8021StreamIdPerPortCountersEntry OBJECT-TYPE
    SYNTAX      Ieee8021StreamIdPerPortCountersEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "A a set of managed objects for stream identification
             counters that are instantiated per port and direction."
    REFERENCE   "9.3"
     INDEX  { ifIndex }
    ::= { ieee8021StreamIdPerPortCountersTable 1 }

Ieee8021StreamIdPerPortCountersEntry ::=
    SEQUENCE {
        ieee8021StreamIdPerPortInputPackets
            Counter64,
        ieee8021StreamIdPerPortOutputPackets
            Counter64
    }

ieee8021StreamIdPerPortInputPackets OBJECT-TYPE
    SYNTAX      Counter64
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
        "A managed object serving as a counter that is incremented once
         for each packet identified by any Stream identification
         function on this port. Its value equals the sum (modulo the
         size of the counters) of all of the
         ieee8021StreamIdPerPortPerStreamInputPackets counters on this
         same port."
    REFERENCE   "9.3.1"
    ::= { ieee8021StreamIdPerPortCountersEntry 1 }

ieee8021StreamIdPerPortOutputPackets OBJECT-TYPE
    SYNTAX      Counter64
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
        "A managed object serving as a counter that is incremented once
         for each packet passed down the stack by any Stream
         identification function on this port. Its value equals the sum
         (modulo the size of the counters) of all of the
         ieee8021StreamIdPerPortPerStreamOutputPackets counters on this
         same port."
    REFERENCE   "9.3.2"
    ::= { ieee8021StreamIdPerPortCountersEntry 2 }

-- ================================================================
-- the ieee8021StreamIdMaskAndMatchPerPortMsduMaskMaxLengthTable
-- ================================================================

ieee8021StreamIdMaskAndMatchPerPortMsduMaskMaxLengthTable OBJECT-TYPE
                        SYNTAX                                      SEQUENCE      OF
Ieee8021StreamIdMaskAndMatchPerPortMsduMaskMaxLengthEntry
    MAX-ACCESS  not-accessible
    STATUS      current
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
    DESCRIPTION
        "A table containing a set of maximum msdu mask length
         parameters that are defined per port."
    REFERENCE   "9.4.1"
    ::= { ieee8021StreamIdMaskAndMatchPerPortMsduMaskMaxLength 8 }

ieee8021StreamIdMaskAndMatchPerPortMsduMaskMaxLengthEntry OBJECT-TYPE
    SYNTAX      Ieee8021StreamIdMaskAndMatchPerPortMsduMaskMaxLengthEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A a set of managed objects specifying the maximum msdu
         mask length per port, applicable to all the instances
         of Mask-And-Match Stream identification functions"
    REFERENCE   "9.4.1"
    INDEX  { ifIndex }
    ::= { ieee8021StreamIdMaskAndMatchPerPortMsduMaskMaxLengthTable 1 }

Ieee8021StreamIdMaskAndMatchPerPortMsduMaskMaxLengthEntry ::=
    SEQUENCE {
        ieee8021StreamIdMaskAndMatchMsduMaskMaxLength
        Integer32
    }

ieee8021StreamIdMaskAndMatchMsduMaskMaxLength OBJECT-TYPE
    SYNTAX      Integer32(2..1984)
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "A managed object specifying the maximum msdu mask length for
         a given port."
    REFERENCE   "9.4.1"
    ::= { ieee8021StreamIdMaskAndMatchPerPortMsduMaskMaxLengthEntry 1 }

-- ================================================================
-- IEEE802 STREAM IDENTIFICATION MIB - Conformance Information
-- ================================================================

ieee8021StreamIdCompliances
    OBJECT IDENTIFIER ::= { ieee8021StreamIdConformance 1 }
ieee8021StreamIdGroups
    OBJECT IDENTIFIER ::= { ieee8021StreamIdConformance 2 }

-- ================================================================
-- units of conformance
-- ================================================================

-- ================================================================
-- the ieee8021StreamIdStreamIdentification group
-- ================================================================

ieee8021StreamIdStreamIdentificationGroup OBJECT-GROUP
    OBJECTS {
        ieee8021StreamIdStreamIdIdentificationTypeSelect,
        ieee8021StreamIdStreamIdIdentificationType,
        ieee8021StreamIdStreamIdIdentificationTypeOUI,
        ieee8021StreamIdStreamIdIdentificationCustomType,
        ieee8021StreamIdStreamIdIdentificationCustomTypeOUI,
        ieee8021StreamIdStreamIdHandle,
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
        ieee8021StreamIdStreamIdInFacOutputPortList,
        ieee8021StreamIdStreamIdOutFacOutputPortList,
        ieee8021StreamIdStreamIdInFacInputPortList,
        ieee8021StreamIdStreamIdOutFacInputPortList,
        ieee8021StreamIdStreamIdInFacOutputPortHandle,
        ieee8021StreamIdStreamIdInFacOutputPortHandleListStatus,
        ieee8021StreamIdStreamIdOutFacOutputPortHandle,
        ieee8021StreamIdStreamIdOutFacOutputPortHandleListStatus,
        ieee8021StreamIdStreamIdInFacInputPortHandle,
        ieee8021StreamIdStreamIdInFacInputPortHandleListStatus,
        ieee8021StreamIdStreamIdOutFacInputPortHandle,
        ieee8021StreamIdStreamIdOutFacInputPortHandleListStatus
    }
    STATUS      current
    DESCRIPTION
        "Objects that are part of the stream identification."
    ::= { ieee8021StreamIdGroups 1 }

-- ================================================================
-- the ieee8021StreamIdNullStreamIdentification group
-- ================================================================

ieee8021StreamIdNullStreamIdentificationGroup OBJECT-GROUP
    OBJECTS {
        ieee8021StreamIdCpeNullDownDestMac,
        ieee8021StreamIdCPENullDownTagged,
        ieee8021StreamIdCpeNullDownVlan
    }
    STATUS      current
    DESCRIPTION
        "Objects that are part of the null stream identification
         method."
    ::= { ieee8021StreamIdGroups 2 }

-- ================================================================
-- the ieee8021StreamIdSrcMacVlanIdentification group
-- ================================================================

ieee8021StreamIdSrcMacVlanIdentificationGroup OBJECT-GROUP
    OBJECTS {
        ieee8021StreamIdCpeSmacVlanDownSrcMac,
        ieee8021StreamIdCpeSmacVlanDownTagged,
        ieee8021StreamIdCpeSmacVlanDownVlan
    }
    STATUS      current
    DESCRIPTION
        "Objects that are part of the Source MAC and VLAN stream
         identification method."
    ::= { ieee8021StreamIdGroups 3 }

-- ================================================================
-- the ieee8021StreamIdActiveDestMacVlanIdentification group
-- ================================================================

ieee8021StreamIdActiveDestMacVlanIdentificationGroup OBJECT-GROUP
    OBJECTS {
        ieee8021StreamIdCpeDmacVlanDownDestMac,
        ieee8021StreamIdCpeDmacVlanDownTagged,
        ieee8021StreamIdCpeDmacVlanDownVlan,
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
        ieee8021StreamIdCpeDmacVlanDownPriority,
        ieee8021StreamIdCpeDmacVlanUpDestMac,
        ieee8021StreamIdCpeDmacVlanUpTagged,
        ieee8021StreamIdCpeDmacVlanUpVlan,
        ieee8021StreamIdCpeDmacVlanUpPriority
    }
    STATUS      current
    DESCRIPTION
        "Objects that are part of the Active Destination MAC and
         VLAN stream identification method."
    ::= { ieee8021StreamIdGroups 4 }

-- ============================================================
-- the ieee8021StreamIdIpStreamIdentification group
-- ============================================================

ieee8021StreamIdIpStreamIdentificationGroup OBJECT-GROUP
    OBJECTS {
        ieee8021StreamIdCpeIpIdDestMac,
        ieee8021StreamIdCpeIpIdTagged,
        ieee8021StreamIdCpeIpIdVlan,
        ieee8021StreamIdCpeIpIdIpSourceType,
        ieee8021StreamIdCpeIpIdIpSource,
        ieee8021StreamIdCpeIpIdIpDestinationType,
        ieee8021StreamIdCpeIpIdIpDestination,
        ieee8021StreamIdCpeIpIdDscp,
        ieee8021StreamIdCpeIpIdNextProtocol,
        ieee8021StreamIdCpeIpIdSourcePort,
        ieee8021StreamIdCpeIpIdDestinationPort
    }
    STATUS      current
    DESCRIPTION
        "Objects that are part of the IP stream identification method."
    ::= { ieee8021StreamIdGroups 5 }

-- ============================================================
-- the ieee8021StreamIdAutoConfiguration group
-- ============================================================

ieee8021StreamIdAutoConfigurationGroup OBJECT-GROUP
    OBJECTS {
        ieee8021StreamIdAutoConfigured,
        ieee8021StreamIdLanPathId,
        ieee8021StreamIdStatus
    }
    STATUS      current
    DESCRIPTION
        "Objects that are used if auto configuration for streams is
        used."
    ::= { ieee8021StreamIdGroups 6 }

-- ============================================================
-- the ieee8021StreamIdPerPortPerStreamCounters group
-- ============================================================

ieee8021StreamIdPerPortPerStreamCountersGroup OBJECT-GROUP
    OBJECTS {
        ieee8021StreamIdPerPortPerStreamInputPackets,
        ieee8021StreamIdPerPortPerStreamOutputPackets
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
    }
    STATUS      current
    DESCRIPTION
        "Objects that provide information ont he count of packets
         handled by stream identification per port, stream and
         direction ."
    ::= { ieee8021StreamIdGroups 7 }

-- ================================================================
-- the ieee8021StreamIdPerPortCounters group
-- ================================================================

ieee8021StreamIdPerPortCountersGroup OBJECT-GROUP
    OBJECTS {
        ieee8021StreamIdPerPortInputPackets,
        ieee8021StreamIdPerPortOutputPackets
    }
    STATUS      current
    DESCRIPTION
        "Objects that provide information on the count of packets
         handled by stream identification per port and direction ."
    ::= { ieee8021StreamIdGroups 8 }

-- ================================================================
-- the ieee8021StreamIdMaskAndMatchIdentification group
-- ================================================================

ieee8021StreamIdMaskAndMatchIdentificationGroup OBJECT-GROUP
    OBJECTS {
        ieee8021StreamIdCpeMmIdDestMacMask,
        ieee8021StreamIdCpeMmIdDestMacMatch,
        ieee8021StreamIdCpeMmIdSrcMacMask,
        ieee8021StreamIdCpeMmIdSrcMacMatch,
        ieee8021StreamIdCpeMmIdMsduMaskLength,
        ieee8021StreamIdCpeMmIdMsduMask,
        ieee8021StreamIdCpeMmIdMsduMatch
    }
    STATUS      current
    DESCRIPTION
        "Objects that are part of the Mask-and-match stream
         identification method."
    ::= { ieee8021StreamIdGroups 9 }

-- ================================================================
-- the ieee8021StreamIdMaskAndMatchPerPortMsduMaskMaxLength group
-- ================================================================

ieee8021StreamIdMaskAndMatchPerPortMsduMaskMaxLengthGroup OBJECT-GROUP
    OBJECTS {
        ieee8021StreamIdMaskAndMatchMsduMaskMaxLength
    }
    STATUS      current
    DESCRIPTION
        "Objects that provide information on the maximum msdu mask
         handled by the Mask-and-match stream identification function
         per port."
    ::= { ieee8021StreamIdGroups 10 }

-- ================================================================
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
-- compliance statements
-- ============================================================

ieee8021StreamIdCompliance MODULE-COMPLIANCE
    STATUS      current
    DESCRIPTION
        "The compliance statement for devices supporting
        stream identification."

    MODULE -- this module
        MANDATORY-GROUPS {
            ieee8021StreamIdStreamIdentificationGroup,
            ieee8021StreamIdNullStreamIdentificationGroup,
            ieee8021StreamIdSrcMacVlanIdentificationGroup,
            ieee8021StreamIdActiveDestMacVlanIdentificationGroup,
            ieee8021StreamIdIpStreamIdentificationGroup,
            ieee8021StreamIdPerPortPerStreamCountersGroup,
            ieee8021StreamIdPerPortCountersGroup,
            ieee8021StreamIdMaskAndMatchIdentificationGroup,
            ieee8021StreamIdMaskAndMatchPerPortMsduMaskMaxLengthGroup
        }

        GROUP ieee8021StreamIdAutoConfigurationGroup
        DESCRIPTION
            "Implementation of this group is mandatory if the
             auto configuration feature is implemented."

    ::= { ieee8021StreamIdCompliances 1 }

END
```

11.5.2 Definitions for the IEEE8021-FRER-MIB

```
IEEE8021-FRER-MIB DEFINITIONS ::= BEGIN

-- ============================================================
-- MIB for support of frame replication and elimination of
-- IEEE 802.1CB Frame Replication and Elimination for Reliability
-- ============================================================

IMPORTS
    OBJECT-GROUP,
    MODULE-COMPLIANCE
        FROM SNMPv2-CONF
    MODULE-IDENTITY,
    OBJECT-TYPE,
    Counter64,
    Unsigned32,
    Integer32
        FROM SNMPv2-SMI
    TEXTUAL-CONVENTION,
    TruthValue,
    RowStatus,
    AutonomousType,
    VariablePointer
        FROM SNMPv2-TC
    ifIndex,
    InterfaceIndex
        FROM IF-MIB
    ieee802dot1mibs
        FROM IEEE8021-TC-MIB
    Ieee8021CBTaggedType,
    Ieee8021CBVlanIdentifier,
    Ieee8021CBLanPathIdType,
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
    ieee8021StreamIdStreamIdHandle
        FROM IEEE8021-STREAM-IDENTIFICATION-MIB
    ;

ieee8021FrerMib MODULE-IDENTITY
    LAST-UPDATED "202112080000Z" -- December 8, 2021
    ORGANIZATION "IEEE 802.1 Working Group"
    CONTACT-INFO
                "WG-URL: http://ieee802.org/1/
                WG-EMail: stds-802-1-l@ieee.org

    Contact: IEEE 802.1 Working Group Chair
    Postal: C/O IEEE 802.1 Working Group
            IEEE Standards Association
            445 Hoes Lane
            Piscataway, NJ 08854
            USA

    E-mail: stds-802-1-chairs@ieee.org"

    DESCRIPTION
        "The Management Information Base module for IEEE 802.1CB Stream
         identification. Unless otherwise indicated, the references in
         this MIB module are to IEEE Std 802.1CB-2017 as amended by
         IEEE Std 802.1CBcv-2021. Copyright (C) IEEE (2021).
         This version of this MIB module is part of IEEE Std 802.1CB-2021;
         see the standard itself for full legal notices."

    REVISION "202112080000Z" -- December 8, 2021
    DESCRIPTION
        "Initial revision published as part of IEEE Std 802.1CBcv-2021."

    ::= { ieee802dot1mibs 35 }

-- ================================================================
-- Textual Conventions
-- ================================================================

Ieee8021CBSequenceRecoveryAlgorithm ::= TEXTUAL-CONVENTION
    STATUS       current
    DESCRIPTION
        "An enumerated value specifying which sequence recovery
         algorithm is to be used for this instance of the Sequence
         recovery function. The enumeration uses an OUI or CID as shown
         in Table 10-1. If the value used is in the range of 0-255,
         the OUI used is 00-80-C2.
         The values 0 and 2-255 are reserved for use by IEEE 802."
    REFERENCE    "10.4.1.5 and Table 10-1"
    SYNTAX      INTEGER {
                    vectorAlgorithm(0),
                    matchAlgorithm(1),
                    nonIEEESpecified(256)
                }

Ieee8021CBSequenceEncodeDecodeType ::= TEXTUAL-CONVENTION
    STATUS       current
    DESCRIPTION
        "An enumerated value indicating the type of encapsulation used
         for this instance of the Sequence encode/decode function. The
         enumeration uses an OUI or CID as shown in Table 10-2. If the
         value used is in the range of 0-255, the OUI used is 00-80-C2.
         The values 0 and 4-255 are reserved for use by IEEE 802."
    REFERENCE    "10.5.1.5 and Table 10-2"
    SYNTAX      INTEGER {
                    reserved(0),
                    rTAG(1),
                    hsrSequenceTag(2),
                    prpSequenceTag(3),
                    nonIEEESpecified(256)
                }

-- ================================================================
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
-- subtrees in the FRER MIB
-- ============================================================

ieee8021FrerNotifications
    OBJECT IDENTIFIER ::= { ieee8021FrerMib 0 }

ieee8021FrerObjects
    OBJECT IDENTIFIER ::= { ieee8021FrerMib 1 }

ieee8021FrerConformance
    OBJECT IDENTIFIER ::= { ieee8021FrerMib 2 }

ieee8021FrerSequenceGeneration
    OBJECT IDENTIFIER ::= { ieee8021FrerObjects 1 }

ieee8021FrerSequenceGenerationHandleList
    OBJECT IDENTIFIER ::= { ieee8021FrerObjects 2 }

ieee8021FrerSequenceRecovery
    OBJECT IDENTIFIER ::= { ieee8021FrerObjects 3 }

ieee8021FrerSequenceRecoveryHandleList
    OBJECT IDENTIFIER ::= { ieee8021FrerObjects 4 }

ieee8021FrerSequenceRecoveryPortHandleList
    OBJECT IDENTIFIER ::= { ieee8021FrerObjects 5 }

ieee8021FrerSequenceIdentification
    OBJECT IDENTIFIER ::= { ieee8021FrerObjects 6 }

ieee8021FrerSequenceIdentificationHandleList
    OBJECT IDENTIFIER ::= { ieee8021FrerObjects 7 }

ieee8021FrerStreamSplit
    OBJECT IDENTIFIER ::= { ieee8021FrerObjects 8 }

ieee8021FrerStreamSplitInputHandleList
    OBJECT IDENTIFIER ::= { ieee8021FrerObjects 9 }

ieee8021FrerStreamSplitOutputHandleList
    OBJECT IDENTIFIER ::= { ieee8021FrerObjects 10 }

ieee8021FrerSequenceAutoconfig
    OBJECT IDENTIFIER ::= { ieee8021FrerObjects 11 }

ieee8021FrerSequenceAutoconfigReceivePortHandleList
    OBJECT IDENTIFIER ::= { ieee8021FrerObjects 12 }

ieee8021FrerSequenceAutoconfigVlanHandleList
    OBJECT IDENTIFIER ::= { ieee8021FrerObjects 13 }

ieee8021FrerSequenceAutoconfigRecoveryPortHandleList
    OBJECT IDENTIFIER ::= { ieee8021FrerObjects 14 }

ieee8021FrerOutputAutoconfig
    OBJECT IDENTIFIER ::= { ieee8021FrerObjects 15 }

ieee8021FrerOutputAutoconfigPortHandleList
    OBJECT IDENTIFIER ::= { ieee8021FrerObjects 16 }

ieee8021FrerPerPortPerStreamCounters
    OBJECT IDENTIFIER ::= { ieee8021FrerObjects 17 }

ieee8021FrerPerPortCounters
    OBJECT IDENTIFIER ::= { ieee8021FrerObjects 18 }

-- ============================================================
-- the ieee8021FrerSequenceGeneration table
-- ============================================================

ieee8021FrerSequenceGenerationTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF Ieee8021FrerSequenceGenerationEntry
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A table containing a set of sequence generation entry objects.
         There is one entry for each sequence generation function."
    REFERENCE   "10.3"
    ::= { ieee8021FrerSequenceGeneration 1 }

ieee8021FrerSequenceGenerationEntry OBJECT-TYPE
    SYNTAX      Ieee8021FrerSequenceGenerationEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A set of managed objects that manage the Streams and direction
         for which a single instance of the Sequence generation function
         is to be placed."
    REFERENCE   "10.3.1"
    INDEX  { ieee8021FrerSequenceGenerationIndex }
    ::= { ieee8021FrerSequenceGenerationTable 1 }

Ieee8021FrerSequenceGenerationEntry ::=
    SEQUENCE {
        ieee8021FrerSequenceGenerationIndex
            Unsigned32,
        ieee8021FrerSequenceGenerationStreamList
            AutonomousType,
        ieee8021FrerSequenceGenerationDirection
            TruthValue,
        ieee8021FrerSequenceGenerationReset
            TruthValue

    }

ieee8021FrerSequenceGenerationIndex OBJECT-TYPE
    SYNTAX      Unsigned32
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "Index for the Sequence Generation table."
    ::= { ieee8021FrerSequenceGenerationEntry 1 }

ieee8021FrerSequenceGenerationStreamList OBJECT-TYPE
    SYNTAX      AutonomousType
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "A list of stream_handle values, corresponding to the values of
         the tsnStreamIdHandle objects in the Stream identity table,
         on which this instance of the Sequence generation function
         is to operate.
         The single instance of the Sequence generation function created
         by this frerSeqGenEntry operates every packet belonging to this
         Stream, regardless of the port on which it is received.
         For each instance of the Sequence generation function present
         in the ieee8021FrerSequenceGenerationTable there exists a list
         of stream_handle values in the
         ieee8021FrerSequenceGenerationHandleListTable."
    REFERENCE   "10.3.1.1"
    ::= { ieee8021FrerSequenceGenerationEntry 2 }

ieee8021FrerSequenceGenerationDirection OBJECT-TYPE
    SYNTAX      TruthValue
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "This managed object indicates whether the Sequence generation
         function is to be placed on the out-facing (True) or
         in-facing (False) side of the port."
    REFERENCE   "10.3.1.2"
    ::= { ieee8021FrerSequenceGenerationEntry 3 }

ieee8021FrerSequenceGenerationReset OBJECT-TYPE
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
    SYNTAX      TruthValue
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "This managed object indicates that the Sequence generation
         function is to be reset by calling its corresponding
         SequenceGenerationReset function. Writing the value True to
         this managed object triggers a reset; writing the value False
         has no effect. When read, this managed object always returns
         the value False."
    REFERENCE   "10.3.1.3"
    ::= { ieee8021FrerSequenceGenerationEntry 4 }

-- ============================================================
-- the ieee8021FrerSequenceGenerationHandleList table
-- ============================================================

ieee8021FrerSequenceGenerationHandleListTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF Ieee8021FrerSequenceGenerationHandleListEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A table containing a list of stream_handle values,
         corresponding to the values of the tsnStreamIdHandle objects in
         the Stream identity table, on which the instance of the
         Sequence generation function referenced in
         ieee8021FrerSequenceGenerationIndex is to operate.
         The single instance of the Sequence generation function created
         by the ieee8021FrerSequenceGenerationEntry referenced operates
         every packet belonging to this Stream, regardless of the port
         on which it is received."
    REFERENCE   "10.3.1.1"
    ::= { ieee8021FrerSequenceGenerationHandleList 2 }

ieee8021FrerSequenceGenerationHandleListEntry OBJECT-TYPE
    SYNTAX      Ieee8021FrerSequenceGenerationHandleListEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A set of managed objects providing the stream_handle values
         corresponding to the values of the tsnStreamIdHandle objects in
         the Stream identity table, on which the instance of the
         Sequence generation function referenced in
         ieee8021FrerSequenceGenerationIndex is to operate."
    REFERENCE   "10.3.1.1"
    INDEX   { ieee8021FrerSequenceGenerationIndex,
              ieee8021FrerSequenceGenerationHandleListIndex
            }
    ::= { ieee8021FrerSequenceGenerationHandleListTable 1 }

Ieee8021FrerSequenceGenerationHandleListEntry ::=
    SEQUENCE {
        ieee8021FrerSequenceGenerationHandleListIndex
            Unsigned32,
        ieee8021FrerSequenceGenerationStreamHandle
            VariablePointer,
        ieee8021FrerSequenceGenerationHandleListStatus
            RowStatus
    }

ieee8021FrerSequenceGenerationHandleListIndex OBJECT-TYPE
    SYNTAX      Unsigned32
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "Index for the Sequence Generation handle list table."
    ::= { ieee8021FrerSequenceGenerationHandleListEntry 1 }

ieee8021FrerSequenceGenerationStreamHandle OBJECT-TYPE
    SYNTAX      VariablePointer
    MAX-ACCESS  read-create
    STATUS      current
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
    DESCRIPTION
        "A pointer to the ieee8021StreamIdStreamIdHandle that is an
         element of the ieee8021FrerSequenceGenerationStreamList
         instance in the ieee8021StreamIdStreamIdentificationTable."
    REFERENCE   "10.3.1.1"
    ::= { ieee8021FrerSequenceGenerationHandleListEntry 2 }

ieee8021FrerSequenceGenerationHandleListStatus OBJECT-TYPE
    SYNTAX      RowStatus
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "Row-Status for the Sequence Generation handle list table."
    ::= { ieee8021FrerSequenceGenerationHandleListEntry 3 }

-- ============================================================
-- the ieee8021FrerSequenceRecovery table
-- ============================================================

ieee8021FrerSequenceRecoveryTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF Ieee8021FrerSequenceRecoveryEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A table containing a set of managed objects for the single
         instance of a Base recovery function and Latent error detection
         function included in the Sequence recovery function or
         Individual recovery function."
    REFERENCE   "10.4"
    ::= { ieee8021FrerSequenceRecovery 3 }

ieee8021FrerSequenceRecoveryEntry OBJECT-TYPE
    SYNTAX      Ieee8021FrerSequenceRecoveryEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A set of managed objects listing the Streams, ports, and
         direction for which instances of a Sequence recovery function
         or Individual recovery function are to be instantiated."
    REFERENCE   "10.4.1"
    INDEX  { ieee8021FrerSequenceRecoveryIndex }
    ::= { ieee8021FrerSequenceRecoveryTable 1 }

Ieee8021FrerSequenceRecoveryEntry ::=
    SEQUENCE {
        ieee8021FrerSequenceRecoveryIndex
            Unsigned32,
        ieee8021FrerSequenceRecoveryStreamList
            AutonomousType,
        ieee8021FrerSequenceRecoveryPortList
            AutonomousType,
        ieee8021FrerSequenceRecoveryDirection
            TruthValue,
        ieee8021FrerSequenceRecoveryReset
            TruthValue,
        ieee8021FrerSequenceRecoveryAlgorithm
            Ieee8021CBSequenceRecoveryAlgorithm,
        ieee8021FrerSequenceRecoveryAlgorithmOUI
            OCTET STRING,
        ieee8021FrerSequenceCustomRecoveryAlgorithm
            Integer32,
        ieee8021FrerSequenceCustomRecoveryAlgorithmOUI
            OCTET STRING,
        ieee8021FrerSequenceRecoveryHistoryLength
            Integer32,
        ieee8021FrerSequenceRecoveryResetMSec
            Unsigned32,
        ieee8021FrerSequenceRecoveryInvalidSequenceValue
            Unsigned32,
        ieee8021FrerSequenceRecoveryTakeNoSequence
            TruthValue,
        ieee8021FrerSequenceRecoveryIndividualRecovery
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
            TruthValue,
        ieee8021FrerSequenceRecoveryLatentErrorDetection
            TruthValue,
        ieee8021FrerSequenceRecoveryLatentErrorDifference
            Integer32,
        ieee8021FrerSequenceRecoveryLatentErrorPeriod
            Integer32,
        ieee8021FrerSequenceRecoveryLatentErrorPaths
            Integer32,
        ieee8021FrerSequenceRecoveryLatentResetPeriod
            Integer32,
        ieee8021FrerSequenceRecoveryStatus
            RowStatus
    }

ieee8021FrerSequenceRecoveryIndex OBJECT-TYPE
    SYNTAX      Unsigned32
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "Index for the Sequence Recovery table."
    ::= { ieee8021FrerSequenceRecoveryEntry 1 }

ieee8021FrerSequenceRecoveryStreamList OBJECT-TYPE
    SYNTAX      AutonomousType
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "A list of the stream_handle values, corresponding to the values
         of the tsnStreamIdHandle objects in the Stream identity table
         to which the system is to apply the instance of the Sequence
         recovery function or Individual recovery function.
         For each entry in the ieee8021FrerSequenceRecoveryTable
         there exists a list of stream_handle values in the
         ieee8021FrerSequenceRecoveryHandleListTable."
    REFERENCE   "10.4.1.1"
    ::= { ieee8021FrerSequenceRecoveryEntry 2 }

ieee8021FrerSequenceRecoveryPortList OBJECT-TYPE
    SYNTAX      AutonomousType
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "The list of ports on each of which the system is to instantiate
         the Sequence recovery function, or from which received packets
         are to be fed to a single instance of the Individual recovery
         function.
         For each entry in the ieee8021FrerSequenceRecoveryTable
         there exists a list of ports in the
         ieee8021FrerSequenceRecoveryPortHandleListTable."
    REFERENCE   "10.4.1.2"
    ::= { ieee8021FrerSequenceRecoveryEntry 3 }

ieee8021FrerSequenceRecoveryDirection OBJECT-TYPE
    SYNTAX      TruthValue
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "This managed object indicates whether the Sequence recovery
         function or Individual recovery function is to be placed on the
         out-facing (True) or in-facing (False) side of the port."
    REFERENCE   "10.4.1.3"
    ::= { ieee8021FrerSequenceRecoveryEntry 4 }

ieee8021FrerSequenceRecoveryReset OBJECT-TYPE
    SYNTAX      TruthValue
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "This managed object indicates that the Sequence recovery
         function or Individual recovery function is to be reset by
         calling its corresponding SequenceGenerationReset function.
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
1           Writing the value True to ieee8021FrerSequenceRecoveryReset
            triggers a reset; writing the value False has no effect.
            When read, ieee8021FrerSequenceRecoveryReset always returns
            the value False."
        REFERENCE   "10.4.1.4"
        ::= { ieee8021FrerSequenceRecoveryEntry 5 }

    ieee8021FrerSequenceRecoveryAlgorithm OBJECT-TYPE
        SYNTAX      Ieee8021CBSequenceRecoveryAlgorithm
        MAX-ACCESS  read-create
        STATUS      current
        DESCRIPTION
            "This managed object is an enumerated value specifying which
             sequence recovery algorithm is to be used for this instance of
             the Sequence recovery function. If the value used is
             nonIEEESpecified(256), the managed objects
             ieee8021FrerSequenceCustomRecoveryAlgorithmOUI and
             ieee8021FrerSequenceCustomRecoveryAlgorithm are used to specify
             OUI/CID and the recovery algorithm defined by the entity owning
             the OUI/CID."
        REFERENCE   "10.4.1.5"
        DEFVAL  { vectorAlgorithm }
        ::= { ieee8021FrerSequenceRecoveryEntry 6 }

    ieee8021FrerSequenceRecoveryAlgorithmOUI OBJECT-TYPE
        SYNTAX      OCTET STRING (SIZE (3))
        MAX-ACCESS  read-only
        STATUS      current
        DESCRIPTION
            "This managed object specifies the OUI/CID used together with
             the sequence recovery algorithm type specified in
             ieee8021FrerSequenceRecoveryAlgorithm.
             It is used if and only if the value for
             ieee8021FrerSequenceRecoveryAlgorithm is in the range of 0-255.
             In this case it always takes the value 00-80-C2."
        REFERENCE   "10.4.1.5"
        ::= { ieee8021FrerSequenceRecoveryEntry 7 }

    ieee8021FrerSequenceCustomRecoveryAlgorithm OBJECT-TYPE
        SYNTAX      Integer32 (256..2147483647)
        MAX-ACCESS  read-create
        STATUS      current
        DESCRIPTION
            "This managed object is an enumerated value specifying which
             sequence recovery algorithm is to be used for this instance of
             the Sequence recovery function. This managed object is used if
             and only if the value for ieee8021FrerSequenceRecoveryAlgorithm
             is 256."
        REFERENCE   "10.4.1.5"
        ::= { ieee8021FrerSequenceRecoveryEntry 8 }

    ieee8021FrerSequenceCustomRecoveryAlgorithmOUI OBJECT-TYPE
        SYNTAX      OCTET STRING (SIZE (3))
        MAX-ACCESS  read-create
        STATUS      current
        DESCRIPTION
            "This managed object specifies the OUI/CID used together with
             the sequence recovery algorithm type specified in
             ieee8021FrerSequenceRecoveryAlgorithm. It is used if and only
             if the value for ieee8021FrerSequenceRecoveryAlgorithm is 256."
        REFERENCE   "10.4.1.5"
        ::= { ieee8021FrerSequenceRecoveryEntry 9 }

    ieee8021FrerSequenceRecoveryHistoryLength OBJECT-TYPE
        SYNTAX      Integer32
        MAX-ACCESS  read-create
        STATUS      current
        DESCRIPTION
            "This managed object specifies how many bits of the
             SequenceHistory variable are to be used. The minimum and the
             default value is 2, maximum is the maximum allowed by the
             implementation. It is not used if the value for
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
1          ieee8021FrerSequenceRecoveryAlgorithm is 1."
     REFERENCE   "10.4.1.6"
     ::= { ieee8021FrerSequenceRecoveryEntry 10 }

 ieee8021FrerSequenceRecoveryResetMSec OBJECT-TYPE
     SYNTAX      Unsigned32
     MAX-ACCESS  read-create
     STATUS      current
     DESCRIPTION
         "This managed object specifies the timeout period in
          milliseconds for the RECOVERY_TIMEOUT event."
     REFERENCE   "10.4.1.7"
     ::= { ieee8021FrerSequenceRecoveryEntry 11 }

 ieee8021FrerSequenceRecoveryInvalidSequenceValue OBJECT-TYPE
     SYNTAX      Unsigned32
     MAX-ACCESS  read-only
     STATUS      current
     DESCRIPTION
         "This managed object specifies a value that cannot be encoded in
          a packet as a value for the sequence_number subparameter,
          i.e., ieee8021FrerSequenceRecoveryInvalidSequenceValue is
          larger than or equal to the value specified in RecovSeqSpace."
     REFERENCE   "10.4.1.8 and 7.4.3.2.1"
     ::= { ieee8021FrerSequenceRecoveryEntry 12 }

 ieee8021FrerSequenceRecoveryTakeNoSequence OBJECT-TYPE
     SYNTAX      TruthValue
     MAX-ACCESS  read-create
     STATUS      current
     DESCRIPTION
         "This managed object specifies whether packets with no
          sequence_number subparameter are to be accepted (True) or
          not (False)."
     REFERENCE   "10.4.1.9"
     DEFVAL  { false }
     ::= { ieee8021FrerSequenceRecoveryEntry 13 }

 ieee8021FrerSequenceRecoveryIndividualRecovery OBJECT-TYPE
     SYNTAX      TruthValue
     MAX-ACCESS  read-create
     STATUS      current
     DESCRIPTION
         "This managed object specifies whether this entry describes a
          Sequence recovery function (False) or Individual recovery
          function (True)."
     REFERENCE   "10.4.1.10"
     ::= { ieee8021FrerSequenceRecoveryEntry 14 }

 ieee8021FrerSequenceRecoveryLatentErrorDetection OBJECT-TYPE
     SYNTAX      TruthValue
     MAX-ACCESS  read-create
     STATUS      current
     DESCRIPTION
         "This managed object indicates whether an instance of the
          Latent error detection function is to be instantiated along
          with the Base recovery function in this recovery function.
          It cannot be set True if
          ieee8021FrerSequenceRecoveryIndividualRecovery is also True;
          an Individual recovery function does not include a Latent
          error detection function."
     REFERENCE   "10.4.1.11"
     ::= { ieee8021FrerSequenceRecoveryEntry 15 }

 ieee8021FrerSequenceRecoveryLatentErrorDifference OBJECT-TYPE
     SYNTAX      Integer32
     MAX-ACCESS  read-create
     STATUS      current
     DESCRIPTION
         "This managed object specifies the maximum difference between
          ieee8021FrerPerPortPerStreamSeqRecoveryDiscardedPackets, and the
          product of ieee8021FrerPerPortPerStreamSeqRecoveryPassedPackets and
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
1          (ieee8021FrerSequenceRecoveryLatentErrorPaths - 1)
           that is allowed. Any larger difference will trigger the
           detection of a latent error by the LatentErrorTest function.
           It is used if and only if
           ieee8021FrerSequenceRecoveryIndividualRecovery is False."
      REFERENCE   "10.4.1.12.1"
      ::= { ieee8021FrerSequenceRecoveryEntry 16 }

  ieee8021FrerSequenceRecoveryLatentErrorPeriod OBJECT-TYPE
      SYNTAX      Integer32
      MAX-ACCESS  read-create
      STATUS      current
      DESCRIPTION
          "This managed object specifies the number of milliseconds that
           are to elapse between instances of running the LatentErrorTest
           function. An implementation can have a minimum value for
           this managed object, below which it cannot be set, but this
           minimum shall be no larger than 1000 ms (1 s).
           Default value 2000 (2 s).
           It is used if and only if
           ieee8021FrerSequenceRecoveryIndividualRecovery is False."
      REFERENCE   "10.4.1.12.2"
      DEFVAL  { 2000 }
      ::= { ieee8021FrerSequenceRecoveryEntry 17 }

  ieee8021FrerSequenceRecoveryLatentErrorPaths OBJECT-TYPE
      SYNTAX      Integer32
      MAX-ACCESS  read-create
      STATUS      current
      DESCRIPTION
          "This managed object specifies the number of paths over which
           FRER is operating for this instance of the Base recovery
           function and Latent error detection function.
           It is used if and only if
           ieee8021FrerSequenceRecoveryIndividualRecovery is False."
      REFERENCE   "10.4.1.12.3"
      ::= { ieee8021FrerSequenceRecoveryEntry 18 }

  ieee8021FrerSequenceRecoveryLatentResetPeriod OBJECT-TYPE
      SYNTAX      Integer32
      MAX-ACCESS  read-create
      STATUS      current
      DESCRIPTION
          "This managed object specifies the number of milliseconds that
           are to elapse between instances of running the LatentErrorReset
           function. An implementation can have a minimum value for
           this managed object, below which it cannot be set, but this
           minimum shall be no larger than 1000 ms (1 s).
           It is used if and only if
           ieee8021FrerSequenceRecoveryIndividualRecovery is False."
      REFERENCE   "10.4.1.12.4"
      DEFVAL  { 30000 }
      ::= { ieee8021FrerSequenceRecoveryEntry 19 }

  ieee8021FrerSequenceRecoveryStatus OBJECT-TYPE
      SYNTAX      RowStatus
      MAX-ACCESS  read-create
      STATUS      current
      DESCRIPTION
          "Row-Status for the Sequence Recovery table."
      ::= { ieee8021FrerSequenceRecoveryEntry 20 }


  -- ============================================================
  -- the ieee8021FrerSequenceRecoveryHandleList table
  -- ============================================================

  ieee8021FrerSequenceRecoveryHandleListTable OBJECT-TYPE
      SYNTAX      SEQUENCE OF Ieee8021FrerSequenceRecoveryHandleListEntry
      MAX-ACCESS  not-accessible
      STATUS      current
      DESCRIPTION
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
        "A table containing a list of stream_handle values,
         corresponding to the values of the tsnStreamIdHandle objects in
         the Stream identity table, to which the system is to apply the
         instance of the Sequence recovery function or Individual
         recovery function referenced in
         ieee8021FrerSequenceRecoveryIndex."
    REFERENCE   "10.4.1.1"
    ::= { ieee8021FrerSequenceRecoveryHandleList 4 }

ieee8021FrerSequenceRecoveryHandleListEntry OBJECT-TYPE
    SYNTAX      Ieee8021FrerSequenceRecoveryHandleListEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A set of managed objects providing the stream_handle values
         corresponding to the values of the tsnStreamIdHandle objects in
         the Stream identity table, to which the system is to apply the
         instance of the Sequence recovery function or Individual
         recovery function referenced in
         ieee8021FrerSequenceRecoveryIndex."
    REFERENCE   "10.4.1.1"
    INDEX   { ieee8021FrerSequenceRecoveryIndex,
              ieee8021FrerSequenceRecoveryHandleListIndex
            }
    ::= { ieee8021FrerSequenceRecoveryHandleListTable 1 }

Ieee8021FrerSequenceRecoveryHandleListEntry ::=
    SEQUENCE {
        ieee8021FrerSequenceRecoveryHandleListIndex
            Unsigned32,
        ieee8021FrerSequenceRecoveryStreamHandle
            VariablePointer,
        ieee8021FrerSequenceRecoveryHandleListStatus
            RowStatus
    }

ieee8021FrerSequenceRecoveryHandleListIndex OBJECT-TYPE
    SYNTAX      Unsigned32
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "Index for the Sequence Recovery handle list table."
    ::= { ieee8021FrerSequenceRecoveryHandleListEntry 1 }

ieee8021FrerSequenceRecoveryStreamHandle OBJECT-TYPE
    SYNTAX      VariablePointer
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "A pointer to the ieee8021StreamIdStreamIdHandle that is an
         element of the ieee8021FrerSequenceRecoveryStreamList instance
         in the ieee8021FrerSequenceRecoveryTable."
    REFERENCE   "10.4.1.1"
    ::= { ieee8021FrerSequenceRecoveryHandleListEntry 2 }

ieee8021FrerSequenceRecoveryHandleListStatus OBJECT-TYPE
    SYNTAX      RowStatus
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "Row-Status for the Sequence Recovery handle list table."
    ::= { ieee8021FrerSequenceRecoveryHandleListEntry 3 }

-- ============================================================
-- the ieee8021FrerSequenceRecoveryPortHandleList table
-- ============================================================
ieee8021FrerSequenceRecoveryPortHandleListTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF Ieee8021FrerSequenceRecoveryPortHandleListEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A table containing a list of ports on each of which the system
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
1          is to instantiate the Sequence recovery function, or from which
           received packets are to be fed to a single instance of the
           Individual recovery function, referenced in
           ieee8021FrerSequenceRecoveryIndex."
      REFERENCE   "10.4.1.2"
      ::= { ieee8021FrerSequenceRecoveryPortHandleList 5 }

  ieee8021FrerSequenceRecoveryPortHandleListEntry OBJECT-TYPE
      SYNTAX      Ieee8021FrerSequenceRecoveryPortHandleListEntry
      MAX-ACCESS  not-accessible
      STATUS      current
      DESCRIPTION
          "A set of managed objects providing the ports on each of which the system
           is to instantiate the Sequence recovery function, or from which
           received packets are to be fed to a single instance of the
           Individual recovery function, referenced in
           ieee8021FrerSequenceRecoveryIndex."
      REFERENCE   "10.4.1.2"
      INDEX   { ieee8021FrerSequenceRecoveryIndex,
                ieee8021FrerSequenceRecoveryPortHandleListIndex
              }
      ::= { ieee8021FrerSequenceRecoveryPortHandleListTable 1 }

  Ieee8021FrerSequenceRecoveryPortHandleListEntry ::=
      SEQUENCE {
          ieee8021FrerSequenceRecoveryPortHandleListIndex
              Unsigned32,
          ieee8021FrerSequenceRecoveryPortHandle
              VariablePointer,
          ieee8021FrerSequenceRecoveryPortHandleListStatus
              RowStatus
      }

  ieee8021FrerSequenceRecoveryPortHandleListIndex OBJECT-TYPE
      SYNTAX      Unsigned32
      MAX-ACCESS  not-accessible
      STATUS      current
      DESCRIPTION
          "Index for the Sequence Recovery Port handle list table."
      ::= { ieee8021FrerSequenceRecoveryPortHandleListEntry 1 }

  ieee8021FrerSequenceRecoveryPortHandle OBJECT-TYPE
      SYNTAX      VariablePointer
      MAX-ACCESS  read-create
      STATUS      current
      DESCRIPTION
          "A pointer to an interface that is an element of the
           ieee8021FrerSequenceRecoveryPortList instance in the
           ieee8021FrerSequenceRecoveryTable."
      REFERENCE   "10.4.1.2"
      ::= { ieee8021FrerSequenceRecoveryPortHandleListEntry 2 }

  ieee8021FrerSequenceRecoveryPortHandleListStatus OBJECT-TYPE
      SYNTAX      RowStatus
      MAX-ACCESS  read-create
      STATUS      current
      DESCRIPTION
          "Row-Status for the Sequence Recovery Port handle list table."
      ::= { ieee8021FrerSequenceRecoveryPortHandleListEntry 3 }

  -- ============================================================
  -- the ieee8021FrerSequenceIdentification table
  -- ============================================================

  ieee8021FrerSequenceIdentificationTable OBJECT-TYPE
      SYNTAX      SEQUENCE OF Ieee8021FrerSequenceIdentificationEntry
      MAX-ACCESS  not-accessible
      STATUS      current
      DESCRIPTION
          "A table containing a set of sequence identification entry
           objects. There is one entry for each port and direction for
           which an instance of the Sequence encode/decode function is to
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
1          be instantiated for a list of Streams."
      REFERENCE   "10.5"
      ::= { ieee8021FrerSequenceIdentification 6 }

  ieee8021FrerSequenceIdentificationEntry OBJECT-TYPE
      SYNTAX      Ieee8021FrerSequenceIdentificationEntry
      MAX-ACCESS  not-accessible
      STATUS      current
      DESCRIPTION
          "A set of managed objects that specify a port and direction on
           which an instance of the Sequence encode/decode function is to
           be instantiated for a list of Streams."
      REFERENCE   "10.5.1"
      INDEX   { ieee8021FrerSequenceIdentificationEncodePort,
                ieee8021FrerSequenceIdentificationEncodeDirection
              }
      ::= { ieee8021FrerSequenceIdentificationTable 1 }

  Ieee8021FrerSequenceIdentificationEntry ::=
      SEQUENCE {
          ieee8021FrerSequenceIdentificationEncodePort
              InterfaceIndex,
          ieee8021FrerSequenceIdentificationEncodeDirection
              TruthValue,
          ieee8021FrerSequenceIdentificationStreamList
              AutonomousType,
          ieee8021FrerSequenceIdentificationEncodeActive
              TruthValue,
          ieee8021FrerSequenceIdentificationEncodeEncapsulationType
              Ieee8021CBSequenceEncodeDecodeType,
          ieee8021FrerSequenceIdentificationEncodeEncapsulationOUI
              OCTET STRING,
          ieee8021FrerSequenceIdentificationCustomEncodeEncapsulationType
              Integer32,
          ieee8021FrerSequenceIdentificationCustomEncodeEncapsulationOUI
              OCTET STRING,
          ieee8021FrerSequenceIdentificationEncodePathIdLanId
              Integer32,
          ieee8021FrerSequenceIdentificationStatus
              RowStatus
      }

  ieee8021FrerSequenceIdentificationEncodePort OBJECT-TYPE
      SYNTAX      InterfaceIndex
      MAX-ACCESS  not-accessible
      STATUS      current
      DESCRIPTION
          "The port on which the system is to place an instance of the
           Sequence encode/decode function."
      REFERENCE   "10.5.1.2"
      ::= { ieee8021FrerSequenceIdentificationEntry 1 }

  ieee8021FrerSequenceIdentificationEncodeDirection OBJECT-TYPE
      SYNTAX      TruthValue
      MAX-ACCESS  not-accessible
      STATUS      current
      DESCRIPTION
          "This managed object indicates whether the Sequence
           encode/decode function is to be placed on the out-facing (True)
           or in-facing (False) side of the port."
      REFERENCE   "10.5.1.3"
      ::= { ieee8021FrerSequenceIdentificationEntry 2 }

  ieee8021FrerSequenceIdentificationStreamList OBJECT-TYPE
      SYNTAX      AutonomousType
      MAX-ACCESS  read-create
      STATUS      current
      DESCRIPTION
          "A list of stream_handles, corresponding to the values of the
           tsnStreamIdHandle objects in the Stream identity table, for
           which the system is to use the same encapsulation for the
           Sequence encode/decode function.
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
1           For each entry in the ieee8021FrerSequenceIdentificationTable
            there exists a list of stream_handle values in the
            ieee8021FrerSequenceIdentificationHandleListTable."
        REFERENCE   "10.5.1.1"
        ::= { ieee8021FrerSequenceIdentificationEntry 3 }

    ieee8021FrerSequenceIdentificationEncodeActive OBJECT-TYPE
        SYNTAX      TruthValue
        MAX-ACCESS  read-create
        STATUS      current
        DESCRIPTION
            "This managed object specifies whether this
             ieee8021FrerSequenceIdentificationEntry is passive (False),
             and therefore is used only to decode (extract information from)
             input packets passing up the protocol stack, or active (True),
             and therefore is used both for recognizing input packets and
             for encoding output packets being passed down the protocol
             stack."
        REFERENCE   "10.5.1.4"
        ::= { ieee8021FrerSequenceIdentificationEntry 4 }

    ieee8021FrerSequenceIdentificationEncodeEncapsulationType OBJECT-TYPE
        SYNTAX      Ieee8021CBSequenceEncodeDecodeType
        MAX-ACCESS  read-create
        STATUS      current
        DESCRIPTION
            "An enumerated value specifying the type of encapsulation used
             for this instance of the Sequence encode/decode function. If
             the value used is nonIEEESpecified(256), the managed objects
             ieee8021FrerSequenceIdentificationCustomEncodeEncapsulationOUI
             and
             ieee8021FrerSequenceIdentificationCustomEncodeEncapsulationType
             are used to specify OUI/CID and the Sequence encode/decode
             method defined by the entity owning the OUI/CID."
        REFERENCE   "10.5.1.5"
        ::= { ieee8021FrerSequenceIdentificationEntry 5 }

    ieee8021FrerSequenceIdentificationEncodeEncapsulationOUI OBJECT-TYPE
        SYNTAX      OCTET STRING (SIZE (3))
        MAX-ACCESS  read-only
        STATUS      current
        DESCRIPTION
            "This managed object specifies the OUI/CID used together with
             the encapsulation type specified in
             ieee8021FrerSequenceIdentificationEncodeEncapsulationType.
             It is used if and only if the value for
             ieee8021FrerSequenceIdentificationEncodeEncapsulationType is
             in the range of 0-255. In this case it always takes the value
             00-80-C2."
        REFERENCE   "10.5.1.5"
        ::= { ieee8021FrerSequenceIdentificationEntry 6 }

    ieee8021FrerSequenceIdentificationCustomEncodeEncapsulationType OBJECT-TYPE
        SYNTAX      Integer32 (256..2147483647)
        MAX-ACCESS  read-create
        STATUS      current
        DESCRIPTION
            "An enumerated value specifying the type of encapsulation used
             for this instance of the Sequence encode/decode function. This
             managed object is used if and only if the value for
             ieee8021FrerSequenceIdentificationEncodeEncapsulationType is
             256."
        REFERENCE   "10.5.1.5"
        ::= { ieee8021FrerSequenceIdentificationEntry 7 }

    ieee8021FrerSequenceIdentificationCustomEncodeEncapsulationOUI OBJECT-TYPE
        SYNTAX      OCTET STRING (SIZE (3))
        MAX-ACCESS  read-create
        STATUS      current
        DESCRIPTION
            "This managed object specifies the OUI/CID used together with
             the encapsulation type specified in
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
1          ieee8021FrerSequenceIdentificationCustomEncodeEncapsulationType.
           It is used if and only if the value for
           ieee8021FrerSequenceIdentificationEncodeEncapsulationType is
           256."
      REFERENCE   "10.5.1.5"
      ::= { ieee8021FrerSequenceIdentificationEntry 8 }

  ieee8021FrerSequenceIdentificationEncodePathIdLanId OBJECT-TYPE
      SYNTAX      Integer32 (0..15)
      MAX-ACCESS  read-create
      STATUS      current
      DESCRIPTION
          "A 4-bit integer value to be placed in the PathId field of an
           HSR sequence tag or the LanId field of a PRP sequence trailer
           added to an output packet."
      REFERENCE   "10.5.1.6"
      ::= { ieee8021FrerSequenceIdentificationEntry 9 }

  ieee8021FrerSequenceIdentificationStatus OBJECT-TYPE
      SYNTAX      RowStatus
      MAX-ACCESS  read-create
      STATUS      current
      DESCRIPTION
          "Row-Status for the Sequence Identification table."
      ::= { ieee8021FrerSequenceIdentificationEntry 10 }

  -- ============================================================
  -- the ieee8021FrerSequenceIdentificationHandleList table
  -- ============================================================

  ieee8021FrerSequenceIdentificationHandleListTable OBJECT-TYPE
      SYNTAX      SEQUENCE OF Ieee8021FrerSequenceIdentificationHandleListEntry
      MAX-ACCESS  not-accessible
      STATUS      current
      DESCRIPTION
          "A table containing a list of stream_handle values,
           corresponding to the values of the tsnStreamIdHandle objects in
           the Stream identity table, for which the system is to use the
           same encapsulation for the Sequence encode/decode function
           referenced in ieee8021FrerSequenceIdentificationEncodePort
           and ieee8021FrerSequenceIdentificationEncodeDirection."
      REFERENCE   "10.5.1.1"
      ::= { ieee8021FrerSequenceIdentificationHandleList 7 }

  ieee8021FrerSequenceIdentificationHandleListEntry OBJECT-TYPE
      SYNTAX      Ieee8021FrerSequenceIdentificationHandleListEntry
      MAX-ACCESS  not-accessible
      STATUS      current
      DESCRIPTION
          "A set of managed objects providing the stream_handle values
           corresponding to the values of the tsnStreamIdHandle objects in
           the Stream identity table, for which the system is to use the
           same encapsulation for the Sequence encode/decode function
           referenced in ieee8021FrerSequenceIdentificationEncodePort
           and ieee8021FrerSequenceIdentificationEncodeDirection."
      REFERENCE   "10.5.1.1"
      INDEX   { ieee8021FrerSequenceIdentificationEncodePort,
                ieee8021FrerSequenceIdentificationEncodeDirection,
                ieee8021FrerSequenceIdentificationHandleListIndex
              }
      ::= { ieee8021FrerSequenceIdentificationHandleListTable 1 }

  Ieee8021FrerSequenceIdentificationHandleListEntry ::=
      SEQUENCE {
          ieee8021FrerSequenceIdentificationHandleListIndex
              Unsigned32,
          ieee8021FrerSequenceIdentificationStreamHandle
              VariablePointer,
          ieee8021FrerSequenceIdentificationHandleListStatus
              RowStatus
      }
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
ieee8021FrerSequenceIdentificationHandleListIndex OBJECT-TYPE
    SYNTAX      Unsigned32
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "Index for the Sequence Identification handle list table."
    ::= { ieee8021FrerSequenceIdentificationHandleListEntry 1 }

ieee8021FrerSequenceIdentificationStreamHandle OBJECT-TYPE
    SYNTAX      VariablePointer
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "A pointer to the ieee8021StreamIdStreamIdHandle that is an
         element of the ieee8021FrerSequenceIdentificationStreamList
         instance in the ieee8021FrerSequenceIdentificationTable."
    REFERENCE   "10.5.1.1"
    ::= { ieee8021FrerSequenceIdentificationHandleListEntry 2 }

ieee8021FrerSequenceIdentificationHandleListStatus OBJECT-TYPE
    SYNTAX      RowStatus
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "Row-Status for the Sequence Identification handle list table."
    ::= { ieee8021FrerSequenceIdentificationHandleListEntry 3 }

-- ============================================================
-- the ieee8021FrerStreamSplit table
-- ============================================================

ieee8021FrerStreamSplitTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF Ieee8021FrerStreamSplitEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A table containing a set of FRER split entry objects. There is
         one entry per Stream splitting function per set of
         stream_handle values."
    REFERENCE   "10.6"
    ::= { ieee8021FrerStreamSplit 8 }

ieee8021FrerStreamSplitEntry OBJECT-TYPE
    SYNTAX      Ieee8021FrerStreamSplitEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A set of managed objects that specify a port and direction
         on which an instance of the Stream splitting function is to be
         instantiated, and the list of stream_handles specifying its
         operation."
    REFERENCE   "10.6.1"
    INDEX   { ieee8021FrerStreamSplitIndex }
    ::= { ieee8021FrerStreamSplitTable 1 }

Ieee8021FrerStreamSplitEntry ::=
    SEQUENCE {
        ieee8021FrerStreamSplitIndex
            Unsigned32,
        ieee8021FrerStreamSplitPort
            InterfaceIndex,
        ieee8021FrerStreamSplitDirection
            TruthValue,
        ieee8021FrerStreamSplitInputIdList
            AutonomousType,
        ieee8021FrerStreamSplitOutputIdList
            AutonomousType
    }

ieee8021FrerStreamSplitIndex OBJECT-TYPE
    SYNTAX      Unsigned32
    MAX-ACCESS  not-accessible
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
    STATUS      current
    DESCRIPTION
        "Index for the Stream Split table."
    ::= { ieee8021FrerStreamSplitEntry 1 }

ieee8021FrerStreamSplitPort OBJECT-TYPE
    SYNTAX      InterfaceIndex
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "The port on which the system is to place an instance of the
         Stream splitting function performing the stream_handle
         translations specified by ieee8021FrerStreamSplitInputIdList
         and ieee8021FrerStreamSplitOutputIdList is to be placed."
    REFERENCE   "10.6.1.1"
    ::= { ieee8021FrerStreamSplitEntry 2 }

ieee8021FrerStreamSplitDirection OBJECT-TYPE
    SYNTAX      TruthValue
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "This managed object indicates whether the instance of the
         Stream splitting function performing the stream_handle
         translations specified by ieee8021FrerStreamSplitInputIdList
         and ieee8021FrerStreamSplitOutputIdList is to be placed on the
         out-facing (True) or in-facing (False) side of the port."
    REFERENCE   "10.6.1.1"
    ::= { ieee8021FrerStreamSplitEntry 3 }

ieee8021FrerStreamSplitInputIdList OBJECT-TYPE
    SYNTAX      AutonomousType
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "A list of stream_handles that are to be split.
         For each entry in the ieee8021FrerStreamSplitTable
         there exists a list of stream_handle values in the
         ieee8021FrerStreamSplitInputHandleListTable."
    REFERENCE   "10.6.1.3"
    ::= { ieee8021FrerStreamSplitEntry 4 }

ieee8021FrerStreamSplitOutputIdList OBJECT-TYPE
    SYNTAX      AutonomousType
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
        "A list of stream_handles into which the input packet is to be
         split, one copy per item in the
         ieee8021FrerStreamSplitOutputIdList.
         For each entry in the ieee8021FrerStreamSplitTable
         there exists a list of stream_handle values in the
         ieee8021FrerStreamSplitOutputHandleListTable."
    REFERENCE   "10.6.1.4"
    ::= { ieee8021FrerStreamSplitEntry 5 }

-- ============================================================
-- the ieee8021FrerStreamSplitInputHandleList table
-- ============================================================
ieee8021FrerStreamSplitInputHandleListTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF Ieee8021FrerStreamSplitInputHandleListEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A table containing a list of stream_handle values that are to
         be split, referenced in ieee8021FrerStreamSplitIndex."
    REFERENCE   "10.6.1.3"
    ::= { ieee8021FrerStreamSplitInputHandleList 9 }

ieee8021FrerStreamSplitInputHandleListEntry OBJECT-TYPE
    SYNTAX      Ieee8021FrerStreamSplitInputHandleListEntry
    MAX-ACCESS  not-accessible
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
    STATUS      current
    DESCRIPTION
        "A set of managed objects providing the stream_handle values
         corresponding to the values of the tsnStreamIdHandle objects in
         the Stream identity table, that are to be split, referenced in
         ieee8021FrerStreamSplitIndex."
    REFERENCE   "10.6.1.3"
    INDEX   { ieee8021FrerStreamSplitIndex,
              ieee8021FrerStreamSplitInputHandleListIndex
            }
    ::= { ieee8021FrerStreamSplitInputHandleListTable 1 }

Ieee8021FrerStreamSplitInputHandleListEntry ::=
    SEQUENCE {
        ieee8021FrerStreamSplitInputHandleListIndex
            Unsigned32,
        ieee8021FrerStreamSplitInputIdHandle
            VariablePointer,
        ieee8021FrerStreamSplitInputHandleListStatus
            RowStatus
    }

ieee8021FrerStreamSplitInputHandleListIndex OBJECT-TYPE
    SYNTAX      Unsigned32
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "Index for the Stream Split Input handle list table."
    ::= { ieee8021FrerStreamSplitInputHandleListEntry 1 }

ieee8021FrerStreamSplitInputIdHandle OBJECT-TYPE
    SYNTAX      VariablePointer
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "A pointer to the ieee8021StreamIdStreamIdHandle that is an
         element of the ieee8021FrerStreamSplitInputIdList instance in
         the ieee8021FrerStreamSplitTable."
    REFERENCE   "10.6.1.3"
    ::= { ieee8021FrerStreamSplitInputHandleListEntry 2 }

ieee8021FrerStreamSplitInputHandleListStatus OBJECT-TYPE
    SYNTAX      RowStatus
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "Row-Status for the Stream Split Input handle list table."
    ::= { ieee8021FrerStreamSplitInputHandleListEntry 3 }

-- ============================================================
-- the ieee8021FrerStreamSplitOutputHandleList table
-- ============================================================
ieee8021FrerStreamSplitOutputHandleListTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF Ieee8021FrerStreamSplitOutputHandleListEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A table containing a list of stream_handles values into which
         the input packet is to be split, one copy per item in this
         table, referenced in ieee8021FrerStreamSplitIndex."
    REFERENCE   "10.6.1.4"
    ::= { ieee8021FrerStreamSplitOutputHandleList 10 }

ieee8021FrerStreamSplitOutputHandleListEntry OBJECT-TYPE
    SYNTAX      Ieee8021FrerStreamSplitOutputHandleListEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A set of managed objects providing the stream_handle values
         corresponding to the values of the tsnStreamIdHandle objects in
         the Stream identity table, into which the input packet is to be
         split, one copy per item in this table, referenced in
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
1          ieee8021FrerStreamSplitIndex."
      REFERENCE   "10.6.1.4"
      INDEX   { ieee8021FrerStreamSplitIndex,
                ieee8021FrerStreamSplitOutputHandleListIndex
              }
      ::= { ieee8021FrerStreamSplitOutputHandleListTable 1 }

  Ieee8021FrerStreamSplitOutputHandleListEntry ::=
      SEQUENCE {
          ieee8021FrerStreamSplitOutputHandleListIndex
              Unsigned32,
          ieee8021FrerStreamSplitOutputIdHandle
              VariablePointer,
          ieee8021FrerStreamSplitOutputHandleListStatus
              RowStatus
      }

  ieee8021FrerStreamSplitOutputHandleListIndex OBJECT-TYPE
      SYNTAX      Unsigned32
      MAX-ACCESS  not-accessible
      STATUS      current
      DESCRIPTION
          "Index for the Stream Split Output handle list table."
      ::= { ieee8021FrerStreamSplitOutputHandleListEntry 1 }

  ieee8021FrerStreamSplitOutputIdHandle OBJECT-TYPE
      SYNTAX      VariablePointer
      MAX-ACCESS  read-create
      STATUS      current
      DESCRIPTION
          "A pointer to the ieee8021StreamIdStreamIdHandle that is an
           element of the ieee8021FrerStreamSplitOutputIdList instance in
           the ieee8021FrerStreamSplitTable."
      REFERENCE   "10.6.1.4"
      ::= { ieee8021FrerStreamSplitOutputHandleListEntry 2 }

  ieee8021FrerStreamSplitOutputHandleListStatus OBJECT-TYPE
      SYNTAX      RowStatus
      MAX-ACCESS  read-create
      STATUS      current
      DESCRIPTION
          "Row-Status for the Stream Split Output handle list table."
      ::= { ieee8021FrerStreamSplitOutputHandleListEntry 3 }

  -- ============================================================
  -- the ieee8021FrerSequenceAutoconfig table
  -- ============================================================

  ieee8021FrerSequenceAutoconfigTable OBJECT-TYPE
      SYNTAX      SEQUENCE OF Ieee8021FrerSequenceAutoconfigEntry
      MAX-ACCESS  not-accessible
      STATUS      current
      DESCRIPTION
          "A table containing a set of managed objects that specify how
           entries are automatically created (and destroyed) in the
           Stream identity table, the Sequence recovery table, and the
           Sequence identification table."
      REFERENCE   "10.7.1"
      ::= { ieee8021FrerSequenceAutoconfig 11 }

  ieee8021FrerSequenceAutoconfigEntry OBJECT-TYPE
      SYNTAX      Ieee8021FrerSequenceAutoconfigEntry
      MAX-ACCESS  not-accessible
      STATUS      current
      DESCRIPTION
          "A set of managed objects relating to a single class of Streams,
           and specifying how entries are created (and destroyed) in the
           Stream identity table, the Sequence recovery table, and the
           Sequence identification table."
      REFERENCE   "10.7.1.1"
      INDEX  { ieee8021FrerSequenceAutoconfigIndex }
      ::= { ieee8021FrerSequenceAutoconfigTable 1 }
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
1
 Ieee8021FrerSequenceAutoconfigEntry ::=
     SEQUENCE {
         ieee8021FrerSequenceAutoconfigIndex
             Unsigned32,
         ieee8021FrerSequenceAutoconfigSequenceEncapsulation
             Ieee8021CBSequenceEncodeDecodeType,
         ieee8021FrerSequenceAutoconfigSequenceEncapsulationOUI
             OCTET STRING,
         ieee8021FrerSequenceAutoconfigCustomSequenceEncapsulation
             Integer32,
         ieee8021FrerSequenceAutoconfigCustomSequenceEncOUI
             OCTET STRING,
         ieee8021FrerSequenceAutoconfigReceivePortList
             AutonomousType,
         ieee8021FrerSequenceAutoconfigTagged
             Ieee8021CBTaggedType,
         ieee8021FrerSequenceAutoconfigVlanList
             AutonomousType,
         ieee8021FrerSequenceAutoconfigRecoveryPortList
             AutonomousType,
         ieee8021FrerSequenceAutoconfigDestructMSec
             Integer32,
         ieee8021FrerSequenceAutoconfigResetMSec
             Unsigned32,
         ieee8021FrerSequenceAutoconfigAlgorithm
             Ieee8021CBSequenceRecoveryAlgorithm,
         ieee8021FrerSequenceAutoconfigAlgorithmOUI
             OCTET STRING,
         ieee8021FrerSequenceAutoconfigCustomAlgorithm
             Integer32,
         ieee8021FrerSequenceAutoconfigCustomAlgorithmOUI
             OCTET STRING,
         ieee8021FrerSequenceAutoconfigHistoryLength
             Integer32,
         ieee8021FrerSequenceAutoconfigCreateIndividual
             TruthValue,
         ieee8021FrerSequenceAutoconfigCreateRecovery
             TruthValue,
         ieee8021FrerSequenceAutoconfigLatentErrorDetection
             TruthValue,
         ieee8021FrerSequenceAutoconfigLatentErrorDifference
             Integer32,
         ieee8021FrerSequenceAutoconfigLatentErrorPeriod
             Integer32,
         ieee8021FrerSequenceAutoconfigLatentErrorResetPeriod
             Integer32,
         ieee8021FrerSequenceAutoconfigStatus
             RowStatus
     }

 ieee8021FrerSequenceAutoconfigIndex OBJECT-TYPE
     SYNTAX      Unsigned32
     MAX-ACCESS  not-accessible
     STATUS      current
     DESCRIPTION
         "Index for the Sequence Autoconfiguration table."
     ::= { ieee8021FrerSequenceAutoconfigEntry 1 }

 ieee8021FrerSequenceAutoconfigSequenceEncapsulation OBJECT-TYPE
     SYNTAX      Ieee8021CBSequenceEncodeDecodeType
     MAX-ACCESS  read-create
     STATUS      current
     DESCRIPTION
         "This managed object specifies which Sequence encode/decode
          function, and therefore, which type sequence_number encoding,
          is to be recognized for the purposes of Autoconfiguration."
     REFERENCE   "10.7.1.1.1"
     ::= { ieee8021FrerSequenceAutoconfigEntry 2 }

 ieee8021FrerSequenceAutoconfigSequenceEncapsulationOUI OBJECT-TYPE
     SYNTAX      OCTET STRING (SIZE (3))
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
1       MAX-ACCESS  read-only
        STATUS      current
        DESCRIPTION
            "This managed object specifies which Sequence encode/decode
             function, and therefore, which type sequence_number encoding,
             is to be recognized for the purposes of Autoconfiguration.
             It is used if and only if the value for
             ieee8021FrerSequenceAutoconfigSequenceEncapsulation is
             in the range of 0-255. In this case it always takes the value
             00-80-C2."
        REFERENCE   "10.7.1.1.1"
        ::= { ieee8021FrerSequenceAutoconfigEntry 3 }

    ieee8021FrerSequenceAutoconfigCustomSequenceEncapsulation OBJECT-TYPE
        SYNTAX      Integer32 (256..2147483647)
        MAX-ACCESS  read-create
        STATUS      current
        DESCRIPTION
            "This managed object specifies which Sequence encode/decode
             function, and therefore, which type sequence_number encoding,
             is to be recognized for the purposes of Autoconfiguration.
             This managed object is used if and only if the value for
             ieee8021FrerSequenceAutoconfigSequenceEncapsulation is
             256."
        REFERENCE   "10.7.1.1.1"
        ::= { ieee8021FrerSequenceAutoconfigEntry 4 }

    ieee8021FrerSequenceAutoconfigCustomSequenceEncOUI OBJECT-TYPE
        SYNTAX      OCTET STRING (SIZE (3))
        MAX-ACCESS  read-create
        STATUS      current
        DESCRIPTION
            "This managed object specifies which Sequence encode/decode
             function, and therefore, which type sequence_number encoding,
             is to be recognized for the purposes of Autoconfiguration.
             This managed object is used if and only if the value for
             ieee8021FrerSequenceAutoconfigSequenceEncapsulation is
             256."
        REFERENCE   "10.7.1.1.1"
        ::= { ieee8021FrerSequenceAutoconfigEntry 5 }

    ieee8021FrerSequenceAutoconfigReceivePortList OBJECT-TYPE
        SYNTAX      AutonomousType
        MAX-ACCESS  read-create
        STATUS      current
        DESCRIPTION
            "The list of ports to which this Sequence Autoconfiguration
             entry applies, and on which Stream identification functions,
             Sequence encode/decode functions, and Individual recovery
             functions are to be autocreated.
             For each entry in the ieee8021FrerSequenceAutoconfigTable
             there exists a list of ports in the
             ieee8021FrerSequenceAutoconfigReceivePortHandleListTable."
        REFERENCE   "10.7.1.1.2"
        ::= { ieee8021FrerSequenceAutoconfigEntry 6 }

    ieee8021FrerSequenceAutoconfigTagged OBJECT-TYPE
        SYNTAX      Ieee8021CBTaggedType
        MAX-ACCESS  read-create
        STATUS      current
        DESCRIPTION
            "An enumerated value indicating whether packets to be matched by
             this Autorecovery entry are permitted to have a VLAN tag."
        REFERENCE   "10.7.1.1.3"
        ::= { ieee8021FrerSequenceAutoconfigEntry 7 }

    ieee8021FrerSequenceAutoconfigVlanList OBJECT-TYPE
        SYNTAX      AutonomousType
        MAX-ACCESS  read-create
        STATUS      current
        DESCRIPTION
            "A list of vlan_identifiers for the packet to match. A null list
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
1          matches all vlan_identifiers.
           For each entry in the ieee8021FrerSequenceAutoconfigTable
           there exists a list of vlan_identifiers in the
           ieee8021FrerSequenceAutoconfigVlanListTable."
      REFERENCE   "10.7.1.1.4"
      ::= { ieee8021FrerSequenceAutoconfigEntry 8 }

  ieee8021FrerSequenceAutoconfigRecoveryPortList OBJECT-TYPE
      SYNTAX      AutonomousType
      MAX-ACCESS  read-create
      STATUS      current
      DESCRIPTION
          "The list of ports on which Sequence recovery functions are to
           be autocreated by this Autorecovery entry.
           For each entry in the ieee8021FrerSequenceAutoconfigTable
           there exists a list of ports in the
           ieee8021FrerSequenceAutoconfigRecoveryPortHandleListTable."
      REFERENCE   "10.7.1.1.5"
      ::= { ieee8021FrerSequenceAutoconfigEntry 9 }

  ieee8021FrerSequenceAutoconfigDestructMSec OBJECT-TYPE
      SYNTAX      Integer32
      MAX-ACCESS  read-create
      STATUS      current
      DESCRIPTION
          "This managed object specifies the number of milliseconds after
           which an idle set of functions created by this Autorecovery
           entry can be destroyed. A value of 0 indicates that idle
           autoconfigured functions are not to be destroyed."
      REFERENCE   "10.7.1.1.6"
      DEFVAL  { 86400000 }
      ::= { ieee8021FrerSequenceAutoconfigEntry 10 }

  ieee8021FrerSequenceAutoconfigResetMSec OBJECT-TYPE
      SYNTAX      Unsigned32
      MAX-ACCESS  read-create
      STATUS      current
      DESCRIPTION
          "This managed object specifies the value used to fill
           ieee8021FrerSequenceRecoveryResetMSec when autoconfiguring
           entries in the Sequence recovery table."
      REFERENCE   "10.7.1.1.7"
      ::= { ieee8021FrerSequenceAutoconfigEntry 11 }

  ieee8021FrerSequenceAutoconfigAlgorithm OBJECT-TYPE
      SYNTAX      Ieee8021CBSequenceRecoveryAlgorithm
      MAX-ACCESS  read-create
      STATUS      current
      DESCRIPTION
          "This managed object specifies the value used to fill
           ieee8021FrerSequenceRecoveryAlgorithm when autoconfiguring
           entries in the Sequence recovery table."
      REFERENCE   "10.7.1.1.8"
      ::= { ieee8021FrerSequenceAutoconfigEntry 12 }

  ieee8021FrerSequenceAutoconfigAlgorithmOUI OBJECT-TYPE
      SYNTAX      OCTET STRING (SIZE (3))
      MAX-ACCESS  read-only
      STATUS      current
      DESCRIPTION
          "This managed object specifies the value used to fill
           ieee8021FrerSequenceRecoveryAlgorithmOUI when autoconfiguring
           entries in the Sequence recovery table."
      REFERENCE   "10.7.1.1.8"
      ::= { ieee8021FrerSequenceAutoconfigEntry 13 }

  ieee8021FrerSequenceAutoconfigCustomAlgorithm OBJECT-TYPE
      SYNTAX      Integer32 (256..2147483647)
      MAX-ACCESS  read-create
      STATUS      current
      DESCRIPTION
          "This managed object specifies the value used to fill
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
1          ieee8021FrerSequenceCustomRecoveryAlgorithm when
           autoconfiguring entries in the Sequence recovery table."
      REFERENCE   "10.7.1.1.8"
      ::= { ieee8021FrerSequenceAutoconfigEntry 14 }

  ieee8021FrerSequenceAutoconfigCustomAlgorithmOUI OBJECT-TYPE
      SYNTAX      OCTET STRING (SIZE (3))
      MAX-ACCESS  read-create
      STATUS      current
      DESCRIPTION
          "This managed object specifies the value used to fill
           ieee8021FrerSequenceCustomRecoveryAlgorithmOUI
           when autoconfiguring entries in the Sequence recovery table."
      REFERENCE   "10.7.1.1.8"
      ::= { ieee8021FrerSequenceAutoconfigEntry 15 }

  ieee8021FrerSequenceAutoconfigHistoryLength OBJECT-TYPE
      SYNTAX      Integer32
      MAX-ACCESS  read-create
      STATUS      current
      DESCRIPTION
          "This managed object specifies the value used to fill
           ieee8021FrerSequenceRecoveryHistoryLength when autoconfiguring
           entries in the Sequence recovery table."
      REFERENCE   "10.7.1.1.9"
      ::= { ieee8021FrerSequenceAutoconfigEntry 16 }

  ieee8021FrerSequenceAutoconfigCreateIndividual OBJECT-TYPE
      SYNTAX      TruthValue
      MAX-ACCESS  read-create
      STATUS      current
      DESCRIPTION
          "This managed object specifies if the receipt of a packet that
           triggers the autoconfiguration of a new entry in the Stream
           Identity table also triggers the instantiation of a new entry
           in the Sequence Generation table for an Individual recovery
           function (true) or not (false)."
      REFERENCE   "10.7.1.1.10"
      ::= { ieee8021FrerSequenceAutoconfigEntry 17 }

  ieee8021FrerSequenceAutoconfigCreateRecovery OBJECT-TYPE
      SYNTAX      TruthValue
      MAX-ACCESS  read-create
      STATUS      current
      DESCRIPTION
          "This managed object specifies if the receipt of a packet that
           triggers the autoconfiguration of a new entry in the Stream
           Identity table also triggers the instantiation of a new entry
           in the Sequence Generation table for a Sequence recovery
           function (true) or not (false)."
      REFERENCE   "10.7.1.1.11"
      ::= { ieee8021FrerSequenceAutoconfigEntry 18 }

  ieee8021FrerSequenceAutoconfigLatentErrorDetection OBJECT-TYPE
      SYNTAX      TruthValue
      MAX-ACCESS  read-create
      STATUS      current
      DESCRIPTION
          "This managed object specifies if the autoconfiguration of a
           new Sequence recovery function also creates an associated
           Latent Error Detection function (true) or not (false)."
      REFERENCE   "10.7.1.1.12"
      ::= { ieee8021FrerSequenceAutoconfigEntry 19 }

  ieee8021FrerSequenceAutoconfigLatentErrorDifference OBJECT-TYPE
      SYNTAX      Integer32
      MAX-ACCESS  read-create
      STATUS      current
      DESCRIPTION
          "This managed object specifies the value used to fill
           ieee8021FrerSequenceRecoveryLatentErrorDifference when
           autoconfiguring entries in the Sequence recovery table."
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
1    REFERENCE   "10.7.1.1.13"
     ::= { ieee8021FrerSequenceAutoconfigEntry 20 }

 ieee8021FrerSequenceAutoconfigLatentErrorPeriod OBJECT-TYPE
     SYNTAX      Integer32
     MAX-ACCESS  read-create
     STATUS      current
     DESCRIPTION
         "This managed object specifies the value used to fill
          ieee8021FrerSequenceRecoveryLatentErrorPeriod when
          autoconfiguring entries in the Sequence recovery table."
     REFERENCE   "10.7.1.1.14"
     ::= { ieee8021FrerSequenceAutoconfigEntry 21 }

 ieee8021FrerSequenceAutoconfigLatentErrorResetPeriod OBJECT-TYPE
     SYNTAX      Integer32
     MAX-ACCESS  read-create
     STATUS      current
     DESCRIPTION
         "This managed object specifies the value used to fill
          ieee8021FrerSequenceRecoveryLatentResetPeriod when
          autoconfiguring entries in the Sequence recovery table."
     REFERENCE   "10.7.1.1.15"
     ::= { ieee8021FrerSequenceAutoconfigEntry 22 }

 ieee8021FrerSequenceAutoconfigStatus OBJECT-TYPE
     SYNTAX      RowStatus
     MAX-ACCESS  read-create
     STATUS      current
     DESCRIPTION
         "Row-Status for the Sequence Autoconfiguration table."
     ::= { ieee8021FrerSequenceAutoconfigEntry 23 }

 -- ============================================================
 -- the ieee8021FrerSequenceAutoconfigReceivePortHandleList
 -- table
 -- ============================================================
 ieee8021FrerSequenceAutoconfigReceivePortHandleListTable OBJECT-TYPE
     SYNTAX      SEQUENCE OF
 Ieee8021FrerSequenceAutoconfigReceivePortHandleListEntry
     MAX-ACCESS  not-accessible
     STATUS      current
     DESCRIPTION
         "A table containing a list of ports to which this
          frerAutSeqEntry applies, and on which Stream identification
          functions, Sequence encode/decode functions, and Individual
          recovery functions are to be autocreated, referenced in
          ieee8021FrerSequenceAutoconfigIndex."
     REFERENCE   "10.7.1.1.2"
     ::= { ieee8021FrerSequenceAutoconfigReceivePortHandleList 12}

 ieee8021FrerSequenceAutoconfigReceivePortHandleListEntry OBJECT-TYPE
     SYNTAX      Ieee8021FrerSequenceAutoconfigReceivePortHandleListEntry
     MAX-ACCESS  not-accessible
     STATUS      current
     DESCRIPTION
         "A set of managed objects providing the ports to which this
          frerAutSeqEntry applies, and on which Stream identification
          functions, Sequence encode/decode functions, and Individual
          recovery functions are to be autocreated, referenced in
          ieee8021FrerSequenceAutoconfigIndex."
     REFERENCE   "10.7.1.1.2"
     INDEX   { ieee8021FrerSequenceAutoconfigIndex,
               ieee8021FrerSequenceAutoconfigReceivePortHandleListIndex
             }
     ::= { ieee8021FrerSequenceAutoconfigReceivePortHandleListTable 1 }

 Ieee8021FrerSequenceAutoconfigReceivePortHandleListEntry ::=
     SEQUENCE {
         ieee8021FrerSequenceAutoconfigReceivePortHandleListIndex
             Unsigned32,
         ieee8021FrerSequenceAutoconfigReceivePortHandle
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
            VariablePointer,
        ieee8021FrerSequenceAutoconfigReceivePortHandleListStatus
            RowStatus
    }

ieee8021FrerSequenceAutoconfigReceivePortHandleListIndex OBJECT-TYPE
    SYNTAX      Unsigned32
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "Index for the Sequence Autoconfiguration Receive Port handle
         list table."
    ::= { ieee8021FrerSequenceAutoconfigReceivePortHandleListEntry 1 }

ieee8021FrerSequenceAutoconfigReceivePortHandle OBJECT-TYPE
    SYNTAX      VariablePointer
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "A pointer to the ieee8021StreamIdStreamIdHandle that is an
         element of the ieee8021FrerSequenceAutoconfigReceivePortList
         instance in the ieee8021FrerSequenceAutoconfigTable."
    REFERENCE   "10.7.1.1.2"
    ::= { ieee8021FrerSequenceAutoconfigReceivePortHandleListEntry 2 }

ieee8021FrerSequenceAutoconfigReceivePortHandleListStatus OBJECT-TYPE
    SYNTAX      RowStatus
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "Row-Status for the Sequence Autoconfiguration Receive Port
         handle list table."
    ::= { ieee8021FrerSequenceAutoconfigReceivePortHandleListEntry 3 }

-- ============================================================
-- the ieee8021FrerSequenceAutoconfigVlanHandleList table
-- ============================================================
ieee8021FrerSequenceAutoconfigVlanListTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF Ieee8021FrerSequenceAutoconfigVlanListEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A table containing a list of vlan_identifiers for the packet to
         match. A null list matches all vlan_identifiers."
    REFERENCE   "10.7.1.1.4"
    ::= { ieee8021FrerSequenceAutoconfigVlanHandleList 13}

ieee8021FrerSequenceAutoconfigVlanListEntry OBJECT-TYPE
    SYNTAX      Ieee8021FrerSequenceAutoconfigVlanListEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A set of managed objects providing the list of vlan_identifiers
         for the packet to match. A null list matches all
         vlan_identifiers."
    REFERENCE   "10.7.1.1.4"
    INDEX   { ieee8021FrerSequenceAutoconfigIndex,
              ieee8021FrerSequenceAutoconfigVlanListIndex
            }
    ::= { ieee8021FrerSequenceAutoconfigVlanListTable 1 }

Ieee8021FrerSequenceAutoconfigVlanListEntry ::=
    SEQUENCE {
        ieee8021FrerSequenceAutoconfigVlanListIndex
            Unsigned32,
        ieee8021FrerSequenceAutoconfigVlan
            Ieee8021CBVlanIdentifier,
        ieee8021FrerSequenceAutoconfigVlanListStatus
            RowStatus
    }

ieee8021FrerSequenceAutoconfigVlanListIndex OBJECT-TYPE
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
1       SYNTAX      Unsigned32
        MAX-ACCESS  not-accessible
        STATUS      current
        DESCRIPTION
            "Index for the Sequence Autoconfiguration VLAN list table."
        ::= { ieee8021FrerSequenceAutoconfigVlanListEntry 1 }

    ieee8021FrerSequenceAutoconfigVlan OBJECT-TYPE
        SYNTAX      Ieee8021CBVlanIdentifier
        MAX-ACCESS  read-create
        STATUS      current
        DESCRIPTION
            "A vlan_identifier for the packet to match."
        REFERENCE   "10.7.1.1.4"
        ::= { ieee8021FrerSequenceAutoconfigVlanListEntry 2 }

    ieee8021FrerSequenceAutoconfigVlanListStatus OBJECT-TYPE
        SYNTAX      RowStatus
        MAX-ACCESS  read-create
        STATUS      current
        DESCRIPTION
            "Row-Status for the Sequence Autoconfiguration VLAN list
             table."
        ::= { ieee8021FrerSequenceAutoconfigVlanListEntry 3 }

    -- ============================================================
    -- the ieee8021FrerSequenceAutoconfigRecoveryPortHandleList
    -- table
    -- ============================================================
    ieee8021FrerSequenceAutoconfigRecoveryPortHandleListTable OBJECT-TYPE
        SYNTAX      SEQUENCE OF
    Ieee8021FrerSequenceAutoconfigRecoveryPortHandleListEntry
        MAX-ACCESS  not-accessible
        STATUS      current
        DESCRIPTION
            "A table containing a list of ports on which Sequence recovery
             functions are to be autocreated by this frerAutSeqEntry,
             referenced in ieee8021FrerSequenceAutoconfigIndex."
        REFERENCE   "10.7.1.1.5"
        ::= { ieee8021FrerSequenceAutoconfigRecoveryPortHandleList 14}

    ieee8021FrerSequenceAutoconfigRecoveryPortHandleListEntry OBJECT-TYPE
        SYNTAX      Ieee8021FrerSequenceAutoconfigRecoveryPortHandleListEntry
        MAX-ACCESS  not-accessible
        STATUS      current
        DESCRIPTION
            "A set of managed objects providing the ports on which Sequence
             recovery functions are to be autocreated by this
             frerAutSeqEntry, referenced in
             ieee8021FrerSequenceAutoconfigIndex."
        REFERENCE   "10.7.1.1.5"
        INDEX   { ieee8021FrerSequenceAutoconfigIndex,
                  ieee8021FrerSequenceAutoconfigRecoveryPortHandleListIndex
                }
        ::= { ieee8021FrerSequenceAutoconfigRecoveryPortHandleListTable 1 }

    Ieee8021FrerSequenceAutoconfigRecoveryPortHandleListEntry ::=
        SEQUENCE {
            ieee8021FrerSequenceAutoconfigRecoveryPortHandleListIndex
                Unsigned32,
            ieee8021FrerSequenceAutoconfigRecoveryPortHandle
                VariablePointer,
            ieee8021FrerSequenceAutoconfigRecoveryPortHandleListStatus
                RowStatus
        }

    ieee8021FrerSequenceAutoconfigRecoveryPortHandleListIndex OBJECT-TYPE
        SYNTAX      Unsigned32
        MAX-ACCESS  not-accessible
        STATUS      current
        DESCRIPTION
            "Index for the Sequence Autoconfiguration Recovery Port handle
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
1          list table."
     ::= { ieee8021FrerSequenceAutoconfigRecoveryPortHandleListEntry 1 }

 ieee8021FrerSequenceAutoconfigRecoveryPortHandle OBJECT-TYPE
     SYNTAX      VariablePointer
     MAX-ACCESS  read-create
     STATUS      current
     DESCRIPTION
         "A pointer to the ieee8021StreamIdStreamIdHandle that is an
          element of the ieee8021FrerSequenceAutoconfigRecoveryPortList
          instance in the ieee8021FrerSequenceAutoconfigTable."
     REFERENCE   "10.7.1.1.5"
     ::= { ieee8021FrerSequenceAutoconfigRecoveryPortHandleListEntry 2 }

 ieee8021FrerSequenceAutoconfigRecoveryPortHandleListStatus OBJECT-TYPE
     SYNTAX      RowStatus
     MAX-ACCESS  read-create
     STATUS      current
     DESCRIPTION
         "Row-Status for the Sequence Autoconfiguration Recovery Port
          handle list table."
     ::= { ieee8021FrerSequenceAutoconfigRecoveryPortHandleListEntry 3 }

 -- ================================================================
 -- the ieee8021FrerOutputAutoconfig table
 -- ================================================================

 ieee8021FrerOutputAutoconfigTable OBJECT-TYPE
     SYNTAX      SEQUENCE OF Ieee8021FrerOutputAutoconfigEntry
     MAX-ACCESS  not-accessible
     STATUS      current
     DESCRIPTION
         "A table containing a set of managed objects that specify how
          active entries are created in the Sequence identification
          table."
     REFERENCE   "10.7.2"
     ::= { ieee8021FrerOutputAutoconfig 15 }

 ieee8021FrerOutputAutoconfigEntry OBJECT-TYPE
     SYNTAX      Ieee8021FrerOutputAutoconfigEntry
     MAX-ACCESS  not-accessible
     STATUS      current
     DESCRIPTION
         "A set of managed objects relating to a single class of Streams
          specifying how active entries are created in the Sequence
          identification table."
     REFERENCE   "10.7.2.1"
     INDEX  { ieee8021FrerOutputAutoconfigIndex }
     ::= { ieee8021FrerOutputAutoconfigTable 1 }

 Ieee8021FrerOutputAutoconfigEntry ::=
     SEQUENCE {
         ieee8021FrerOutputAutoconfigIndex
             Unsigned32,
         ieee8021FrerOutputAutoconfigPortList
             AutonomousType,
         ieee8021FrerOutputAutoconfigEncapsulation
             Ieee8021CBSequenceEncodeDecodeType,
         ieee8021FrerOutputAutoconfigEncapsulationOUI
             OCTET STRING,
         ieee8021FrerOutputAutoconfigCustomEncapsulation
             Integer32,
         ieee8021FrerOutputAutoconfigCustomEncapsulationOUI
             OCTET STRING,
         ieee8021FrerOutputAutoconfigLanPathId
             Ieee8021CBLanPathIdType,
         ieee8021FrerOutputAutoconfigStatus
             RowStatus
     }

 ieee8021FrerOutputAutoconfigIndex OBJECT-TYPE
     SYNTAX      Unsigned32
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
1     MAX-ACCESS  not-accessible
      STATUS      current
      DESCRIPTION
          "Index for the Output Autoconfiguration table."
      ::= { ieee8021FrerOutputAutoconfigEntry 1 }

  ieee8021FrerOutputAutoconfigPortList OBJECT-TYPE
      SYNTAX      AutonomousType
      MAX-ACCESS  read-create
      STATUS      current
      DESCRIPTION
          "The list of ports to which this Output Autoconfiguration entry
           applies, and on which active Sequence encode/decode functions
           are to be autocreated.
           For each entry in the ieee8021FrerOutputAutoconfigTable
           there exists a list of ports in the
           ieee8021FrerOutputAutoconfigPortHandleListTable."
      REFERENCE   "10.7.2.1.1"
      ::= { ieee8021FrerOutputAutoconfigEntry 2 }

  ieee8021FrerOutputAutoconfigEncapsulation OBJECT-TYPE
      SYNTAX      Ieee8021CBSequenceEncodeDecodeType
      MAX-ACCESS  read-create
      STATUS      current
      DESCRIPTION
           "This managed object specifies which Sequence encode/decode
            function, and therefore, which type sequence_number encoding,
            is to be used for autoconfigured Streams on the ports in
            ieee8021FrerSequenceAutoconfigReceivePortList."
      REFERENCE   "10.7.2.1.2"
      ::= { ieee8021FrerOutputAutoconfigEntry 3 }

  ieee8021FrerOutputAutoconfigEncapsulationOUI OBJECT-TYPE
      SYNTAX      OCTET STRING (SIZE (3))
      MAX-ACCESS  read-only
      STATUS      current
      DESCRIPTION
          "This managed object specifies which Sequence encode/decode
           function, and therefore, which type sequence_number encoding,
           is to be used for autoconfigured Streams on the ports in
           ieee8021FrerSequenceAutoconfigReceivePortList.
           It is used if and only if the value for
           ieee8021FrerOutputAutoconfigEncapsulation is in the
           range of 0-255. In this case it always takes the value
           00-80-C2."
      REFERENCE   "10.7.2.1.2"
      ::= { ieee8021FrerOutputAutoconfigEntry 4 }

  ieee8021FrerOutputAutoconfigCustomEncapsulation OBJECT-TYPE
      SYNTAX      Integer32 (256..2147483647)
      MAX-ACCESS  read-create
      STATUS      current
      DESCRIPTION
          "This managed object specifies which Sequence encode/decode
           function, and therefore, which type sequence_number encoding,
           is to be used for autoconfigured Streams on the ports in
           ieee8021FrerSequenceAutoconfigReceivePortList. This
           managed object is used if and only if the value for
           ieee8021FrerOutputAutoconfigEncapsulation is 256."
      REFERENCE   "10.7.2.1.2"
      ::= { ieee8021FrerOutputAutoconfigEntry 5 }

  ieee8021FrerOutputAutoconfigCustomEncapsulationOUI OBJECT-TYPE
      SYNTAX      OCTET STRING (SIZE (3))
      MAX-ACCESS  read-create
      STATUS      current
      DESCRIPTION
          "This managed object specifies which Sequence encode/decode
           function, and therefore, which type sequence_number encoding,
           is to be used for autoconfigured Streams on the ports in
           ieee8021FrerSequenceAutoconfigReceivePortList. This
           managed object is used if and only if the value for
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
1          ieee8021FrerOutputAutoconfigEncapsulation is 256."
     REFERENCE   "10.7.2.1.2"
     ::= { ieee8021FrerOutputAutoconfigEntry 6 }

 ieee8021FrerOutputAutoconfigLanPathId OBJECT-TYPE
     SYNTAX      Ieee8021CBLanPathIdType
     MAX-ACCESS  read-create
     STATUS      current
     DESCRIPTION
         "An integer specifying a path or LAN. If and only if
          ieee8021FrerOutputAutoconfigEncapsulation specifies HSR
          or PRP this managed object specifies the LanId or PathId value
          to be inserted into the HSR sequence tag or PRP sequence
          trailer of autoconfigured packets transmitted on the ports
          in ieee8021FrerSequenceAutoconfigReceivePortList."
     REFERENCE   "10.7.2.1.3"
     ::= { ieee8021FrerOutputAutoconfigEntry 7 }

 ieee8021FrerOutputAutoconfigStatus OBJECT-TYPE
     SYNTAX      RowStatus
     MAX-ACCESS  read-create
     STATUS      current
     DESCRIPTION
         "Row-Status for the Output Autoconfiguration table."
     ::= { ieee8021FrerOutputAutoconfigEntry 8 }

 -- ============================================================
 -- the ieee8021FrerOutputAutoconfigPortHandleList table
 -- ============================================================
 ieee8021FrerOutputAutoconfigPortHandleListTable OBJECT-TYPE
     SYNTAX      SEQUENCE OF Ieee8021FrerOutputAutoconfigPortHandleListEntry
     MAX-ACCESS  not-accessible
     STATUS      current
     DESCRIPTION
         "A table containing a list of ports to which this
          frerAutOutEntry applies, and on which active Sequence
          encode/decode functions are to be autocreated, referenced in
          ieee8021FrerOutputAutoconfigIndex."
     REFERENCE   "10.7.2.1.1"
     ::= { ieee8021FrerOutputAutoconfigPortHandleList 16}

 ieee8021FrerOutputAutoconfigPortHandleListEntry OBJECT-TYPE
     SYNTAX      Ieee8021FrerOutputAutoconfigPortHandleListEntry
     MAX-ACCESS  not-accessible
     STATUS      current
     DESCRIPTION
         "A set of managed objects providing the ports to which this
          frerAutOutEntry applies, and on which active Sequence
          encode/decode functions are to be autocreated, referenced in
          ieee8021FrerOutputAutoconfigIndex."
     REFERENCE   "10.7.2.1.1"
     INDEX   { ieee8021FrerOutputAutoconfigIndex,
               ieee8021FrerOutputAutoconfigPortHandleListIndex
             }
     ::= { ieee8021FrerOutputAutoconfigPortHandleListTable 1 }

 Ieee8021FrerOutputAutoconfigPortHandleListEntry ::=
     SEQUENCE {
         ieee8021FrerOutputAutoconfigPortHandleListIndex
             Unsigned32,
         ieee8021FrerOutputAutoconfigPortHandle
             VariablePointer,
         ieee8021FrerOutputAutoconfigPortHandleListStatus
             RowStatus
     }

 ieee8021FrerOutputAutoconfigPortHandleListIndex OBJECT-TYPE
     SYNTAX      Unsigned32
     MAX-ACCESS  not-accessible
     STATUS      current
     DESCRIPTION
         "Index for the Output Autoconfiguration Port handle list
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
1          table."
      ::= { ieee8021FrerOutputAutoconfigPortHandleListEntry 1 }

  ieee8021FrerOutputAutoconfigPortHandle OBJECT-TYPE
      SYNTAX      VariablePointer
      MAX-ACCESS  read-create
      STATUS      current
      DESCRIPTION
          "A pointer to the ieee8021StreamIdStreamIdHandle that is an
           element of the ieee8021FrerOutputAutoconfigPortList instance in
           the ieee8021FrerOutputAutoconfigTable."
      REFERENCE   "10.7.2.1.1"
      ::= { ieee8021FrerOutputAutoconfigPortHandleListEntry 2 }

  ieee8021FrerOutputAutoconfigPortHandleListStatus OBJECT-TYPE
      SYNTAX      RowStatus
      MAX-ACCESS  read-create
      STATUS      current
      DESCRIPTION
          "Row-Status for the Output Autoconfiguration Port handle list
           table."
      ::= { ieee8021FrerOutputAutoconfigPortHandleListEntry 3 }

  -- ============================================================
  -- the ieee8021FrerPerPortPerStreamCounters table
  -- ============================================================

  ieee8021FrerPerPortPerStreamCountersTable OBJECT-TYPE
      SYNTAX      SEQUENCE OF Ieee8021FrerPerPortPerStreamCountersEntry
      MAX-ACCESS  not-accessible
      STATUS      current
      DESCRIPTION
              "A table containing a set of counters for frame replication
               and elimination for reliability that are instantiated per
               port, Stream, and direction."
      REFERENCE   "10.8"
      ::= { ieee8021FrerPerPortPerStreamCounters 17 }

  ieee8021FrerPerPortPerStreamCountersEntry OBJECT-TYPE
      SYNTAX      Ieee8021FrerPerPortPerStreamCountersEntry
      MAX-ACCESS  not-accessible
      STATUS      current
      DESCRIPTION
              "A a set of managed objects for FRER counters that are
               instantiated per port, Stream, and direction."
      REFERENCE   "10.8"
       INDEX  { ifIndex,
                ieee8021StreamIdStreamIdHandle,
                ieee8021FrerPerPortPerStreamDirection
              }
      ::= { ieee8021FrerPerPortPerStreamCountersTable 1 }

  Ieee8021FrerPerPortPerStreamCountersEntry ::=
      SEQUENCE {
          ieee8021FrerPerPortPerStreamDirection
              TruthValue,
          ieee8021FrerPerPortPerStreamSeqGenResets
              Counter64,
          ieee8021FrerPerPortPerStreamSeqRecoveryOutOfOrderPackets
              Counter64,
          ieee8021FrerPerPortPerStreamSeqRecoveryRoguePackets
              Counter64,
          ieee8021FrerPerPortPerStreamSeqRecoveryPassedPackets
              Counter64,
          ieee8021FrerPerPortPerStreamSeqRecoveryDiscardedPackets
              Counter64,
          ieee8021FrerPerPortPerStreamSeqRecoveryLostPackets
              Counter64,
          ieee8021FrerPerPortPerStreamSeqRecoveryTaglessPackets
              Counter64,
          ieee8021FrerPerPortPerStreamSeqRecoveryResets
              Counter64,
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
1         ieee8021FrerPerPortPerStreamSeqRecoveryLatentErrorResets
              Counter64,
          ieee8021FrerPerPortPerStreamSeqEncErroredPackets
              Counter64
      }

  ieee8021FrerPerPortPerStreamDirection OBJECT-TYPE
      SYNTAX      TruthValue
      MAX-ACCESS  not-accessible
      STATUS      current
      DESCRIPTION
          "A managed object specifying the facing of a given Stream
           on a port. The Stream can be either in-facing (False) or
           out-facing (True)."
      REFERENCE   "10.8"
      ::= { ieee8021FrerPerPortPerStreamCountersEntry 1 }

  ieee8021FrerPerPortPerStreamSeqGenResets OBJECT-TYPE
      SYNTAX      Counter64
      MAX-ACCESS  read-only
      STATUS      current
      DESCRIPTION
          "A managed object serving as a counter that is incremented each
           time the SequenceGenerationReset function is called."
      REFERENCE   "10.8.2"
      ::= { ieee8021FrerPerPortPerStreamCountersEntry 2 }

  ieee8021FrerPerPortPerStreamSeqRecoveryOutOfOrderPackets OBJECT-TYPE
      SYNTAX      Counter64
      MAX-ACCESS  read-only
      STATUS      current
      DESCRIPTION
          "A managed object serving as a counter that is incremented once
           for each packet accepted out-of-order by the
           VectorRecoveryAlgorithm or MatchRecoveryAlgorithm. Out-of-order
           means that the packet's sequence number is not one more than
           the previous packet received."
      REFERENCE   "10.8.3"
      ::= { ieee8021FrerPerPortPerStreamCountersEntry 3 }

  ieee8021FrerPerPortPerStreamSeqRecoveryRoguePackets OBJECT-TYPE
      SYNTAX      Counter64
      MAX-ACCESS  read-only
      STATUS      current
      DESCRIPTION
          "A managed object serving as a counter that is incremented once
           for each packet discarded by the VectorRecoveryAlgorithm
           because its sequence_number subparameter is more than
           ieee8021FrerSequenceRecoveryHistoryLength from RecovSeqNum."
      REFERENCE   "10.8.4"
      ::= { ieee8021FrerPerPortPerStreamCountersEntry 4 }

  ieee8021FrerPerPortPerStreamSeqRecoveryPassedPackets OBJECT-TYPE
      SYNTAX      Counter64
      MAX-ACCESS  read-only
      STATUS      current
      DESCRIPTION
          "A managed object serving as a counter that is incremented once
           for each packet passed up the stack by the
           VectorRecoveryAlgorithm or MatchRecoveryAlgorithm."
      REFERENCE   "10.8.5"
      ::= { ieee8021FrerPerPortPerStreamCountersEntry 5 }

  ieee8021FrerPerPortPerStreamSeqRecoveryDiscardedPackets OBJECT-TYPE
      SYNTAX      Counter64
      MAX-ACCESS  read-only
      STATUS      current
      DESCRIPTION
          "A managed object serving as a counter that is incremented once
           for each packet discarded due to a duplicate sequence number by
           the VectorRecoveryAlgorithm or MatchRecoveryAlgorithm."
      REFERENCE   "10.8.6"
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
    ::= { ieee8021FrerPerPortPerStreamCountersEntry 6 }

ieee8021FrerPerPortPerStreamSeqRecoveryLostPackets OBJECT-TYPE
    SYNTAX      Counter64
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
        "A managed object serving as a counter that is incremented once
         for each packet lost by the VectorRecoveryAlgorithm. A packet
         is counted as lost if its sequence number is not received on
         any ingress port"
    REFERENCE   "10.8.7"
    ::= { ieee8021FrerPerPortPerStreamCountersEntry 7 }

ieee8021FrerPerPortPerStreamSeqRecoveryTaglessPackets OBJECT-TYPE
    SYNTAX      Counter64
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
        "A managed object serving as a counter that is incremented once
         for each packet received by the VectorRecoveryAlgorithm that
         has no sequence_number subparameter."
    REFERENCE   "10.8.8"
    ::= { ieee8021FrerPerPortPerStreamCountersEntry 8 }

ieee8021FrerPerPortPerStreamSeqRecoveryResets OBJECT-TYPE
    SYNTAX      Counter64
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
        "A managed object serving as a counter that is incremented once
         each time the SequenceRecoveryReset function is called."
    REFERENCE   "10.8.9"
    ::= { ieee8021FrerPerPortPerStreamCountersEntry 9 }

ieee8021FrerPerPortPerStreamSeqRecoveryLatentErrorResets OBJECT-TYPE
    SYNTAX      Counter64
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
        "A managed object serving as a counter that is incremented once
         each time the LatentErrorReset function is called."
    REFERENCE   "10.8.10"
    ::= { ieee8021FrerPerPortPerStreamCountersEntry 10 }

ieee8021FrerPerPortPerStreamSeqEncErroredPackets OBJECT-TYPE
    SYNTAX      Counter64
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
        "A managed object serving as a counter that is incremented once
         each time the Sequence encode/decode function receives a packet
         that it is unable to decode successfully."
    REFERENCE   "10.8.11"
    ::= { ieee8021FrerPerPortPerStreamCountersEntry 11 }

-- ============================================================
-- the ieee8021FrerPerPortCounters table
-- ============================================================

ieee8021FrerPerPortCountersTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF Ieee8021FrerPerPortCountersEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A table containing a set of counters for frame replication
         and elimination for reliability that are instantiated per
         port and direction."
    REFERENCE   "10.9"
    ::= { ieee8021FrerPerPortCounters 18 }

ieee8021FrerPerPortCountersEntry OBJECT-TYPE
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
1     SYNTAX      Ieee8021FrerPerPortCountersEntry
      MAX-ACCESS  not-accessible
      STATUS      current
      DESCRIPTION
          "A a set of managed objects for FRER counters that are
           instantiated per port and direction."
      REFERENCE   "10.9"
       INDEX  { ifIndex }
      ::= { ieee8021FrerPerPortCountersTable 1 }

  Ieee8021FrerPerPortCountersEntry ::=
      SEQUENCE {
          ieee8021FrerPerPortSeqRecoveryPassedPackets
              Counter64,
          ieee8021FrerPerPortfrerCpSeqRecoveryDiscardPackets
              Counter64,
          ieee8021FrerPerPortfrerCpSeqEncErroredPackets
              Counter64
      }

  ieee8021FrerPerPortSeqRecoveryPassedPackets OBJECT-TYPE
      SYNTAX      Counter64
      MAX-ACCESS  read-only
      STATUS      current
      DESCRIPTION
          "A managed object serving as a counter that is incremented once
           for each packet passed up the stack by the
           VectorRecoveryAlgorithm or MatchRecoveryAlgorithm. Its value
           equals the sum (modulo the size of the counters) of all of the
           ieee8021FrerPerPortPerStreamSeqRecoveryPassedPackets counters on
           this same port."
      REFERENCE   "10.9.1"
      ::= { ieee8021FrerPerPortCountersEntry 1 }

  ieee8021FrerPerPortfrerCpSeqRecoveryDiscardPackets OBJECT-TYPE
      SYNTAX      Counter64
      MAX-ACCESS  read-only
      STATUS      current
      DESCRIPTION
          "A managed object serving as a counter that is incremented once
           for each packet discarded due to a duplicate sequence number or
           for being a rogue packet by any VectorRecoveryAlgorithm or
           MatchRecoveryAlgorithm on this port. Its value equals the sum
           (modulo the size of the counters) of all of the
           ieee8021FrerPerPortPerStreamSeqRecoveryRoguePackets and
           ieee8021FrerPerPortPerStreamSeqRecoveryDiscardedPackets counters on
           this same port."
      REFERENCE   "10.9.2"
      ::= { ieee8021FrerPerPortCountersEntry 2 }

  ieee8021FrerPerPortfrerCpSeqEncErroredPackets OBJECT-TYPE
      SYNTAX      Counter64
      MAX-ACCESS  read-only
      STATUS      current
      DESCRIPTION
          "A managed object serving as a counter that is incremented once
           each time the Sequence encode/decode function receives a packet
           that it is unable to decode successfully. Its value equals the
           sum (modulo the size of the counters) of all of the
           ieee8021FrerPerPortPerStreamSeqEncErroredPackets counters on
           this same port."
      REFERENCE   "10.9.3"
      ::= { ieee8021FrerPerPortCountersEntry 3 }

  -- ============================================================
  -- IEEE802 FRER MIB - Conformance Information
  -- ============================================================

  ieee8021FrerCompliances
      OBJECT IDENTIFIER ::= { ieee8021FrerConformance 1 }
  ieee8021FrerGroups
      OBJECT IDENTIFIER ::= { ieee8021FrerConformance 2 }
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
1
 -- ================================================================
 -- units of conformance
 -- ================================================================

 -- ================================================================
 -- the ieee8021FrerSequenceGeneration group
 -- ================================================================

 ieee8021FrerSequenceGenerationGroup OBJECT-GROUP
     OBJECTS {
         ieee8021FrerSequenceGenerationStreamList,
         ieee8021FrerSequenceGenerationDirection,
         ieee8021FrerSequenceGenerationReset,
         ieee8021FrerSequenceGenerationStreamHandle,
         ieee8021FrerSequenceGenerationHandleListStatus
     }
     STATUS      current
     DESCRIPTION
         "Objects that are part of the FRER sequence generation
          functionality."
     ::= { ieee8021FrerGroups 1 }

 -- ================================================================
 -- the ieee8021FrerSequenceRecovery group
 -- ================================================================

 ieee8021FrerSequenceRecoveryGroup OBJECT-GROUP
     OBJECTS {
         ieee8021FrerSequenceRecoveryStreamList,
         ieee8021FrerSequenceRecoveryPortList,
         ieee8021FrerSequenceRecoveryDirection,
         ieee8021FrerSequenceRecoveryReset,
         ieee8021FrerSequenceRecoveryAlgorithm,
         ieee8021FrerSequenceRecoveryAlgorithmOUI,
         ieee8021FrerSequenceCustomRecoveryAlgorithm,
         ieee8021FrerSequenceCustomRecoveryAlgorithmOUI,
         ieee8021FrerSequenceRecoveryHistoryLength,
         ieee8021FrerSequenceRecoveryResetMSec,
         ieee8021FrerSequenceRecoveryInvalidSequenceValue,
         ieee8021FrerSequenceRecoveryTakeNoSequence,
         ieee8021FrerSequenceRecoveryIndividualRecovery,
         ieee8021FrerSequenceRecoveryLatentErrorDetection,
         ieee8021FrerSequenceRecoveryLatentErrorDifference,
         ieee8021FrerSequenceRecoveryLatentErrorPeriod,
         ieee8021FrerSequenceRecoveryLatentErrorPaths,
         ieee8021FrerSequenceRecoveryLatentResetPeriod,
         ieee8021FrerSequenceRecoveryStatus,
         ieee8021FrerSequenceRecoveryStreamHandle,
         ieee8021FrerSequenceRecoveryHandleListStatus,
         ieee8021FrerSequenceRecoveryPortHandle,
         ieee8021FrerSequenceRecoveryPortHandleListStatus
     }
     STATUS      current
     DESCRIPTION
         "Objects that are part of the FRER sequence recovery
          functionality."
     ::= { ieee8021FrerGroups 2 }

 -- ================================================================
 -- the ieee8021FrerSequenceIdentification group
 -- ================================================================

 ieee8021FrerSequenceIdentificationGroup OBJECT-GROUP
     OBJECTS {
         ieee8021FrerSequenceIdentificationStreamList,
         ieee8021FrerSequenceIdentificationEncodeActive,
         ieee8021FrerSequenceIdentificationEncodeEncapsulationType,
         ieee8021FrerSequenceIdentificationEncodeEncapsulationOUI,
         ieee8021FrerSequenceIdentificationCustomEncodeEncapsulationType,
         ieee8021FrerSequenceIdentificationCustomEncodeEncapsulationOUI,
         ieee8021FrerSequenceIdentificationEncodePathIdLanId,
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
1           ieee8021FrerSequenceIdentificationStatus,
            ieee8021FrerSequenceIdentificationStreamHandle,
            ieee8021FrerSequenceIdentificationHandleListStatus
        }
        STATUS      current
        DESCRIPTION
            "Objects that are part of the FRER sequence identification
             functionality."
        ::= { ieee8021FrerGroups 3 }

    -- ============================================================
    -- the ieee8021FrerStreamSplit group
    -- ============================================================

    ieee8021FrerStreamSplitGroup OBJECT-GROUP
        OBJECTS {
            ieee8021FrerStreamSplitPort,
            ieee8021FrerStreamSplitDirection,
            ieee8021FrerStreamSplitInputIdList,
            ieee8021FrerStreamSplitOutputIdList,
            ieee8021FrerStreamSplitInputIdHandle,
            ieee8021FrerStreamSplitInputHandleListStatus,
            ieee8021FrerStreamSplitOutputIdHandle,
            ieee8021FrerStreamSplitOutputHandleListStatus
        }
        STATUS      current
        DESCRIPTION
            "Objects that are part of the FRER Stream split functionality."
        ::= { ieee8021FrerGroups 4 }

    -- ============================================================
    -- the ieee8021FrerSequenceAutoconfig group
    -- ============================================================

    ieee8021FrerSequenceAutoconfigGroup OBJECT-GROUP
        OBJECTS {
            ieee8021FrerSequenceAutoconfigSequenceEncapsulation,
            ieee8021FrerSequenceAutoconfigSequenceEncapsulationOUI,
            ieee8021FrerSequenceAutoconfigCustomSequenceEncapsulation,
            ieee8021FrerSequenceAutoconfigCustomSequenceEncOUI,
            ieee8021FrerSequenceAutoconfigReceivePortList,
            ieee8021FrerSequenceAutoconfigTagged,
            ieee8021FrerSequenceAutoconfigVlanList,
            ieee8021FrerSequenceAutoconfigRecoveryPortList,
            ieee8021FrerSequenceAutoconfigDestructMSec,
            ieee8021FrerSequenceAutoconfigResetMSec,
            ieee8021FrerSequenceAutoconfigAlgorithm,
            ieee8021FrerSequenceAutoconfigAlgorithmOUI,
            ieee8021FrerSequenceAutoconfigCustomAlgorithm,
            ieee8021FrerSequenceAutoconfigCustomAlgorithmOUI,
            ieee8021FrerSequenceAutoconfigHistoryLength,
            ieee8021FrerSequenceAutoconfigCreateIndividual,
            ieee8021FrerSequenceAutoconfigCreateRecovery,
            ieee8021FrerSequenceAutoconfigLatentErrorDetection,
            ieee8021FrerSequenceAutoconfigLatentErrorDifference,
            ieee8021FrerSequenceAutoconfigLatentErrorPeriod,
            ieee8021FrerSequenceAutoconfigLatentErrorResetPeriod,
            ieee8021FrerSequenceAutoconfigStatus,
            ieee8021FrerSequenceAutoconfigReceivePortHandle,
            ieee8021FrerSequenceAutoconfigReceivePortHandleListStatus,
            ieee8021FrerSequenceAutoconfigVlan,
            ieee8021FrerSequenceAutoconfigVlanListStatus,
            ieee8021FrerSequenceAutoconfigRecoveryPortHandle,
            ieee8021FrerSequenceAutoconfigRecoveryPortHandleListStatus
        }
        STATUS      current
        DESCRIPTION
            "Objects that are part of the FRER sequence autoconfiguration
            functionality."
        ::= { ieee8021FrerGroups 5 }

    -- ============================================================
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
1-- the ieee8021FrerOutputAutoconfig group
 -- ============================================================

 ieee8021FrerOutputAutoconfigGroup OBJECT-GROUP
     OBJECTS {
         ieee8021FrerOutputAutoconfigPortList,
         ieee8021FrerOutputAutoconfigEncapsulation,
         ieee8021FrerOutputAutoconfigEncapsulationOUI,
         ieee8021FrerOutputAutoconfigCustomEncapsulation,
         ieee8021FrerOutputAutoconfigCustomEncapsulationOUI,
         ieee8021FrerOutputAutoconfigLanPathId,
         ieee8021FrerOutputAutoconfigStatus,
         ieee8021FrerOutputAutoconfigPortHandle,
         ieee8021FrerOutputAutoconfigPortHandleListStatus
     }
     STATUS      current
     DESCRIPTION
         "Objects that are part of the FRER output autoconfiguration
         functionality."
     ::= { ieee8021FrerGroups 6 }

 -- ============================================================
 -- the ieee8021FrerPerPortPerStreamCounters group
 -- ============================================================

 ieee8021FrerPerPortPerStreamCountersGroup OBJECT-GROUP
     OBJECTS {
         ieee8021FrerPerPortPerStreamSeqGenResets,
         ieee8021FrerPerPortPerStreamSeqRecoveryOutOfOrderPackets,
         ieee8021FrerPerPortPerStreamSeqRecoveryRoguePackets,
         ieee8021FrerPerPortPerStreamSeqRecoveryPassedPackets,
         ieee8021FrerPerPortPerStreamSeqRecoveryDiscardedPackets,
         ieee8021FrerPerPortPerStreamSeqRecoveryLostPackets,
         ieee8021FrerPerPortPerStreamSeqRecoveryTaglessPackets,
         ieee8021FrerPerPortPerStreamSeqRecoveryResets,
         ieee8021FrerPerPortPerStreamSeqRecoveryLatentErrorResets,
         ieee8021FrerPerPortPerStreamSeqEncErroredPackets
     }
     STATUS      current
     DESCRIPTION
         "Objects that provide information on the count of packets
          handled or actions taken by FRER algorithms per port, Stream,
          and direction."
     ::= { ieee8021FrerGroups 7 }

 -- ============================================================
 -- the ieee8021FrerPerPortCounters group
 -- ============================================================

 ieee8021FrerPerPortCountersGroup OBJECT-GROUP
     OBJECTS {
         ieee8021FrerPerPortSeqRecoveryPassedPackets,
         ieee8021FrerPerPortfrerCpSeqRecoveryDiscardPackets,
         ieee8021FrerPerPortfrerCpSeqEncErroredPackets
     }
     STATUS      current
     DESCRIPTION
         "Objects that provide information on the count of packets
          handled by FRER per port and direction."
     ::= { ieee8021FrerGroups 8 }

 -- ============================================================
 -- compliance statements
 -- ============================================================

 ieee8021FrerCompliance MODULE-COMPLIANCE
     STATUS      current
     DESCRIPTION
         "The compliance statement for devices supporting
         Stream identification."

     MODULE -- this module
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
        MANDATORY-GROUPS {
            ieee8021FrerSequenceGenerationGroup,
            ieee8021FrerSequenceRecoveryGroup,
            ieee8021FrerSequenceIdentificationGroup,
            ieee8021FrerStreamSplitGroup,
            ieee8021FrerPerPortPerStreamCountersGroup,
            ieee8021FrerPerPortCountersGroup
        }

        GROUP ieee8021FrerSequenceAutoconfigGroup
        DESCRIPTION
            "Implementation of this group is mandatory if the
             auto configuration feature is implemented."

        GROUP ieee8021FrerOutputAutoconfigGroup
        DESCRIPTION
            "Implementation of this group is mandatory if the
             auto configuration feature is implemented."

    ::= { ieee8021FrerCompliances 1 }

END
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

12. YANG Data Model

This clause specifies YANG data models that provide control and monitoring for all features of this standard. Specifically:

a) Stream Identification
b) Frame Replication and Elimination for Reliability

The YANG data management models are derived from UML models specified in 12.2. The UML models are based on Clause 9 and Clause 10.

NOTE 1—OMG UML 2.5 [B14] conventions are used in this clause as a representation to convey model structure and relationships.

NOTE 2—The MIB modules specified in Clause 11 were also derived from Clause 9 and Clause 10. Consequently, the capabilities and structure of the YANG data models are closely aligned with their MIB representations. However, the YANG data models have not been derived from the MIB modules, and no attempt has been made to include data or modeling constructs that might appear in the MIB modules but not in the information models.

12.1 YANG Framework

This clause has been developed according to the YANG guidelines published in IETF RFC 6087 [B7] as applicable to IEEE standards.

The YANG framework applies hierarchy in the following areas:

a) The uniform resource name (URN), as specified in IEEE Std 802d [B3].
b) The YANG objects form a hierarchy of configuration and operational data structures that define the YANG model. These hierarchical relationships are described in 12.2.

The general YANG framework hierarchy is illustrated in Figure 12-1.

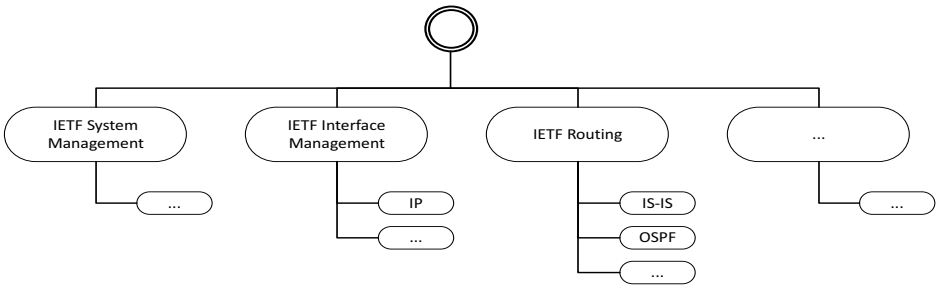


Figure 12-1—General YANG hierarchy

Network interfaces are central to the management of protocols supported over said interface. Thus, it is important to establish a common data model for how interfaces are identified, configured, and monitored. The IETF Interface Management model (IETF RFC 8343) defines a generic YANG data model for the management of network interfaces. Consequently, IEEE Std 802.1CB augments the generic interfaces data model defined by the IETF Interface Management model (IETF RFC 8343).

In addition, two high level YANG objects are defined to support IEEE Std 802.1CB features that exist at device level.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

The YANG hierarchical structure that incorporates the IEEE Std 802.1CB YANG models supported by this standard is represented in Figure 12-2. Objects that are added by this standard are indicated by a darker shading.

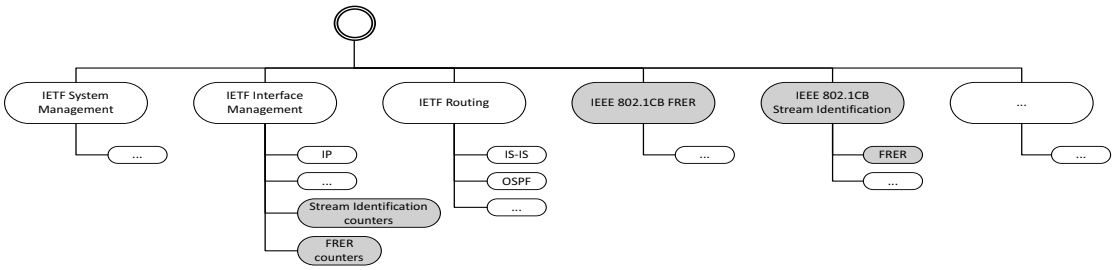


Figure 12-2—YANG root hierarchy with IEEE Std 802.1CB YANG modules

12.2 IEEE Std 802.1CB YANG model

The YANG data models are based on the management models outlined in Clause 9 and Clause 10. A UML representation of the management model is provided in the following subclauses.

The purpose of a UML-like[18] diagram is to express the model design on a single piece of paper. The structure of the UML-like representation shows the name of the object followed by a list of properties for the object. The properties indicate their type and accessibility. It should be noted that the UML-like representation is meant to express simplified semantics for the properties. It is not meant to provide the specific datatype as used to encode the object in either MIB or YANG. In the UML-like representation, a box with a white background represents information that comes from sources outside of the IEEE. A box with a gray background represents objects that are defined by this IEEE Standard.

12.2.1 Stream Identification model

The UML data model for Stream identification is illustrated in Figure 12-3.

NOTE—The {XOR} UML constraint denotes that the stream-identity parameters can be either one of the identification-types described on the right-hand side of the diagram.

[18] A description of the UML-like diagrams used in this clause is provided at https://1.ieee802.org/uml-like-diagrams

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

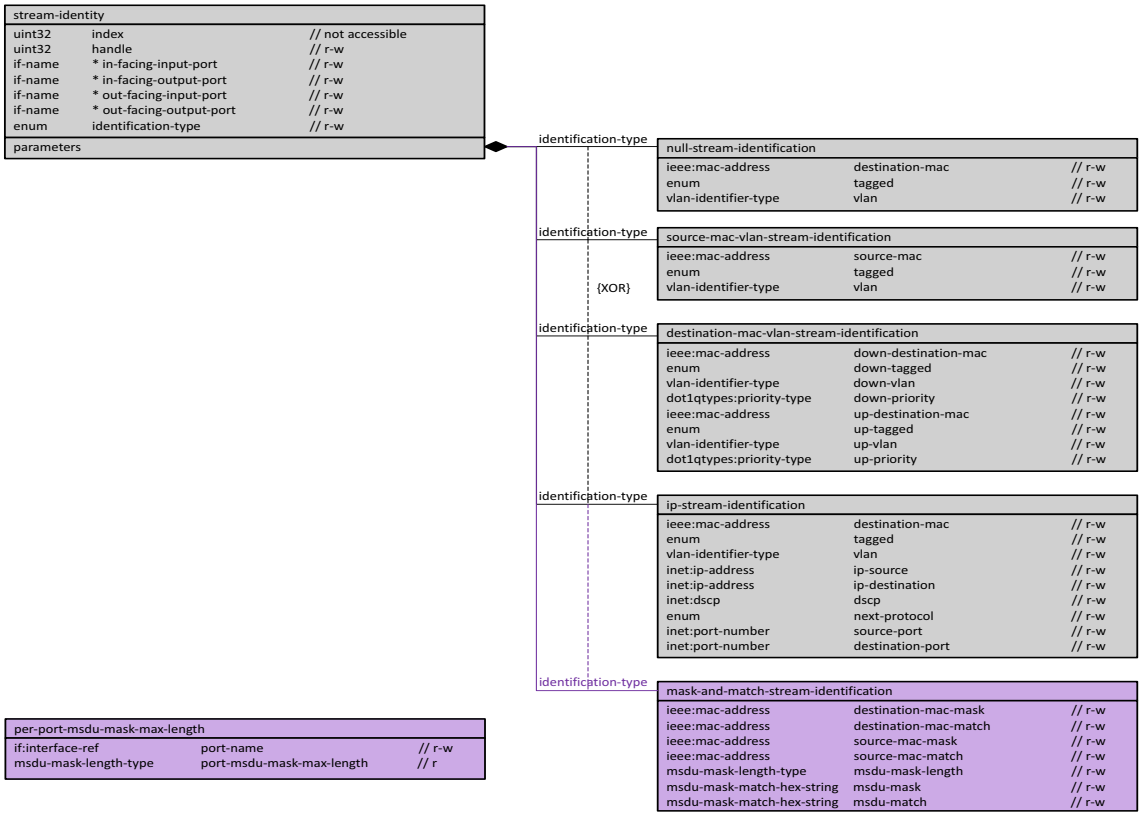


Figure 12-3—Stream identification model

The counters introduced with Stream identification augment the statistics from the Interface Management (IETF RFC 8343 [B11]) YANG model. The UML data model for the Stream identification counters is illustrated in Figure 12-4.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

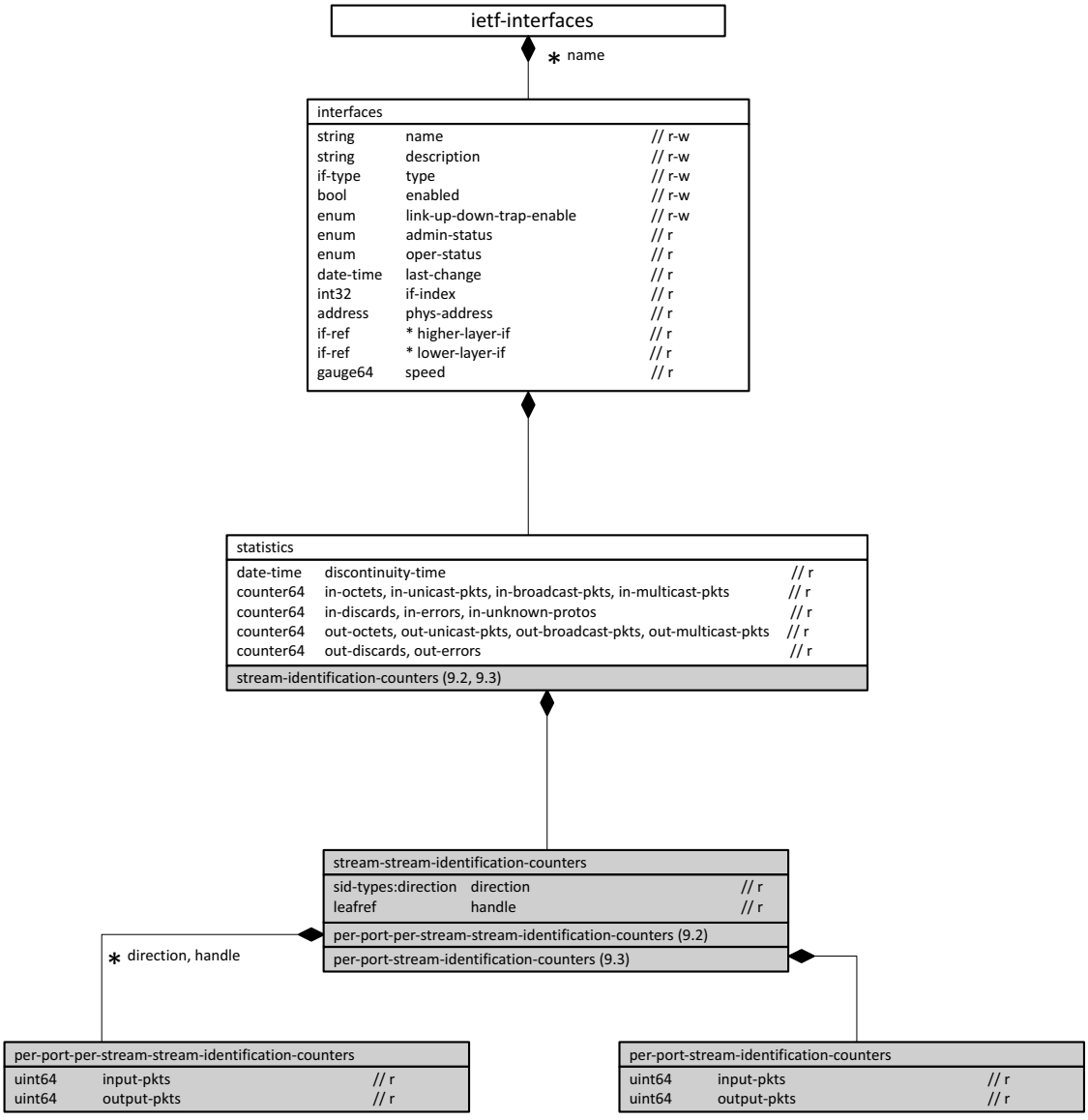


Figure 12-4—Stream identification counters model

12.2.2 Frame Replication and Elimination for Reliability model

The UML data model for FRER is illustrated in Figure 12-5. ~~Additionally, FRER augments the Stream identification YANG model as illustrated in Figure 12-6.~~

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

sequence-generation (10.3)		
uint32	index	// r-w
handle-ref	* stream	// r-w
boolean	direction-out-facing	// r-w
boolean	reset	// r-w

sequence-recovery (10.4)		
uint32	index	// r-w
handle-ref	* stream	// r-w
if-ref	* port	// r-w
boolean	direction-out-facing	// r-w
boolean	reset	// r-w
sequence-recovery-algorithm	algorithm	// r-w
sequence-history-length	history-length	// r-w
uint32	reset-timeout	// r-w
uint32	invalid-sequence-value	// r
boolean	take-no-sequence	// r-w
boolean	individual-recovery	// r-w
boolean	latent-error-detection	// r-w
int32	latent-error-difference	// r-w
uint32	latent-error-period	// r-w
uint16	latent-error-paths	// r-w
uint32	latent-error-reset-period	// r-w

sequence-identification (10.5)		
handle-ref	* stream	// r-w
if-ref	port	// r-w
boolean	direction-out-facing	// r-w
boolean	active	// r-w
enum	encapsulation	// r-w
int8	path-id-lan-id	// r-w

stream-split (10.6)		
uint32	index	// r-w
if-ref	port	// r-w
boolean	direction-out-facing	// r-w
leaf-ref	* input-id	// r-w
leaf-ref	* output-id	// r-w

~~sequence-autoconfiguration (10.7.1)~~		
~~uint32~~	~~index~~	~~// r-w~~
enum	sequence-encapsulation	// r-w
if-ref	* receive-port	// r-w
enum	tagged	// r-w
dot1qtypes:vlanid	* vlan	// r-w
if-ref	* recovery-port	// r-w
uint64	destruction-interval	// r-w
uint64	reset-interval	// r-w
sequence-recovery-algorithm	algorithm	// r-w
sequence-history-length	history-length	// r-w
boolean	create-individual	// r-w
boolean	create-recovery	// r-w
boolean	latent-error-detection	// r-w
int32	latent-error-difference	// r-w
uint32	latent-error-period	// r-w
uint32	latent-error-reset-period	// r-w

~~output-autoconfiguration (10.7.2)~~		
~~uint32~~	~~index~~	~~// r-w~~
if-ref	* port	// r-w
enum	encapsulation	// r-w
int8	lan-path-id	// r-w

Figure 12-5—FRER model

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

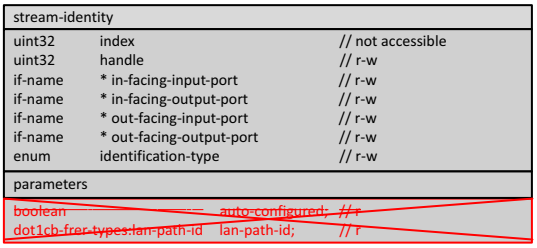


Figure 12-6—FRER Stream identification augmentation

The counters introduced with FRER augment the statistics from the Interface Management (IETF RFC 8343 [B11]) YANG model. The UML data model for the FRER counters is illustrated in Figure 12-7.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

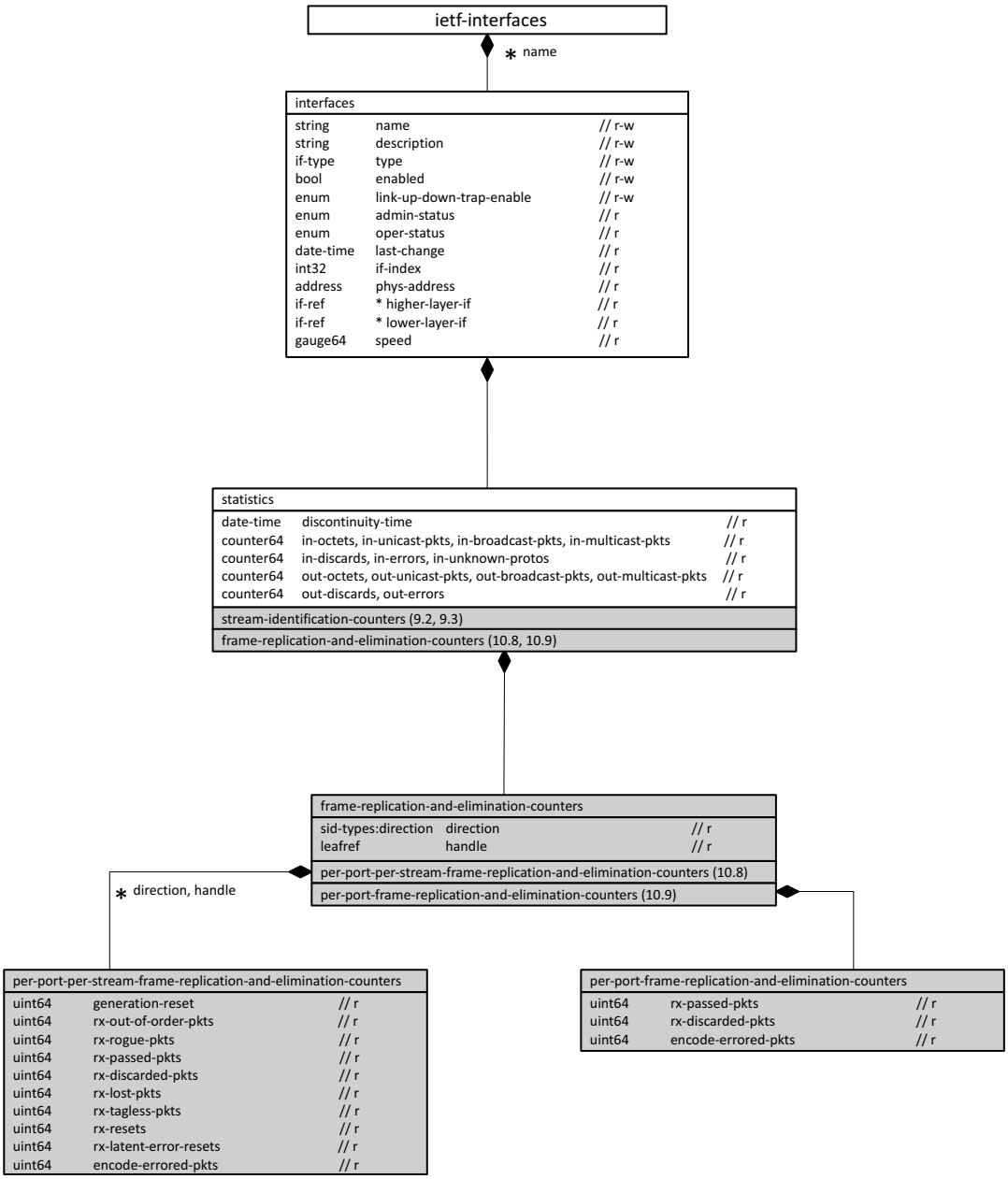


Figure 12-7—FRER counters model

12.3 Structure of the YANG model

IEEE Std 802.1CB YANG models are divided into a number of YANG modules. A summary of the modules contained in this clause is represented in Table 12-1.

The modules in Table 12-1 can be used to create a Stream Identification model or an FRER model.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Table 12-1—Description of the YANG modules

Module	References	Notes
ieee802-dot1cb-stream-identification-types	12.6.2.1	General type definitions used by IEEE Std 802.1CB Stream identification.
ieee802-dot1cb-stream-identification	12.6.2.2	YANG model for Stream identification.
ieee802-dot1cb-mask-and-match	12.6.2.3	YANG model for the extended stream identification.
ieee802-dot1cb-frer-types	12.6.2.4	General type definitions used by IEEE Std 802.1CB frame replication and elimination for reliability.
ieee802-dot1cb-frer	12.6.2.5	YANG model for FRER.

12.3.1 Structure of the ieee802-dot1cb-stream-identification YANG module

The Stream Identification model consists of the *ieee802-dot1cb-stream-identification* YANG module along with all the dependencies (YANG imports) that the module uses. Clause 12.6.1.1 contains the YANG data schema tree for the *ieee802-dot1cb-stream-identification* module.

The high-level structure of the *ieee802-dot1cb-stream-identification* YANG module is found in Table 12-2.

The list of YANG modules directly imported by the *ieee802-dot1cb-stream-identification* YANG module is found in Table 12-3.

Table 12-2—ieee802-dot1cb-stream-identification structure and relationship to this standard

Module	References	Notes
ieee802-dot1cb-stream-identification	9	—
stream-identity	9.1	Stream identity management within a system.
per-port-counters	9.3	Per-port counters for Stream identification.
per-port-per-stream-counters	9.2	Per-port-per-stream counters for Stream identification.

Table 12-3—YANG module dependencies for the Stream identification model

YANG module
ieee802-types
ieee802-dot1q-types
ietf-inet-types
ietf-interfaces
ieee802-dot1cb-stream-identification-types

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

To complete the model, all the dependencies from the imported modules must also be identified. The process to determine all the dependencies can be done through tooling. For example, if the pyang [B15] or yanglint tool [B16] is used on the *ieee802-dot1cb-stream-identification* YANG module, the tooling will try to include all the imports and produce an error message if an import is missing. The YANG Catalog [B17] search tools and/or the YANG Catalog's github repository [B18] can be used to find the missing imports.

12.3.2 Structure of the ieee802-dot1cb-mask-and-match YANG module

The Extended Stream Identification model consists of the *ieee802-dot1cb-mask-and-match* YANG module along with all the dependencies (YANG imports) that the module uses. 12.6.1.2 contains the YANG data schema tree for the *ieee802-dot1cb-mask-and-match* module.

The high-level structure of the *ieee802-dot1cb-mask-and-match* YANG module is found in Table 12-4.

The list of YANG modules directly imported by the *ieee802-dot1cb-mask-and-match* YANG module is found in Table 12-5.

Table 12-4—ieee802-dot1cb-mask-and-match structure and relationship to this standard

Module	References	Notes
ieee802-dot1cb-mask-and-match		—
stream-identity parameters	9.1.6	Mask-and-match stream identification parameters.
per-port-msdu-mask-max-length	9.4	Per-port stream identification parameter.

Table 12-5—YANG module dependencies for the Extended Stream identification model

YANG module
ieee802-types
ietf-interfaces
ieee802-dot1cb-stream-identification-types
ieee802-dot1cb-stream-identification

To complete the model, all the dependencies from the imported modules must also be identified. The process to determine all the dependencies can be done through tooling. For example, if the pyang [B15] or yanglint tool [B16] is used on the *ieee802-dot1cb-mask-and-match* YANG module, the tooling will try to include all the imports and produce an error message if an import is missing. The YANG Catalog [B17] search tools and/or the YANG Catalog repository [B18] can be used to find the missing imports.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

12.3.3 Structure of the ieee802-dot1cb-frer YANG module

The FRER model consists of the *ieee802-dot1cb-frer* YANG module along with all the dependencies (YANG imports) that the module uses. Clause 12.6.1.3 contains the YANG data schema tree for the *ieee802-dot1cb-frer* module.

The high-level structure of the *ieee802-dot1cb-frer* YANG module is found in Table 12-6.

The list of YANG modules directly imported by the *ieee802-dot1cb-frer* YANG module is found in Table 12-7.

Table 12-6—ieee802-dot1cb-frer structure and relationship to this standard

Module	References	Notes
ieee802-dot1cb-frer	10	—
sequence-generation	10.3	Sequence generation management within a system.
sequence-recovery	10.4	Sequence recovery management within a system.
sequence-identification	10.5	Sequence identification management within a system.
stream-split	10.6	Stream splitting management within a system.
~~autoconfiguration~~	~~10.7~~	~~Autoconfiguration management within a system.~~
per-port-counters	10.9	Per-port counters for FRER.
per-port-per-stream-counters	10.8	Per-port-per-stream counters for FRER.

Table 12-7—YANG module dependencies for the FRER model

YANG module
ieee802-dot1q-types
ietf-interfaces
ieee802-dot1cb-stream-identification-types
ieee802-dot1cb-stream-identification
ieee802-dot1cb-frer-types
ieee802-dot1cb-frer

To complete the model, all the dependencies from the imported modules must also be identified. The process to determine all the dependencies can be done through tooling. For example, if the pyang [B15] or yanglint tool [B16] is used on the *ieee802-dot1cb-frer* YANG module, the tooling will try to include all the

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

imports and produce an error message if an import is missing. The YANG Catalog [B17] search tools and/or the YANG Catalog's github repository [B18] can be used to find the missing imports.

12.4 Relationship to other YANG modules

In order to facilitate interoperable management of the features defined in this standard by remote means, 12.6 contains a complete set of YANG modules. This clause describes how the YANG modules defined in this standard are related to the YANG modules that are imported.

12.4.1 IEEE 802 Types Module

The *ieee802-types* YANG module provides reusable types that are used in IEEE 802 standards.

The type for mac-addresses defined in *ieee802-types* has a pattern that allows upper and lower case letters. To avoid issues with string comparison, it is suggested to only use upper case for the letters in the hexadecimal numbers. Implementers using code comparing MAC addresses should note that there is still an issue with a difference between the IETF mac-address definition and the IEEE mac-address definition.

12.4.2 IEEE 802.1Q Types Module

The *ieee802-dot1q-types* YANG module provides reusable types that are used in IEEE 802.1 standards.

12.4.3 IETF Inet Types Module

The *ietf-inet-types* YANG module (IETF RFC 6021) provides reusable data types to be used with the YANG data modeling language.

12.4.4 IETF Interfaces YANG Module

The *ietf-interfaces* YANG module (IETF RFC 8343) contains a set of YANG definitions for managing network interfaces. This document models a port as an interface. Counters in the YANG modules defined in this document augment the interface statistics provided by *ietf-interfaces*.

12.4.5 IEEE 802.1CB Stream Identification Types YANG module

The *ieee802-dot1cb-stream-identification-types* YANG module provides reusable types that are used for IEEE 802.1CB Stream Identification.

12.4.6 IEEE 802.1CB Stream Identification YANG module

The *ieee802-dot1cb-stream-identification* YANG module defined in this standard provides Stream identification functionality and a Stream identification handle. The *ieee802-dot1cb-frer* YANG module that is also defined in this standard uses the Stream identification handle and also augments the Stream identification functionality.

12.4.7 IEEE 802.1CB Extended Stream identification module

The *ieee802-dot1cb-mask-and-match* YANG module defined in this standard provides an additional Stream identification functionality by augmenting the Stream identification functionality provided by the *ieee802-dot1cb-stream-identification* module.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

12.4.8 IEEE 802.1CB FRER Types YANG module

The *ieee802-dot1cb-frer-types* YANG module provides reusable types that are used for IEEE 802.1CB frame replication and elimination for reliability.

12.5 Security Considerations

The YANG modules defined in this clause are designed to be accessed via a network configuration protocol, e.g., NETCONF protocol (IETF RFC 6242 [B9]). In the case of NETCONF, the lowest NETCONF layer is the secure transport layer and the mandatory to implement secure transport is SSH (IETF RFC 6242 [B9]). The NETCONF access control model (IETF RFC 6536 [B10]) provides the means to restrict access for particular NETCONF users to a preconfigured subset of all available NETCONF protocol operations and content.

It is the responsibility of a system's implementor and administrator to ensure that the protocol entities in the system that support NETCONF, and any other remote configuration protocols that make use of these YANG modules, are properly configured to allow access only to those principals (users) that have legitimate rights to read or write data nodes. This standard does not specify how the credentials of those users are to be stored or validated.

12.5.1 Security Considerations of the ieee802-dot1cb-stream-identification YANG module

There are a number of management objects defined in the *ieee802-dot1cb-stream-identification* YANG module that are configurable (i.e., read-write) and/or operational (i.e., read-only). Such objects may be considered sensitive or vulnerable in some network environments. A network configuration protocol, such as NETCONF (IETF RFC 6241 [B8]), can support protocol operations that can edit or delete YANG module configuration data (e.g., edit-config, delete-config, copy-config). If this is done in a non-secure environment without proper protection, then negative effects on the network operation is possible.

12.5.2 Security Considerations of the ieee802-dot1cb-frer YANG module

There are a number of management objects defined in the *ieee802-dot1cb-frer* YANG module that are configurable (i.e., read-write) and/or operational (i.e., read-only). Such objects may be considered sensitive or vulnerable in some network environments. A network configuration protocol, such as NETCONF (IETF RFC 6241 [B8]), can support protocol operations that can edit or delete YANG module configuration data (e.g., edit-config, delete-config, copy-config). If this is done in a non-secure environment without proper protection, then negative effects on the network operation are possible.

12.6 Definition of IEEE Std 802.1CB YANG modules[19] [20]

The structure of the YANG modules related to this standard is described in 12.3. In the following YANG module definitions, if any discrepancy between the DESCRIPTION text and the corresponding definition in any other part of this standard occurs, the definitions outside this subclause take precedence.

[19]Copyright release for YANG modules: Users of this standard may freely reproduce the YANG modules contained in this subclause so that they can be used for their intended purpose.

[20]An ASCII version of the YANG modules are attached to the PDF version of this standard, and can be obtained by Web browser from the IEEE 802.1 Website at https://1.ieee802.org/yang-modules/.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

12.6.1 YANG data scheme tree definitions

A simplified graphical representation of the data model is used in this document. The meaning of the symbols in these diagrams is as follows:

- Brackets "[" and "]" enclose list keys.
- Abbreviations before data node names: "rw" means configuration (read-write), and "ro" means state data (read-only).
- Symbols after data node names: "?" means an optional node, "!" means a presence container, and "*" denotes a list and leaf-list.
- Parentheses enclose choice and case nodes, and case nodes are also marked with a colon (":").
- Ellipsis ("...") stands for contents of subtrees that are not shown.

12.6.1.1 YANG data scheme definition for ieee802-dot1cb-stream-identification YANG module

```
module: ieee802-dot1cb-stream-identification
    +--rw stream-identity* [index]
       +--rw index                                    uint32
       +--rw handle                                   uint32
       +--rw in-facing
       |  +--rw input-port*    if:interface-ref
       |  +--rw output-port*   if:interface-ref
       +--rw out-facing
       |  +--rw input-port*    if:interface-ref
       |  +--rw output-port*   if:interface-ref
       +--rw (parameters)
          +--:(null-stream-identification)
          |  +--rw null-stream-identification
          |     +--ro identification-type
          |     |  +--ro type-number?   dot1cb-sid-types:stream-id-function
          |     |  +--ro oui-cid?       string
          |     +--rw destination-mac?       ieee:mac-address
          |     +--rw tagged?                vlan-tag-identification-type
          |     +--rw vlan?                  vlan-identifier-type
          +--:(smac-vlan-stream-identification)
          |  +--rw smac-vlan-stream-identification
          |     +--ro identification-type
          |     |  +--ro type-number?   dot1cb-sid-types:stream-id-function
          |     |  +--ro oui-cid?       string
          |     +--rw source-mac?            ieee:mac-address
          |     +--rw tagged?                vlan-tag-identification-type
          |     +--rw vlan?                  vlan-identifier-type
          +--:(dmac-vlan-stream-identification)
          |  +--rw dmac-vlan-stream-identification
          |     +--ro identification-type
          |     |  +--ro type-number?   dot1cb-sid-types:stream-id-function
          |     |  +--ro oui-cid?       string
          |     +--rw down
          |     |  +--rw destination-mac?   ieee:mac-address
          |     |  +--rw tagged?            vlan-tag-identification-type
          |     |  +--rw vlan?              vlan-identifier-type
          |     |  +--rw priority?          dot1qtypes:priority-type
          |     +--rw up
          |        +--rw destination-mac?   ieee:mac-address
          |        +--rw tagged?            vlan-tag-identification-type
          |        +--rw vlan?              vlan-identifier-type
          |        +--rw priority?          dot1qtypes:priority-type
          +--:(ip-stream-identification)
          |  +--rw ip-stream-identification
          |     +--ro identification-type
          |     |  +--ro type-number?   dot1cb-sid-types:stream-id-function
          |     |  +--ro oui-cid?       string
          |     +--rw destination-mac?       ieee:mac-address
          |     +--rw tagged?                vlan-tag-identification-type
          |     +--rw vlan?                  vlan-identifier-type
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
        |     +--rw ip-source?             inet:ip-address
        |     +--rw ip-destination?        inet:ip-address
        |     +--rw dscp?                  inet:dscp
        |     +--rw next-protocol?         enumeration
        |     +--rw source-port?           inet:port-number
        |     +--rw destination-port?      inet:port-number
        +--:(organization-specific)
           +--rw organization-specific
              +--rw identification-type
                 +--rw type-number?   int32
                 +--rw oui-cid?       string

  augment /if:interfaces/if:interface/if:statistics:
    +--ro stream-id
       +--ro per-port-counters
       |  +--ro input-pkts?    uint64
       |  +--ro output-pkts?   uint64
       +--ro per-port-per-stream-counters* [direction-out-facing handle]
          +--ro direction-out-facing    dot1cb-sid-types:direction
          +--ro handle                  -> /stream-identity/handle
          +--ro input-pkts?             uint64
          +--ro output-pkts?            uint64
```

12.6.1.2 YANG data scheme definition for ieee802-dot1cb-mask-and-match YANG module

```
module: ieee802-dot1cb-mask-and-match
  +--rw per-port-msdu-mask-max-length* [port-name]
     +--rw port-name                    if:interface-ref
     +--ro port-msdu-mask-max-length?   msdu-mask-length-type

  augment /dot1cb-sid:stream-identity/dot1cb-sid:parameters:
     +--rw mask-and-match-stream-identification
       +--ro identification-type
       |  +--ro type-number?   dot1cb-sid-types:stream-id-function
       |  +--ro oui-cid?       string
       +--rw destination-mac-mask?    ieee802:mac-address
       +--rw destination-mac-match?   ieee802:mac-address
       +--rw source-mac-mask?         ieee802:mac-address
       +--rw source-mac-match?        ieee802:mac-address
       +--rw msdu-mask-length?        msdu-mask-length-type
       +--rw msdu-mask?               msdu-mask-match-hex-string
       +--rw msdu-match?              msdu-mask-match-hex-string
```

12.6.1.3 YANG data scheme definition for ieee802-dot1cb-frer YANG module

```
module: ieee802-dot1cb-frer
  +--rw frer
     +--rw sequence-generation* [index]
|  +--rw index                    uint32
     |  +--rw stream*                  -> /dot1cb-sid:stream-identity/handle
     |  +--rw direction-out-facing?    dot1cb-sid-types:direction
     |  +--rw reset?                   boolean
     +--rw sequence-recovery* [index]
     |  +--rw index                                    uint32
     |  +--rw stream*
     |  |       -> /dot1cb-sid:stream-identity/handle
     |  +--rw port*                                    if:interface-ref
     |  +--rw direction-out-facing?                    dot1cb-sid-types:direction
     |  +--rw reset?                                   boolean
     |  +--rw algorithm
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
|  |  +--rw (algorithm)?
|  |     +--:(vector)
|  |     |  +--rw vector!
|  |     |     +--ro type-number?   dot1cb-frer-types:seq-rcvy-algorithm
|  |     |     +--ro oui-cid?       string
|  |     +--:(match)
|  |     |  +--rw match!
|  |     |     +--ro type-number?   dot1cb-frer-types:seq-rcvy-algorithm
|  |     |     +--ro oui-cid?       string
|  |     +--:(organization-specific)
|  |        +--rw organization-specific!
|  |           +--rw type-number?   int32
|  |           +--rw oui-cid?       string
|  +--rw history-length?                        sequence-history-length
|  +--rw reset-timeout?                         uint32
|  +--ro invalid-sequence-value?                uint32
|  +--rw take-no-sequence?                      boolean
|  +--rw individual-recovery?                   boolean
|  +--rw latent-error-detection?                boolean
|  +--rw latent-error-detection-parameters
|     +--rw difference?     int32
|     +--rw period?         uint32
|     +--rw paths?          uint16
|     +--rw reset-period?   uint32
+--rw sequence-identification* [port direction-out-facing]
|  +--rw stream*                  -> /dot1cb-sid:stream-identity/handle
|  +--rw port                     if:interface-ref
|  +--rw direction-out-facing     dot1cb-sid-types:direction
|  +--rw active?                  boolean
|  +--rw encapsulation
|  |  +--rw (encapsulation)?
|  |     +--:(r-tag)
|  |     |  +--rw r-tag!
|  |     |     +--ro type-number?   dot1cb-frer-types:seq-encaps-method
|  |     |     +--ro oui-cid?       string
|  |     +--:(hsr-sequence-tag)
|  |     |  +--rw hsr-sequence-tag!
|  |     |     +--ro type-number?   dot1cb-frer-types:seq-encaps-method
|  |     |     +--ro oui-cid?       string
|  |     +--:(prp-sequence-tag)
|  |     |  +--rw prp-sequence-tag!
|  |     |     +--ro type-number?   dot1cb-frer-types:seq-encaps-method
|  |     |     +--ro oui-cid?       string
|  |     +--:(organization-specific)
|  |        +--rw organization-specific!
|  |           +--rw type-number?   int32
|  |           +--rw oui-cid?       string
|  +--rw path-id-lan-id?          lan-path-id
+--rw stream-split* [index]
|  +--rw index                    uint32
|  +--rw port                     if:interface-ref
|  +--rw direction-out-facing     dot1cb-sid-types:direction
|  +--rw input-id*                -> /dot1cb-sid:stream-identity/handle
|  +--rw output-id*               -> /dot1cb-sid:stream-identity/handle
+--rw autoconfiguration {auto-configuration}?
|  +--rw sequence* [index]
|  |  +--rw index                                uint32
|  |  +--rw sequence-encapsulation
|  |  |  +--rw (encapsulation)?
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
1  |  |  |  +--:(r-tag)
2  |  |  |  |  +--rw r-tag!
3  |  |  |  |     +--ro type-number?
4  |  |  |  |     |        dot1cb-frer-types:seq-encaps-method
5  |  |  |  |     +--ro oui-cid?        string
6  |  |  |  +--:(hsr-sequence-tag)
7  |  |  |  |  +--rw hsr-sequence-tag!
8  |  |  |  |     +--ro type-number?
9  |  |  |  |     |        dot1cb-frer-types:seq-encaps-method
10 |  |  |  |     +--ro oui-cid?        string
11 |  |  |  +--:(prp-sequence-tag)
12 |  |  |  |  +--rw prp-sequence-tag!
13 |  |  |  |     +--ro type-number?
14 |  |  |  |     |        dot1cb-frer-types:seq-encaps-method
15 |  |  |  |     +--ro oui-cid?        string
16 |  |  |  +--:(organization-specific)
17 |  |  |     +--rw organization-specific!
18 |  |  |        +--rw type-number?    int32
19 |  |  |        +--rw oui-cid?        string
20 |  |  +--rw receive-port*                  if:interface-ref
21 |  |  +--rw tagged?                        enumeration
22 |  |  +--rw vlan*                          dot1qtypes:vlanid
23 |  |  +--rw recovery-port*                 if:interface-ref
24 |  |  +--rw destruction-interval?          uint64
25 |  |  +--rw reset-interval?                uint64
26 |  |  +--rw algorithm
27 |  |  |  +--rw (algorithm)?
28 |  |  |  +--:(vector)
29 |  |  |  |  +--rw vector!
30 |  |  |  |     +--ro type-number?
31 |  |  |  |     |        dot1cb-frer-types:seq-rcvy-algorithm
32 |  |  |  |     +--ro oui-cid?        string
33 |  |  |  +--:(match)
34 |  |  |  |  +--rw match!
35 |  |  |  |     +--ro type-number?
36 |  |  |  |     |        dot1cb-frer-types:seq-rcvy-algorithm
37 |  |  |  |     +--ro oui-cid?        string
38 |  |  |  +--:(organization-specific)
39 |  |  |     +--rw organization-specific!
40 |  |  |        +--rw type-number?    int32
41 |  |  |        +--rw oui-cid?        string
42 |  |  +--rw history-length?                sequence-history-length
43 |  |  +--rw create-individual?             boolean
44 |  |  +--rw create-recovery?               boolean
45 |  |  +--rw latent-error-detection?        boolean
46 |  |  +--rw latent-error-difference?       int32
47 |  |  +--rw latent-error-period?           uint32
48 |  |  +--rw latent-error-reset-period?     uint32
49 |  +--rw output* [index]
50 |     +--rw index              uint32
51 |     +--rw port*              if:interface-ref
52 |     +--rw encapsulation
53 |     |  +--rw (encapsulation)?
54 |     |  +--:(r-tag)
55 |     |  |  +--rw r-tag!
56 |     |  |     +--ro type-number?
57 |     |  |     |        dot1cb-frer-types:seq-encaps-method
58 |     |  |     +--ro oui-cid?        string
59 |     |  +--:(hsr-sequence-tag)
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
|       |   +--rw hsr-sequence-tag!
|       |      +--ro type-number?
|       |      |        dot1cb-frer-types:seq-encaps-method
|       |      +--ro oui-cid?        string
|       +--:(prp-sequence-tag)
|       |   +--rw prp-sequence-tag!
|       |      +--ro type-number?
|       |      |        dot1cb-frer-types:seq-encaps-method
|       |      +--ro oui-cid?        string
|       +--:(organization-specific)
|          +--rw organization-specific!
|             +--rw type-number?   int32
|             +--rw oui-cid?        string
|       +--rw lan-path-id?      lan-path-id

augment /dot1cb-sid:stream-identity:
   +--ro auto-configured?   boolean {auto-configuration}?
   +--rw lan-path-id?       lan-path-id {auto-configuration}?
augment /if:interfaces/if:interface/if:statistics:
  +--ro frer
     +--ro per-port-counters
     |  +--ro rx-passed-pkts?        uint64
     |  +--ro rx-discarded-pkts?     uint64
     |  +--ro encode-errored-pkts?   uint64
     +--ro per-port-per-stream-counters* [direction-out-facing handle]
        +--ro direction-out-facing      dot1cb-sid-types:direction
        +--ro handle
        |       -> /dot1cb-sid:stream-identity/handle
        +--ro generation-reset?         uint64
        +--ro rx-out-of-order-pkts?     uint64
        +--ro rx-rogue-pkts?            uint64
        +--ro rx-passed-pkts?           uint64
        +--ro rx-discarded-pkts?        uint64
        +--ro rx-lost-pkts?             uint64
        +--ro rx-tagless-pkts?          uint64
        +--ro rx-resets?                uint64
        +--ro rx-latent-error-resets?   uint64
        +--ro encode-errored-pkts?      uint64
```

12.6.2 YANG data module definitions

12.6.2.1 Definition for the ieee802-dot1cb-stream-identification-types YANG module

```
module ieee802-dot1cb-stream-identification-types {
  yang-version "1.1";
  namespace
    urn:ieee:std:802.1Q:yang:ieee802-dot1cb-stream-identification-types;
  prefix dot1cb-sid-types;
  organization
    "Institute of Electrical and Electronics Engineers";
  contact
    "WG-URL: http://ieee802.org/1/
    WG-EMail: stds-802-1-l@ieee.org

    Contact: IEEE 802.1 Working Group Chair
    Postal: C/O IEEE 802.1 Working Group
         IEEE Standards Association
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
        445 Hoes Lane
        Piscataway, NJ 08854
        USA

    E-mail: stds-802-1-chairs@ieee.org";
  description
    "Management objects that control the stream identification from IEEE
    Std 802.1CB-2017. This YANG data model conforms to the Network
    Management Datastore Architecture defined in RFC 8342."; Copyright
    (C)IEEE (2021). This version of this YANG module is part of IEEE Std
    802.1CBdb-2021; see the draft itself for full legal notices.
  revision 2021-12-09 {
    description
      "Published as part of IEEE Std 802.1CBdb-2021.
      Added the stream identification type used for the Mask-and-match
      identification method.";
    reference
      "IEEE Std 802.1CBdb-2021, Frame Replication and Elimination for
      Reliability - Extended Stream identification functions.";
  }
  revision 2021-12-08 {
    description
      "Published as part of IEEE Std 802.1CBcv-2021. Initial version.";
    reference
      "IEEE Std 802.1CBcv-2021, Frame Replication and Elimination for
      Reliability - FRER YANG Data Model and Management Information Base
      Module.";
  }
  identity strid-idty {
    description
      "Root identity for all stream identification types";
  }
  typedef direction {
    type boolean;
    description
      "A boolean object indicating whether the direction is out-facing
      (True) or in-facing (False).";
    reference
      "10.4.1.3 of IEEE Std 802.1CB-2017";
  }
  typedef stream-id-function {
    type enumeration {
      enum reserved {
        value 0;
        description
          "Reserved value.";
      }
      enum null-stream {
        value 1;
        description
          "The stream identification type used for the Null Stream
          identification method.";
      }
      enum smac-vlan {
        value 2;
        description
          "The stream identification type used for the Source MAC and
          VLAN Stream identification method.";
      }
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
      enum dmac-vlan {
        value 3;
        description
          "The stream identification type used for the Active Destination
          MAC and VLAN Stream identification method.";
      }
      enum ip {
        value 4;
        description
          "The stream identification type used for the IP Stream
          identification method.";
      }
       enum mask-and-match {
        value 5;
        description
          "The stream identification type used for the Mask-and-match
          identification method.";
      }
    }
    description
      "An enumerated value indicating the method used to identify packets
      belonging to a Stream.";
    reference
      "9.1.1.6 of IEEE Std 802.1CBdb-2021";
  }
}

```

12.6.2.2 Definition for the ieee802-dot1cb-stream-identification YANG module

```
module ieee802-dot1cb-stream-identification {
  yang-version "1.1";
  namespace urn:ieee:std:802.1Q:yang:ieee802-dot1cb-stream-identification;
  prefix dot1cb-sid;
  import ieee802-types {
    prefix ieee;
  }
  import ieee802-dot1q-types {
    prefix dot1qtypes;
  }
  import ietf-inet-types {
    prefix inet;
  }
  import ietf-interfaces {
    prefix if;
  }
  import ieee802-dot1cb-stream-identification-types {
    prefix dot1cb-sid-types;
  }
  organization
    "Institute of Electrical and Electronics Engineers";
  contact
    "WG-URL: http://ieee802.org/1/
    WG-EMail: stds-802-1-l@ieee.org

    Contact: IEEE 802.1 Working Group Chair
    Postal: C/O IEEE 802.1 Working Group
            IEEE Standards Association
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
            445 Hoes Lane
            Piscataway, NJ 08854
            USA

    E-mail: stds-802-1-chairs@ieee.org";
  description
    "Management objects that control the Stream identification from IEEE
    Std 802.1CB-2017. This YANG data model conforms to the Network
    Management Datastore Architecture defined in RFC 8342. Copyright (C)
    IEEE (2021). This version of this YANG module is part of IEEE Std
    802.1CBcv; see the standard itself for full legal notices.";
revision 2026-03-06 {
    description
      "Published as part of IEEE Std 802.1CB-2017-Rev.";
    reference
      "IEEE Std 802.1CB-2017-Rev, Frame Replication and Elimination for
      Reliability - Revision.";
  }
revision 2021-12-08 {
    description
      "Published as part of IEEE Std 802.1CBcv-2021. Initial version.";
    reference
      "IEEE Std 802.1CBcv-2021, Frame Replication and Elimination for
      Reliability - FRER YANG Data Model and Management Information Base
      Module.";
  }
  identity null-stream-identification {
    base dot1cb-sid-types:strid-idty;
    description
      "Null Stream identification";
  }
  identity smac-vlan-stream-identification {
    base dot1cb-sid-types:strid-idty;
    description
      "Source MAC and VLAN Stream identification";
  }
  identity dmac-vlan-stream-identification {
    base dot1cb-sid-types:strid-idty;
    description
      "Active Destination MAC and VLAN Stream identification";
  }
  identity ip-stream-identification {
    base dot1cb-sid-types:strid-idty;
    description
      "IP Stream identification";
  }
  typedef vlan-tag-identification-type {
    type enumeration {
      enum tagged {
        value 1;
        description
          "A frame must have a VLAN tag to be recognized as belonging to
          the Stream.";
      }
      enum priority {
        value 2;
        description
          "A frame must be untagged, or have a VLAN tag with a VLAN ID =
          0 to be recognized as belonging to the Stream.";
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
      }
      enum all {
        value 3;
        description
          "A frame is recognized as belonging to the Stream whether
          tagged or not.";
      }
    }
    description
      "Enumeration describing how a Stream can be identified using the
      VLAN tag.";
  }
  typedef vlan-identifier-type {
    type uint16 {
      range "0 .. 4095";
    }
    description
      "Specifies the vlan_identifier. A value of 0 indicates that the
      vlan_identifier carries a special meaning.";
  }
  list stream-identity {
    key "index";
    description
      "The Stream identity table consists of a set of tsnStreamIdEntry
      objects, each relating to a single Stream, specifying the points in
      the system where Stream identification functions are to be
      instantiated. Each entry in the Stream identity table has a
      tsnStreamIdHandle object specifying a stream_handle value and one
      or more tsnStreamIdEntry objects describing one identification
      method for that Stream. If a single Stream has multiple
      identification methods, perhaps (but not necessarily) on different
      ports, then there can be multiple tsnStreamIdEntry objects with the
      same value for the tsnStreamIdHandle. If the HSR or PRP method or
      the Sequence encode/decode function is applied to a packet, then
      the LanId or PathId fields are also used to identify the Stream to
      which the packet belongs.";
    reference
      "9.1. of IEEE Std 802.1CB-2017";
    leaf index {
      type uint32;
      description
        "If a single Stream has multiple identification methods, perhaps
        (but not necessarily) on different ports, then there can be
        multiple tsnStreamIdEntry objects with the same value for the
        tsnStreamIdHandle";
    }
    leaf handle {
      type uint32;
      mandatory true;
      description
        "The objects in a given entry of the Stream identity table are
        used to control packets whose stream_handle subparameter is equal
        to the entry's tsnStreamIdHandle object. The specific values used
        in the tsnStreamIdHandle object are not necessarily used in the
        system; they are used only to relate the various management
        objects in Clause 9 and Clause 10.";
      reference
        "9.1.1.1 of IEEE Std 802.1CB-2017";
    }
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
container in-facing {
  description
    "Container for in-facing Stream identification functions.";
  leaf-list input-port {
    type if:interface-ref;
    description
      "The list of ports on which an in-facing Stream identification
      function using this identification method is to be placed for
      this Stream in the input (coming from the system forwarding
      function) direction. Any number of tsnStreamIdEntry objects can
      list the same port for the same tsnStreamIdHandle in its
      tsnStreamIdInFacInputPortList.";
    reference
      "9.1.1.4 of IEEE Std 802.1CB-2017";
  }
  leaf-list output-port {
    type if:interface-ref;
    description
      "The list of ports on which an in-facing Stream identification
      function using this identification method is to be placed for
      this Stream in the output (towards the system forwarding
      function) direction. At most one tsnStreamIdEntry can list a
      given port for a given tsnStreamIdHandle in its
      tsnStreamIdInFacOutputPortList.";
    reference
      "9.1.1.2 of IEEE Std 802.1CB-2017";
  }
}
container out-facing {
  description
    "Container for out-facing Stream identification functions.";
  leaf-list input-port {
    type if:interface-ref;
    description
      "The list of ports on which an out-facing Stream identification
      function using this identification method is to be placed for
      this Stream in the input (coming from the physical interface)
      direction. Any number of tsnStreamIdEntry objects can list the
      same port for the same tsnStreamIdHandle in its
      tsnStreamIdOutFacInputPortList.";
    reference
      "9.1.1.5 of IEEE Std 802.1CB-2017";
  }
  leaf-list output-port {
    type if:interface-ref;
    description
      "The list of ports on which an out-facing Stream identification
      function using this identification method is to be placed for
      this Stream in the output (towards the physical interface)
      direction. At most one tsnStreamIdEntry can list a given port
      for a given tsnStreamIdHandle in its
      tsnStreamIdOutFacOutputPortList.";
    reference
      "9.1.1.3 of IEEE Std 802.1CB-2017";
  }
}
choice parameters {
  mandatory true;
  description
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
      "The number of controlling parameters for a Stream identification
      method, their types and values, are specific to the
      tsnStreamIdIdentificationType.";
    reference
      "9.1.1.7 of IEEE Std 802.1CB-2017";
    container null-stream-identification {
      description
        "When instantiating an instance of the Null Stream
        identification function for a particular input Stream, the
        managed objects in this container serve as the
        tsnStreamIdParameters managed object.";
      reference
        "9.1.2 of IEEE Std 802.1CB-2017";
      container identification-type {
        config false;
        description
          "The identification type indicating the method used to
          identify packets belonging to the Stream. The identification
          type contains a type number and an Organizationally Unique
          Identifier (OUI) or Company ID (CID) to identify the
          organization defining the identification method.";
        reference
          "9.1.1.6 of IEEE Std 802.1CB-2017";
        leaf type-number {
          type dot1cb-sid-types:stream-id-function;
          default "null-stream";
          description
            "The Stream identification type used for the Null Stream
            identification method.";
          reference
            "9.1.1.6 of IEEE Std 802.1CB-2017";
        }
        leaf oui-cid {
          type string {
            pattern "[0-9A-F]{2}(-[0-9A-F]{2}){2}";
          }
          default "00-80-C2";
          description
            "The Organizationally Unique Identifier (OUI) or Company ID
            (CID) to identify the organization defining the
            identification method. For identification methods defined
            in IEEE Std 802.1CB-2017 the OUI/CID is always 00-80-C2.";
          reference
            "9.1.1.6 of IEEE Std 802.1CB-2017";
        }
      }
      leaf destination-mac {
        type ieee:mac-address;
        description
          "Specifies the destination_address that identifies a packet
          in an EISS indication primitive, to the Null Stream
          identification function. The ieee:mac-address type has a
          pattern that allows upper and lower case letters. To avoid
          issues with string comparison, it is suggested to only use
          upper case for the letters in the hexadecimal numbers. There
          is still an issue with a difference between the IETF
          mac-address definition and the IEEE mac-address definition,
          so consider that if implementing code that compares
          mac-addresses.";
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
        reference
          "9.1.2.1 of IEEE Std 802.1CB-2017";
      }
      leaf tagged {
        type vlan-tag-identification-type;
        description
          "An enumerated value indicating whether a packet in an EISS
          indication primitive to the Null Stream identification
          function is permitted to have a VLAN tag.";
        reference
          "9.1.2.2 of IEEE Std 802.1CB-2017";
      }
      leaf vlan {
        type vlan-identifier-type;
        description
          "Specifies the vlan_identifier parameter that identifies a
          packet in an EISS indication primitive to the Null Stream
          identification function. A value of 0 indicates that the
          vlan_identifier parameter is ignored on EISS indication
          primitives.";
        reference
          "9.1.2.3 of IEEE Std 802.1CB-2017";
      }
    }
    container smac-vlan-stream-identification {
      description
        "When instantiating an instance of the Source MAC and VLAN
        Stream identification function for a particular input Stream,
        the managed objects in the following subclauses serve as the
        tsnStreamIdParameters managed object.";
      reference
        "9.1.3 of IEEE Std 802.1CB-2017";
      container identification-type {
        config false;
        description
          "The identification type indicating the method used to
          identify packets belonging to the Stream. The identification
          type contains a type number and an Organizationally Unique
          Identifier (OUI) or Company ID (CID) to identify the
          organization defining the identification method.";
        reference
          "9.1.1.6 of IEEE Std 802.1CB-2017";
        leaf type-number {
          type dot1cb-sid-types:stream-id-function;
          default "smac-vlan";
          description
            "The Stream identification type used for the Source MAC and
            VLAN Stream identification method.";
          reference
            "9.1.1.6 of IEEE Std 802.1CB-2017";
        }
        leaf oui-cid {
          type string {
            pattern "[0-9A-F]{2}(-[0-9A-F]{2}){2}";
          }
          default "00-80-C2";
          description
            "The Organizationally Unique Identifier (OUI) or Company ID
            (CID) to identify the organization defining the
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
            identification method. For identification methods defined
            in IEEE Std 802.1CB-2017 the OUI/CID is always 00-80-C2.";
          reference
            "9.1.1.6 of IEEE Std 802.1CB-2017";
        }
      }
      leaf source-mac {
        type ieee:mac-address;
        description
          "Specifies the source_address that identifies a packet in an
          EISS indication primitive, to the Source MAC and VLAN Stream
          identification function. The ieee:mac-address type has a
          pattern that allows upper and lower case letters. To avoid
          issues with string comparison, it is suggested to only use
          upper case for the letters in the hexadecimal numbers. There
          is still an issue with a difference between the IETF
          mac-address definition and the IEEE mac-address definition,
          so consider that if implementing code that compares
          mac-addresses.";
        reference
          "9.1.3.1 of IEEE Std 802.1CB-2017";
      }
      leaf tagged {
        type vlan-tag-identification-type;
        description
          "An enumerated value indicating whether a packet in an EISS
          indication primitive to the Source MAC and VLAN Stream
          identification function is permitted to have a VLAN tag.";
        reference
          "9.1.3.2 of IEEE Std 802.1CB-2017";
      }
      leaf vlan {
        type vlan-identifier-type;
        description
          "Specifies the vlan_identifier parameter that identifies a
          packet in an EISS indication primitive to the Source MAC and
          VLAN Stream identification function. A value of 0 indicates
          that the vlan_identifier parameter is ignored on EISS
          indication primitives.";
        reference
          "9.1.3.3 of IEEE Std 802.1CB-2017";
      }
    }
    container dmac-vlan-stream-identification {
      description
        "When instantiating an instance of the Active Destination MAC
        and VLAN Stream identification function for a particular output
        Stream, the managed objects in the following subclauses, along
        with those listed in 9.1.2, serve as the tsnStreamIdParameters
        managed object.";
      reference
        "9.1.4 of IEEE Std 802.1CB-2017";
      container identification-type {
        config false;
        description
          "The identification type indicating the method used to
          identify packets belonging to the Stream. The identification
          type contains a type number and an Organizationally Unique
          Identifier (OUI) or Company ID (CID) to identify the
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
          organization defining the identification method.";
        reference
          "9.1.1.6 of IEEE Std 802.1CB-2017";
        leaf type-number {
          type dot1cb-sid-types:stream-id-function;
          default "dmac-vlan";
          description
            "The Stream identification type used for the Active
            Destination MAC and VLAN Stream identification method.";
          reference
            "9.1.1.6 of IEEE Std 802.1CB-2017";
        }
        leaf oui-cid {
          type string {
            pattern "[0-9A-F]{2}(-[0-9A-F]{2}){2}";
          }
          default "00-80-C2";
          description
            "The Organizationally Unique Identifier (OUI) or Company ID
            (CID) to identify the organization defining the
            identification method. For identification methods defined
            in IEEE Std 802.1CB-2017 the OUI/CID is always 00-80-C2.";
          reference
            "9.1.1.6 of IEEE Std 802.1CB-2017";
        }
      }
      container down {
        description
          "Container for all parameters which are sent to lower layers.";
        leaf destination-mac {
          type ieee:mac-address;
          description
            "Specifies the destination_address parameter to use in the
            EISS request primitive for output packets sent to lower
            layers by the Active Destination MAC and VLAN Stream
            identification function, and the destination_address that
            identifies an input packet in an EISS indication primitive
            to the Active Destination MAC and VLAN Stream
            identification function. The ieee:mac-address type has a
            pattern that allows upper and lower case letters. To avoid
            issues with string comparison, it is suggested to only use
            upper case for the letters in the hexadecimal numbers.
            There is still an issue with a difference between the IETF
            mac-address definition and the IEEE mac-address definition,
            so consider that if implementing code that compares
            mac-addresses.";
          reference
            "9.1.4.1 of IEEE Std 802.1CB-2017";
        }
        leaf tagged {
          type vlan-tag-identification-type;
          description
            "An enumerated value indicating whether a packet in an EISS
            indication or request primitive between the Active
            Destination MAC and VLAN Stream identification function and
            the lower layers is to have a VLAN tag. This variable is
            not used in an FRER C-component. See 8.4.";
          reference
            "9.1.4.2 of IEEE Std 802.1CB-2017";
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
      }
      leaf vlan {
        type vlan-identifier-type;
        description
          "Specifies the vlan_identifier parameter to use in the EISS
          request primitive for output packets sent to lower layers
          by the Active Destination MAC and VLAN Stream
          identification function, and the vlan_identifier that
          identifies an input packet in an EISS indication primitive
          to the Active Destination MAC and VLAN Stream
          identification function. A value of 0 indicates that the
          vlan_identifier parameter is ignored on EISS indication
          primitives.";
        reference
          "9.1.4.3 of IEEE Std 802.1CB-2017";
      }
      leaf priority {
        type dot1qtypes:priority-type;
        description
          "Specifies the priority parameter to use in the EISS
          request primitive for output packets sent to lower layers
          by the Active Destination MAC and VLAN Stream
          identification function for all packets in a particular
          Stream.";
        reference
          "9.1.4.4 of IEEE Std 802.1CB-2017";
      }
    }
    container up {
      description
        "Container for all parameters which are offered to higher
        layers.";
      leaf destination-mac {
        type ieee:mac-address;
        description
          "Specifies the destination_address parameter to use in the
          EISS indication primitive for input packets offered to
          upper layers by the Active Destination MAC and VLAN Stream
          identification layer. This address replaces the address
          that was used to identify the packet
          (tsnCpeDmacVlanDownDestMac). The ieee:mac-address type has
          a pattern that allows upper and lower case letters. To
          avoid issues with string comparison, it is suggested to
          only use upper case for the letters in the hexadecimal
          numbers. There is still an issue with a difference between
          the IETF mac-address definition and the IEEE mac-address
          definition, so consider that if implementing code that
          compares mac-addresses.";
        reference
          "9.1.4.5 of IEEE Std 802.1CB-2017";
      }
      leaf tagged {
        type vlan-tag-identification-type;
        description
          "An enumerated value indicating whether a packet in an EISS
          indication or request primitive between the Active
          Destination MAC and VLAN Stream identification function and
          the upper layers is to have a VLAN tag. This variable is
          used only by an end system and not by a relay system.";
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
        reference
          "9.1.4.6 of IEEE Std 802.1CB-2017";
      }
      leaf vlan {
        type vlan-identifier-type;
        description
          "Specifies the vlan_identifier parameter to use in the EISS
          indication primitive for packets offered to upper layers,
          or the VLAN ID field for an IEEE 802.1Q tag in an ISS
          mac_service_data_unit. This address replaces the VLAN ID
          that was used to identify the packet
          (tsnCpeDmacVlanDownVlan).";
        reference
          "9.1.4.7 of IEEE Std 802.1CB-2017";
      }
      leaf priority {
        type dot1qtypes:priority-type;
        description
          "Specifies the priority parameter to use in the EISS
          indication primitive for packets offered to upper layers.";
        reference
          "9.1.4.8 of IEEE Std 802.1CB-2017";
      }
    }
  }
  container ip-stream-identification {
    description
      "When instantiating an instance of the IP Stream identification
      function, the parameters in the following subclauses replace
      the tsnStreamIdParameters managed object.";
    reference
      "9.1.5 of IEEE Std 802.1CB-2017";
    container identification-type {
      config false;
      description
        "The identification type indicating the method used to
        identify packets belonging to the Stream. The identification
        type contains a type number and an Organizationally Unique
        Identifier (OUI) or Company ID (CID) to identify the
        organization defining the identification method.";
      reference
        "9.1.1.6 of IEEE Std 802.1CB-2017";
      leaf type-number {
        type dot1cb-sid-types:stream-id-function;
        default "ip";
        description
          "The Stream identification type used for the IP Stream
          identification method.";
        reference
          "9.1.1.6 of IEEE Std 802.1CB-2017";
      }
      leaf oui-cid {
        type string {
          pattern "[0-9A-F]{2}(-[0-9A-F]{2}){2}";
        }
        default "00-80-C2";
        description
          "The Organizationally Unique Identifier (OUI) or Company ID
          (CID) to identify the organization defining the
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
            identification method. For identification methods defined
            in IEEE Std 802.1CB-2017 the OUI/CID is always 00-80-C2.";
          reference
            "9.1.1.6 of IEEE Std 802.1CB-2017";
        }
      }
      leaf destination-mac {
        type ieee:mac-address;
        description
          "Specifies the destination_address parameter that identifies
          a packet in an EISS indication primitive. A value of all 0
           indicates that the MAC destination address is to be ignored
           on packets received from lower layers
           The ieee:mac-address type has a pattern that allows upper and
          lower case letters. To avoid issues with string comparison,
          it is suggested to only use upper case for the letters in the
          hexadecimal numbers. There is still an issue with a
          difference between the IETF mac-address definition and the
          IEEE mac-address definition, so consider that if implementing
          code that compares mac-addresses.";
        reference
          "9.1.5.1 of IEEE Std 802.1CB-2017";
      }
      leaf tagged {
        type vlan-tag-identification-type;
        description
          "An enumerated value indicating whether a packet in an EISS
          indication or request primitive to the IP Stream
          identification function is to have a VLAN tag.";
        reference
          "9.1.5.2 of IEEE Std 802.1CB-2017";
      }
      leaf vlan {
        type vlan-identifier-type;
        description
          "Specifies the vlan_identifier parameter that identifies a
          packet in an EISS indication primitive. ~~A value of 0
          indicates that the frame is not to have a VLAN tag.~~ A value
          of 0 indicates that the vlan identifier parameter is ignored
          on EISS indication primitives.";
        reference
          "9.1.5.3 of IEEE Std 802.1CB-2017";
      }
      leaf ip-source {
        type inet:ip-address;
        description
          "Specifies the IPv4 (RFC 791) or IPv6 (RFC 2460) source
          address parameter that must be matched to identify packets
          coming up from lower layers. ~~An address~~A ~~value~~ of all 0 indicates
          that the IP source address is to be ignored on packets
          received from lower layers.";
        reference
          "9.1.5.4 of IEEE Std 802.1CB-2017";
      }
      leaf ip-destination {
        type inet:ip-address;
        description
          "Specifies the IPv4 (RFC 791) or IPv6 (RFC 2460) destination
          address parameter that must be matched to identify packets
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
        coming up from lower layers.";
      reference
        "9.1.5.5 of IEEE Std 802.1CB-2017";
    }
    leaf dscp {
      type inet:dscp;
      description
        "Specifies the IPv4 (RFC 791) or IPv6 (RFC 2460)
        differentiated services codepoint (DSCP, RFC 2474) that must
        be matched to identify packets coming up from the lower
        layers. A value of 64 decimal indicates that the DSCP is to
        be ignored on packets received from lower layers.";
      reference
        "9.1.5.6 of IEEE Std 802.1CB-2017";
    }
    leaf next-protocol {
      type enumeration {
        enum none {
          description
            "No protocol is specified";
        }
        enum udp {
          description
            "UDP is specified as the next protocol.";
          reference
            "RFC 768";
        }
        enum tcp {
          description
            "TCP is specified as the next protocol.";
          reference
            "RFC 793";
        }
        enum sctp {
          description
            "SCTP is specified as the next protocol.";
          reference
            "RFC 4960";
        }
      }
      description
        "Specifies the IP next protocol parameter that must be
        matched to identify packets coming up from lower layers. The
        value of this parameter must specify either none, UDP (RFC
        768), TCP (RFC 793), or SCTP (RFC 4960). If “none,” then the
        tsnCpeIpIdSourcePort and tsnCpeIpIdDestinationPort managed
        objects are not used.";
      reference
        "9.1.5.7 of IEEE Std 802.1CB-2017";
    }
    leaf source-port {
      type inet:port-number;
      description
        "Specifies the TCP or UDP Source Port parameter that must be
        matched to identify packets coming up from lower layers. A
        value of 0 indicates that the Source Port number of the
        packet is to be ignored on packets received from lower
        layers.";
      reference
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
            "9.1.5.8 of IEEE Std 802.1CB-2017";
        }
        leaf destination-port {
          type inet:port-number;
          description
            "Specifies the TCP or UDP Destination Port parameter that
            must be matched to identify packets coming up from lower
            layers. A value of 0 indicates that the Destination Port
            number of the packet is to be ignored on packets received
            from lower layers.";
          reference
            "9.1.5.9 of IEEE Std 802.1CB-2017";
        }
      }
      container organization-specific {
        description
          "This container allows to select Stream identification methods
          that are defined by entities outside of IEEE 802.1.";
        reference
          "9.1.1.6 of IEEE Std 802.1CB-2017";
        container identification-type {
          description
            "The identification type indicating the method used to
            identify packets belonging to the Stream. The identification
            type contains a type number and an Organizationally Unique
            Identifier (OUI) or Company ID (CID) to identify the
            organization defining the identification method.";
          reference
            "9.1.1.6 of IEEE Std 802.1CB-2017";
          leaf type-number {
            type int32 {
              range "256..max";
            }
            description
              "The type number used for an identification method defined
              by an entity owning the OUI or CID for this identification
              type.";
            reference
              "9.1.1.6 of IEEE Std 802.1CB-2017";
          }
          leaf oui-cid {
            type string {
              pattern "[0-9A-F]{2}(-[0-9A-F]{2}){2}";
            }
            description
              "The Organizationally Unique Identifier (OUI) or Company ID
              (CID) to identify the organization defining the
              identification method.";
            reference
              "9.1.1.6 of IEEE Std 802.1CB-2017";
          }
        }
      }
    }
  }
  augment "/if:interfaces/if:interface/if:statistics" {
    description
      "The following counters are the counters for Stream identification.
      All counters are unsigned integers. If used on links faster than
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
    650 000 000 bits per second, they shall be 64 bits in length to
    ensure against excessively short wrap times.";
  reference
    "9.2 of IEEE Std 802.1CB-2017
    9.3 of IEEE Std 802.1CB-2017";
  container stream-id {
    description
      "This container contains the per-port as well as the
      per-port-per-stream counters for Stream identification.";
    reference
      "9.2 of IEEE Std 802.1CB-2017
      9.3 of IEEE Std 802.1CB-2017";
    container per-port-counters {
      config false;
      description
        "Contains the per-port counters for Stream identification. The
        following counters are instantiated for each port on which the
        Stream identification function is configured. The counters are
        indexed by port number.";
      reference
        "9.3 of IEEE Std 802.1CB-2017";
      leaf input-pkts {
        type uint64;
        config false;
        description
          "The tsnCpSidInputPackets counter is incremented once for
          each packet identified by any Stream identification function
          on this port. Its value equals the sum (modulo the size of
          the counters) of all of the tsnCpsSidInputPackets counters on
          this same port.";
        reference
          "9.3.1 of IEEE Std 802.1CB-2017";
      }
      leaf output-pkts {
        type uint64;
        config false;
        description
          "The tsnCpSidOutputPackets counter is incremented once for
          each packet passed down the stack by any Stream
          identification function on this port. Its value equals the
          sum (modulo the size of the counters) of all of the
          tsnCpsSidOutputPackets counters on this same port.";
        reference
          "9.3.2 of IEEE Std 802.1CB-2017";
      }
    }
    list per-port-per-stream-counters {
      key "direction-out-facing handle";
      config false;
      description
        "Contains the per-port-per-stream counters for Stream
        identification. The following counters are instantiated for
        each port on which the Stream identification function is
        configured. The counters are indexed by port number, facing
        (in-facing or out-facing), and stream_handle value
        (tsnStreamIdHandle).";
      reference
        "9.2 of IEEE Std 802.1CB-2017";
      leaf direction-out-facing {
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
          type dot1cb-sid-types:direction;
          description
            "An object indicating whether the counters apply to
            out-facing (True) or in-facing (False).";
        }
        leaf handle {
          type leafref {
            path '/stream-identity/handle';
          }
          description
            "The according tsnStreamIdHandle for these counters.";
        }
        leaf input-pkts {
          type uint64;
          description
            "The tsnCpsSidInputPackets counter is incremented once for
            each packet identified by the Stream identification function.";
          reference
            "9.2.1 of IEEE Std 802.1CB-2017";
        }
        leaf output-pkts {
          type uint64;
          description
            "The tsnCpsSidOutputPackets counter is incremented once for
            each packet passed down the stack by the Stream
            identification function.";
          reference
            "9.2.2 of IEEE Std 802.1CB-2017";
        }
      }
    }
  }
}
```

12.6.2.3 Definition for the ieee802-dot1cb-mask-and-match module

```
module ieee802-dot1cb-mask-and-match {
  yang-version 1.1;
  namespace "urn:ieee:std:802.1CB:yang:ieee802-dot1cb-mask-and-match";
  prefix dot1cb-mm-sid;

  import ieee802-dot1cb-stream-identification {
    prefix dot1cb-sid;
  }

  import ieee802-dot1cb-stream-identification-types {
    prefix dot1cb-sid-types;
  }

  import ieee802-types {
    prefix ieee802;
  }

  import ietf-interfaces {
    prefix if;
  }
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
  organization
    "Institute of Electrical and Electronics Engineers";
  contact
    "WG-URL: http://ieee802.org/1/
     WG-EMail: stds-802-1-l@ieee.org

     Contact: IEEE 802.1 Working Group Chair
     Postal: C/O IEEE 802.1 Working Group
             IEEE Standards Association
             445 Hoes Lane
             Piscataway, NJ 08854
             USA

     E-mail: stds-802-1-chairs@ieee.org";

  description
    "Management objects that control the Mask-and-match
     stream identification function from IEEE Std 802.1CBdb-2021.
     Augmentation to the ieee802-dot1cb-stream-identification module.      This
YANG data model conforms to the Network Management Datastore
     Architecture defined in RFC 8342. Copyright (C) IEEE (2021). This
     version of this YANG module is part of IEEE Std 802.1CBdb-2021;
     see the draft itself for full legal notices.";

  revision 2021-12-09 {
    description
      "Published as part of IEEE Std 802.1CBdb-2021.
       Initial version.";
  reference
      "IEEE Std 802.1CBdb-2021, Frame Replication and Elimination for
      Reliability - Extended Stream identification functions.";
  }

  typedef msdu-mask-match-hex-string {
    type string {
      pattern '[0-9a-fA-F]{2}(-[0-9a-fA-F]{2}){1,1983}';
    }
    description
      "Hex string representing a series of 2 to 1984 octets, each
       separated by a '-' separator";
  }

  typedef msdu-mask-length-type {
    type uint16 {
      range "2..1984";
    }
    description
      "unsigned integer between 2 and 1984";
}

  identity mask-and-match-stream-identification {
    base dot1cb-sid-types:strid-idty;
    description
      "Mask-and-match Stream Identification";
  }

  augment "/dot1cb-sid:stream-identity/dot1cb-sid:parameters" {
    description
      "Controlling parameters for the Mask-and-match Stream
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
    identification method";
  container mask-and-match-stream-identification {
    description
      "When instantiating an instance of the Mask-and-match Stream
       identification function (6.8), the parameters in the following
       subclauses replace the tsnStreamIdParameters managed object
       (9.1.1.7).";
    reference
      "9.1.6 of IEEE Std 802.1CBdb-2021";
    container identification-type {
      config false;
      description
        "The identification type indicating the method used to
        identify packets belonging to the Stream. The
        identification type contains a type number and an
        Organizationally Unique Identifier (OUI) or Company ID
        (CID) to identify the organization defining the
        identification method.";
      reference
        "9.1.1.6 of IEEE Std 802.1CB-2017";
      leaf type-number {
        type dot1cb-sid-types:stream-id-function;
        default "mask-and-match";
        description
          "The stream identification type used for the Mask-and-match
          Stream identification method.";
        reference
          "9.1.1.6 of IEEE Std 802.1CBdb-2021";
      }
      leaf oui-cid {
        type string {
          pattern "[0-9A-F]{2}(-[0-9A-F]{2}){2}";
        }
        default "00-80-C2";
        description
          "The Organizationally Unique Identifier (OUI) or Company ID
          (CID) to identify the organization defining the
          identification method. For identification methods defined
          in IEEE Std 802.1CB-2017 the OUI/CID is always 00-80-C2.";
        reference
          "9.1.1.6 of IEEE Std 802.1CB-2017";
        }
    }
    leaf destination-mac-mask {
      type ieee802:mac-address;
      description
        "Specifies a 48-bit mask. A bitwise AND operation is performed
         between destination-mac-mask and the
         destination_address_parameter passed by the ISS indication
         primitive to the Mask-and-match Stream identification
         function. The resulting 48-bit information is the masked
         destination_address that is used as input for the instance of
         the Mask-and-match Stream identification function. If
         destination-mac-mask has a value of 0, the destination-address
         parameter is ignored.";
      reference
        "9.1.6.1 of IEEE Std 802.1CBdb-2021";
    }
    leaf destination-mac-match {
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
        when '../destination-mac-mask';
        type ieee802:mac-address;
        description
          "Specifies the 48-bit value of the masked destination_address,
           to be matched by the instance of the Mask-and-match Stream
           identification function.";
        reference
          "9.1.6.2 of IEEE Std 802.1CBdb-2021";
      }
      leaf source-mac-mask {
        type ieee802:mac-address;
        description
          "Specifies a 48-bit mask. A bitwise AND operation is performed
           between source-mac-mask and the source_address_parameter
           passed by the ISS indication primitive to the Mask-and-match
           Stream identification function. The resulting 48-bit
           information is the masked source_address that is used as input
           for the instance of the Mask-and-match Stream identification
           function. If source-mac-mask has a value of 0, the
           source-address parameter is ignored.";
        reference
          "9.1.6.3 of IEEE Std 802.1CBdb-2021";
      }
      leaf source-mac-match {
        when '../source-mac-mask';
        type ieee802:mac-address;
        description
          "Specifies the 48-bit value of the masked source_address, to be
           matched by the instance of the Mask-and-match Stream
           identification function.";
        reference
          "9.1.6.4 of IEEE Std 802.1CBdb-2021";
      }
      leaf msdu-mask-length {
        type msdu-mask-length-type;
        description
          "Integer value that specifies the length, in octets, of
           tsnCpeMmIdMsduMask (9.1.6.6) and tsnCpeMmIdMsduMatch
           (9.1.6.7). The behavior of the Mask-and-match Stream
           identification function is undefined on any port where the
           value of tsnCpeMmIdMsduMaskLength exceeds the per-port
           mac_service_data_unit mask maximum length (9.4.1).
           tsnCpeMmIdMsduMaskLength has a minimum value of 2. If the
           length of the frame's mac_service_data_unit is shorter than
           tsnCpeMmIdMsduMaskLength, then the identification fails.";
        reference
          "9.1.6.5 of IEEE Std 802.1CBdb-2021";
      }
      leaf msdu-mask {
        type msdu-mask-match-hex-string;
        description
          "Specifies a mask of msdu-mask-length bytes. A bitwise AND
           operation is performed between the msdu-mask-length first
           bytes of the mac_service_data_unit parameter passed by the ISS
           indication primitive to the Mask-and-match Stream
           identification function. The resulting 'msdu-mask-length'-byte
           information is the masked mac_service_data_unit that is used
           as input for the instance of the Mask-and-match Stream
           identification function. If msdu-mask has a value of 0, the
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
          mac-service-data-unit parameter is ignored.";
        reference
          "9.1.6.6 of IEEE Std 802.1CBdb-2021";
      }
      leaf msdu-match {
        when '../msdu-mask';
        type msdu-mask-match-hex-string;
        description
          "Specifies the 'msdu-mask-length'-byte value of the masked
           mac_service_data_unit to be matched by the instance of the
           Mask-and-match Stream identification function";
        reference
          "9.1.6.7 of IEEE Std 802.1CBdb-2021";
      }
    }
  }

  list per-port-msdu-mask-max-length {
    key "port-name";
    description
      "Table of the per-port maximum msdu mask lengths";
    reference
      "9.4.1 of IEEE Std 802.1CBdb-2021";
    leaf port-name {
      type if:interface-ref;
      description
        "Identification of the port";
      reference
        "9.4.1 of IEEE Std 802.1CBdb-2021";
    }
    leaf port-msdu-mask-max-length {
      type msdu-mask-length-type;
      config false;
      description
        "tsnPpMmIdMsduMaskMaxLength is a read-only variable defined for
         each port the Mask-and-match Stream identification function is
         to be ran on. It defines the maximum length a
         mac_service_data_unit mask can have. It is an integer value,
         in octets, with a minimum value of 2.";
      reference
        "9.4.1 of IEEE Std 802.1CBdb-2021";
    }
  }
}
```

12.6.2.4 Definition for the ieee802-dot1cb-frer-types YANG module

```
module ieee802-dot1cb-frer-types {
  yang-version "1.1";
  namespace urn:ieee:std:802.1Q:yang:ieee802-dot1cb-frer-types;
  prefix dot1cb-frer-types;
  organization
    "Institute of Electrical and Electronics Engineers";
  contact
    "WG-URL: http://ieee802.org/1/
    WG-EMail: stds-802-1-l@ieee.org

    Contact: IEEE 802.1 Working Group Chair
    Postal: C/O IEEE 802.1 Working Group
            IEEE Standards Association
            445 Hoes Lane
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
1             Piscataway, NJ 08854
              USA

    E-mail: stds-802-1-chairs@ieee.org";
  description
    "Management objects that control the frame replication and
    elimination from IEEE Std 802.1CB-2017. This YANG data model conforms
    to the Network Management Datastore Architecture defined in RFC 8342.
    Copyright (C) IEEE (2021). This version of this YANG module is part
    of IEEE Std 802.1CBcv; see the standard itself for full legal
    notices.";
  revision 2021-12-08 {
    description
      "Published as part of IEEE Std 802.1CBcv-2021. Initial version.";
    reference
      "IEEE Std 802.1CBcv-2021, Frame Replication and Elimination for
      Reliability - FRER YANG Data Model and Management Information Base
      Module.";
  }
  typedef seq-rcvy-algorithm {
    type enumeration {
      enum vector {
        value 0;
        description
          "The sequence recovery type used for the Vector Recovery
          Algorithm.";
      }
      enum match {
        value 1;
        description
          "The sequence recovery type used for the Match Recovery
          Algorithm.";
      }
    }
    description
      "An enumerated value specifying which sequence recovery algorithm
      is to be used for an instance of the Sequence recovery function.";
    reference
      "10.4.1.5 of IEEE Std 802.1CB-2017";
  }
  typedef seq-encaps-method {
    type enumeration {
      enum reserved {
        value 0;
        description
          "Reserved value.";
      }
      enum r-tag {
        value 1;
        description
          "The sequence encode decode type used for the R_TAG
          encode/decode method.";
      }
      enum hsr-seq-tag {
        value 2;
        description
          "The sequence encode decode type used for the HSR encode/decode
          method.";
      }
      enum prp-seq-trailer {
        value 3;
        description
          "The sequence encode decode type used for the PRP encode/decode
          method.";
      }
    }
    description
      "An enumerated value indicating the type of encapsulation used for
      an instance of the Sequence encode/decode function.";
    reference
      "10.5.1.5 of IEEE Std 802.1CB-2017";
  }
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
}
```

12.6.2.5 Definition for the ieee802-dot1cb-frer YANG module

```
module ieee802-dot1cb-frer {
  yang-version "1.1";
  namespace urn:ieee:std:802.1Q:yang:ieee802-dot1cb-frer;
  prefix dot1cb-frer;
  import ieee802-dot1cb-stream-identification {
    prefix dot1cb-sid;
  }
  import ieee802-dot1cb-stream-identification-types {
    prefix dot1cb-sid-types;
  }
  import ieee802-dot1cb-frer-types {
    prefix dot1cb-frer-types;
  }
  import ieee802-dot1q-types {
    prefix dot1qtypes;
  }
  import ietf-interfaces {
    prefix if;
  }
  organization
    "Institute of Electrical and Electronics Engineers";
  contact
    "WG-URL: http://ieee802.org/1/
    WG-EMail: stds-802-1-l@ieee.org

    Contact: IEEE 802.1 Working Group Chair
    Postal: C/O IEEE 802.1 Working Group
            IEEE Standards Association
            445 Hoes Lane
            Piscataway, NJ 08854
            USA

    E-mail: stds-802-1-chairs@ieee.org";
  description
    "Management objects that control the frame replication and
    elimination from IEEE Std 802.1CB-2017. This YANG data model conforms
    to the Network Management Datastore Architecture defined in RFC 8342.
    Copyright (C) IEEE (2021). This version of this YANG module is part
    of IEEE Std 802.1CBcv; see the standard itself for full legal
    notices.";
  revision 2026-08-19 {
    description
      "Published as part of IEEE Std 802.1CB-2027. Removal of the YANG
      data model's parts related to autoconfiguration.";
    reference
      "IEEE Std 802.1CB-2027, Frame Replication and Elimination for
      Reliability.";
  }
  revision 2025-11-12 {
    description
      "Published as part of IEEE Std 802.1CB-2017/Cor1-2026.";
    reference
      "IEEE Std 802.1CB-2017/Cor1-2026, Frame Replication and Elimination for
      Reliability - Corrigendum 1.";
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
}
revision 2021-12-08 {
  description
    "Published as part of IEEE Std 802.1CBcv-2021. Initial version.";
  reference
    "IEEE Std 802.1CBcv-2021, Frame Replication and Elimination for
    Reliability - FRER YANG Data Model and Management Information Base
    Module.";
}
feature auto-configuration {
  description
    "Autoconfiguration of entries in the tables for Stream identity
    table, sequence recovery table, and sequence identification table.";
  reference
    "7.11 of IEEE Std 802.1CB-2017
    10.7 of IEEE Std 802.1CB-2017";
}
typedef lan-path-id {
  type int8;
  description
    "An integer specifying a path or LAN. If and only if a packet
    matches an entry in the Sequence identification table that
    specifies HSR or PRP in its frerSeqEncEncapsType object,
    tsnStreamIdLanPathId specifies the LanId or PathId value that must
    be matched for this tsnStreamIdEntry to apply. A value of -1
    indicates that the LanId or PathId are to be ignored.";
  reference
    "10.22 of IEEE Std 802.1CB-2017";
}
typedef sequence-history-length {
  type uint32 {
    range "2..max";
  }
  description
    "An integer specifying how many bits of the SequenceHistory
    variable are to be used.";
  reference
    "7.4.3.2.2 of IEEE Std 802.1CB-2017";
}
grouping sequence-encode-decode {
  description
    "The sequence-encode-decode grouping indicates the type of
    encapsulation used for this instance of the Sequence encode/decode
    function.";
  reference
    "10.5.1.5 of IEEE Std 802.1CB-2017";
  choice encapsulation {
    description
      "This choice indicates the type of encapsulation used for this
      instance of the Sequence encode/decode function. The
      encapsulation includes an Organizationally Unique Identifier
      (OUI) or Company ID (CID) to identify the organization defining
      the Sequence encode/decode method.";
    reference
      "10.5.1.5 of IEEE Std 802.1CB-2017";
    container r-tag {
      presence "true";
      description
        "R-TAG";
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
      reference
        "7.8 of IEEE Std 802.1CB-2017";
      leaf type-number {
        type dot1cb-frer-types:seq-encaps-method;
        default "r-tag";
        config false;
        description
          "The type number used for the R-TAG encode/decode method.";
        reference
          "10.5.1.5 of IEEE Std 802.1CB-2017";
      }
      leaf oui-cid {
        type string {
          pattern "[0-9A-F]{2}(-[0-9A-F]{2}){2}";
        }
        default "00-80-C2";
        config false;
        description
          "The Organizationally Unique Identifier (OUI) or Company ID
          (CID) to identify the organization defining the encode/decode
          method. For encode/decode methods defined in IEEE Std
          802.1CB-2017 the OUI/CID is always 00-80-C2.";
        reference
          "10.5.1.5 of IEEE Std 802.1CB-2017";
      }
    }
    container hsr-sequence-tag {
      presence "true";
      description
        "HSR sequence tag";
      reference
        "7.9 of IEEE Std 802.1CB-2017";
      leaf type-number {
        type dot1cb-frer-types:seq-encaps-method;
        default "hsr-seq-tag";
        config false;
        description
          "The type number used for the HSR sequence tag encode/decode
          method.";
        reference
          "10.5.1.5 of IEEE Std 802.1CB-2017";
      }
      leaf oui-cid {
        type string {
          pattern "[0-9A-F]{2}(-[0-9A-F]{2}){2}";
        }
        default "00-80-C2";
        config false;
        description
          "The Organizationally Unique Identifier (OUI) or Company ID
          (CID) to identify the organization defining the encode/decode
          method. For encode/decode methods defined in IEEE Std
          802.1CB-2017 the OUI/CID is always 00-80-C2.";
        reference
          "10.5.1.5 of IEEE Std 802.1CB-2017";
      }
    }
    container prp-sequence-tag {
      presence "true";
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
        description
          "PRP sequence trailer";
        reference
          "7.10 of IEEE Std 802.1CB-2017";
        leaf type-number {
          type dot1cb-frer-types:seq-encaps-method;
          default "prp-seq-trailer";
          config false;
          description
            "The type number used for the PRP sequence trailer
            encode/decode method.";
          reference
            "10.5.1.5 of IEEE Std 802.1CB-2017";
        }
        leaf oui-cid {
          type string {
            pattern "[0-9A-F]{2}(-[0-9A-F]{2}){2}";
          }
          default "00-80-C2";
          config false;
          description
            "The Organizationally Unique Identifier (OUI) or Company ID
            (CID) to identify the organization defining the encode/decode
            method. For encode/decode methods defined in IEEE Std
            802.1CB-2017 the OUI/CID is always 00-80-C2.";
          reference
            "10.5.1.5 of IEEE Std 802.1CB-2017";
        }
      }
      container organization-specific {
        presence "true";
        description
          "This container allows to select Sequence encode/decode types
          that are defined by entities outside of IEEE 802.1.";
        reference
          "10.5.1.5 of IEEE Std 802.1CB-2017";
        leaf type-number {
          type int32 {
            range "256..max";
          }
          description
            "The type number used for an encode/decode method defined by
            an entity owning the OUI or CID for this encapsulation type.";
          reference
            "10.5.1.5 of IEEE Std 802.1CB-2017";
        }
        leaf oui-cid {
          type string {
            pattern "[0-9A-F]{2}(-[0-9A-F]{2}){2}";
          }
          description
            "The Organizationally Unique Identifier (OUI) or Company ID
            (CID) to identify the organization defining the encode/decode
            method.";
          reference
            "10.5.1.5 of IEEE Std 802.1CB-2017";
        }
      }
    }
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
}
grouping sequence-recovery-algorithm {
  description
    "The sequence-recovery-algorithm grouping specifies which sequence
    recovery algorithm is to be used for this instance of the Sequence
    recovery function.";
  reference
    "10.4.1.5 of IEEE Std 802.1CB-2017";
  choice algorithm {
    description
      "This choice indicates the sequence recovery algorithm used for
      this instance of the Sequence recovery function. It includes an
      Organizationally Unique Identifier (OUI) or Company ID (CID) to
      identify the organization defining the sequence recovery
      algorithm.";
    reference
      "10.4.1.5 of IEEE Std 802.1CB-2017";
    container vector {
      presence "true";
      description
        "Vector Recovery Algorithm.";
      reference
        "10.4.1.5 of IEEE Std 802.1CB-2017";
      leaf type-number {
        type dot1cb-frer-types:seq-rcvy-algorithm;
        default "vector";
        config false;
        description
          "The type number used for the VectorRecoveryAlgorithm.";
        reference
          "10.4.1.5 of IEEE Std 802.1CB-2017";
      }
      leaf oui-cid {
        type string {
          pattern "[0-9A-F]{2}(-[0-9A-F]{2}){2}";
        }
        default "00-80-C2";
        config false;
        description
          "The Organizationally Unique Identifier (OUI) or Company ID
          (CID) to identify the organization defining the sequence
          recovery algorithm. For sequence recovery algorithms defined
          in IEEE Std 802.1CB-2017 the OUI/CID is always 00-80-C2.";
        reference
          "10.4.1.5 of IEEE Std 802.1CB-2017";
      }
    }
    container match {
      presence "true";
      description
        "Match Recovery Algorithm.";
      reference
        "10.4.1.5 of IEEE Std 802.1CB-2017";
      leaf type-number {
        type dot1cb-frer-types:seq-rcvy-algorithm;
        default "match";
        config false;
        description
          "The type number used for the MatchRecoveryAlgorithm.";
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
        reference
          "10.4.1.5 of IEEE Std 802.1CB-2017";
      }
      leaf oui-cid {
        type string {
          pattern "[0-9A-F]{2}(-[0-9A-F]{2}){2}";
        }
        default "00-80-C2";
        config false;
        description
          "The Organizationally Unique Identifier (OUI) or Company ID
          (CID) to identify the organization defining the sequence
          recovery algorithm. For sequence recovery algorithms defined
          in IEEE Std 802.1CB-2017 the OUI/CID is always 00-80-C2.";
        reference
          "10.4.1.5 of IEEE Std 802.1CB-2017";
      }
    }
    container organization-specific {
      presence "true";
      description
        "This container allows to select sequence recovery algorithms
        that are defined by entities outside of IEEE 802.1.";
      reference
        "10.4.1.5 of IEEE Std 802.1CB-2017";
      leaf type-number {
        type int32 {
          range "256..max";
        }
        description
          "The type number used for a sequence recovery algorithm
          defined by an entity owning the OUI or CID for this algorithm
          type.";
        reference
          "10.4.1.5 of IEEE Std 802.1CB-2017";
      }
      leaf oui-cid {
        type string {
          pattern "[0-9A-F]{2}(-[0-9A-F]{2}){2}";
        }
        description
          "The Organizationally Unique Identifier (OUI) or Company ID
          (CID) to identify the organization defining the sequence
          recovery algorithm.";
        reference
          "10.4.1.5 of IEEE Std 802.1CB-2017";
      }
    }
  }
}
container frer {
  description
    "The managed objects that control Stream identification are
    described in Clause 9 of IEEE Std 802.1CB-2017. The managed objects
    that control FRER are described in Clause 10 of IEEE Std
    802.1CB-2017 as follows:

     a) General requirements on the behavior of counters are in 10.1.
     b) The various tables of managed objects that can manage, in
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

detail, each individual Stream, are described in five subclauses, including:

~~1) Additions to the Stream identity table required for Autoconfiguration.~~

~~2~~1) The Sequence generation table that configures instances of the Sequence generation function;

~~3~~2) The Sequence recovery table that configures instances of the Individual recovery function, the Sequence recovery function, and the Latent error detection function;

~~4~~3) The Sequence identification table that configures instances of the Sequence encode/decode function; and

~~5~~4) The Stream split table that configures instances of the Stream splitting function.

~~c) The managed objects that support the automatic configuration, upon receipt of a packet, of entries in the first four of the preceding tables, are described in the subclause on Autoconfiguration.~~

~~d~~c) The per-port, per-Stream packet counters that are kept by FRER functions for inspection by network management entities are described in 10.8, and the per-port (totaled over all Streams) counters in 10.9.

The managed objects in the subclauses under 9.1 make it possible to configure more than one encapsulation for the same stream_handle subparameter on the same port. Similarly, the managed objects in the subclauses under 10.3 and 10.4 make it possible to configure more than one Sequence encode/decode function or more than one Sequence generation function for the same stream_handle subparameter. [The same value of stream_handle can be in the frerSeqGenStreamList of more than one frerSeqGenEntry or in the frerSeqRcvyStreamList of more than one frerSeqRcvyEntry.] A system shall return an error if an attempt is made to configure conflicting requirements upon that system.";

```
  reference
    "Clause 10 of IEEE Std 802.1CB-2017";
  list sequence-generation {
    key "index";
    description
      "There is one Sequence generation table in a system, and one
      entry in the Sequence generation table for each Sequence
      generation function. Each frerSeqGenEntry lists the Streams and
      direction for which a single instance of the Sequence generation
      function is to be placed.";
    reference
      "10.3 of IEEE Std 802.1CB-2017";
    leaf index {
      type uint32;
      description
        "Clause 10 of IEEE Std 802.1CB-2017 states that the same Stream
        handle can be present multiple times in the sequence generation
        table. Therefore this index leaf is being used to uniquely
        identify an entry in the sequence generation table.";
    }
    leaf-list stream {
      type leafref {
        path '/dot1cb-sid:stream-identity/dot1cb-sid:handle';
      }
      min-elements 1;
      description
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
        "A list of stream_handle values, corresponding to the values of
        the tsnStreamIdHandle objects in the Stream identity table, on
        which this instance of the Sequence generation function is to
        operate. The single instance of the Sequence generation
        function created by this frerSeqGenEntry operates every packet
        belonging to this Stream, regardless of the port on which it is
        received.";
      reference
        "10.3.1.1 of IEEE Std 802.1CB-2017";
    }
    leaf direction-out-facing {
      type dot1cb-sid-types:direction;
      description
        "An object indicating whether the Sequence generation function
        is to be placed on the out-facing (True) or in-facing (False)
        side of the port.";
      reference
        "10.3.1.2 of IEEE Std 802.1CB-2017";
    }
    leaf reset {
      type boolean;
      description
        "A Boolean object indicating that the Sequence generation
        function is to be reset by calling its corresponding
        SequenceGenerationReset function. Writing the value True to
        frerSeqGenReset triggers a reset; writing the value False has
        no effect. When read, frerSeqGenReset always returns the value
        False.";
      reference
        "10.3.1.3 of IEEE Std 802.1CB-2017";
    }
  }
  list sequence-recovery {
    key "index";
    description
      "There is one Sequence recovery table in a system, and one entry
      in the Sequence recovery table for each Sequence recovery
      function or Individual recovery function that can also be
      present. The entry describes a set of managed objects for the
      single instance of a Base recovery function and Latent error
      detection function included in the Sequence recovery function or
      Individual recovery function. Each frerSeqRcvyEntry lists the
      Streams, ports, and direction for which instances of a Sequence
      recovery function or Individual recovery function are to be
      instantiated.";
    reference
      "10.4 of IEEE Std 802.1CB-2017";
    leaf index {
      type uint32;
      description
        "Clause 10 of IEEE Std 802.1CB-2017 states that the same Stream
        handle can be present multiple times in the sequence recovery
        table. Therefore this index leaf is being used to uniquely
        identify an entry in the sequence recovery table.";
    }
    leaf-list stream {
      type leafref {
        path '/dot1cb-sid:stream-identity/dot1cb-sid:handle';
      }
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
      min-elements 1;
      description
        "A list of the stream_handle values, corresponding to the
        values of the tsnStreamIdHandle objects in the Stream identity
        table, to which the system is to apply the instance of the
        Sequence recovery function or Individual recovery function.";
      reference
        "10.4.1.1 of IEEE Std 802.1CB-2017";
    }
    leaf-list port {
      type if:interface-ref;
      min-elements 1;
      description
        "The list of ports on each of which the system is to
        instantiate the Sequence recovery function, or from which
        received packets are to be fed to a single instance of the
        Individual recovery function.";
      reference
        "10.4.1.2 of IEEE Std 802.1CB-2017";
    }
    leaf direction-out-facing {
      type dot1cb-sid-types:direction;
      description
        "An object indicating whether the Sequence recovery function or
        Individual recovery function is to be placed on the out-facing
        (True) or in-facing (False) side of the port.";
      reference
        "10.4.1.3 of IEEE Std 802.1CB-2017";
    }
    leaf reset {
      type boolean;
      default "false";
      description
        "A Boolean object indicating that the Sequence recovery
        function or Individual recovery function is to be reset by
        calling its corresponding SequenceGenerationReset function.
        Writing the value True to frerSeqRcvyReset triggers a reset;
        writing the value False has no effect. When read,
        frerSeqRcvyReset always returns the value False.";
      reference
        "10.4.1.4 of IEEE Std 802.1CB-2017";
    }
    container algorithm {
      description
        "This object is an enumerated value specifying which sequence
        recovery algorithm is to be used for this instance of the
        Sequence recovery function. The enumeration uses an OUI or CID
        as shown in Table 10-1. The default value for
        frerSeqRcvyAlgorithm is Vector_Alg (00-80-C2, 0).";
      reference
        "10.4.1.5 of IEEE Std 802.1CB-2017";
      uses sequence-recovery-algorithm;
    }
    leaf history-length {
      type sequence-history-length;
      default "2";
      description
        "An integer specifying how many bits of the SequenceHistory
        variable are to be used. The minimum and the default value is
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
        2, maximum is the maximum allowed by the implementation. [Not
        used if frerSeqRcvyAlgorithm = Match_Alg (00-80-C2, 1).]";
      reference
        "10.4.1.6 of IEEE Std 802.1CB-2017";
    }
    leaf reset-timeout {
      type uint32;
      units "ms";
      description
        "An unsigned integer specifying the timeout period in
        milliseconds for the RECOVERY_TIMEOUT event.";
      reference
        "10.4.1.7 of IEEE Std 802.1CB-2017";
    }
    leaf invalid-sequence-value {
      type uint32;
      config false;
      description
        "A read-only unsigned integer value that cannot be encoded in a
        packet as a value for the sequence_number subparameter, i.e.,
        frerSeqRcvyInvalidSequenceValue is larger than or equal to
        RecovSeqSpace.";
      reference
        "10.4.1.8 of IEEE Std 802.1CB-2017";
    }
    leaf take-no-sequence {
      type boolean;
      default "false";
      description
        "A Boolean value specifying whether packets with no
        sequence_number subparameter are to be accepted (True) or not
        (False). Default value False.";
      reference
        "10.4.1.9 of IEEE Std 802.1CB-2017";
    }
    leaf individual-recovery {
      type boolean;
      description
        "A Boolean value specifying whether this entry describes a
        Sequence recovery function or Individual recovery function.
         a) True: The entry describes an Individual recovery function.
            Packets discarded by the SequenceGenerationAlgorithm will
            cause the variable RemainingTicks to be reset. There is no
            Latent error detection function associated with this entry,
            so frerSeqRcvyLatentErrorDetection cannot also be True.
         b) False: The entry describes a Sequence recovery function.
            Packets discarded by the SequenceGenerationAlgorithm will
            not cause the variable RemainingTicks to be reset.";
      reference
        "10.4.1.10 of IEEE Std 802.1CB-2017";
    }
    leaf latent-error-detection {
      type boolean;
      description
        "A Boolean value indicating whether an instance of the Latent
        error detection function is to be instantiated along with the
        Base recovery function in this Sequence recovery function or
        Individual recovery function. frerSeqRcvyLatentErrorDetection
        cannot be set True if frerSeqRcvyIndividualRecovery is also
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
        True; an Individual recovery function does not include a Latent
        error detection function.";
      reference
        "10.4.1.11 of IEEE Std 802.1CB-2017";
    }
    container latent-error-detection-parameters {
      description
        "The objects in the following subclauses are present if and
        only if frerSeqRcvyIndividualRecovery is False.";
      reference
        "10.4.1.12 of IEEE Std 802.1CB-2017";
      leaf difference {
        type int32;
        description
          "An integer specifying the maximum difference between
          frerCpsSeqRcvyDiscardedPackets, and the product of
          frerCpsSeqRcvyPassedPackets and (frerSeqRcvyLatentErrorPaths
          - 1) that is allowed. Any larger difference will trigger the
          detection of a latent error by the LatentErrorTest function.";
        reference
          "10.4.1.12.1 of IEEE Std 802.1CB-2017";
      }
      leaf period {
        type uint32;
        units "ms";
        default "2000";
        description
          "The integer number of milliseconds that are to elapse
          between instances of running the LatentErrorTest function. An
          implementation can have a minimum value for
          frerSeqRcvyLatentErrorPeriod, below which it cannot be set,
          but this minimum shall be no larger than 1000 ms (1 s).
          Default value 2000 (2 s).";
        reference
          "10.4.1.12.2 of IEEE Std 802.1CB-2017";
      }
      leaf paths {
        type uint16;
        description
          "The integer number of paths over which FRER is operating for
          this instance of the Base recovery function and Latent error
          detection function.";
        reference
          "10.4.1.12.3 of IEEE Std 802.1CB-2017";
      }
      leaf reset-period {
        type uint32;
        units "ms";
        default "30000";
        description
          "The integer number of milliseconds that are to elapse
          between instances of running the LatentErrorReset function.
          An implementation can have a minimum value for
          LatentErrorReset, below which it cannot be set, but this
          minimum shall be no larger than 1000 ms (1 s). Default value
          30000 (30 s).";
        reference
          "10.4.1.12.4 of IEEE Std 802.1CB-2017";
      }
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
    }
  }
  list sequence-identification {
    key "port direction-out-facing";
    description
      "There is one Sequence identification table per system, and one
      entry in the Sequence identification table for each port and
      direction for which an instance of the Sequence encode/decode
      function is to be created. Each entry in the Sequence
      identification table specifies a port and direction on which an
      instance of the Sequence encode/decode function is to be
      instantiated for a list of Streams.";
    reference
      "10.5 of IEEE Std 802.1CB-2017";
    leaf-list stream {
      type leafref {
        path '/dot1cb-sid:stream-identity/dot1cb-sid:handle';
      }
      min-elements 1;
      description
        "A list of stream_handles, corresponding to the values of the
        tsnStreamIdHandle objects in the Stream identity table, for
        which the system is to use the same encapsulation for the
        Sequence encode/decode function.";
      reference
        "10.5.1.1 of IEEE Std 802.1CB-2017";
    }
    leaf port {
      type if:interface-ref;
      description
        "The port on which the system is to place an instance of the
        Sequence encode/decode function.";
      reference
        "10.5.1.2 of IEEE Std 802.1CB-2017";
    }
    leaf direction-out-facing {
      type dot1cb-sid-types:direction;
      description
        "An object indicating whether the Sequence encode/decode
        function is to be placed on the out-facing (True) or in-facing
        (False) side of the port.";
      reference
        "10.5.1.3 of IEEE Std 802.1CB-2017";
    }
    leaf active {
      type boolean;
      description
        "A Boolean value specifying whether this frerSeqEncEntry is
        passive (False), and therefore is used only to decode (extract
        information from) input packets passing up the protocol stack,
        or active (True), and therefore is used both for recognizing
        input packets and for encoding output packets being passed down
        the protocol stack.";
      reference
        "10.5.1.4 of IEEE Std 802.1CB-2017";
    }
    container encapsulation {
      description
        "An enumerated value indicating the type of encapsulation used
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
        for this instance of the Sequence encode/decode function. The
        type includes an OUI or CID.";
      reference
        "10.5.1.5 of IEEE Std 802.1CB-2017";
      uses sequence-encode-decode;
    }
    leaf path-id-lan-id {
      type lan-path-id;
      description
        "A 4-bit integer value to be placed in the PathId field of an
        HSR sequence tag or the LanId field of a PRP sequence trailer
        added to an output packet. This managed object is used only if:
         a) The HSR sequence tag or the PRP sequence trailer is
            selected by the frerSeqEncEncapsType object; and
         b) frerSeqEncActive is False (passive)";
      reference
        "10.5.1.6 of IEEE Std 802.1CB-2017";
    }
  }
  list stream-split {
    key "index";
    description
      "There is one Stream split table per system, with one
      frerSplitEntry per Stream splitting function per set of
      stream_handle values. Each entry in the Stream split table
      specifies a port and direction on which an instance of the Stream
      splitting function is to be instantiated, and the list of
      stream_handles specifying its operation.";
    reference
      "10.6 of IEEE Std 802.1CB-2017";
    leaf index {
      type uint32;
      description
        "Entry in the Stream split table.";
      reference
        "10.6 of IEEE Std 802.1CB-2017";
    }
    leaf port {
      type if:interface-ref;
      description
        "The port on which the system is to place an instance of the
        Stream splitting function performing the stream_handle
        translations specified by frerSplitInputIdList and
        frerSplitOutputIdList is to be placed.";
      reference
        "10.6.1.1 of IEEE Std 802.1CB-2017";
    }
    leaf direction-out-facing {
      type dot1cb-sid-types:direction;
      description
        "An object indicating whether the instance of the Stream
        splitting function performing the stream_handle translations
        specified by frerSplitInputIdList and frerSplitOutputIdList is
        to be placed on the out-facing (True) or in-facing (False) side
        of the port.";
      reference
        "10.6.1.2 of IEEE Std 802.1CB-2017";
    }
    leaf-list input-id {
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
        type leafref {
          path '/dot1cb-sid:stream-identity/dot1cb-sid:handle';
        }
        min-elements 1;
        description
          "A list of stream_handles (tsnStreamIdHandle values) that are
          to be split.";
        reference
          "10.6.1.3 of IEEE Std 802.1CB-2017";
      }
      leaf-list output-id {
        type leafref {
          path '/dot1cb-sid:stream-identity/dot1cb-sid:handle';
        }
        min-elements 1;
        description
          "A list of stream_handles (tsnStreamIdHandle values) into which
          the input packet is to be split, one copy per item in the
          frerSplitOutputIdList.";
        reference
          "10.6.1.4 of IEEE Std 802.1CB-2017";
      }
    }
    container autoconfiguration {
      if-feature "auto-configuration";
      description
        "Container for autoconfiguration managed objects.";
      reference
        "10.7 of IEEE Std 802.1CB-2017";
      list sequence {
        key "index";
        description
          "There is one Sequence autoconfiguration table per system. It
          contains any number of table entries. No two (or more) entries
          in the Sequence autoconfiguration table can have the same
          values for frerAutSeqSeqEncaps, frerAutSeqTagged, and
          frerAutSeqVlan on any given port. Each frerAutSeqEntry object
          relates to a single class of Streams, and specifies how entries
          are created (and destroyed) in the Stream identity table, the
          Sequence recovery table, and the Sequence identification table.";
        reference
          "10.7.1 of IEEE Std 802.1CB-2017";
        leaf index {
          type uint32;
          description
            "Entry in the sequence list referencing to a single class of
            Streams.";
        }
        container sequence_encapsulation {
          description
            "An enumerated value, specifying which Sequence encode/decode
            function, and therefore, which type sequence_number encoding,
            is to be recognized for the purposes of Autoconfiguration.";
          reference
            "10.7.1.1.1 of IEEE Std 802.1CB-2017";
          uses sequence-encode-decode;
        }
        leaf-list receive_port {
          type if:interface-ref;
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
        min-elements 1;
        description
          "The list of ports to which this frerAutSeqEntry applies, and
          on which Stream identification functions, Sequence
          encode/decode functions, and Individual recovery functions
          are to be autocreated.";
        reference
          "10.7.1.1.2 of IEEE Std 802.1CB-2017";
      }
      leaf tagged {
        type enumeration {
          enum tagged {
            value 1;
            description
              "A frame must have a VLAN tag to be matched.";
          }
          enum priority {
            value 2;
            description
              "A frame must be untagged, or have a VLAN tag with a VLAN
              ID = 0 to be matched.";
          }
          enum all {
            value 3;
            description
              "A frame is matched whether tagged or not.";
          }
        }
        description
          "An enumerated value indicating whether packets to be matched
          by this frerAutSeqEntry are permitted to have a VLAN tag.";
        reference
          "10.7.1.1.3 of IEEE Std 802.1CB-2017";
      }
      leaf-list vlan {
        type dot1qtypes:vlanid;
        description
          "A list of vlan_identifiers for the packet to match. A null
          list matches all vlan_identifiers.";
        reference
          "10.7.1.1.4 of IEEE Std 802.1CB-2017";
      }
      leaf-list recovery_port {
        type if:interface-ref;
        min-elements 1;
        description
          "The list of ports on which Sequence recovery functions are
          to be autocreated by this frerAutSeqEntry.";
        reference
          "10.7.1.1.5 of IEEE Std 802.1CB-2017";
      }
      leaf destruction_interval {
        type uint64;
        units "ms";
        default "86400000";
        description
          "An integer number of milliseconds after which an idle set of
          functions created by this frerAutSeqEntry can be destroyed. A
          value of 0 indicates that idle autoconfigured functions are
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
            not to be destroyed. Default value is 86 400 000 decimal (one
            day).";
          reference
            "10.7.1.1.6 of IEEE Std 802.1CB-2017";
        }
        leaf reset-interval {
          type uint64;
          units "ms";
          description
            "The value used to fill frerSeqRcvyResetMSec when
            autoconfiguring entries in the Sequence recovery table.";
          reference
            "10.7.1.1.7 of IEEE Std 802.1CB-2017";
        }
        container algorithm {
          description
            "The value used to fill frerSeqRcvyAlgorithm when
            autoconfiguring entries in the Sequence recovery table.";
          reference
            "10.7.1.1.8 of IEEE Std 802.1CB-2017";
          uses sequence-recovery-algorithm;
        }
        leaf history-length {
          type sequence-history-length;
          default "2";
          description
            "The value used to fill frerSeqRcvyHistoryLength when
            autoconfiguring entries in the Sequence recovery table.";
          reference
            "10.7.1.1.9 of IEEE Std 802.1CB-2017";
        }
        leaf create-individual {
          type boolean;
          description
            "A Boolean value. If True, the receipt of a packet that
            triggers the autoconfiguration of a new tsnStreamIdEntry also
            triggers the instantiation of a frerSeqRcvyEntry for an
            Individual recovery function.";
          reference
            "10.7.1.1.10 of IEEE Std 802.1CB-2017";
        }
        leaf create-recovery {
          type boolean;
          description
            "A Boolean value. If True, the receipt of a packet that
            triggers the autoconfiguration of a new tsnStreamIdEntry can
            also trigger the instantiation of a frerSeqRcvyEntry for a
            Sequence recovery function.";
          reference
            "10.7.1.1.11 of IEEE Std 802.1CB-2017";
        }
        leaf latent-error-detection {
          type boolean;
          description
            "A Boolean value. If True, the autoconfiguration of a new
            Sequence recovery function also creates an associated Latent
            Error Detection function.";
          reference
            "10.7.1.1.12 of IEEE Std 802.1CB-2017";
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
        }
        leaf latent-error-difference {
          type int32;
          description
            "The value used to fill frerSeqRcvyLatentErrorDifference when
             autoconfiguring entries in the Sequence recovery table.";
          reference
            "10.7.1.1.13 of IEEE Std 802.1CB-2017";
        }
        leaf latent-error-period {
          type uint32;
          units "ms";
          default "2000";
          description
            "The value used to fill frerSeqRcvyLatentErrorPeriod when
             autoconfiguring entries in the Sequence recovery table.";
          reference
            "10.7.1.1.14 of IEEE Std 802.1CB-2017";
        }
        leaf latent-error-reset-period {
          type uint32;
          units "ms";
          default "30000";
          description
            "The value used to fill frerSeqRcvyLatentResetPeriod when
             autoconfiguring entries in the Sequence recovery table.";
          reference
            "10.7.1.1.15 of IEEE Std 802.1CB-2017";
        }
      }
      list output {
        key "index";
        description
          "There is one Output autoconfiguration table per system. It
          contains any number of frerAutOutEntry objects, each relating
          to a single class of Streams specifying how active entries are
          created in the Sequence identification table. No two (or more)
          entries in the Output autoconfiguration table can include the
          same port in their frerAutSeqReceivePortList objects.";
        reference
          "10.7.2 of IEEE Std 802.1CB-2017";
        leaf index {
          type uint32;
          description
            "Entry in the output list referencing to a single class of
             Streams.";
        }
        leaf-list port {
          type if:interface-ref;
          min-elements 1;
          description
            "The list of ports to which this frerAutOutEntry applies, and
             on which active Sequence encode/decode functions are to be
             autocreated.";
          reference
            "10.7.2.1.1 of IEEE Std 802.1CB-2017";
        }
        container encapsulation {
          description
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
            "An enumerated value, specifying which Sequence encode/decode
            function, and therefore, which type sequence_number encoding,
            is to be used for autoconfigured Streams on the ports in
            frerAutSeqReceivePortList.";
          reference
            "10.7.2.1.2 of IEEE Std 802.1CB-2017";
          uses sequence-encode-decode;
        }
        leaf lan-path-id {
          type lan-path-id;
          description
            "An integer specifying a path or LAN. If and only if
            frerAutOutEncaps specifies HSR or PRP frerAutOutLanPathId
            specifies the LanId or PathId value to be inserted into the
            HSR sequence tag or PRP sequence trailer of autoconfigured
            packets transmitted on the ports in
            frerAutSeqReceivePortList.";
          reference
            "10.7.2.1.3 of IEEE Std 802.1CB-2017";
        }
      }
    }
  }
  augment "/dot1cb-sid:stream-identity" {
    if-feature "auto-configuration";
    description
      "Two managed objects augment each tsnStreamIdEntry in the Stream
      identity table when Managed objects for autoconfiguration is
      implemented.";
    reference
      "10.2 of IEEE Std 802.1CB-2017";
    leaf auto-configured {
      type boolean;
      config false;
      description
        "A read-only Boolean value, supplied by the system, specifying
        whether this entry was created explicitly (False) or via the
        Sequence autoconfiguration table.";
      reference
        "10.2.1 of IEEE Std 802.1CB-2017";
    }
    leaf lan-path-id {
      type lan-path-id;
      description
        "An integer specifying a path or LAN. If and only if a packet
        matches an entry in the Sequence identification table that
        specifies HSR or PRP in its frerSeqEncEncapsType object,
        tsnStreamIdLanPathId specifies the LanId or PathId value that
        must be matched for this tsnStreamIdEntry to apply. A value of -1
        indicates that the LanId or PathId are to be ignored.";
      reference
        "10.2.2 of IEEE Std 802.1CB-2017";
    }
  }
}
augment "/if:interfaces/if:interface/if:statistics" {
  description
    "The following counters are the counters for frame replication and
    elimination for reliability. All counters are unsigned integers. If
    used on links faster than 650 000 000 bits per second, they shall
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
    be 64 bits in length to ensure against excessively short wrap
    times.

    A Stream identification component shall implement the first two
    counters provided in the stream-identification YANG module per-port
    counters, input-packets and output-packets; the remainder of the
    counters in the frer YANG module module are optional for such a
    system.";
  reference
    "10.8 of IEEE Std 802.1CB-2017
    10.9 of IEEE Std 802.1CB-2017";
  container frer {
    description
      "This container contains the per-port as well as the
      per-port-per-stream counters for frame replication and
      elimination for reliability.";
    reference
      "10.8 of IEEE Std 802.1CB-2017
      10.9 of IEEE Std 802.1CB-2017";
    container per-port-counters {
      config false;
      description
        "Contains the per-port counters for frame replication and
        elimination for reliability. The following counters are
        instantiated for each port on which any of the Stream
        identification function, Sequencing function, or Sequence
        encode/decode function is configured. The counters are indexed
        by port number.";
      leaf rx-passed-pkts {
        type uint64;
        config false;
        description
          "The frerCpSeqRcvyPassedPackets counter is incremented once
          for each packet passed up the stack by the
          VectorRecoveryAlgorithm or MatchRecoveryAlgorithm. Its value
          equals the sum (modulo the size of the counters) of all of
          the frerCpsSeqRcvyPassedPackets counters on this same port.";
        reference
          "10.9.1 of IEEE Std 802.1CB-2017";
      }
      leaf rx-discarded-pkts {
        type uint64;
        config false;
        description
          "The frerCpSeqRcvyDiscardPackets counter is incremented once
          for each packet discarded due to a duplicate sequence number
          or for being a rogue packet by any VectorRecoveryAlgorithm or
          MatchRecoveryAlgorithm on this port. Its value equals the sum
          (modulo the size of the counters) of all of the
          frerCpsSeqRcvyRoguePackets and frerCpsSeqRcvyDiscardedPackets
          counters on this same port.";
        reference
          "10.9.2 of IEEE Std 802.1CB-2017";
      }
      leaf encode-errored-pkts {
        type uint64;
        config false;
        description
          "The frerCpSeqEncErroredPackets counter is incremented once
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
        each time the Sequence encode/decode function receives a
        packet that it is unable to decode successfully. Its value
        equals the sum (modulo the size of the counters) of all of
        the frerCpsSeqEncErroredPackets counters on this same port.";
      reference
        "10.9.2 of IEEE Std 802.1CB-2017";
    }
  }
  list per-port-per-stream-counters {
    key "direction-out-facing handle";
    config false;
    description
      "Contains the per-port-per-stream counters for frame
      replication and elimination for reliability. The following
      counters are instantiated for each port on which any of the
      Stream identification function, Sequencing function, or
      Sequence encode/decode function is configured. The counters are
      indexed by port number, facing (in-facing or out-facing), and
      stream_handle value.";
    reference
      "10.8 of IEEE Std 802.1CB-2017";
    leaf direction-out-facing {
      type dot1cb-sid-types:direction;
      description
        "An object indicating whether the counters apply to
        out-facing (True) or in-facing (False).";
    }
    leaf handle {
      type leafref {
        path '/dot1cb-sid:stream-identity/dot1cb-sid:handle';
      }
      description
        "The according tsnStreamIdHandle for the counters.";
    }
    leaf generation-reset {
      type uint64;
      config false;
      description
        "The frerCpsSeqGenResets counter is incremented each time the
        SequenceGenerationReset function is called.";
      reference
        "10.8.2 of IEEE Std 802.1CB-2017";
    }
    leaf rx-out-of-order-pkts {
      type uint64;
      config false;
      description
        "The frerCpsSeqRcvyOutOfOrderPackets counter is incremented
        once for each packet accepted out-of-order by the
        VectorRecoveryAlgorithm or MatchRecoveryAlgorithm.
        Out-of-order means that the packet's sequence number is not
        one more than the previous packet received.";
      reference
        "10.8.3 of IEEE Std 802.1CB-2017";
    }
    leaf rx-rogue-pkts {
      type uint64;
      config false;
      description
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
        "The frerCpsSeqRcvyRoguePackets counter is incremented once
        for each packet discarded by the VectorRecoveryAlgorithm
        because its sequence_number subparameter is more than
        frerSeqRcvyHistoryLength from RecovSeqNum.";
      reference
        "10.8.4 of IEEE Std 802.1CB-2017";
    }
    leaf rx-passed-pkts {
      type uint64;
      config false;
      description
        "The frerCpsSeqRcvyPassedPackets counter is incremented once
        for each packet passed up the stack by the
        VectorRecoveryAlgorithm or MatchRecoveryAlgorithm.";
      reference
        "10.8.5 of IEEE Std 802.1CB-2017";
    }
    leaf rx-discarded-pkts {
      type uint64;
      config false;
      description
        "The frerCpsSeqRcvyDiscardedPackets counter is incremented
        once for each packet discarded due to a duplicate sequence
        number by the VectorRecoveryAlgorithm or
        MatchRecoveryAlgorithm.";
      reference
        "10.8.6 of IEEE Std 802.1CB-2017";
    }
    leaf rx-lost-pkts {
      type uint64;
      config false;
      description
        "The frerCpsSeqRcvyLostPackets counter is incremented once
        for each packet lost by the VectorRecoveryAlgorithm. A packet
        is counted as lost if its sequence number is not received on
        any ingress port.

        NOTE—If per-source sequence numbering is used,
        frerCpsSeqRcvyLostPackets can count, as lost, packets that
        were sent to another destination, but not lost.";
      reference
        "10.8.7 of IEEE Std 802.1CB-2017";
    }
    leaf rx-tagless-pkts {
      type uint64;
      config false;
      description
        "The frerCpsSeqRcvyTaglessPackets counter is incremented once
        for each packet received by the VectorRecoveryAlgorithm that
        has no sequence_number subparameter.";
      reference
        "10.8.8 of IEEE Std 802.1CB-2017";
    }
    leaf rx-resets {
      type uint64;
      config false;
      description
        "The frerCpsSeqRcvyResets counter is incremented once each
        time the SequenceRecoveryReset function is called.";
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
          reference
            "10.8.9 of IEEE Std 802.1CB-2017";
        }
        leaf rx-latent-error-resets {
          type uint64;
          config false;
          description
            "The frerCpsSeqRcvyLatentErrorResets counter is incremented
            once each time the LatentErrorReset function is called.";
          reference
            "10.8.10 of IEEE Std 802.1CB-2017";
        }
        leaf encode-errored-pkts {
          type uint64;
          config false;
          description
            "The frerCpsSeqEncErroredPackets counter is incremented once
            each time the Sequence encode/decode function receives a
            packet that it is unable to decode successfully.";
          reference
            "10.8.11 of IEEE Std 802.1CB-2017";
        }
      }
    }
  }
}
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Annex A

(normative)

Protocol Implementation Conformance Statement (PICS) proforma

A.1 Introduction[21]

The supplier of an implementation that is claimed to conform to Clause 7 shall complete the following protocol implementation conformance statement (PICS) proforma.

A completed PICS proforma is the PICS for the implementation in question. The PICS is a statement of which capabilities and options of the protocol have been implemented. A PICS is included at the end of each clause as appropriate. The PICS can be used for a variety of purposes by various parties, including the following:

a) As a checklist by the protocol implementor, to reduce the risk of failure to conform to the standard through oversight;
b) As a detailed indication of the capabilities of the implementation, stated relative to the common basis for understanding provided by the standard PICS proforma, by the supplier and acquirer, or potential acquirer, of the implementation;
c) As a basis for initially checking the possibility of interworking with another implementation by the user, or potential user, of the implementation (note that, while interworking can never be guaranteed, failure to interwork can often be predicted from incompatible PICS);
d) As the basis for selecting appropriate tests against which to assess the claim for conformance of the implementation, by a protocol tester.

A.1.1 Abbreviations and special symbols

The following symbols are used in the PICS proforma:

M	mandatory field/function
!	negation
O	optional field/function
O.<n>	optional field/function, but at least one of the group of options labeled by the same numeral <n> is required
O/<n>	optional field/function, but one and only one of the group of options labeled by the same numeral <n> is required
X	prohibited field/function
<item>:	simple-predicate condition, dependent on the support marked for <item>
<item1>*<item2>:	AND-predicate condition, the requirement must be met if both optional items are implemented
<item1>+<item2>:	OR-predicate condition, the requirement must be met if either of the optional items are implemented

[21] *Copyright release for PICS proformas:* Users of this standard may freely reproduce the PICS proforma in this subclause so that it can be used for its intended purpose and may further publish the completed PICS.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

A.1.2 Instructions for completing the PICS proforma

The first part of the PICS proforma, Implementation Identification and Protocol Summary, is to be completed as indicated with the information necessary to identify fully both the supplier and the implementation.

The main part of the PICS proforma is a fixed-format questionnaire divided into subclauses, each containing a group of items. Answers to the questionnaire items are to be provided in the right-most column, either by simply marking an answer to indicate a restricted choice (usually Yes, No, or Not Applicable), or by entering a value or a set or range of values. (Note that there are some items where two or more choices from a set of possible answers can apply; all relevant choices are to be marked.)

Each item is identified by an item reference in the first column; the second column contains the question to be answered; the third column contains the reference or references to the material that specifies the item in the main body of the standard; the sixth column contains values and/or comments pertaining to the question to be answered. The remaining columns record the status of the items—whether the support is mandatory, optional, or conditional—and provide the space for the answers.

The supplier may also provide, or be required to provide, further information, categorized as either Additional Information or Exception Information. When present, each kind of further information is to be provided in a further subclause of items labeled A<i> or X<i>, respectively, for cross-referencing purposes, where <i> is any unambiguous identification for the item (e.g., simply a numeral); there are no other restrictions on its format or presentation.

A completed PICS proforma, including any Additional Information and Exception Information, is the protocol implementation conformance statement for the implementation in question.

Note that where an implementation is capable of being configured in more than one way, according to the items listed under Major Capabilities/Options, a single PICS may be able to describe all such configurations. However, the supplier has the choice of providing more than one PICS, each covering some subset of the implementation's configuration capabilities, if that would make presentation of the information easier and clearer.

A.1.3 Additional information

Items of Additional Information allow a supplier to provide further information intended to assist the interpretation of the PICS. It is not intended or expected that a large quantity will be supplied, and the PICS can be considered complete without any such information. Examples might be an outline of the ways in which a (single) implementation can be set up to operate in a variety of environments and configurations; or a brief rationale, based perhaps upon specific application needs, for the exclusion of features that, although optional, are nonetheless commonly present in implementations.

References to items of Additional Information may be entered next to any answer in the questionnaire, and may be included in items of Exception Information.

A.1.4 Exceptional information

It may occasionally happen that a supplier will wish to answer an item with mandatory or prohibited status (after any conditions have been applied) in a way that conflicts with the indicated requirement. No preprinted answer will be found in the Support column for this; instead, the supplier is required to write into the Support column an X<i> reference to an item of Exception Information, and to provide the appropriate rationale in the Exception item itself.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

An implementation for which an Exception item is required in this way does not conform to this standard.

Note that a possible reason for the situation described above is that a defect in the standard has been reported, a correction for which is expected to change the requirement not met by the implementation.

A.1.5 Conditional items

The PICS proforma contains a number of conditional items. These are items for which both the applicability of the item itself, and its status if it does apply—mandatory, optional, or prohibited—are dependent upon whether or not certain other items are supported.

Individual conditional items are indicated by a conditional symbol of the form "<item>:<s>" in the Status column, where "<item>" is an item reference that appears in the first column of the table for some other item, and "<s>" is a status symbol, M (Mandatory), O (Optional), or X (Not Applicable).

If the item referred to by the conditional symbol is marked as supported, then 1) the conditional item is applicable, 2) its status is given by "<s>", and 3) the support column is to be completed in the usual way. Otherwise, the conditional item is not relevant and the Not Applicable (N/A) answer is to be marked.

Each item whose reference is used in a conditional symbol is indicated by an asterisk in the Item column.

A.1.6 Identification

A.1.6.1 Implemention identification

Supplier (Note 1)	
Contact point for queries about the PICS (Note 1)	
Implementation Name(s) and Version(s) (Notes 1 and 3)	
Other information necessary for full identification—e.g., name(s) and version(s) of machines and/or operating system names (Note 2)	
NOTE 1—Required for all implementations. NOTE 2—May be completed as appropriate in meeting the requirements for the identification. NOTE 3—The terms Name and Version should be interpreted appropriately to correspond with a supplier's terminology (e.g., Type, Series, Model).	

A.1.6.2 Protocol summary

Identification of protocol specification	IEEE Std 802.1CB, IEEE Standard for Frame Replication and Elimination for Reliability
Identification of amendments and corrigenda to the PICS proforma that have been completed as part of the PICS	Amd : _____ Cor: _____ Amd : _____ Cor: _____
Have any exceptions been noted? (See A.1.4. The answer, "Yes" means that the implementation does not conform to IEEE Std 802.1CB)	Yes [ ] No [ ]

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

A.2 PICS proforma for Frame Replication and Elimination for Reliability[22]

A.2.1 Major capabilities/options

Item	Feature	Subclause	Value/Comment	Status	Support
BG	FRER C-component implemented?	5.15		O	Yes [ ] No [ ]
IS	Stream identification system implemented?	5.3, 5.4, 5.5	One or more of IS, TE, LE, and/or RS must be answered "Yes."	O.1	Yes [ ] No [ ]
TE	Talker end system implemented?	5.6, 5.7, 5.8		O.1	Yes [ ] No [ ]
LE	Listener end system implemented?	5.9, 5.10, 5.11		O.1	Yes [ ] No [ ]
RS	Relay system implemented?	5.12, 5.13, 5.14, 5.15:b, 5.15:c		BG:M + O.1	Yes [ ] No [ ]
MIB	Does the system implementation support management operations using SMIv2 MIB modules?	5.8, 5.11, 5.14		O	Yes [ ] No [ ]
YANG	Does the implementation support management operations using the YANG modules?	5.8, 5.11, 5.14		O	Yes [ ] No [ ]

A.2.2 Stream identification component

Item	Feature	Subclause	Value/Comment	Status	Support
IS1	Can the system identify frames using the Null Stream identification function?	5.3:b, 6.4		IS: M	Yes [ ]
IS2	Does the system implement the required managed objects of Clause 9?	5.3:c, 9		IS: M	Yes [ ]
IS3	Can the system identify frames using the Source MAC and VLAN Stream identification function ?	5.4:a, 6.5		IS: O	Yes [ ] No [ ] ___[a]
IS4	Can the system encode frames using the Active Destination MAC and VLAN Stream identification?	5.4:b, 6.6		IS: O	Yes [ ] No [ ] ___[a]
IS5	Can the system identify packets using the IP Stream identification?	5.5:c, 6.7		IS: O	Yes [ ] No [ ]

[22] *Copyright release for PICS proformas:* Users of this standard may freely reproduce the PICS proforma in this annex so that it can be used for its intended purpose and may further publish the completed PICS.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Item	Feature	Subclause	Value/Comment	Status	Support
IS6	Can the system identify packets using the Mask-and-match Stream identification ?	5.5:d, 6.8		IS: O	Yes [ ] No [ ]
IS7	For what additional Stream decodings can the system be configured?	5.5:e		IS: O	—
IS8	Explain the limits on which ports the above features can be configured.	5.5:a		IS: O	—
IS9	Explain the limits on the number of Streams for which the above features can be configured.	5.5:b		IS: O	—

[a]If "No," supply a reason why.

A.2.3 Talker end system

Item	Feature	Subclause	Value/Comment	Status	Support
TE1	Can the system identify frames using the Null Stream identification function?	5.6:b, 6.4		TE: M	Yes [ ]
TE2	Can the system be configured with a Sequence generation function?	5.6:c, 7.4.1		TE: M	Yes [ ]
TE3	Can the system be configured with a Sequence encode/decode function?	5.6:d, 7.7		TE: M	Yes [ ]
TE4	Does the system implement the managed objects of Clause 9 and Clause 10 (10.7 not required)?	5.6:e, 9, 10		TE: M	Yes [ ]
TE5	Can the system identify frames using the Source MAC and VLAN Stream identification function ?	5.7:a, 6.5		TE: O	Yes [ ] No [ ] —[a]
TE6	Can the system encode frames using the Active Destination MAC and VLAN Stream identification?	5.7:b, 6.6		TE: O	Yes [ ] No [ ] —[a]
TE7	Can the system be configured with a Stream splitting function?	5.7:c, 7.6		TE: O	Yes [ ] No [ ] —[a]
TE8	Can the system identify packets using the IP Stream identification?	5.8:c, 6.7		TE: O	Yes [ ] No [ ]
TE9	Can the system identify packets using the Mask-and-match Stream identification ?	5.8:d, 6.8		TE: O	Yes [ ] No [ ]

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Item	Feature	Subclause	Value/Comment	Status	Support
TE10	For what additional Stream decodings can the system be configured?	5.8:e		TE: O	—
TE11	Can the system encode frames using HSR sequence tag?	5.8:f, 7.8		TE: O	Yes [ ] No [ ]
TE12	Can the system encode frames using PRP sequence trailer?	5.8:g, 7.9		TE: O	Yes [ ] No [ ]
TE13	For what additional Sequence encode/decode functions can the system be configured?	5.8:h		TE: O	—
TE14	Explain the limits on which ports the above features can be configured.	5.8:a		TE: O	—
TE15	Explain the limits on the number of Streams for which the above features can be configured.	5.8:b		TE: O	—

[a]If "No," supply a reason why.

A.2.4 Listener end system

Item	Feature	Subclause	Value/Comment	Status	Support
LE1	Can the system identify frames using the Null Stream identification?	5.9:b, 6.4		LE: M	Yes [ ]
LE2	Can the system be configured with at least two Individual recovery functions?	5.9:c, 7.4.5		LE: M	Yes [ ]
LE3	Can the system be configured with at least one Sequence recovery function using the MatchRecoveryAlgorithm?	5.9:c, 7.4.3, 7.4.3.5		LE: M	Yes [ ]
LE4	Does the system support the Sequence recovery function using the VectorRecoveryAlgorithm with a value of frerSeqRcvyHistoryLength ≥ 2?	5.9:c, 7.4.3, 7.4.3.4		LE: M	Yes [ ]
LE5	Can the system be configured with at least two Individual recovery functions using the MatchRecoveryAlgorithm?	5.9:d, 7.4.5, 7.4.3.5		LE: M	Yes [ ]
LE6	Can the system be configured with a Sequence decoding function?	5.9:e, 7.7		LE: M	Yes [ ]
LE7	Does the system implement the managed objects of Clause 9 and Clause 10 (10.7 not required)?	5.9:f, 9, 10		LE: M	Yes [ ]

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Item	Feature	Subclause	Value/Comment	Status	Support
LE8	Does the Base recovery function process a frame before its FCS has been verified?	7.4.2		LE: M	No [ ]
LE9	Can the system identify frames using the Source MAC and VLAN Stream identification function ?	5.10:a, 6.5		LE: O	Yes [ ] No [ ] __[a]
LE10	Can the system decode frames using the Active Destination MAC and VLAN Stream identification?	5.10:b, 6.6		LE: O	Yes [ ] No [ ] __[a]
LE11	Can the system decode packets using the IP Stream identification?	5.11:c, 6.7		LE: O	Yes [ ] No [ ]
LE12	Can the system identify packets using the Mask-and-match Stream identification ?	5.11:d, 6.8		LE: O	Yes [ ] No [ ]
LE13	For what additional Stream decodings can the system be configured?	5.11:e		LE: O	—
LE14	Can the system decode frames using HSR sequence tag?	5.11:f, 7.8		LE: O	Yes [ ] No [ ]
LE15	Can the system decode frames using PRP sequence trailer?	5.11:g, 7.9		LE: O	Yes [ ] No [ ]
LE16	For what additional Sequence decodings can the system be configured?	5.11:h		LE: O	—
LE17	Can the system be configured with at least two Individual recovery functions using the VectorRecoveryAlgorithm?	5.11:i, 7.4.5, 7.4.3.4		LE: O	Yes [ ] No [ ]
LE18	Explain the limits on which ports the above features can be configured.	5.11:a		LE: O	—
LE19	Explain the limits on the number of Streams for which the above features can be configured.	5.11:b		LE: O	—

[a]If "No," supply a reason why.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

A.2.5 Relay system

Item	Feature	Subclause	Value/Comment	Status	Support
RS1	Can the system identify frames using the Null Stream identification function?	5.12:b, 6.4		RS: M	Yes [ ]
RS2	Can the system be configured with a Sequence generation function?	5.12:c, 7.4.1		RS: M	Yes [ ]
RS3	Can the system be configured with at least two Individual recovery functions?	5.12:e, 7.4.5		RS: M	Yes [ ]
RS4	Can the system be configured with at least one Sequence recovery function using the MatchRecoveryAlgorithm?	5.12:e, 7.4.3, 7.4.3.5		RS: M	Yes [ ]
RS5	Does the system support the Sequence recovery function using the VectorRecoveryAlgorithm with a value of frerSeqRcvyHistoryLength ≥ 2?	5.12:e, 7.4.3, 7.4.3.4		RS: M	Yes [ ]
RS6	Can the system be configured with at least two Individual recovery functions using the MatchRecoveryAlgorithm?	5.12:f, 7.4.5, 7.4.3.5		RS: M	Yes [ ]
RS7	Can the system be configured with a Sequence encode/decode function?	5.12:d, 7.7		RS: M	Yes [ ]
RS8	Does the system implement the managed objects of Clause 9 and Clause 10 (including 10.7)?	5.12:g, 9, 10		RS: M	Yes [ ]
RS9	Does the Base recovery function process a frame before its FCS has been verified?	7.4.2		RS: M	No [ ]
RS10	Can the system identify frames using the Source MAC and VLAN Stream identification function ?	5.13:a, 6.5		RS: O	Yes [ ] No [ ] ___[a]
RS11	Can the system encode/decode frames using the Active Destination MAC and VLAN Stream identification?	5.13:b, 6.6		RS: O	Yes [ ] No [ ] ___[a]
RS12	Can the system identify packets using the IP Stream identification?	5.13:c:1, 6.7		RS: O	Yes [ ] No [ ] ___[a]
RS13	Can the system identify packets using the Mask-and-match Stream identification ?	5.13:c:2, 6.8		RS: O	Yes [ ] No [ ] ___[a]

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Item	Feature	Subclause	Value/Comment	Status	Support
RS14	For what additional Stream identification functions can the system be configured?	5.14:c		RS: O	—
RS15	Can the Stream splitting function be configured on the system?	5.14:d, 7.6		RS: O	Yes [ ] No [ ]
RS16	Can the system encode/decode frames using HSR sequence tag?	5.14:e, 7.8		RS: O	Yes [ ] No [ ]
RS17	Can the system encode/decode frames using PRP sequence trailer?	5.14:f, 7.9		RS: O	Yes [ ] No [ ]
RS18	For what additional Sequence encode/decode functions can the system be configured?	5.14:g		RS: O	—
RS19	Can the system be configured with at least two Individual recovery functions using the VectorRecoveryAlgorithm?	5.14:i, 7.4.5, 7.4.3.4		RS: O	Yes [ ] No [ ]
RS20	~~Can the system be configured for Autoconfiguration via the Managed objects for autoconfiguration?~~	~~5.14:j, 7.10, 10.7~~		~~RS: O~~	~~Yes [ ]~~ ~~No [ ]~~
RS21	Explain the limits on which ports the above features can be configured.	5.14:a		RS: O	—
RS22	Explain the limits on the number of Streams for which the above features can be configured.	5.14:b		RS: O	—
RS23	Explain the limits on whether the above features can be configured at in-facing or out-facing positions.	5.14:h		RS: O	—

[a]If "No," supply a reason why.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

A.2.6 FRER 802.1Q C-component

Item	Feature	Subclause	Value/Comment	Status	Support
CB1	Does the FRER 802.1Q C-component conform to the required and optional behaviors required of an IEEE 802.1Q C-VLAN component?	5.15:a		BG:M	Yes [ ]
CB1	Does the FRER 802.1Q C-component conform to the placement of the FRER functions of Clause 8?	5.15:d, 8		BG:M	Yes [ ]
CB2	Does the FRER 802.1Q C-component implement all required managed objects?	8.4		BG:M	Yes [ ]

A.2.7 Common requirements

Item	Feature	Subclause	Value/Comment	Status	Support
COM1	Does the R-TAG use the specified EtherType?	7.7.1		M	Yes [ ]
COM1	Is the Reserved field of the R-TAG transmitted as 0 and ignored on receipt?	7.1.1:d		M	Yes [ ]
COM2	Do all managed object counters roll over to 0 from their maximum value?	10.1		M	Yes [ ]
COM3	Can the system be configured with a latent error detection function?	7.4.4	Latent error detection required if sequence recovery supported	LE+RS: M	N/A [ ] Y [ ]
COM4	Is the minimum value for frerSeqRcvyLatentErrorPeriod no larger than 1 s?	10.4.1.12.2		LE+RS: M	N/A [ ] Y [ ]
COM5	Is the RemainingTicks decremented at least 100 ticks/s?	7.4.3.2.5		LE+RS: M	N/A [ ] Y [ ]
COM6	Does the system return an error if an attempt is made to configure conflicting requirements?	10		M	Yes [ ]
COM7	Is the minimum supported value of frerSeqRcvyLatentResetPeriod no larger than 1 s?	10.4.1.12.4		LE+RS: M	N/A [ ] Y [ ]
COM8	Are all counters 64 bits in length on links faster than 650 000 000 bits/s?	10.8, 10.9		M	N/A [ ] Y [ ]

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

A.2.8 Management Information Base (MIB)

Item	Feature	Subclause	Value/Comment	Status	Support
	If item MIB is not supported, mark N/A.				N/A [ ]
MIB1	Is the IEEE8021-STREAM-IDENTIFICATION-MIB module fully supported (per its MODULECOMPLIANCE)?	11.5.1		MIB:M	Yes [ ] No [ ]
MIB2	Is the IEEE8021-FRER-MIB module fully supported (per its MODULECOMPLIANCE)?	11.5.2		MIB:O	Yes [ ] No [ ]

A.2.9 YANG

Item	Feature	Subclause	Value/Comment	Status	Support
	If item YANG is not supported, mark N/A.				N/A [ ]
YANG1	Is the *ieee802-dot1cb-stream-identification-types* module supported?	12.6.2.1		YANG:M	Yes [ ] No [ ]
YANG2	Is the *ieee802-dot1cb-stream-identification* module supported?	12.6.2.2		YANG:M	Yes [ ] No [ ]
YANG3	Is the *ieee802-dot1cb-frer-types* module supported?	12.6.2.4		YANG:O YANG*YANG4:M	Yes [ ] No [ ]
YANG4	Is the *ieee802-dot1cb-frer* module supported?	12.6.2.5		YANG:O	Yes [ ] No [ ]
YANG5	Is the *ieee802-dot1cb-mask-and-match* module supported?	12.6.2.3		YANG:O	Yes [ ] No [ ]

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Annex B

(informative)

Interoperability with other standards

IEC 62439-3 defines High-availability Seamless Redundancy (HSR) and the Parallel Redundancy Protocol (PRP). IEEE Std 802.1CB Frame Replication and Elimination for Reliability (FRER) has features that enable, though the procedures are not specified by this standard, to achieve a degree of interoperation between systems employing IEEE Std 802.1CB, and systems employing some other, similar protocol, including HSR and PRP, and RFC 3985 pseudowires (see [B6]).

B.1 Sequence number size

HSR, PRP, and pseudowires all use 16-bit sequence numbers. This standard supports the same sized sequence_number subparameter, in order to be maximally compatible.

B.2 Per-Stream versus per-source sequencing

IEC 62439-3 HSR/PRP uses one variable per source MAC address to sequence Ethernet frames. That is, frames belonging to different flows, and sent to different destinations, all share a single sequence number space. FRER, as defined in this standard, can be configured in the same way, or can be configured to generate a separate sequence number space for each Stream. For interoperability with HSR or PRP, per-source sequence numbering should be configured by the user.

A problem needs to be considered if one configures a VectorRecoveryAlgorithm (7.4.3.4) to receive Intermittent Streams (item c in 7.1.1) transmitted by an HSR/PRP end system. The HSR/PRP end system uses a single sequence number, not one per Stream. If that end system is transmitting more than one Stream, and some Listener is receiving less than all of those Streams, then that Listener can observe gaps in the sequence_number subparameters of the received packets, which will be counted in frerCpsSeqRcvyLostPackets (10.8.7). For both Intermittent Streams and Bulk Streams, the Sequence recovery function's frerSeqRcvyHistoryLength managed object (10.4.1.6) has to be large enough to accommodate both the delivery time difference and the maximum possible gap due to packets sent to other destinations, because packets received outside the window defined by frerSeqRcvyHistoryLength are discarded. If frerSeqRcvyHistoryLength is not large enough, duplicate packets will be delivered.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Annex C

(informative)

Frame Replication and Elimination for Reliability in systems

The building blocks described in Clause 6 and Clause 7 can be put together in many ways to achieve particular goals. A few examples are given in this annex. These examples are chosen, rather arbitrarily, to illustrate the range of applicability of FRER; they are not meant to constrain its use or to describe the only way to accomplish any given goal.

C.1 Example 1: End-to-end FRER

Illustrated in Figure C-1 is a simple network utilizing FRER for a single Compound Stream. In this example, all FRER functions are confined to the two end systems B and G. “Split” indicates that the Stream splitting function (7.6) is acting on transmitted packets. “Seq.” and “Rec.” indicate whether the Sequencing function is acting on transmitted packets (Sequence generation function, 7.4.1) or received packets (Sequence recovery function, 7.4.1, 7.4.3).

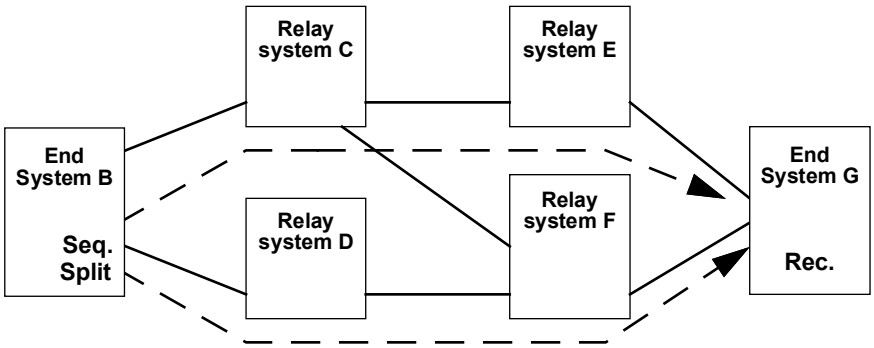


Figure C-1—Dual-homed end systems using Link Aggregation

In this example, both end systems utilize IEEE 802.1AX Link Aggregation along with FRER in order to connect their two ports (each) to a single protocol stack, as shown in Figure C-2 (end system B) and Figure C-3 (end system G). In end system B, each packet is replicated and given two different stream_handle subparameter values. The two stream_handles result in the two packets being assigned two different VLAN IDs. Based on those VLAN IDs, the two packets are both transmitted, on different physical ports, by Link Aggregation. The Distributed Resilient Network Interconnect (DRNI) feature of IEEE Std 802.1AX allows each end system’s aggregation to terminate in two relay systems.

In this example, the two-port end systems never relay packets from one port to the other; they are strictly end systems, and never act as relay systems. See C.2 and C.6 for similar examples that use alternative methods for building an end system.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

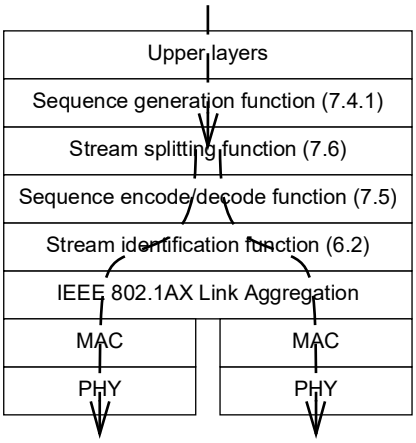


Figure C-2—Protocol stack for End System B in Figure C-1

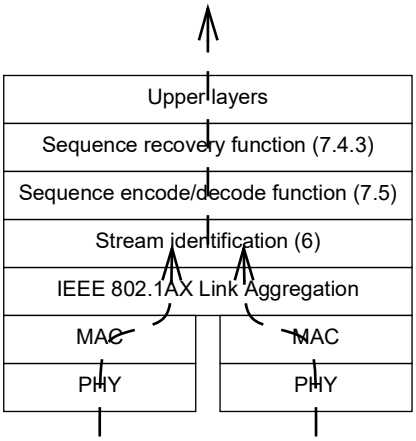


Figure C-3—Protocol stack for End System G in Figure C-1 and Figure C-4

C.2 Example 2: Various stack positions

Illustrated in Figure C-4 is a simple network utilizing FRER for a single Compound Stream, in which a relay system serves as a proxy for an end system that has no FRER capability (see item h in 7.1.1). In this figure, the numbers (1, 2, 3) are points in item e in 7.1.1. “Split” indicates that the Compound Stream is split into two streams by the ordinary bridge multicast mechanism, with the {VLAN, destination address} altered by Stream identification functions on output. [No Stream splitting function (7.6) is used.] “Seq.” and “Rec.” indicate whether the Sequencing function is acting on transmitted packets (sequence generation) or received packets (sequence recovery).

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

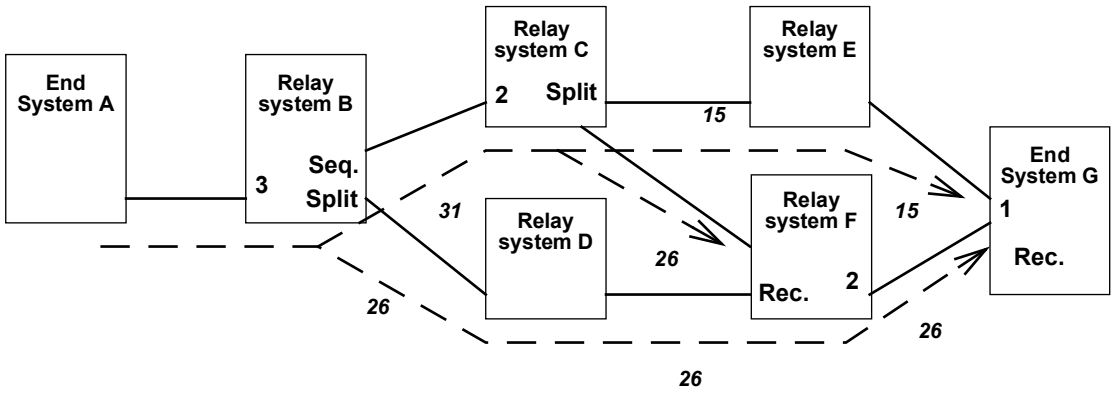


Figure C-4—IEEE Std 802.1CB flexible positioning

In Figure C-4, End System A is transmitting a Stream, but has no FRER functions. Relay system B transforms that Stream into a Compound Stream by sequencing the packets and splitting them into Member Streams *26* and *31* to go to relay systems C and D. Relay system C further splits the Stream into Streams *15* and *26* to go to relay systems E and F. (No sequencing is required; the packets were sequenced by relay system B.) Relay system F combines the two Member Streams *26* from relay systems C and D, outputting a single copy of each Packet on Stream *26* to End System G. End System G merges the two remaining Member Streams *15* and *26*, and discards the extras.

NOTE—In this example, stream_handle values are shown on the wires, being relayed from system to system. This is for the convenience of the example. In an actual network, the stream_handle is an arbitrary integer that has meaning only within a system, and is mapped to or from an external representation by the Stream identification functions (6.2) in the various systems. That external representation can, but does not necessarily, include an explicit field corresponding to a stream_handle subparameter.

Figure C-5 illustrates relay system B in Figure C-4. As the packets enter from the left, from End System A, they pass first through a Stream identification function [IP Stream identification (6.7) or Mask-and-match Stream identification (6.8)], which identifies the Stream. The Stream Transfer Function delivers the packet with all TSN parameters, including the stream_handle subparameter, to the Sequence generation function (7.4.1, marked "Seq." in Figure C-4), which adds a sequence_number subparameter with a steadily-increasing integer sequence value (modulo the size of the packet field carrying the sequence_number). The sequence_number subparameter is encapsulated into the packet by the Sequence encode/decode function (7.5). A Stream identification function [this time, In-facing Active Destination MAC and VLAN Stream identification (6.6)] modifies the two packets' destination MAC addresses and VLANs for identification through the bridged network. Relay system B's forwarding function then outputs the two packets on two different ports. The external form of the packets are labeled differently, as indicated by the italic numbers *26* and *31* in Figure C-4.

Figure C-6 illustrates relay system C in Figure C-4. The packets on Stream *31* enter from the left, from relay system B. On output, they pass first through a passive Stream identification (Clause 6) function, which identifies the Stream. The Stream Transfer Function delivers the packet with all TSN parameters, including the stream_handle subparameter, to an active Stream identification (Clause 6) function. No Sequencing function, Sequence generation function, or Base recovery functions are needed, because the packets have already been sequenced (by relay system B) and none are being discarded. Stream identification encapsulates the two packets differently on the two output ports (only one output port is shown in Figure C-6), marking them as belonging to Streams *15* and *26*, and relay system C's forwarding function outputs the two packets on two different ports.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

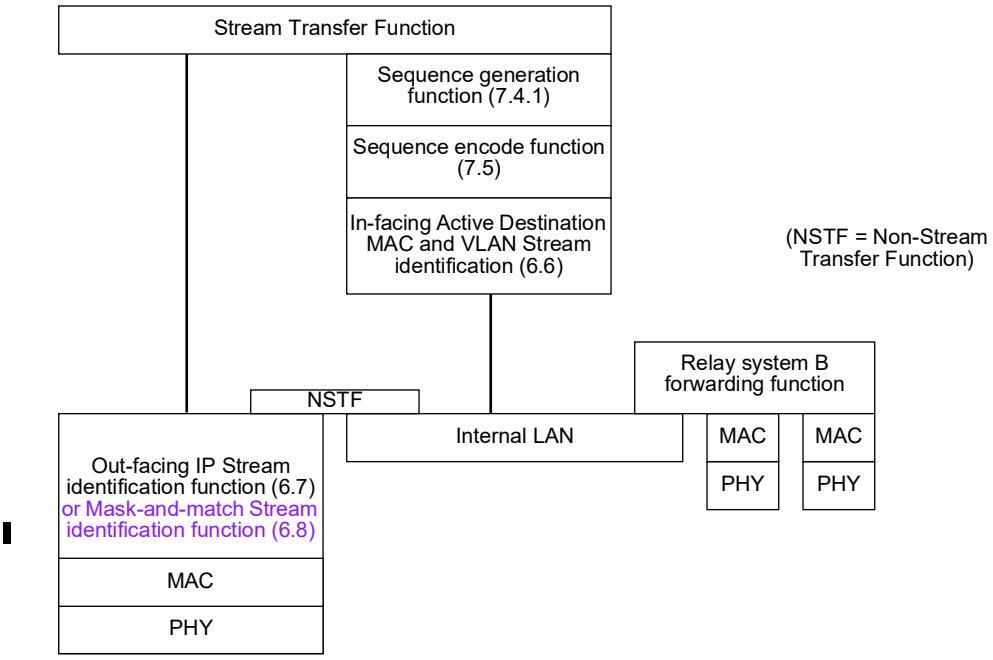


Figure C-5—Protocol stack for relay system B, proxying for End System A, in Figure C-4

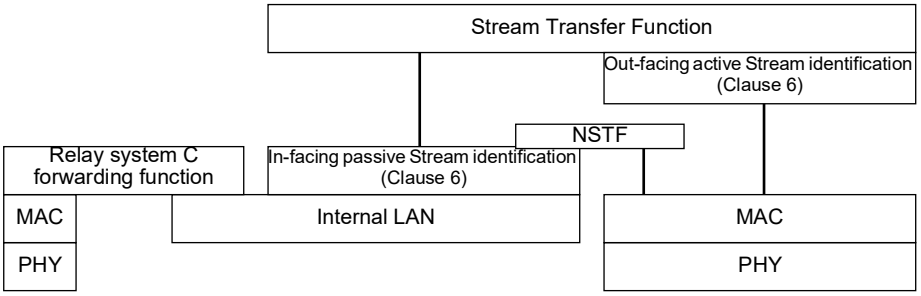


Figure C-6—Protocol stack for relay system C in Figure C-4

Figure C-7 illustrates relay system F in Figure C-4. In this case, the TSN layers are on the output port, rather than the input port, as was the case for relay system B. Data packets from both Streams *26* are brought together simply by the fact that they are both output to the same port. On that port, the In-facing passive Stream identification function gives them both the same stream_handle subparameter, effectively merging them into a single Stream. A Sequence encode/decode function (7.5) extracts the sequence_number subparameters from the packets, so that the Sequence recovery function (7.4.3) can discard the replicates. The Stream Transfer Function delivers all of these parameters to a Sequencing function and an Out-facing active Stream identification function that re-encapsulate the packets, marking them all as belonging to Stream *26*, the same as when they were received. In this particular example, unlike Figure C-5, no translation of Stream identification functions is being performed.

Finally, the protocol stack in the right-hand End System G of Figure C-4 is shown in Figure C-3. We assume in this example that relay systems E and F use IEEE Std 802.1AX Distributed Resilient Network Interface (DRNI), and that End System G uses IEEE 802.1AX Link Aggregation, in order to make End System G's two physical ports appear, to the network, to be a single port, even though they are connected to two different relay systems. The packets on Streams *15* and *26* are brought together by Link Aggregation. The

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Stream identification function assigns the same stream_handle subparameter to packets of both Streams, effectively merging them into a single Stream. They then pass through the Sequence recovery function, which deletes the replicates for delivery to the upper layers. See Figure C-2 for a similar example of a Talker, and C.6 for another type of two-port end system.

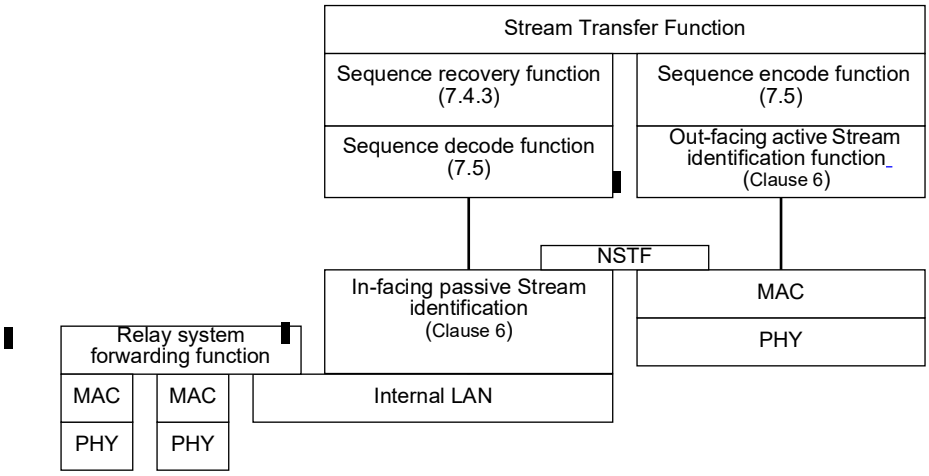


Figure C-7—Protocol stack for relay system F in Figure C-4

C.3 Example 3: Ladder redundancy

Illustrated in Figure C-8 is a network implementing "ladder redundancy." This network will continue to function in the face of multiple failures, because the Stream is repeatedly split and remerged.

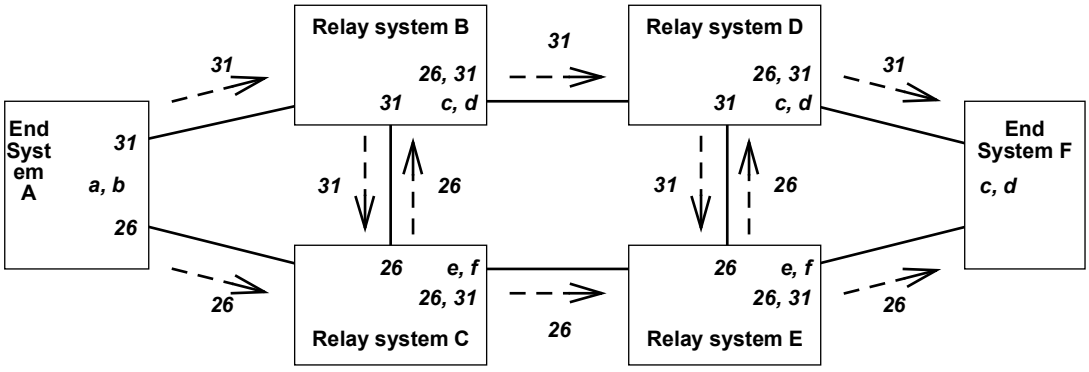


Figure C-8—Ladder redundancy

In Figure C-8, the Talker end system sequences Stream 31, then splits it into two Streams 26 and 31, sending one on each of its two ports. The network "ladder" has two "rails," an upper (relay systems B and D) and a lower (relay systems C and E). The "rungs" are the connections B–C and D–E. Each relay system sends the Stream received on the rail from the left both to the right, along the rail, and up or down, over the rung. It forwards the packets received from the rung only to the right, along the rail. At each output port to the rail, the end systems have a Sequence recovery function (7.4.3) to eliminate duplicates.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Each of the relay systems in Figure C-8 uses the protocol stack illustrated in Figure C-7.

C.4 Example 4: Multicast trees

Illustrated in Figure C-9 is a multicast FRER application. Each Stream is a multicast Stream. There are two paths to get from the Talker to each Listener; no single failure can disrupt both paths to any relay system. Presumably, each relay system has a Sequence recovery function (7.4.3) on each port to each Listener, so that each Listener receives only one copy of each packet of the Stream.

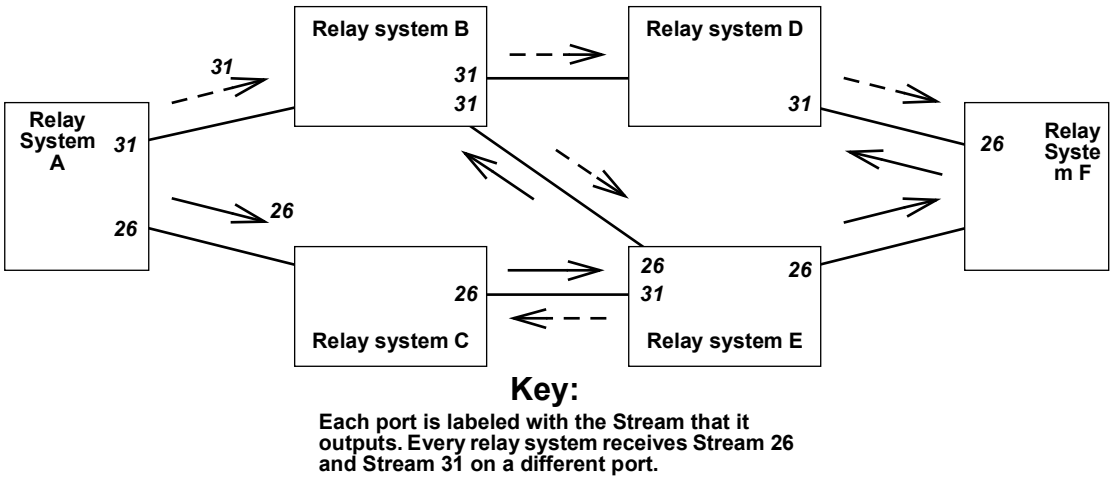


Figure C-9—Multicast trees

NOTE—The reader may find it informative to see that not all packets are replicated on all links in Figure C-9. It is not necessary to use each link twice; it is only necessary to get each packet to each relay system twice.

C.5 Example 5: Protocol interworking

Figure C-10 illustrates a simple protocol interworking function in one port of a relay system. In this example, two different encapsulation schemes **1** and **2** are used for the two legs of the Stream Transfer Function, so that packets are transformed from using one encapsulation to using the other encapsulation as they pass through the port. No additional functions, e.g., a Sequence recovery function (7.4.3) are shown, although they would be perfectly admissible. If this were a port of a bridge attached to an end system, encapsulation **1** could be the Active Destination MAC and VLAN Stream identification (6.6), and encapsulation **2** could be the IP Stream identification (6.7) or the Mask-and-match Stream identification (6.8). The net result for the end system could be to make a specific unicast IP Stream use a specific multicast destination address and VLAN, in order to direct the packet through a specific path through the bridged network. Presumably, a similar interworking pair at the other end of the Stream would restore the packet to its original destination MAC address and VLAN

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

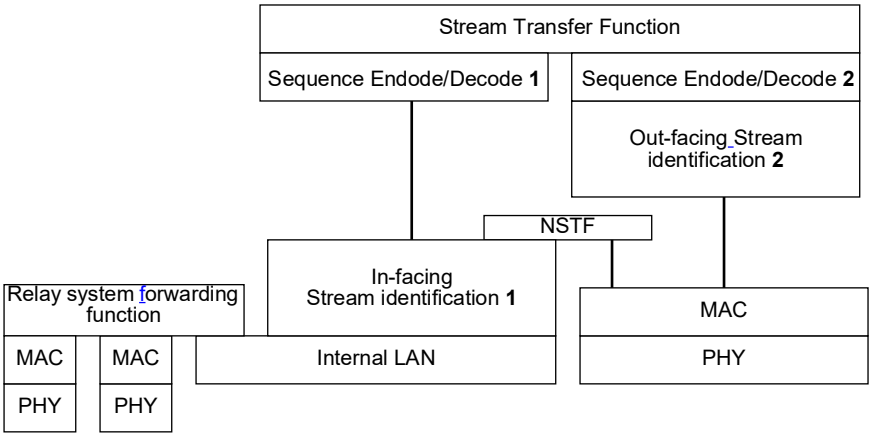


Figure C-10—Protocol interworking

C.6 Example 6: Chained two-port end systems

Illustrated in Figure C-11 is a simple network utilizing FRER for a single Compound Stream. In this example, all FRER functions are confined to the two end systems B and G. "Seq." and "Rec." indicate whether the Sequencing function is acting on transmitted packets (sequence generation) or received packets (sequence recovery). In this example, unlike that in C.1, both end systems are composite systems; each uses a separate one-port end system (B1, G1) attached to a three-port FRER C-component (B2, G2) in order to connect their two ports (each) to a single protocol stack, as shown in Figure C-12.

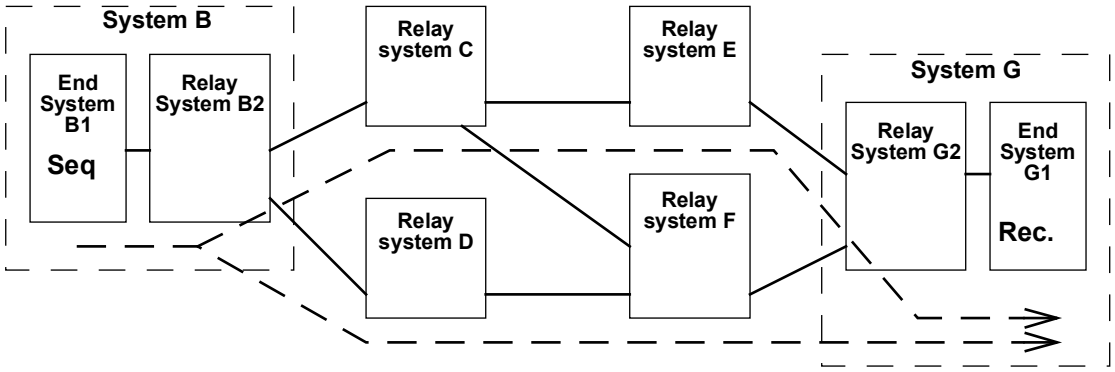


Figure C-11—Dual-homed end systems using 3-port bridge

In this example, we assume that the relay systems are all FRER C-components, and that the network has been configured so that:

a) The Stream packets carry a multicast destination MAC address;

b) Bridge B2 has been configured to transmit the (multicast) Member Stream on both ports; and

c) The other Bridges, especially G2, have been configured to allow both Member Streams to enter Bridge G2, but that Bridge G2 passes the Stream to end system G1 only (that is, the normal rules for a multicast tree are broken).

Given this configuration, no Stream splitting function (7.6) is necessary for end system B1; packet duplication is handled by the multicast forwarding functions performed in its associated relay system B2.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

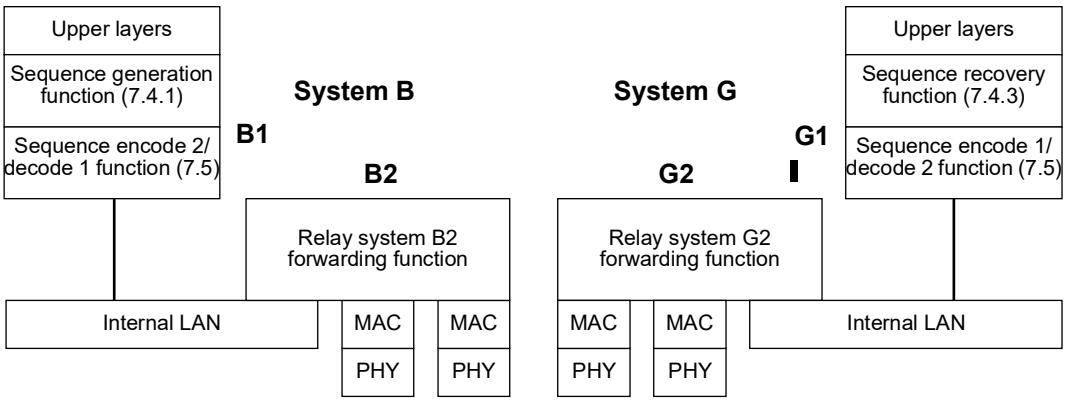


Figure C-12—Protocol stacks for Systems B and G in Figure C-11

Unlike the example in C.1, the two-port composite end systems' Bridge functions (B2, G2) are required to play their part in the network as Bridges, operating protocols and forwarding packets.

C.7 Cautions

Explicit path creation carries a clear danger when combined with automatic network forwarding, e.g., as defined by IEEE Std 802.1Q. Figure C-13 shows an explicit tree (the dashed arrows) established in relay system D that sends a particular Stream to relay system B and end system F. (Perhaps, at one time, the Talker was attached to relay system D and the Listeners were towards B and F.) Now, end system A starts transmitting that Stream along an IEEE 802.1Q spanning tree. When the Stream hits relay system D, it is directed back to relay system B, and the Stream loops, multiplying the packets until the bandwidth is saturated. Note also that, if this is a bridged network, relay system B (a bridge) learns conflicting information about the path to the Stream's source; System A's source MAC address is being received both from end system A and relay system D.

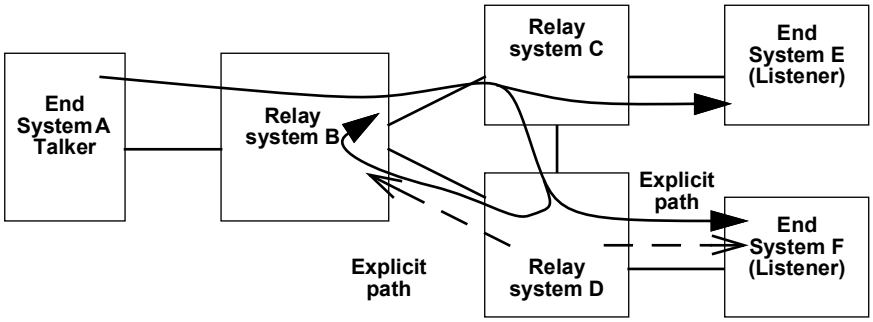


Figure C-13—Explicit path causing a loop

In order to avoid this situation in a bridged network, the application or individual that sets up the explicit paths can configure them so that they are kept, end-to-end, on separate VLANs from traffic that is not following explicit paths. On the explicit path VLANs, source learning is not performed, and destination addresses not found in the filtering database are discarded, not flooded.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

C.8 Balancing tag insertion and removal

A Sequence encode/decode function will often add something to a packet. The R-TAG described in 7.7, for example, adds six octets. In a network in which not all end systems contain a Sequence encode/decode function (7.5), instances of that function can be configured in relay systems adjacent to the end systems lacking the layer, in order to remove the extra information (e.g., the R-TAG) from the packets before delivery to the end system. If the tag is not removed, and the end system has no Sequence encode/decode function, the end system will be unable to parse the received packet. It is up to the network administrator and/or configuration and management software to make sure that this does not happen.

C.9 FRER and reserved bandwidth

Satisfying the Zero congestion loss goal (item k in 7.1.1) can require the forwarding of packets to be delayed. The problem is that the different paths taken by packets in a Compound Stream can take different times to reach various Base recovery functions (7.4.2). If these times are very different, and if the faster path fails and then recovers, the receiving Sequence recovery function can be presented with a double-rate load of packets that all have to be delivered.

As an example, Figure C-14 illustrates a network with two Member Streams being merged into a single Stream by a Sequence recovery function. The two paths are of different lengths such that, typically, there are 40 more packets in flight on one path than on the other.

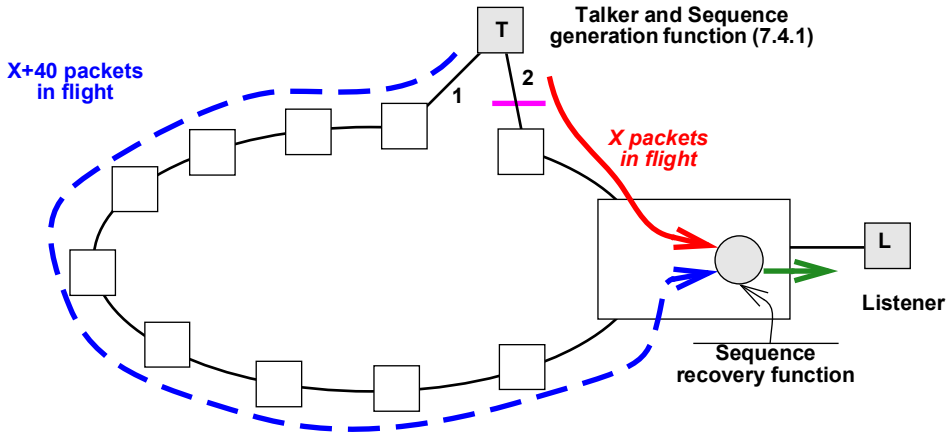


Figure C-14—Example of Long and short paths

Let us examine a hypothetical sequence of events as Link 2 in Figure C-14 fails, and then recovers. In the following list, the sequence numbers of packets taking the path through Link 2 (the short path) are in *italics*, and packets that are discarded by the Sequence recovery function are marked with asterisks (*).

a) The network is in a steady state. The Sequence recovery function receives packets: *1040*, 1000*, *1041*, 1001*, *1042*, 1002*, …
That is, excepting the occasional random packet loss (not shown), the packets taking the *short path* (Link 2) are passed on, and those taking the long path (Link 1) are discarded.

b) Link 2 fails sometime later (4000 packets later). The Sequence recovery function receives: *5040*, 5000* (failure occurs), 5001*, 5002*, …
That is, for a while at least, no packets are passed on; those in the slow path have already been seen.

c) The discards continue with no packets passed until the backlog on the slow path is emptied: 5038*, 5039*, 5040*, 5041, 5042, 5043, …
Normal output rates then resume, with all packets coming from the long path (Link 1).

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

d) When the *short path* (Link 2) heals (3000 packets later), the Sequence recovery function receives: 7998, 7999, (link heals) *8040*, 8000, *8041*, 8001, *8042*, 8002, … Note that the Sequence recovery function is now passing packets at twice the normal rate, until the 40-packet backlog on the long (Link 1) path has been passed along.

e) Finally, the backlog is exhausted, and the Sequence recovery function resumes the normal output rate, passing packets from the *short path* (Link 2) and discarding those from the long path (Link 1): *8078*, 8038, *8079*, 8039, *8080*, 8040*, *8081*, 8041*, *8082*, 8042*, …

Zero congestion loss goal (item k in 7.1.1) can require that packets be output to the Listener at a fixed maximum rate over some observation interval smaller than the transmission time of this 80-packet burst. (See, for example, Clauses 34 and 35 of IEEE Std 802.1Q-2022.) In that case, some means of buffering, and thus delaying the packets in this burst, is required at or near the Sequence recovery function.

This buffering and/or delaying is beyond the scope of this standard. We can observe, however, that the extra buffering required by FRER in order to prevent congestion loss when bandwidth reservation is employed, due to this problem, can be calculated separately from the buffering required for the non-FRER case, based on the bandwidth of the Compound Stream and the difference in worst-case delivery latency along the two (or more) Member Streams' paths.

C.10 Use of the Individual recovery function

The Individual recovery function is described in 7.4.5, and a sample network employing the Individual recovery function is illustrated in Figure 7-3. The primary use of the Individual recovery function is to prevent stale data from being transmitted in the event that one of the Member Streams of a Compound Stream fails such that the same packet is sent over and over again. Imagine, in Figure 7-3, that the leftmost system fails on its upper port, resending the packet with sequence_number 5 over and over again on Member Stream 1. The VectorRecoveryAlgorithm (7.4.3.4) of the Base recovery functions (7.4.2) in the two systems receiving the Member Streams 1 and 2 will discard the repeated packets until the sequence_number subparameter on the good Member Stream 2 wraps around after 65 536 packets. When sequence number 5 gets accepted due to the history window covering it again, the next packet 5 of Stream 1 is accepted and relayed to the next stage. The desired Member Stream 2's packet 5 is discarded when it arrives a number of packets later, depending on the window size and the number of Member Streams. If Individual recovery functions are configured for Member Stream 1 as shown by the triangles in the two systems receiving that Stream, then all of the packet 5s after the first will be discarded there, and never reach the Base recovery functions.

C.11 ~~Use of autoconfiguration~~

~~Every last detail of the operation of FRER on each individual Stream can be configured by using the managed objects in Clause 9 and Clause 10. Often, however, there is enough regularity in the use of FRER across various Streams in a network that Autoconfiguration (7.10) can be used. There are four steps to using Autoconfiguration in a given relay system or end system, as follows:~~

a) ~~Deciding on which port(s) packets belonging to Member Streams will be received and transmitted, using what translations are required that do not use active Stream identification functions (C.11.1).~~
b) ~~Deciding which packets can trigger Autoconfiguration (C.11.2).~~
c) ~~Deciding which method for encoding the sequence_number subparameter will be used for input and output on each port (C.11.3).~~
d) ~~Deciding on whether Individual recovery functions, Base recovery functions, or both, are to be automatically instantiated, and what controlling parameters are to be used for each instantiation (C.11.4).~~

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

~~NOTE—Although the following examples describe interoperation between HSR/PRP and the R-TAG, such interoperability is not the primary reason for defining autoconfiguration. Autoconfiguration greatly simplifies the amount of configuration required for networks that use only the R-TAG.~~

C.11.1 ~~Routing and labeling Member Streams~~

~~Figure C-15 illustrates an example network with four Member Streams constituting a Compound Stream. This example is similar to that illustrated in Figure 7-3.~~

~~In general, the selection of paths taken by Member Streams through the network is not determined by FRER, but by other standards. Autoconfiguration does not provide a means of establishing active Stream identification functions to perform per-Stream identification transformations. In Figure C-15, let us assume for the sake of example that Member Stream 1 and Member Stream 2 both have the same destination MAC address, but are on two special VLANs 1000 and 1001, used exclusively in this network for fully managed pinned-down paths. That is, these two VLANs are not controlled by the topology protocols defined in IEEE Std 802.1Q. Let us further suppose that Member Stream 3 and Member Stream 4 are similar, but both use the same VLANs 2000. (Presumably, source address learning is disabled on this VLAN.)~~

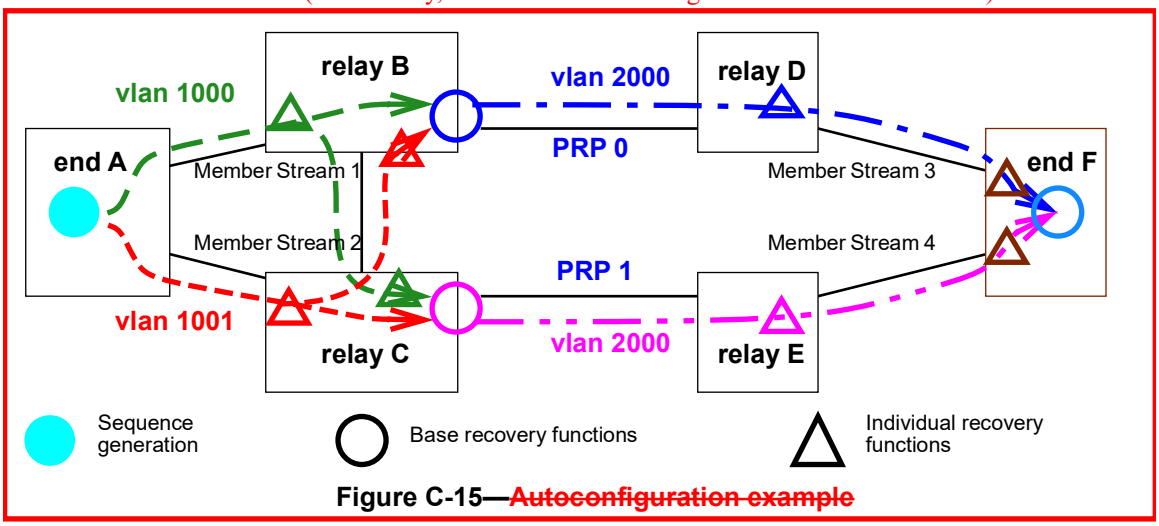


Figure C-15—~~Autoconfiguration example~~

~~Looking at relay system B in Figure C-15, we can see that all packets in Member Stream 3 are to be output on VLAN 2000 with a PRP sequence trailer specifying LanId 0. That means that packets from Member Streams 1 and 2 that are accepted by the Sequence recovery function in relay system B must be translated from VLAN 1000 or 1001 to VLAN 2000. The Autoconfiguration facility of 7.10 does not automatically setup or change vlan_identifiers. Relay system B must be configured to perform a VLAN ID translation on its right-hand port, translating both VLAN 1000 and VLAN 1001 to VLAN 2000 on output. If there are a great many Compound Streams, then presumably they would share the same VLAN assignments and thus make use of the same VLAN translations. IEEE 802.1Q bridges have such a VLAN ID translation capability.~~

~~Although not shown in Figure C-15, we can imagine that there are Streams with Talkers in the right side of the figure and listeners on the left. Additional VLAN mapping would then be required on the left and bottom ports of relay system B to map VLAN 2000 to VLAN 1000 and VLAN 1001. If the VLAN situation were significantly more complex, then the network might not be suitable for Autoconfiguration.~~

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

C.11.2 ~~Recognizing packets that trigger autoconfiguration~~

~~In the example in Figure C-15, let us consider only relay system B, and only packets that follow the same routes as Member Streams 1 and 2. In that case, we expect relay system B to receive packets suitable for Autoconfiguration only on its left and bottom ports, all encoded with the R-TAG (7.7). Thus, an administrator would construct a frerAutSeqEntry (10.7.1.1) in the Sequence autoconfiguration table specifying that tag on those ports, for VLANs 1000 and 1001. This entry will, when a matching packet is received, create a tsnStreamIdEntry (9.1.1) in the Sequence identification table (10.5) using Source MAC and VLAN Stream identification (6.5). Since the R-TAG contains no indication as to which Member Stream a packet belongs, the entry is tied to the port and VLAN on which the packet was received. In this way, all of the Member Streams from end system A that take the path of Member Stream 1 are assigned one stream_handle, and all those following the path of Member Stream 2 get a second stream_handle.~~

~~Again, we can imagine that there are Streams with Talkers in the right side of Figure C-15 and listeners on the left. A second frerAutSeqEntry is required in the Sequence autoconfiguration table is required to recognize, for Autoconfiguration, the packets coming in on relay system B's right-hand port, and create entries in the Sequence identification table. This single entry specifies packets encoded with the PRP sequence trailer (see a further explanation of this in C.11.3), and includes all PRP ports—in this example, just the one. A tsnStreamIdEntry created for HSR or PRP by this entry that assigns packets a stream_handle would not be tied to the port and VLAN on which the packet was received, but to all ports covered by the tsnStreamIdEntry, and to the LanId or PathId contained in the packet. That is, the contents of the HSR sequence tag or the PRP sequence trailer override the port number.~~

C.11.3 ~~Per-port packet decoding and encoding~~

~~The example in Figure C-15 also illustrates a second use of Autoconfiguration. In this network, everything to the right of relay systems B and C, including the rightmost ports on relay systems B and C, use the PRP sequence trailer (7.9), whereas all ports on relay systems A, B, and C to the left use the R-TAG (7.7). Packets transmitted to the right by relay system B, and all packets transmitted by system D and the upper port of system F are marked as being on LanId 0, and by other ports on the right side of the figure as being on LanId 1. Packets transmitted by any other port on relay systems B and C, or by system A, use the R-TAG. These distinctions are made using the Output autoconfiguration table (10.7.2). Using this table, each port is assigned an output encapsulation, including the LanId or PathId for PRP or HSR, respectively. Again, if the usage is too complex (e.g., one Stream is LanId 0 on a port, while another is LanId 1 on that same port), Autoconfiguration cannot be used.~~

~~Note that the administrator could configure the Sequence autoconfiguration tables and Output autoconfiguration tables in relay systems D and E to translate among the R-TAG, the HSR sequence tag, and the PRP sequence trailer for the purpose of interworking, even if no Individual recovery functions or Base recovery functions were desired in those relay systems.~~

C.11.4 ~~Individual and Sequence recovery functions~~

~~Given the explanations in C.11.1, C.11.2, and C.11.3, the reader can consult 7.10.2 to understand how Individual recovery functions and Base recovery functions are configured as a result of Autoconfiguration. They are set up, along with the input (passive) Stream identification functions, by entries in the Sequence autoconfiguration table (10.7.1). The reader may find the following facts helpful:~~

a) ~~At most one instance of the Individual recovery function is instantiated for each stream_handle associated with an autocreated tsnStreamIdEntry (9.1.1).~~
b) ~~One instance of the Sequence recovery function is instantiated on each expected output port (frerAutSeqRecoveryPortList, 10.7.1.1.5) for each Compound Stream.~~

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

c) ~~Compound Streams are inferred by associating Member Streams with the same Source MAC address and VLAN ID.~~

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Annex D

(informative)

Mask-and-match Stream identification function

D.1 Example 1: Generic Stream identification for industrial communication networks

This example illustrates a way to identify a stream of an industrial communication network protocol, described in IEC 61158-1:2019 [B1], whose frames are characterized by:

— being untagged
— the 12 LSBs of their destination MAC address having a value of 0xFAB
— carrying an EtherType of value 0x88B6
— having a payload starting with a 2-octet application-specific field of value 0xBABE

The tsnStreamIdParameters of the tsnStreamIdEntry (9.1.1) corresponding to the instance of the Mask-and-match Stream identification function that identifies this particular Stream are shown in Table D-1

Table D-1—Managed objects values for example 1

Managed object	Value
tsnCpeMmIdDestMacMask	00-00-00-00-0F-FF
tsnCpeMmIdDestMacMatch	0F-AB
tsnCpeMmIdSrcMacMask	00-00-00-00-00-00
tsnCpeMmIdSrcMacMatch	N/A
tsnCpeMmIdMsduMask	FF-FF-FF-FF
tsnCpeMmIdMsduMaskLength	4
tsnCpeMmIdMsduMatch	88-B6-BA-BE
tsnPpMmIdMsduMaskMaxLength	8

D.2 Example 2: MPLS DetNet flow identification

IETF Deterministic Networking (DetNet) operates at the IP layer and delivers service over lower layer technologies such as MPLS. DetNet accomplishes these goals by dedicating network resources such as link bandwidth and buffer space to DetNet flows and/or classes of DetNet flows, and by replicating packets along multiple paths.

MPLS encapsulated DetNet flows can be carried over network technologies that can provide DetNet the required level of service, such as IEEE 802.1 TSN sub-networks.

In a TSN sub network TSN supporting an MPLS DetNet network (See section 6 in [B13]), the identification of DetNet flows can be necessary. In such a case, one or more DetNet flows are supported by a TSN Stream. Therefore, the identification of Streams supporting DetNet flows is needed.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

This example illustrates a way to identify packets of a TSN Stream supporting an MPLS encapsulated DetNet flow. Frames of such Stream are characterized by:

- their destination MAC address having a value of 02-00-B0-C1-19-71 (MAC address of the MPLS next-hop node)
- being C-tagged (EtherType: 81-00), with VLAN-ID of 0x042
- carrying MPLS unicast payload (EtherType: 88-48)
- having a payload starting with a label stack comprising three labels, of which the 2 bottom ones have the value of 0x0900D (F-label) and 0x0FACE (A-label). (For DetNet label definitions see section 4.2 in [B12].)

The tsnStreamIdParameters of the tsnStreamIdEntry (9.1.1) corresponding to the instance of the Mask-and-match Stream identification function that identifies this particular Stream are shown in Table D-2

Table D-2—Managed objects values for example 2

Managed object	Value
tsnCpeMmIdDestMacMask	FF-FF-FF-FF-FF-FF
tsnCpeMmIdDestMacMatch	02-00-b0-c1-19-71
tsnCpeMmIdSrcMacMask	00-00-00-00-00-00
tsnCpeMmIdSrcMacMatch	N/A
tsnCpeMmIdMsduMask	FF-FF-0F-FF-FF-FF 00-80-00-00-00-8F FF-FF-00-8F-FF-FF
tsnCpeMmIdMsduMaskLength	18
tsnCpeMmIdMsduMatch	81-00-00-42-88-48 00-00-00-00-00-00 90-0D-00-80-FA-CE
tsnPpMmIdMsduMaskMaxLength	20

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Annex E

(informative)

Autoconfiguration

E.1 Introduction to autoconfiguration

In order to satisfy the goal of Ease of use (item l in 7.1.1), there are two methods by which Stream identification functions (6.2), Individual recovery functions (7.4.5), Base recovery functions (7.4.2), and Sequence encode/decode functions (7.5) can be configured in a system:

a) Explicit configuration of entries in the Stream identity table (9.1), Sequence recovery table (10.4), Sequence identification table (10.5); and

b) Autoconfiguration of entries in these tables using the Sequence autoconfiguration table (E.3.2.1) and Output autoconfiguration table (E.3.2.2).

Autoconfiguration can only be used with Source MAC and VLAN Stream identification (6.5) and the MatchRecoveryAlgorithm (7.4.3.5).

Whenever a packet is received that has a sequence_number subparameter encoded in a manner identified in an entry in the Sequence autoconfiguration table, but for which there is no tsnStreamIdEntry (9.1.1) in the Stream identity table, a new entry in the Stream identity table is created for the packet's Stream, and optionally, a new entry in the Sequence recovery table for an Individual recovery function for that Stream. If a second or subsequent such Stream identity table entry is created that shares all the same key parameters as another autoconfigured tsnStreamIdEntry, except for port number (tsnStreamIdOutFacInputPortList, 9.1.1.5) and/or LanId/PathId (tsnStreamIdLanPathId, E.3.1.2), then another entry in the Sequence recovery table, this time for a Sequence recovery function, is created (for the second Member Stream) or is expanded (for subsequent Member Streams) for what is now known to be a Compound Stream.

The Sequence autoconfiguration table controls whether Individual recovery functions, Base recovery functions, neither, or both, are created automatically. Stream identification functions are always created. If the R-TAG (7.7) is used to encode the sequence_number, entries autoconfigured in the Sequence identification table, and thus the Individual recovery functions (7.4.5) created, are keyed to the individual ports and VLANs on which the first packet of each Member Stream is received. If the HSR sequence tag (7.8) or PRP sequence trailer (7.9) is used, entries autoconfigured in the Sequence identification table and the autocreated Individual recovery functions are keyed to the PathId or LanId of the first-received packet of each Member Stream, and span all autoconfigurable ports.

In order to support the goals of Ease of use (item l in 7.1.1) and Interoperability (item g in 7.1.1) among systems using the HSR sequence tag (7.8), the PRP sequence trailer (7.9), and the R-TAG (7.7), passive (packet decoding) entries are also automatically created in the Sequence identification table to instantiate Sequence encode/decode functions as governed by the Sequence autoconfiguration table (E.3.2.1). Typically, the Output autoconfiguration table (E.3.2.2) is configured to construct active (packet encoding) Sequence encode/decode functions on every port on which a Stream might be output. See E.4.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

E.2 Autoconfigured Stream identity table entry creation and destruction

NOTE 1—The following description, if implemented naively, might well result in a relay system delaying the transmission of the first-received packet of a Member Stream for an unacceptable length of time. The purpose of the following description is to describe the externally visible behavior of a system solely in terms of the managed objects in Clause 9, Clause 10 and Clause E.3, not to constrain an implementation. It is expected that an actual implementation is likely to construct databases in software, firmware, and/or hardware that combine elements of the managed objects in Clause 9, Clause 10 and Clause E.3, and to perform up-front preliminary configuration, so as to optimize the reaction time when a packet is actually received.

Autoconfiguration is triggered by the receipt of a packet that matches an entry in the Sequence autoconfiguration table (10.7.1) as follows:

a) If a packet received on an out-facing port matches an entry in the Stream identity table (9.1), it is processed according to the managed objects in 10.3, 10.4, 10.5, and 10.6.

NOTE 2—It does not matter whether the Stream identity table entry was configured explicitly, or was created via Autoconfiguration.

b) If the packet does not match any entry in the Stream identity table at all, the packet is examined to see if it matches an entry in the Sequence autoconfiguration table. A match between the packet and a frerAutSeqEntry (E.3.2.1.1) in the Sequence autoconfiguration table occurs if all of the following match conditions are met:

   1) The packet has a sequence_number encapsulated in the manner specified by frerAutSeqSeqEncaps (E.3.2.1.1.1).
   2) The packet was received on a port in frerAutSeqReceivePortList (E.3.2.1.1.2).
   3) The packet's VLAN tag (or lack thereof) matches frerAutSeqTagged (E.3.2.1.1.3).
   4) Either the vlan_identifier parameter is in the list in frerAutSeqVlan (E.3.2.1.1.4), or that list is empty.

c) If the packet does not match an entry in the Sequence autoconfiguration table, no further Autoconfiguration processing takes place. Otherwise (the packet did not match any tsnStreamIdEntry, but does match a frerAutSeqEntry), Autoconfiguration processing proceeds.

d) The system creates (autoconfigures) new entries in the Stream identity table (9.1), the Sequence recovery table (10.4). and the Sequence identification table (10.5) using the matched frerAutSeqEntry as follows:

   1) One new tsnStreamIdEntry (9.1.1) is created in the Stream identity table as follows:
      i) tsnStreamIdHandle (9.1.1.1) = an unused stream_handle value chosen by the implementation.
      ii) tsnStreamIdInFacOutputPortList (9.1.1.2) = empty.
      iii) tsnStreamIdOutFacOutputPortList (9.1.1.3) = empty.
      iv) tsnStreamIdInFacInputPortList (9.1.1.4) = empty.
      v) If frerAutSeqSeqEncaps equals either HSR or PRP, tsnStreamIdOutFacInputPortList (9.1.1.5) = frerAutSeqReceivePortList (E.3.2.1.1.2). Otherwise, frerSeqRcvyPortList (10.4.1.2) = only the port on which the packet was received.
      vi) tsnStreamIdIdentificationType (9.1.1.6) = 00-80-C2, 2 = Source MAC and VLAN Stream identification (6.5).
      vii) tsnStreamIdAutoconfigured (E.3.1.1) = True.
      viii) If frerAutSeqSeqEncaps equals either HSR or PRP, tsnStreamIdLanPathId (E.3.1.2) = the PathId or LanId field from the packet. Otherwise, tsnStreamIdLanPathId is not used.
      ix) tsnCpeSmacVlanDownSrcMac (9.1.3.1) = the source_mac_address parameter of the received packet.
      x) tsnCpeSmacVlanDownTagged (9.1.3.2) = frerAutSeqTagged (E.3.2.1.1.3).
      xi) tsnCpeSmacVlanDownVlan (9.1.3.3) = the vlan_identifier parameter of the received packet.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

2) If and only if frerAutSeqCreateIndividual (E.3.2.1.1.10) is True, a frerSeqRcvyEntry (10.4.1) is created in the Sequence recovery table (10.4) as follows:
   i) frerSeqRcvyStreamList (10.4.1.1) = the stream_handle from item d:1:i, above.
   ii) frerSeqRcvyPortList (10.4.1.2) = the same list as tsnStreamIdOutFacInputPortList (9.1.1.5), described in item d:1:v, above. (See E.4.)
   iii) frerSeqRcvyDirection (10.4.1.3) = True (out-facing).
   iv) frerSeqRcvyReset (10.4.1.4) = True. (The state machine is reset when created.)
   v) frerSeqRcvyAlgorithm (10.4.1.5) = Match_Alg (00-80-C2, 1, see Table 10-1).
   vi) frerSeqRcvyHistoryLength (10.4.1.6) is not used.
   vii) frerSeqRcvyResetMSec (10.4.1.7) = frerAutSeqResetMSec (E.3.2.1.1.7).
   viii) frerSeqRcvyInvalidSequenceValue (10.4.1.8) is a read-only value chosen by the system.
   ix) frerSeqRcvyTakeNoSequence (10.4.1.9) = True.
   x) frerSeqRcvyIndividualRecovery (10.4.1.10) = True.
   xi) frerSeqRcvyLatentErrorDetection (10.4.1.11) = False.
3) One new passive frerSeqEncEntry (10.5.1) is created in the Sequence identification table to decode further received packets as follows:
   i) frerSeqEncStreamList (10.5.1.1) = the stream_handle from item d:1:i, above.
   ii) frerSeqEncPort (10.5.1.2) = frerAutSeqReceivePortList (E.3.2.1.1.2).
   iii) frerSeqEncDirection (10.5.1.3) = True (out-facing).
   iv) frerSeqEncActive (10.5.1.4) = False (passive).
   v) frerSeqEncEncapsType (10.5.1.5) = frerAutSeqSeqEncaps (E.3.2.1.1.1).
   vi) If frerAutSeqSeqEncaps == 00-80-C2, 2 (HSR), frerSeqEncPathIdLanId (10.5.1.6) = the PathId field of the HSR sequence tag (7.8) found in the packet. If frerAutSeqSeqEncaps == 00-80-C2, 3 (PRP), frerSeqEncPathIdLanId = the LanId field of the PRP sequence trailer (7.9) found in the packet. Otherwise, frerSeqEncPathIdLanId is not used.

   If a frerSeqEncEntry with the same port list, direction, and encapsulation type is present, a new entry need not be created; the packet's stream_handle can be added to that frerSeqEncEntry's frerSeqEncStreamList.
4) One new active frerSeqEncEntry (10.5.1) is created in the Sequence identification table for each frerAutOutEntry (E.3.2.2.1) in the Output autoconfiguration table (E.3.2.2) to format transmitted packets for this packet's Stream as follows:
   i) frerSeqEncStreamList (10.5.1.1) = the stream_handle from item d:1:i, above.
   ii) frerSeqEncPort (10.5.1.2) = frerAutOutPortList (E.3.2.2.1.1).
   iii) frerSeqEncDirection (10.5.1.3) = True (out-facing).
   iv) frerSeqEncEncapsType (10.5.1.5) = frerAutOutEncaps (E.3.2.2.1.2).
   v) frerSeqEncPathIdLanId (10.5.1.6) = frerAutOutLanPathId (E.3.2.2.1.3).

   If a frerSeqEncEntry with the same port list, direction, encapsulation type, and PathId/LanId is present, a new entry need not be created; the packet's stream_handle can be added to that frerSeqEncEntry's frerSeqEncStreamList.

e) If none of the ports to which the received packet is to be output are listed in the frerAutSeqEntry's frerAutSeqRecoveryPortList (E.3.2.1.1.5), or if frerAutSeqCreateRecovery (E.3.2.1.1.11) is false, Autoconfiguration processing is terminated. No Sequence recovery function need be instantiated.

f) The system next scans the Stream identity table looking for a tsnStreamIdEntry that nearly matches the received packet, in order to discover whether there are other Streams similar to the received packet's Stream such that they are all Member Streams of a Compound Stream. "Nearly matches" means that all of the following conditions are met:
   1) tsnStreamIdIdentificationType (9.1.1.6) == 00-80-C2, 2 == Source MAC and VLAN Stream identification (6.5).
   2) The packet's stream_handle is not equal to tsnStreamIdHandle (9.1.1.1).
   3) The source_mac_address parameter of the packet == tsnCpeSmacVlanDownSrcMac (9.1.3.1).

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

4) Either tsnCpeSmacVlanDownVlan (9.1.3.3) is in the frerAutSeqVlan (E.3.2.1.1.4) list, or that list is empty.

g) If no tsnStreamIdEntry is found that meets these criteria, then Autoconfiguration processing is terminated. No Sequence recovery function need be instantiated.

h) If a tsnStreamIdEntry is found that does meet these criteria, then either Base recovery functions need to be instantiated, or existing instantiations need to be modified to include the received packet's Member Stream. To determine which, the system searches the Sequence recovery table for a frerSeqRcvyEntry that meets all of the following criteria:

1) frerSeqRcvyStreamList contains the stream_handle of the found tsnStreamIdEntry.
2) frerSeqRcvyIndividualRecovery (10.4.1.10) == False == Sequence recovery function.
3) frerSeqRcvyPortList (10.4.1.2) == frerAutSeqRecoveryPortList (E.3.2.1.1.5).
4) For a relay system, frerSeqRcvyDirection (10.4.1.3) is False (in-facing). For an end system, True (out-facing).

i) If the Sequence recovery table search (item h, above) is found, then the following steps are taken:

1) The packet's stream_handle is added to the frerSeqRcvyEntry's frerSeqRcvyStreamList (10.4.1.1).
2) If frerAutSeqLatErrDetection (E.3.2.1.1.12) is True, then the following steps are taken:
   i) frerSeqRcvyLatentErrorPaths (10.4.1.12.3) is incremented by 1.
   ii) The LatentErrorReset function (7.4.4.3) is called for the instantiation of the Latent error detection function (7.4.4) on every port in the frerSeqRcvyEntry's frerSeqRcvyPortList (10.4.1.2).

j) Otherwise (the Sequence recovery table search in item h, above, failed to find a frerSeqRcvyEntry), a new frerSeqRcvyEntry is created as follows:

1) frerSeqRcvyStreamList (10.4.1.1) = two stream_handle values, the stream_handle from the received packet (item d:1:i, above) and the tsnStreamIdHandle (9.1.1.1) from the tsnStreamIdEntry found in item f, above.
2) frerSeqRcvyPortList (10.4.1.2) = frerAutSeqRecoveryPortList (E.3.2.1.1.5).
3) frerSeqRcvyDirection (10.4.1.3) = False (in-facing) for a relay system, or True (out-facing) for an end system.
4) frerSeqRcvyReset (10.4.1.4) = True. (The state machine is reset when created.)
5) frerSeqRcvyAlgorithm (10.4.1.5) = frerAutSeqAlgorithm (E.3.2.1.1.8).
6) frerSeqRcvyHistoryLength (10.4.1.6) = frerAutSeqAlgorithm (E.3.2.1.1.8). This value is not used if frerSeqRcvyAlgorithm (10.4.1.5) == Match_Alg (00-80-C2, 1, see Table 10-1).
7) frerSeqRcvyResetMSec (10.4.1.7) = frerAutSeqResetMSec (E.3.2.1.1.7).
8) frerSeqRcvyInvalidSequenceValue (10.4.1.8) is a read-only value chosen by the system.
9) frerSeqRcvyTakeNoSequence (10.4.1.9) = True.
10) frerSeqRcvyIndividualRecovery (10.4.1.10) = False.
11) frerSeqRcvyLatentErrorDetection (10.4.1.11) = frerAutSeqLatErrDetection (E.3.2.1.1.12).
12) if frerAutSeqLatErrDetection is True, then the following assignments are made:
   i) frerSeqRcvyLatentErrorDifference (10.4.1.12.1) = frerAutSeqLatErrDifference (E.3.2.1.1.13).
   ii) frerSeqRcvyLatentErrorPeriod (10.4.1.12.2) = frerAutSeqLatErrPeriod (E.3.2.1.1.14).
   iii) frerSeqRcvyLatentErrorPaths (10.4.1.12.3) = 2.
   iv) frerSeqRcvyLatentResetPeriod (10.4.1.12.4) = frerAutSeqLatErrResetPeriod (E.3.2.1.1.15).

k) The packet is then processed according to the newly created entries.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

l) After the reception of the packet that triggers Autoconfiguration for a stream_handle, if none of the Individual recovery functions (7.4.5) or Base recovery functions (7.4.2) instantiated by a given frerAutSeqEntry (E.3.2.1.1) increment any packet counters (10.8) for longer than the period specified by the frerAutSeqEntry's frerAutSeqDestructMSec (E.3.2.1.1.6) object, the system can destroy the state machines created by that frerAutSeqEntry.

NOTE 3—As Member Streams are found, instances of the Base recovery function are created, and the number of Member Flows increased to enable Latent Error Detection. If the destruct timer (frerAutSeqDestructMSec, E.3.2.1.1.6) is not much larger than the normal reset period for the state machines (frerSeqRcvyResetMSec, 10.4.1.7), there is a risk that the state machines will be destroyed due to a temporary service interruption, that one or more of the Member Streams will fail to be restored after the interruption, and the purpose of Latent Error Detection will be defeated.

E.3 Autoconfiguration management

E.3.1 Additional tsnStreamIdEntry managed objects

Two managed objects augment each tsnStreamIdEntry (9.1.1) in the Stream identity table (9.1) when Managed objects for autoconfiguration (E.3.2) is implemented.

E.3.1.1 tsnStreamIdAutoconfigured

A read-only Boolean value, supplied by the system, specifying whether this entry was created explicitly (False) or via the Sequence autoconfiguration table (E.3.2.1, True).

E.3.1.2 tsnStreamIdLanPathId

An integer specifying a path or LAN. If and only if a packet matches an entry in the Sequence identification table (10.5) that specifies HSR or PRP in its frerSeqEncEncapsType (10.5.1.5) object, tsnStreamIdLanPathId specifies the LanId or PathId value that must be matched for this tsnStreamIdEntry to apply. A value of –1 indicates that the LanId or PathId are to be ignored.

E.3.2 Managed objects for autoconfiguration

E.3.2.1 Sequence autoconfiguration table

There is one Sequence autoconfiguration table per system. It contains any number of table entries (E.3.2.1.1). No two (or more) entries in the Sequence autoconfiguration table can have the same values for frerAutSeqSeqEncaps (E.3.2.1.1.1), frerAutSeqTagged (E.3.2.1.1.3), and frerAutSeqVlan (E.3.2.1.1.4) on any given port (E.3.2.1.1.2).

E.3.2.1.1 frerAutSeqEntry

Each frerAutSeqEntry objects (E.3.2.1.1) relates to a single class of Streams, and specifies how entries are created (and destroyed) in the Stream identity table (9.1), the Sequence recovery table (10.4), and the Sequence identification table (10.5).

E.3.2.1.1.1 frerAutSeqSeqEncaps

An enumerated value from Table 10-2, specifying which Sequence encode/decode function, and therefore, which type sequence_number encoding, is to be recognized for the purposes of Autoconfiguration.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

E.3.2.1.1.2 frerAutSeqReceivePortList

The list of ports to which this frerAutSeqEntry applies, and on which Stream identification functions (6.2), Sequence encode/decode functions (7.5), and Individual recovery functions (7.4.5) are to be autocreated.

E.3.2.1.1.3 frerAutSeqTagged

An enumerated value indicating whether packets to be matched by this frerAutSeqEntry are permitted to have a VLAN tag. It can take the following values:

1) **tagged:** A frame must have a VLAN tag to be matched.
2) **priority:** A frame must be untagged, or have a VLAN tag with a VLAN ID = 0 to be matched.
3) **all:** A frame is matched whether tagged or not.

E.3.2.1.1.4 frerAutSeqVlan

A list of vlan_identifiers for the packet to match. A null list matches all vlan_identifiers.

E.3.2.1.1.5 frerAutSeqRecoveryPortList

The list of ports on which Base recovery functions (7.4.2) are to be autocreated by this frerAutSeqEntry.

E.3.2.1.1.6 frerAutSeqDestructMSec

An integer number of milliseconds after which an idle set of functions created by this frerAutSeqEntry can be destroyed. A value of 0 indicates that idle autoconfigured functions are not to be destroyed. Default value is 86 400 000 decimal (one day).

E.3.2.1.1.7 frerAutSeqResetMSec

The value used to fill frerSeqRcvyResetMSec (10.4.1.7) when autoconfiguring entries in the Sequence recovery table.

E.3.2.1.1.8 frerAutSeqAlgorithm

The value used to fill frerSeqRcvyAlgorithm (10.4.1.5) when autoconfiguring entries in the Sequence recovery table.

E.3.2.1.1.9 frerAutSeqHistoryLength

The value used to fill frerSeqRcvyHistoryLength (10.4.1.6) when autoconfiguring entries in the Sequence recovery table.

E.3.2.1.1.10 frerAutSeqCreateIndividual

A Boolean value. If True, the receipt of a packet that triggers the autoconfiguration of a new tsnStreamIdEntry also triggers the instantiation of a frerSeqRcvyEntry for an Individual recovery function.

E.3.2.1.1.11 frerAutSeqCreateRecovery

A Boolean value. If True, the receipt of a packet that triggers the autoconfiguration of a new tsnStreamIdEntry can also trigger the instantiation of a frerSeqRcvyEntry for a Sequence recovery function.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

E.3.2.1.1.12 frerAutSeqLatErrDetection

A Boolean value. If True, the autoconfiguration of a new Sequence recovery function also creates an associated Latent Error Detection function.

E.3.2.1.1.13 frerAutSeqLatErrDifference

The value used to fill frerSeqRcvyLatentErrorDifference (10.4.1.12.1) when autoconfiguring entries in the Sequence recovery table.

E.3.2.1.1.14 frerAutSeqLatErrPeriod

The value used to fill frerSeqRcvyLatentErrorPeriod (10.4.1.12.2) when autoconfiguring entries in the Sequence recovery table.

E.3.2.1.1.15 frerAutSeqLatErrResetPeriod

The value used to fill frerSeqRcvyLatentResetPeriod (10.4.1.12.4) when autoconfiguring entries in the Sequence recovery table.

E.3.2.2 Output autoconfiguration table

There is one Output autoconfiguration table per system. It contains any number of frerAutOutEntry objects (E.3.2.2.1), each relating to a single class of Streams specifying how active entries are created in the Sequence identification table (10.5).

E.3.2.2.1 frerAutOutEntry

No two (or more) entries in the Output autoconfiguration table can include the same port in their frerAutSeqReceivePortList objects (E.3.2.1.1.2).

E.3.2.2.1.1 frerAutOutPortList

The list of ports to which this frerAutOutEntry applies, and on which active Sequence encode/decode functions (7.5) are to be autocreated.

E.3.2.2.1.2 frerAutOutEncaps

An enumerated value from Table 10-2, specifying which Sequence encode/decode function, and therefore, which type sequence_number encoding, is to be used for autoconfigured Streams on the ports in frerAutSeqReceivePortList (E.3.2.1.1.2).

E.3.2.2.1.3 frerAutOutLanPathId

An integer specifying a path or LAN. If and only if frerAutOutEncaps (E.3.2.2.1.2) specifies HSR or PRP frerAutOutLanPathId specifies the LanId or PathId value to be inserted into the HSR sequence tag or PRP sequence trailer of autoconfigured packets transmitted on the ports in frerAutSeqReceivePortList (E.3.2.1.1.2).

E.4 Use of autoconfiguration

Every last detail of the operation of FRER on each individual Stream can be configured by using the managed objects in Clause 9 and Clause 10. Often, however, there is enough regularity in the use of FRER

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

across various Streams in a network that Autoconfiguration can be used. There are four steps to using Autoconfiguration in a given relay system or end system, as follows:

a) Deciding on which port(s) packets belonging to Member Streams will be received and transmitted, using what translations are required that do not use active Stream identification functions (E.4.1).
b) Deciding which packets can trigger Autoconfiguration (E.4.2).
c) Deciding which method for encoding the sequence_number subparameter will be used for input and output on each port (E.4.3).
d) Deciding on whether Individual recovery functions, Base recovery functions, or both, are to be automatically instantiated, and what controlling parameters are to be used for each instantiation (E.4.4).

NOTE—Although the following examples describe interoperation between HSR/PRP and the R-TAG, such interoperability is not the primary reason for defining autoconfiguration. Autoconfiguration greatly simplifies the amount of configuration required for networks that use only the R-TAG.

E.4.1 Routing and labeling Member Streams

Figure E-1 illustrates an example network with four Member Streams constituting a Compound Stream. This example is similar to that illustrated in Figure 7-3.

In general, the selection of paths taken by Member Streams through the network is not determined by FRER, but by other standards. Autoconfiguration does not provide a means of establishing active Stream identification functions to perform per-Stream identification transformations. In Figure E-1, let us assume for the sake of example that Member Stream 1 and Member Stream 2 both have the same destination MAC address, but are on two special VLANs 1000 and 1001, used exclusively in this network for fully managed pinned-down paths. That is, these two VLANs are not controlled by the topology protocols defined in IEEE Std 802.1Q. Let us further suppose that Member Stream 3 and Member Stream 4 are similar, but both use the same VLANs 2000. (Presumably, source address learning is disabled on this VLAN.)

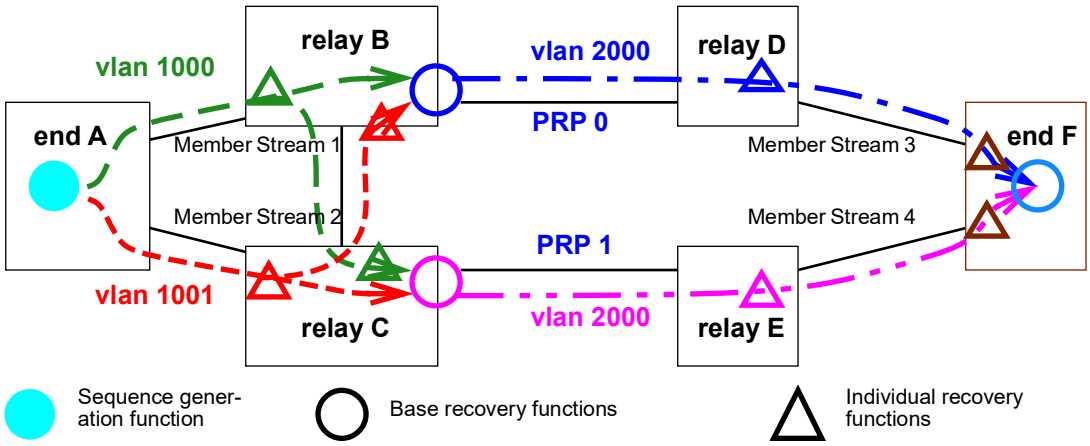


Figure E-1—Autoconfiguration example

Looking at relay system B in Figure E-1, we can see that all packets in Member Stream 3 are to be output on VLAN 2000 with a PRP sequence trailer specifying LanId 0. That means that packets from Member Streams 1 and 2 that are accepted by the Sequence recovery function in relay system B must be translated from VLAN 1000 or 1001 to VLAN 2000. The Autoconfiguration facility does not automatically setup or change vlan_identifiers. Relay system B must be configured to perform a VLAN ID translation on its right-hand port, translating both VLAN 1000 and VLAN 1001 to VLAN 2000 on output. If there are a great many Compound Streams, then presumably they would share the same VLAN assignments and thus make use of the same VLAN translations. IEEE 802.1Q bridges have such a VLAN ID translation capability.

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Although not shown in Figure E-1, we can imagine that there are Streams with Talkers in the right side of the figure and listeners on the left. Additional VLAN mapping would then be required on the left and bottom ports of relay system B to map VLAN 2000 to VLAN 1000 and VLAN 1001. If the VLAN situation were significantly more complex, then the network might not be suitable for Autoconfiguration.

E.4.2 Recognizing packets that trigger autoconfiguration

In the example in Figure E-1, let us consider only relay system B, and only packets that follow the same routes as Member Streams 1 and 2. In that case, we expect relay system B to receive packets suitable for Autoconfiguration only on its left and bottom ports, all encoded with the R-TAG (7.7). Thus, an administrator would construct a frerAutSeqEntry (E.3.2.1.1) in the Sequence autoconfiguration table specifying that tag on those ports, for VLANs 1000 and 1001. This entry will, when a matching packet is received, create a tsnStreamIdEntry (9.1.1) in the Sequence identification table (10.5) using Source MAC and VLAN Stream identification (6.5). Since the R-TAG contains no indication as to which Member Stream a packet belongs, the entry is tied to the port and VLAN on which the packet was received. In this way, all of the Member Streams from end system A that take the path of Member Stream 1 are assigned one stream_handle, and all those following the path of Member Stream 2 get a second stream_handle.

Again, we can imagine that there are Streams with Talkers in the right side of Figure E-1 and listeners on the left. A second frerAutSeqEntry is required in the Sequence autoconfiguration table is required to recognize, for Autoconfiguration, the packets coming in on relay system B's right-hand port, and create entries in the Sequence identification table. This single entry specifies packets encoded with the PRP sequence trailer (see a further explanation of this in E.4.3), and includes all PRP ports—in this example, just the one. A tsnStreamIdEntry created for HSR or PRP by this entry that assigns packets a stream_handle would not be tied to the port and VLAN on which the packet was received, but to all ports covered by the tsnStreamIdEntry, and to the LanId or PathId contained in the packet. That is, the contents of the HSR sequence tag or the PRP sequence trailer override the port number.

E.4.3 Per-port packet decoding and encoding

The example in Figure E-1 also illustrates a second use of Autoconfiguration. In this network, everything to the right of relay systems B and C, including the rightmost ports on relay systems B and C, use the PRP sequence trailer (7.9), whereas all ports on relay systems A, B, and C to the left use the R-TAG (7.7). Packets transmitted to the right by relay system B, and all packets transmitted by system D and the upper port of system F are marked as being on LanId 0, and by other ports on the right side of the figure as being on LanId 1. Packets transmitted by any other port on relay systems B and C, or by system A, use the R-TAG. These distinctions are made using the Output autoconfiguration table (E.3.2.2). Using this table, each port is assigned an output encapsulation, including the LanId or PathId for PRP or HSR, respectively. Again, if the usage is too complex (e.g., one Stream is LanId 0 on a port, while another is LanId 1 on that same port), Autoconfiguration cannot be used.

Note that the administrator could configure the Sequence autoconfiguration tables and Output autoconfiguration tables in relay systems D and E to translate among the R-TAG, the HSR sequence tag, and the PRP sequence trailer for the purpose of interworking, even if no Individual recovery functions or Base recovery functions were desired in those relay systems.

E.4.4 Individual and Sequence recovery functions

Given the explanations in E.4.1, E.4.2, and E.4.3, the reader can consult E.2 to understand how Individual recovery functions and Base recovery functions are configured as a result of Autoconfiguration. They are set

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

up, along with the input (passive) Stream identification functions, by entries in the Sequence autoconfiguration table (E.3.2.1). The reader may find the following facts helpful:

a) At most one instance of the Individual recovery function is instantiated for each stream_handle associated with an autocreated tsnStreamIdEntry (9.1.1).
b) One instance of the Sequence recovery function is instantiated on each expected output port (frerAutSeqRecoveryPortList, E.3.2.1.1.5) for each Compound Stream.
c) Compound Streams are inferred by associating Member Streams with the same Source MAC address and VLAN ID.

E.5 Autoconfiguration YANG model

The UML data model for Autoconfiguration is illustrated in Figure E-2 and Figure E-3. It consists in augmentations of the Stream identification and FRER models (12.2.1 and 12.2.2).

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

sequence-generation (10.3)		
uint32	index	// r-w
handle-ref	* stream	// r-w
boolean	direction-out-facing	// r-w
boolean	reset	// r-w

sequence-recovery (10.4)		
uint32	index	// r-w
handle-ref	* stream	// r-w
if-ref	* port	// r-w
boolean	direction-out-facing	// r-w
boolean	reset	// r-w
sequence-recovery-algorithm	algorithm	// r-w
sequence-history-length	history-length	// r-w
uint32	reset-timeout	// r-w
uint32	invalid-sequence-value	// r
boolean	take-no-sequence	// r-w
boolean	individual-recovery	// r-w
boolean	latent-error-detection	// r-w
int32	latent-error-difference	// r-w
uint32	latent-error-period	// r-w
uint16	latent-error-paths	// r-w
uint32	latent-error-reset-period	// r-w

sequence-identification (10.5)		
handle-ref	* stream	// r-w
if-ref	port	// r-w
boolean	direction-out-facing	// r-w
boolean	active	// r-w
enum	encapsulation	// r-w
int8	path-id-lan-id	// r-w

stream-split (10.6)		
uint32	index	// r-w
if-ref	port	// r-w
boolean	direction-out-facing	// r-w
leaf-ref	* input-id	// r-w
leaf-ref	* output-id	// r-w

output-autoconfiguration (E.3.1)		
uint32	index	// r-w
if-ref	* port	// r-w
enum	encapsulation	// r-w
int8	lan-path-id	// r-w

sequence-autoconfiguration (E.3.2)		
uint32	index	// r-w
enum	sequence-encapsulation	// r-w
if-ref	* receive-port	// r-w
enum	tagged	// r-w
dot1qtypes:vlanid	* vlan	// r-w
if-ref	* recovery-port	// r-w
uint64	destruction-interval	// r-w
uint64	reset-interval	// r-w
sequence-recovery-algorithm	algorithm	// r-w
sequence-history-length	history-length	// r-w
boolean	create-individual	// r-w
boolean	create-recovery	// r-w
boolean	latent-error-detection	// r-w
int32	latent-error-difference	// r-w
uint32	latent-error-period	// r-w
uint32	latent-error-reset-period	// r-w

Figure E-2—FRER augmentation

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

1

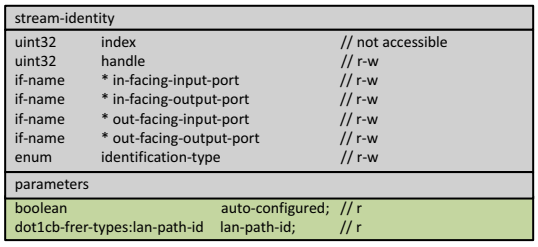


Figure E-3—FRER Stream identification augmentation

2 E.5.1 Structure of the ieee802-dot1cb-autoconfiguration YANG module

3 The FRER autoconfiguration model consists of the *ieee802-dot1cb-autoconf* YANG module along with all
4 the dependencies (YANG imports) that the module uses. E.5.2.1 contains the YANG data schema tree for the
5 *ieee802-dot1cb-autoconf* module.

6 The high-level structure of the *ieee802-dot1cb-autoconf* YANG module is found in Table E-4.

7 The list of YANG modules directly imported by the *ieee802-dot1cb-autoconf* YANG module is found in
8 Table E-5.

Table E-4—ieee802-dot1cb-autoconfiguration structure and relationship to this standard

Module	References	Notes
ieee802-dot1cb-mask-and-match		—
additional stream-identity parameters	E.3.1	Autoconfiguration parameters.
autocofiguration tables parameters	E.3.2	Autoconfiguration parameters.

9

Table E-5—YANG module dependencies for the autoconfiguration model

YANG module
ietf-interfaces
ieee802-dot1cb-stream-identification-types
ieee802-dot1cb-stream-identification
ieee802-dot1cb-frer-types
ieee802-dot1cb-frer

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

To complete the model, all the dependencies from the imported modules must also be identified. The process to determine all the dependencies can be done through tooling. For example, if the pyang [B15] or yanglint tool [B16] is used on the *ieee802-dot1cb-autoconf* YANG module, the tooling will try to include all the imports and produce an error message if an import is missing. The YANG Catalog [B17] search tools and/or the YANG Catalog repository [B18] can be used to find the missing imports.

E.5.2 Definition of the ieee802-dot1cb-autoconf YANG module

E.5.2.1 YANG data scheme definition for the ieee802-dot1cb-autoconf YANG module

```
module: ieee802-dot1cb-autoconf
  augment /dot1cb-frer:frer:
     +--rw autoconfiguration {auto-configuration}?
        +--rw sequence* [index]
        |  +--rw index                          uint32
        |  +--rw sequence-encapsulation
        |  |  +--rw (encapsulation)?
        |  |     +--:(r-tag)
        |  |     |  +--rw r-tag!
        |  |     |     +--ro type-number?
        |  |     |     |       dot1cb-frer-types:seq-encaps-method
        |  |     |     +--ro oui-cid?       string
        |  |     +--:(hsr-sequence-tag)
        |  |     |  +--rw hsr-sequence-tag!
        |  |     |     +--ro type-number?
        |  |     |     |       dot1cb-frer-types:seq-encaps-method
        |  |     |     +--ro oui-cid?       string
        |  |     +--:(prp-sequence-tag)
        |  |     |  +--rw prp-sequence-tag!
        |  |     |     +--ro type-number?
        |  |     |     |       dot1cb-frer-types:seq-encaps-method
        |  |     |     +--ro oui-cid?       string
        |  |     +--:(organization-specific)
        |  |        +--rw organization-specific!
        |  |           +--rw type-number?   int32
        |  |           +--rw oui-cid?       string
        |  +--rw receive-port*                if:interface-ref
        |  +--rw tagged?                      enumeration
        |  +--rw vlan*                        dot1qtypes:vlanid
        |  +--rw recovery-port*               if:interface-ref
        |  +--rw destruction-interval?        uint64
        |  +--rw reset-interval?              uint64
        |  +--rw algorithm
        |  |  +--rw (algorithm)?
        |  |     +--:(vector)
        |  |     |  +--rw vector!
        |  |     |     +--ro type-number?
        |  |     |     |       dot1cb-frer-types:seq-rcvy-algorithm
        |  |     |     +--ro oui-cid?       string
        |  |     +--:(match)
        |  |     |  +--rw match!
        |  |     |     +--ro type-number?
        |  |     |     |       dot1cb-frer-types:seq-rcvy-algorithm
        |  |     |     +--ro oui-cid?       string
        |  |     +--:(organization-specific)
        |  |        +--rw organization-specific!
        |  |           +--rw type-number?   int32
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
        |  |           +--rw oui-cid?       string
        |  +--rw history-length?              sequence-history-length
        |  +--rw create-individual?           boolean
        |  +--rw create-recovery?             boolean
        |  +--rw latent-error-detection?      boolean
        |  +--rw latent-error-difference?     int32
        |  +--rw latent-error-period?         uint32
        |  +--rw latent-error-reset-period?   uint32
        +--rw output* [index]
           +--rw index            uint32
           +--rw port*            if:interface-ref
           +--rw encapsulation
           |  +--rw (encapsulation)?
           |     +--:(r-tag)
           |     |  +--rw r-tag!
           |     |     +--ro type-number?
           |     |     |       dot1cb-frer-types:seq-encaps-method
           |     |     +--ro oui-cid?       string
           |     +--:(hsr-sequence-tag)
           |     |  +--rw hsr-sequence-tag!
           |     |     +--ro type-number?
           |     |     |       dot1cb-frer-types:seq-encaps-method
           |     |     +--ro oui-cid?       string
           |     +--:(prp-sequence-tag)
           |     |  +--rw prp-sequence-tag!
           |     |     +--ro type-number?
           |     |     |       dot1cb-frer-types:seq-encaps-method
           |     |     +--ro oui-cid?       string
           |     +--:(organization-specific)
           |        +--rw organization-specific!
           |           +--rw type-number?   int32
           |           +--rw oui-cid?       string
           +--rw lan-path-id?     lan-path-id

  augment /dot1cb-sid:stream-identity:
    +--ro auto-configured?   boolean {auto-configuration}?
    +--rw lan-path-id?       lan-path-id {auto-configuration}?
```

E.5.2.2 ieee802-dot1cb-autoconf YANG module

```
module ieee802-dot1cb-autoconf {
  yang-version "1.1";
  namespace urn:ieee:std:802.1Q:yang:ieee802-dot1cb-autoconf;
  prefix dot1cb-autoconf;

  import ieee802-dot1cb-frer {
    prefix dot1cb-frer;
  }
  import ieee802-dot1cb-frer-types {
    prefix dot1cb-frer-types;
  }
  import ieee802-dot1cb-stream-identification {
    prefix dot1cb-sid;
  }
  import ieee802-dot1cb-stream-identification-types {
    prefix dot1cb-sid-types;
  }
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
import ietf-interfaces {
  prefix if;
}

organization
  "Institute of Electrical and Electronics Engineers";
contact
  "WG-URL: http://ieee802.org/1/
  WG-EMail: stds-802-1-l@ieee.org

  Contact: IEEE 802.1 Working Group Chair
  Postal: C/O IEEE 802.1 Working Group
          IEEE Standards Association
          445 Hoes Lane
          Piscataway, NJ 08854
          USA

  E-mail: stds-802-1-chairs@ieee.org";
description
  "Management objects that support the automatic configuration, upon receipt
     of a packet, of entries in the following tables: Stream identity table,
     Sequence generation table, Sequence recovery table and Sequence
     identification table. They are described in Clauses E.3.1 and E.3.2 of
     IEEE Std 802.1CB-2027.";

revision 2026-08-18 {
  description
    "Published as part of IEEE Std 802.1CB-2027, Frame Replication and
       Elimination for Reliability.";
  reference
    "Annex E of IEEE Std 802.1CB-2027.";
}

feature auto-configuration {
  description
    "Autoconfiguration of entries in the tables for Stream identity
    table, sequence recovery table, and sequence identification table.";
  reference
    "Annex E of IEEE Std 802.1CB-2027";
}

augment "/dot1cb-frer:frer" {
  container autoconfiguration {
    if-feature "auto-configuration";
    description
      "Container for Autoconfiguration managed objects.";
    reference
      "Annex E of IEEE Std 802.1CB-2027";
    list sequence {
      key "index";
      description
        "There is one Sequence autoconfiguration table per system. It
        contains any number of table entries. No two (or more) entries
        in the Sequence autoconfiguration table can have the same
        values for frerAutSeqSeqEncaps, frerAutSeqTagged, and
        frerAutSeqVlan on any given port. Each frerAutSeqEntry object
        relates to a single class of Streams, and specifies how entries
        are created (and destroyed) in the Stream identity table, the
        Sequence recovery table, and the Sequence identification table.";
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
        reference
          "E.3.2.1 of IEEE Std 802.1CB-2027";
        leaf index {
          type uint32;
          description
            "Entry in the sequence list referencing to a single class of
            Streams.";
        }
        container sequence-encapsulation {
          description
            "An enumerated value, specifying which Sequence encode/decode
            function, and therefore, which type sequence_number encoding,
            is to be recognized for the purposes of Autoconfiguration.";
          reference
            "E.3.2.1.1.1 of IEEE Std 802.1CB-2027";
          uses sequence-encode-decode;
        }
        leaf-list receive-port {
          type if:interface-ref;
          min-elements 1;
          description
            "The list of ports to which this frerAutSeqEntry applies, and
            on which Stream identification functions, Sequence
            encode/decode functions, and Individual recovery functions
            are to be autocreated.";
          reference
            "E.3.2.1.1.2 of IEEE Std 802.1CB-2027";
        }
        leaf tagged {
          type enumeration {
            enum tagged {
              value 1;
              description
                "A frame must have a VLAN tag to be matched.";
            }
            enum priority {
              value 2;
              description
                "A frame must be untagged, or have a VLAN tag with a VLAN
                ID = 0 to be matched.";
            }
            enum all {
              value 3;
              description
                "A frame is matched whether tagged or not.";
            }
          }
          description
            "An enumerated value indicating whether packets to be matched
            by this frerAutSeqEntry are permitted to have a VLAN tag.";
          reference
            "E.3.2.1.1.3 of IEEE Std 802.1CB-2027";
        }
        leaf-list vlan {
          type dot1qtypes:vlanid;
          description
            "A list of vlan_identifiers for the packet to match. A null
            list matches all vlan_identifiers.";
          reference
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
        "E.3.2.1.1.4 of IEEE Std 802.1CB-2027";
    }
    leaf-list recovery-port {
      type if:interface-ref;
      min-elements 1;
      description
        "The list of ports on which Sequence recovery functions are
        to be autocreated by this frerAutSeqEntry.";
      reference
        "E.3.2.1.1.5 of IEEE Std 802.1CB-2027";
    }
    leaf destruction-interval {
      type uint64;
      units "ms";
      default "86400000";
      description
        "An integer number of milliseconds after which an idle set of
        functions created by this frerAutSeqEntry can be destroyed. A
        value of 0 indicates that idle autoconfigured functions are
        not to be destroyed. Default value is 86 400 000 decimal (one
        day).";
      reference
        "E.3.2.1.1.6 of IEEE Std 802.1CB-2027";
    }
    leaf reset-interval {
      type uint64;
      units "ms";
      description
        "The value used to fill frerSeqRcvyResetMSec when
        autoconfiguring entries in the Sequence recovery table.";
      reference
        "E.3.2.1.1.7 of IEEE Std 802.1CB-2027";
    }
    container algorithm {
      description
        "The value used to fill frerSeqRcvyAlgorithm when
        autoconfiguring entries in the Sequence recovery table.";
      reference
        "E.3.2.1.1.8 of IEEE Std 802.1CB-2027";
      uses sequence-recovery-algorithm;
    }
    leaf history-length {
      type sequence-history-length;
      default "2";
      description
        "The value used to fill frerSeqRcvyHistoryLength when
        autoconfiguring entries in the Sequence recovery table.";
      reference
        "E.3.2.1.1.9 of IEEE Std 802.1CB-2027";
    }
    leaf create-individual {
      type boolean;
      description
        "A Boolean value. If True, the receipt of a packet that
        triggers the autoconfiguration of a new tsnStreamIdEntry also
        triggers the instantiation of a frerSeqRcvyEntry for an
        Individual recovery function.";
      reference
        "E.3.2.1.1.10 of IEEE Std 802.1CB-2027";
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
        }
        leaf create-recovery {
          type boolean;
          description
            "A Boolean value. If True, the receipt of a packet that
            triggers the autoconfiguration of a new tsnStreamIdEntry can
            also trigger the instantiation of a frerSeqRcvyEntry for a
            Sequence recovery function.";
          reference
            "E.3.2.1.1.11 of IEEE Std 802.1CB-2027";
        }
        leaf latent-error-detection {
          type boolean;
          description
            "A Boolean value. If True, the autoconfiguration of a new
            Sequence recovery function also creates an associated Latent
            Error Detection function.";
          reference
            "E.3.2.1.1.12 of IEEE Std 802.1CB-2027";
        }
        leaf latent-error-difference {
          type int32;
          description
            "The value used to fill frerSeqRcvyLatentErrorDifference when
            autoconfiguring entries in the Sequence recovery table.";
          reference
            "E.3.2.1.1.13 of IEEE Std 802.1CB-2027";
        }
        leaf latent-error-period {
          type uint32;
          units "ms";
          default "2000";
          description
            "The value used to fill frerSeqRcvyLatentErrorPeriod when
            autoconfiguring entries in the Sequence recovery table.";
          reference
            "E.3.2.1.1.14 of IEEE Std 802.1CB-2027";
        }
        leaf latent-error-reset-period {
          type uint32;
          units "ms";
          default "30000";
          description
            "The value used to fill frerSeqRcvyLatentResetPeriod when
            autoconfiguring entries in the Sequence recovery table.";
          reference
            "E.3.2.1.1.15 of IEEE Std 802.1CB-2027";
        }
      }
      list output {
        key "index";
        description
          "There is one Output autoconfiguration table per system. It
          contains any number of frerAutOutEntry objects, each relating
          to a single class of Streams specifying how active entries are
          created in the Sequence identification table. No two (or more)
          entries in the Output autoconfiguration table can include the
          same port in their frerAutSeqReceivePortList objects.";
        reference
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
          "E.3.2.2 of IEEE Std 802.1CB-2027";
        leaf index {
          type uint32;
          description
            "Entry in the output list referencing to a single class of
            Streams.";
        }
        leaf-list port {
          type if:interface-ref;
          min-elements 1;
          description
            "The list of ports to which this frerAutOutEntry applies, and
            on which active Sequence encode/decode functions are to be
            autocreated.";
          reference
            "E.3.2.2.1.1 of IEEE Std 802.1CB-2027";
        }
        container encapsulation {
          description
            "An enumerated value, specifying which Sequence encode/decode
            function, and therefore, which type sequence_number encoding,
            is to be used for autoconfigured Streams on the ports in
            frerAutSeqReceivePortList.";
          reference
            "E.3.2.2.1.2 of IEEE Std 802.1CB-2027";
          uses sequence-encode-decode;
        }
        leaf lan-path-id {
          type lan-path-id;
          description
            "An integer specifying a path or LAN. If and only if
            frerAutOutEncaps specifies HSR or PRP frerAutOutLanPathId
            specifies the LanId or PathId value to be inserted into the
            HSR sequence tag or PRP sequence trailer of autoconfigured
            packets transmitted on the ports in
            frerAutSeqReceivePortList.";
          reference
            "E.3.2.2.1.3 of IEEE Std 802.1CB-2027";
        }
      }
    }
  }
  augment "/dot1cb-sid:stream-identity" {
    if-feature "auto-configuration";
    description
      "Two managed objects augment each tsnStreamIdEntry in the Stream
      identity table when Managed objects for autoconfiguration is
      implemented.";
    reference
      "E.3.1 of IEEE Std 802.1CB-2027";
    leaf auto-configured {
      type boolean;
      config false;
      description
        "A read-only Boolean value, supplied by the system, specifying
        whether this entry was created explicitly (False) or via the
        Sequence autoconfiguration table.";
      reference
        "E.3.1.1 of IEEE Std 802.1CB-2027";
```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

```
    }
    leaf lan-path-id {
      type lan-path-id;
      description
        "An integer specifying a path or LAN. If and only if a packet
        matches an entry in the Sequence identification table that
        specifies HSR or PRP in its frerSeqEncEncapsType object,
        tsnStreamIdLanPathId specifies the LanId or PathId value that
        must be matched for this tsnStreamIdEntry to apply. A value of -1
        indicates that the LanId or PathId are to be ignored.";
      reference
        "E.3.1.2 of IEEE Std 802.1CB-2027";
    }
  }
}

```


Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

Annex F

(informative)

Bibliography

Bibliographical references are resources that provide additional or helpful material but do not need to be understood or used to implement this standard. Reference to these resources is made for informational use only.

[B1] IEC 61158-1:2019, Industrial communication networks—Fieldbus specifications—Part 1: Overview and guidance for the IEC 61158 and IEC 61784 series.

[B2] IEEE Std 802.1BA-2021, IEEE Standard for Local and Metropolitan Area Networks—Audio Video Bridging (AVB) Systems.[23, 24]

[B3] IEEE Std 802d-2017, IEEE Standard for Local and Metropolitan Area Networks: Overview and Architecture—Allocation of Uniform Resource Name (URN) Values in IEEE 802® Standards.[25, 26]

[B4] IEEE Std 802.3™, IEEE Standard for Ethernet.

[B5] IEEE Std 1722™-2016, IEEE Standard for a Transport Protocol for Time-Sensitive Applications in Bridged Local Area Networks.

[B6] IETF RFC 3985, Bryant, S., Ed., and P. Pate, Ed., Pseudo Wire Emulation Edge-to-Edge (PWE3) Architecture, DOI 10.17487/RFC3985, March 2005.[27]

[B7] IETF RFC 6087, Guidelines for Authors and Reviewers of YANG Data Model Documents, Bierman, A., Jan. 2011.[28]

[B8] IETF RFC 6241, Network Configuration Protocol (NETCONF), Enns, R., M. Bjorklund, J. Schoenwaelder, and A. Bierman, Jun. 2011.

[B9] IETF RFC 6242, Using the NETCONF Protocol over Secure Shell (SSH), Wasserman, M., Jun. 2011.

[B10] IETF RFC 6536, Network Configuration Protocol (NETCONF) Access Control Model, Bierman, A., and M. Bjorklund, Mar. 2012.

[B11] IETF RFC 8343, A YANG Data Model for Interface Management, Bjorklund, M., Mar. 2018.

[B12] IETF RFC 8964, B. Varga, Ed., J. Farkas, L. Berger, A. Malis, S. Bryant, J. Korhonen, Deterministic Networking (DetNet) Data Plane: MPLS, January 2021.

[23]The IEEE standards or products referred to in Annex F are trademarks owned by The Institute of Electrical and Electronics Engineers, Incorporated.

[24]IEEE publications are available from The Institute of Electrical and Electronics Engineers (http://standards.ieee.org).

[25] IEEE publications are available from the Institute of Electrical and Electronics Engineers (https://standards.ieee.org/).

[26] The IEEE standards or products referred to in Annex F are trademarks of the Institute of Electrical and Electronics Engineers, Incorporated.

[27]IETF documents (i.e., RFCs) are available for download at http://www.rfc-archive.org/.

[28] IETF RFCs are available from the Internet Engineering Task Force (https://www.rfc-archive.org/).

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.

[B13] IETF RFC 9037, B. Varga, Ed., J. Farkas, A. Malis, S. Bryant, Deterministic Networking (DetNet) Data Plane: MPLS over IEEE 802.1 Time-Sensitive Networking (TSN), June 2021.

[B14] OMG Unified Modeling Language (OMG UML), Version 2.5, March 2015.[29]

[B15] pyang, pyang GitHub repository, https://github.com/mbj4668/pyang

[B16] yanglint, libyang GitHub repository, https://github.com/CESNET/libyang

[B17] YANG Catalog, YANG catalog website, https://yangcatalog.org/

[B18] YANG Catalog, YANG Catalog GitHub repository, https://github.com/YangModels/yang

[29] OMG documents are available from the Object Management Group (https://www.omg.org/).

Copyright © 2026 IEEE. All rights reserved.
This is an unapproved IEEE Standards Draft, subject to change.